



IMS
Institut für Mikroelektronische Systeme
Fachgebiet Architekturen und Systeme



Leibniz
Universität
Hannover

Masterarbeit

Implementierung und Evaluierung eines KI-Beschleunigers für einen RISC-V-Core
für Edge Computing

Leibniz Universität Hannover
Fakultät für Elektrotechnik und Informatik
Institut für Mikroelektronische Systeme
Fachgebiet Architekturen und Systeme

Omar Abedmouleh, B. Sc.

25. Februar 2026





IMS
Institut für Mikroelektronische Systeme
Fachgebiet Architekturen und Systeme



Leibniz
Universität
Hannover

Masterarbeit

Implementierung und Evaluierung eines KI-Beschleunigers für einen RISC-V-Core für Edge Computing

Implementation and evaluation of an AI accelerator for a RISC-V core for edge
computing

Leibniz Universität Hannover
Fakultät für Elektrotechnik und Informatik
Institut für Mikroelektronische Systeme
Fachgebiet Architekturen und Systeme

vorgelegt von

Omar Abedmouleh, B. Sc.
Matrikelnummer 10032429

Erstprüfer: Prof. Dr.-Ing. Holger Blume
Zweitprüfer: Prof. Dr.-Ing. Bernhard Wicht
Betreuer: Keivan Fayyazifard, M. Sc.

Hannover, 25. Februar 2026

Hannover, 25. Jul 2025

Masterarbeit

für Herrn Abed Mouleh

Implementierung und Evaluierung eines KI-Beschleunigers für einen RISC-V-Core für Edge Computing

In modernen Anwendungen gewinnt Edge Computing zunehmend an Bedeutung, da es eine lokale Datenverarbeitung mit geringer Latenz ermöglicht und gleichzeitig die Anforderungen an die Netzwerkbandbreite reduziert. Besonders in Bereichen wie autonomem Fahren, modernen Kommunikationssystemen (5G/6G) und anderen datenintensiven Szenarien ist die schnelle und effiziente Ausführung von KI- und ML-Algorithmen ein zentraler Erfolgsfaktor. Diese Algorithmen erlauben es, komplexe Datenströme in Echtzeit auszuwerten und unmittelbar Entscheidungen am Ort der Datenentstehung zu treffen.

Um KI-Anwendungen effizient am Edge ausführen zu können, sind spezialisierte Hardwarelösungen notwendig. Allgemeine Prozessorarchitekturen stoßen dabei häufig an ihre Grenzen. Ein Lösungsansatz besteht in der Entwicklung und Integration von KI-Beschleunigern, die bestimmte Rechenoperationen – wie Matrixmultiplikationen oder neuronale Netzwerkoperationen – hardwareseitig optimieren. Die offene RISC-V-Architektur bietet hier große Flexibilität, da sie sich durch benutzerdefinierte Erweiterungen gezielt an spezifische Anforderungen anpassen lässt. So kann die Kombination aus RISC-V-Core und spezialisierten Beschleunigern eine leistungs- und energieeffiziente Plattform für KI-Workloads im Edge-Bereich schaffen.

Das Fachgebiet „Architekturen und Systeme“ des Instituts für Mikroelektronische Systeme (IMS) beschäftigt sich mit der Entwicklung applikationsspezifischer Hardware und der systematischen Exploration von Architektorentwurfsräumen. Ein aktueller Forschungsschwerpunkt ist die Integration und Evaluierung von KI-Beschleunigern im Kontext RISC-V-basierter Edge-Plattformen. Ziel ist es, durch gezielte Hardware-Erweiterungen – etwa durch Matrix-Multiplikations- oder MAC-Einheiten – die Rechenleistung und Energieeffizienz bei AI/ML-Anwendungen zu erhöhen. Anhand typischer ML-Workloads sollen dabei konkrete Aussagen über Performancesteigerung und Effizienzgewinne getroffen werden.

Herr Abed Mouleh implementiert im Rahmen dieser Arbeit einen KI-Beschleuniger, der als AXI-basiertes Hardwaremodul realisiert wird. Der Beschleuniger verfügt über eine Konfigurationsschnittstelle sowie über einen DMA-Controller zur eigenständigen Datenübertragung zwischen Hauptspeicher und internen SRAM-Strukturen. Durch die standardisierte AXI-Anbindung ist das Modul prinzipiell in RISC-V-basierte SoC-Architekturen integrierbar und kann als Coprozessor in eine entsprechende Prozessorplattform eingebunden werden. Die Funktionalität und numerische Korrektheit werden mittels einer dedizierten Verifikationsumgebung sowie durch Vergleich mit einer Software-Referenz überprüft. Abschließend wird eine Front-End-Synthese der erweiterten Architektur unter Verwendung des ASAP7 PDK durchgeführt, um die Auswirkungen auf Leistung und Ressourceneffizienz zu evaluieren.

Auf gute Dokumentation ist zu achten. Die eingereichten Exemplare sowie die Ergebnisse der Arbeit bleiben Eigentum des Instituts.

Prof. Dr.-Ing. Holger Blume

Prof. Dr.-Ing Holger Blume
blume@ims.uni-hannover.de

Appelstraße 4 | 30167 Hannover
fon 0511 762-19640 | fax 0511 762-19601
www.ims.uni-hannover.de | info@ims.uni-hannover.de

Eigenständigkeitserklärung

Ich versichere hiermit, dass ich die vorstehende Arbeit selbstständig angefertigt, keine anderen als die angegebenen Hilfsmittel benutzt und sowohl wörtliche, als auch sinngemäß entlehnte Stellen als solche kenntlich gemacht habe. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

Omar Abedmouleh, B. Sc.

Hannover, den 25. Februar 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	VIII
Abbildungsverzeichnis	XIII
Tabellenverzeichnis	XIV
1 Einleitung	1
2 Grundlagen	3
2.1 Convolutional Neural Network	3
2.1.1 Convolution	3
2.1.2 Depthwise Convolution	5
2.1.3 Pointwise Convolution	7
2.1.4 Pooling	9
2.1.5 Aktivierungsfunktionen	9
2.2 Systolisches Array	10
2.2.1 Datenfluss im systolischen Array	10
2.3 Festkommazahlen und Quantisierung	13
2.4 Modell und Quantisierungsvarianten	14
2.4.1 Trade-offs von Full-INT8 gegenüber FP32	14
3 Stand der Technik	16
3.1 SoC-Integration	16
3.2 Systolische Arrays und Datenfluss	16
3.3 Effiziente CNN-Operatoren und Quantisierung	17
3.4 Erweiterte Optimierungsansätze	17
4 Implementierung	19
4.1 Übersicht der Architektur	19
4.1.1 Zielplattform und Integrationskonzept (RISC-V SoC)	21
4.2 AXI Module	22
4.2.1 Steuer- und Datenanbindung über AXI	22
4.2.2 Konfigurationsregister und AXI4-Lite Interface	23
4.2.3 DMA-Controller	27
4.3 Implementierung des Arithmetikmoduls	30
4.3.1 Arithmetikmodul: Überblick und Einordnung	30
4.3.2 Processing Element	31
4.3.3 Arithmetic Controller	38
4.3.4 Systolic Array Module	39
4.3.5 ReLU-Module	47

4.4	On-Chip-Speicher	47
4.4.1	I/O-SRAM	47
4.4.2	Weight-SRAM	53
4.4.3	Bias/Shift/Mult-SRAM	54
4.5	Fetching Module	55
4.5.1	Fetching Module: Überblick und Einordnung	55
4.5.2	Line Buffer und Fenstersammlung	56
4.5.3	Fetching-Controller	58
4.6	Processing Module	77
4.6.1	Daten- und Speicherpfad im Processing Module	77
4.6.2	Datenfluss und Rückschreiben	78
4.6.3	Pooling Module	80
5	Validierung	85
5.1	Validierungsstrategie	85
5.2	Validierung der Teilfunktionalitäten	85
5.3	Validierung der drei Betriebsmodi	87
5.4	Gesamtvalidierung des MobileNetV1-Datenpfads	92
6	Evaluation	94
6.1	Front-End-Synthese	94
6.1.1	Konfigurationen	94
6.1.2	Flächenergebnisse	95
6.1.3	Timing-Ergebnisse	97
6.2	Performanceanalyse	97
6.2.1	MAC Throughput	97
6.2.2	Utilization	100
6.3	Vergleich mit Referenzsystem	103
6.4	Diskussion	108
7	Zusammenfassung und Ausblick	111
	Literaturverzeichnis	115

Abkürzungsverzeichnis

KERNEL_ELEMS_MAX	Max. Kernel-Länge.
i_cell_en	Lokales Enable der aktuellen PE-Zelle.
i_kernel_elems	Anzahl MAC-Schritte pro Ausgabepixel.
i_kernel	8-bit Kernelwert am PE-Eingang.
i_pipeline_en	Globales Pipeline-Enable.
i_pixel	8-bit Eingabepixelwert am PE-Eingang.
i_requant_shift	Shift-Faktor zur Requantisierung.
o_cell_en	Registrierte Weitergabe von i_cell_en zum Nachbar-PE.
o_kernel	Registrierte Weitergabe von i_kernel zum Nachbar-PE.
o_pixel_result	8-bit Ausgabepixel des PEs.
o_pixel_valid	Gültigkeitssignal für o_pixel.
o_pixel	Registrierte Weitergabe des Eingabepixels i_pixel zum Nachbar-PE.
ASIC	Application Specific Integrated Circuit.
C_{in}	Anzahl der Eingangskanäle einer Feature-Map.
CNN	Convolutional Neural Network.
C_{out}	Anzahl der Ausgabekanäle einer Faltungs- bzw. Convolution-Schicht.
CPU	Central Processing Unit.
CSR	Control and Status Register.
ctx	linearisierter Kontextindex $ctx(w, h, c)$ mit fester räumlicher Position und Eingangskanal.
CW	Convolutional Window.
CWN	Anzahl möglicher Fensterpositionen.
DL	Deep Learning.
DMA	Direct Memory Access.
DNN	Deep Neural Network.
Festkomma	Zahlenformat mit festem Skalierungsfaktor.
FIFO	First In First Out.
FP16	16-Bit-Gleitkommaformat.
FP32	32-Bit-Gleitkommaformat.
FPGA	Field Programmable Gate Array.
FSM	Finite State Machine.
GEMM	General Matrix–Matrix Multiplication.
GPU	Graphics Processing Unit.
H	Höhe einer Feature-Map.

I/O	Input/Output.
INFH	Input Feature Map Height.
INFMAP	Input Feature Map.
INFW	Input Feature Map Width.
INT16	16-Bit-Ganzzahlenformat.
INT8	8-Bit-Ganzzahlenformat.
IS	Input-Stationary.
K	Anzahl der Ausgabekanäle (Filter) einer Faltungsschicht; in dieser Arbeit gilt $K = C_{\text{out}}$.
KE	Kernel Elements pro Filter; $KE = KH \cdot KW \cdot C_{\text{in}}$.
KH	Kernel Height.
KI	Künstliche Intelligenz.
KN	Anzahl der Filter (Kernel Number); entspricht der Anzahl der Ausgabekanäle OCHN.
KW	Kernel Width.
MAC	Multiply–Accumulate.
MLP	Multi-Layer Perceptron.
MSB	Most Significant Bit, höchstwertiges Bit eines binären Wortes, das bei vorzeichenbehafteten Darstellungen das Vorzeichen kodiert.
OCHE	Output Channel Height Elements; entspricht der Anzahl Fensterpositionen CWN.
OCHN	Anzahl der Ausgabekanäle (Output Channel Number); in dieser Arbeit $OCHN = C_{\text{out}}$.
OFH	Output Feature Map Height.
OFMAP	Output Feature Map.
OFW	Output Feature Map Width.
OS	Output-Stationary.
P	Padding (Nullauffüllung).
PE	Processing Element.
PFMAP	Pooling Feature Map.
POH	Pooling Feature Map Height.
POW	Pooling Feature Map Width.
RAM	Random Access Memory.
ReLU	Rectified Linear Unit.
ROM	Read-Only Memory.
RTL	Register-Transfer Level.
SA	systolisches Array.
Sigmoid	Logistische Aktivierungsfunktion.
SoC	System-on-Chip.
Softmax	Softmax-Funktion.
Stride	Schrittweite der Faltung.
tanh	Hyperbolische Tangens-Funktion.
TPU	Tensor Processing Unit.
W	Breite einer Feature-Map.

WHC Anzahl der Kontexte $WHC = W \cdot H \cdot C$ einer Schicht.
WS Weight-Stationary.

Abbildungsverzeichnis

2.1	MobileNet-Architektur (nach [1]).	3
2.2	Schematische Darstellung einer Faltung: ein Kernel gleitet über die Eingabe-Feature-Maps und erzeugt eine Ausgabe-Feature-Map.	4
2.3	Beispiel einer Standard-Faltung: Eine Eingabe mit drei Kanälen wird mit 16 Filtern der Größe $3 \times 3 \times 3$ gefaltet und erzeugt eine Ausgabe mit 16 Kanälen. Nach [2].	5
2.4	Prinzip der Depthwise-Convolution: kanalweise räumliche Faltung ohne Kanalmischung.	6
2.5	Beispiel einer Depthwise-Convolution: Jeder Eingangskanal wird separat mit einem 3×3 -Kernel gefaltet; es entsteht eine Ausgabe mit gleicher Kanalzahl. Nach [2].	7
2.6	Prinzip der Pointwise-Convolution (1×1): pro Pixelposition werden die C_{in} Eingangskanäle linear zu C_{out} Ausgabekanälen kombiniert.	7
2.7	Beispiel einer Pointwise-Convolution: $3 \times 3 \times 3$ gefaltet mit $(1 \times 1 \times 3) \times 16$ ergibt $3 \times 3 \times 16$. Nach [2].	8
2.8	Beispiel für kanalweises Max-Pooling (2×2 , Schrittweite 2); die Output Feature Map (OFMAP) wird zur Pooling Feature Map (PFMAP) verdichtet.	9
2.9	Abbildung der 2D-Faltung auf eine Matrizenmultiplikation: links das Aufrollen der Eingabe-Patches, in der Mitte das Stapeln der Filter, rechts die entstehenden Ausgabespalten.	11
2.10	Output-Stationary-Datenfluss (OS): Teilsummen bleiben in den PEs, Eingaben werden von links und Gewichte von oben eingespeist und lokal akkumuliert. . . .	13
4.1	Systemübersicht des CNN-Beschleunigers im Testbench-Aufbau.	20
4.2	Zustandsdiagramm des DMA-Controllers mit sequenziellen Ladephasen (Input, Weights, Bias, Shift, Mult) sowie optionalem Write-Back.	27
4.3	Processing Element: Datenpfad und Steuerlogik im PE.	32
4.4	Mul-Block zur 32×32 -Multiplikation: Zerlegung in Low-/High-Anteile, Bildung der vier Teilprodukte $P_1..P_4$ und anschließende Zusammenführung im Add-Block. . .	35
4.5	Add-Block: Zusammensetzen des 64-bit Zwischenprodukts a aus den registrierten Teilprodukten $P_1..P_4$	36
4.6	Zustandsautomat des Arithmetic Controllers zur Laufzeitsteuerung der Rechenpipeline.	38
4.7	Datenfluss im Convolution-Modus (<i>output-stationary</i>): Aktivierungen (von links) und Gewichte (von oben) strömen durch das Array, während die partiellen Summen im jeweiligen PE verbleiben und über K_{elems} MAC-Schritte akkumuliert werden; die farbigen Zeilen zeigen aufeinanderfolgende Fensterpositionen $Window_Pos_i$ über alle Kanäle.	40

4.8	Datenfluss im Pointwise-Modus (1×1): Für eine feste Pixelposition (x, y) werden die Kanalwerte seriell eingespeist, während pro Takt ein Gewichtsvektor für <code>cfg_tile_c</code> Ausgabekanäle von oben in das Array gestreamt wird. In der aktuellen Implementierung ist <code>cfg_tile_r = 1</code> , sodass ausschließlich die oberste PE-Zeile aktiv ist.	43
4.9	Datenfluss im Depthwise-Modus (output-stationary): pro Kanal wird ein separater Faltungsfilter verwendet. Die Aktivierungen werden zeilenweise (links) und die zugehörigen Kanal-Gewichte spaltenweise (oben) eingespeist; aktiv sind ausschließlich die diagonalen PEs $PE_{r,r}$	45
4.10	MUX-basierte Realisierung der ReLU-Aktivierung: negative Werte werden auf 0 abgebildet, nicht-negative Werte unverändert durchgereicht.	47
4.11	Zustandsautomat der Feature-I/O-SRAM für das Streaming-Lesen: <code>S_IDLE</code> wartet auf <code>start</code> . In <code>S_STREAM</code> werden die (gepaddingten) Bildkoordinaten (x, y) traversiert und Read-Anforderungen an die synchronen Kanalbänke gestellt; die gelesenen Werte stehen aufgrund der 1-Takt-Read-Latenz im Folgetakt zur Verfügung und werden als lane-paralleles Pixelwort <code>px_out_lane</code> ausgegeben. Die Übernahme durch den nachfolgenden Pfad wird über das Handshake <code>px_valid/px_ready</code> gesteuert. In <code>S_DONE</code> wird <code>done_frame</code> gesetzt und anschließend nach <code>S_IDLE</code> zurückgekehrt.	50
4.12	Softwareseitig erzeugtes Weight-Layout in der Weight-SRAM (Zielordnung $[c][h][w][k]$ mit k als schnellster Dimension): Für ein fixes (c, h, w) liegen die Gewichte über die Output-Kanäle k direkt hintereinander. Pro Fetch werden <code>cfg_tile_c</code> aufeinanderfolgende Output-Kanal-Gewichte als Tile genutzt.	54
4.13	Prinzip der Line-Buffer-Pipeline zur 3×3 -Fensterbildung aus einem Pixelstream: zwei Zeilenpuffer halten vorherige Bildzeilen, während Schieberegister die letzten Spaltenwerte der drei beteiligten Zeilen speichern [3].	56
4.14	Zustandsautomat des <code>fetching_module</code> . Der Controller trennt die Erzeugung der Fenster (Scan-Phase) von der nachfolgenden, taktsynchronen Streaming-Phase, in der pro Kontext <code>cfg_tile_r</code> Aktivierungen und <code>cfg_tile_c</code> Gewichte (inkl. Bias/Shift/Mult/Mult pro Block) bereitgestellt werden.	59
4.15	Scan-Vorgehensweise im Zustand <code>S_SCAN</code> : kanalsynchrone Fensterausgabe über mehrere Line-Buffer, Packen von <code>cfg_tile_r</code> Fensterpositionen pro Wort und Schreiben der gepackten Wörter in wortadressierte Window-RAM-Instanzen. Beispiel: <code>cfg_cin = 3</code> , <code>cfg_tile_r = 4</code> ; die vierte Instanz bleibt ungenutzt, da nur <code>ch < cfg_cin</code> beschrieben wird und Pack-Puffer vorab deterministisch auf 0 gesetzt werden.	61
4.16	Pointwise-Datenerfassung in <code>S_SCAN</code> für <code>LANES_MAX = 4</code> und <code>cfg_cin = 8</code> : zwei Beats werden nacheinander aufgenommen und im <code>pw_buffer</code> als <code>Beat_0</code> (Kanäle $c = 0..3$) und <code>Beat_1</code> (Kanäle $c = 4..7$) abgelegt.	64
4.17	Gewichtestreaming: Initialisierung <code>raddr_pointer = base_addr + blk_idx</code> und Kontextschritte mit Sprungweite <code>total_blks</code>	66
4.18	Convolution: Lesen eines gepackten Window-RAM-Worts pro Kanal (<code>cv_word_addr = grp_idx</code>) und Byte-Auswahl über <code>pix_sel / chan_sel</code>	68
4.19	Depthwise: Pro Gruppe wird dieselbe Fensterposition über alle Kanäle gelesen; Fensterwahl im gepackten Wort über <code>dw_lane_sel</code> und Wortwahl über <code>dw_word_addr</code>	70

4.20	Pointwise-Streaming aus dem Puffer <code>pw_buffer</code> am Beispiel <code>LANES_MAX = 4</code> und <code>cfg_cin = 8</code> : Das Arbeitspixel wird in zwei Beats gepuffert (Beat 0: $c = 0..3$, Beat 1: $c = 4..7$) und anschließend kanalweise seriell in <code>ifmap_array[0]</code> ausgegeben.	71
4.21	Übergangslogik für Gruppen- und Blockwechsel. Links: nächstes Gruppenwort innerhalb desselben Blocks. Rechts: Blockwechsel (inkl. Pointer-/Index-Umschaltung) mit anschließendem Bias/Shift/Mult-Laden (Convolution), neuem Scan (Depthwise) bzw. Pixel-Fortschritt (Pointwise).	74
4.22	Write-Back-FSM für das Rückschreiben ohne Pooling: Nach <code>done_acc_pulse</code> iteriert die FSM deterministisch über das Tile und schreibt die Ergebnisse byteweise in die I/O-SRAM.	79
4.23	Übersicht über das Befüllen der Pooling-Lines pro Gruppe und die Bildung eines Pooling-Stripes aus zwei OFMAP-Zeilen. Pro Kanaloffset c existieren zwei Lines (<code>pool_line0</code> für die gerade Zeile, <code>pool_line1</code> für die folgende ungerade Zeile).	81
4.24	Sliding der Pooling-Unit über einen Stripe: Für einen Kanaloffset c wird ein 2×2 -Fenster aus <code>pool_line0/1</code> gelesen, gepoolt und ausgegeben; danach wird x fortgeschaltet. Nach Abschluss der Stripe-Breite wird zum nächsten Kanaloffset c gewechselt.	83
4.25	Pooling-Controller-FSM: <code>S_RESET</code> $\rightarrow$ <code>S_IDLE</code> (Warten auf <code>start_pool</code>) $\rightarrow$ <code>S_POOLING</code> (Stripe verarbeiten) $\rightarrow$ <code>S_IDLE</code> .	84
6.1	Layerweiser Vergleich des mittleren MAC-Throughputs (MAC/s) für 4×4 und 8×8 .	100
6.2	Layerweiser Vergleich für 4×4 und 8×8 .	101
6.3	Kritischer Pfad der untersuchten Konfigurationen.	104
6.4	Ausführungszeit der MobileNetV1-Inferenz für Ibex (Scalar/Vektor) und den KI-Beschleuniger (4×4 / 8×8).	105
6.5	Flächenbedarf der untersuchten Konfigurationen.	106
6.6	Konzept zur Blockverarbeitung bei <i>Convolution</i> mit $C_{in} > LANES_MAX$: Zwischenspeicherung und Akkumulation von Partialsummen über mehrere Kanalblöcke.	110

Tabellenverzeichnis

2.1	TFLite-Varianten für MobileNetV1 auf EuroSAT: Genauigkeit und Modellgröße. .	14
4.1	CSR-Registerübersicht (Byte-adressiert)	23
4.2	AXI4-Lite-Kanäle und zugehörige Signale [4]	26
4.3	AXI4-Master-Schnittstelle des DMA-Controllers: verwendete Kanäle und Signale	30
5.1	Zusätzlich getestete Stride-/Padding-Kombinationen (außerhalb MobileNetV1). .	92
6.1	Parameterübersicht für die Synthese (128×128). In den Spalten der Array-Größen ändern sich ausschließlich LANES_MAX, COLS_MAX sowie die daraus abgeleiteten Maximalwerte; alle übrigen Parameter bleiben konstant.	95
6.2	Flächenergebnisse für verschiedene Array-Konfigurationen. Die Fläche ist als <i>Total cell area</i> in μm^2 und in mm^2 angegeben.	95
6.3	Timing-Ergebnisse für verschiedene Array-Konfigurationen.	97
6.4	Layerweiser MAC-Durchsatz für die 4×4 -Konfiguration.	99
6.5	Layerweiser MAC-Durchsatz für die 8×8 -Konfiguration.	99
6.6	Layerweise Utilization für 4×4 und 8×8	101
6.7	Kritischer Pfad, minimale Taktperiode und maximale Taktfrequenz.	104
6.8	Ausführungszeit der MobileNetV1-Inferenz.	105
6.9	Speedup und Zeitverkürzung.	105
6.10	Flächenbedarf der untersuchten Konfigurationen (Ki-Beschleuniger ohne RAM-Makros)	106
6.11	Skalierungsfaktor α_A relativ zu verschiedenen Baselines (Ki-Beschleuniger ohne RAM-Makros).	107
6.12	Zyklusanteile nach Layer-Typ für MobileNetV1.	107
6.13	VGG16-Architektur (Input $224 \times 224 \times 3$) [5].	109
6.14	LeNet-5-Architektur (Input $32 \times 32 \times 1$) [6].	109

1 Einleitung

Edge Computing gewinnt zunehmend an Bedeutung: Daten sollen möglichst nah am Entstehungsort mit geringer Latenz verarbeitet werden, um Netzwerkbandbreite zu sparen und Echtzeit-Entscheidungen zu ermöglichen. Das ist entscheidend u. a. im autonomen Fahren, in 5G/6G-Kommunikationssystemen und in anderen datenintensiven Anwendungen. Wenn insbesondere Convolutional Neural Network (CNN)-Algorithmen effizient am Rand ausgeführt und beschleunigt werden, erreicht man die Ziele des Edge Computings: schnelle Inferenz, weniger Datenverkehr und bessere Energieeffizienz [7, 8].

CNN sind zentrale Bausteine in Computer Vision, Objekterkennung und Bildverarbeitung [7, 8]. Mit ihrem Erfolg sind jedoch Parametermengen und Rechenaufwand stark gewachsen. Gleichzeitig verlieren klassische Skalierungseffekte (Moore/Dennard) an Wirkung; allgemeine Plattformen stoßen bei Leistung und Energieeffizienz an Grenzen [9, 10].

CPUs sind für kontrollflusslastige, heterogene Aufgaben optimiert und liefern hohe Single-Thread-Leistung. Für datenparallele CNN-Workloads sind sie jedoch im Nachteil: begrenzte Vektorbreiten, geringere Parallelität im Vergleich zu spezialisierten Arrays und die Memory-Wall verhindern eine dauerhafte, energieeffiziente Auslastung—insbesondere im Edge. GPUs bieten zwar massive Parallelität, sind jedoch nicht speziell auf typische Deep Learning (DL)-Datenflüsse zugeschnitten und weisen im Edge-Kontext häufig eine zu hohe Leistungsaufnahme auf [11].

Daher rücken spezialisierte Beschleuniger auf Field Programmable Gate Array (FPGA)- und Application Specific Integrated Circuit (ASIC)-Basis in den Fokus: Rechenkerne und Datenpfade lassen sich gezielt auf CNN-Operationen zuschneiden und erreichen so ein besseres Verhältnis aus Durchsatz und Energiebedarf [12]. Eine besonders geeignete Mikroarchitektur ist das SA: lokale Kommunikation, feste Datenflüsse, hohe Operandreuse und schlanke Steuerlogik erlauben hohen Durchsatz bei begrenzter Speicherbandbreite [13].

In dieser Arbeit liegt der Fokus auf der Beschleunigung von KI-Anwendungen im Edge-Umfeld durch einen hardwarebasierten Beschleuniger auf Grundlage eines systolischen Arrays. Dazu werden die charakteristischen Eigenschaften von CNN hinsichtlich Rechen- und Datenflusstruktur analysiert, um zu verstehen, wie sich diese effizient auf Hardware abbilden lassen; die daraus abgeleiteten Wiederverwendungs- und Parallelitätsmuster werden gezielt ausgenutzt, um den Durchsatz zu erhöhen und die Datenbewegung zu reduzieren. Im Rahmen dieser Arbeit wird hierfür ein KI-Beschleuniger als *AXI-basiertes Hardwaremodul* realisiert, der eine Konfigurationsschnittstelle zur Laufzeitparametrierung bereitstellt und einen DMA-Controller integriert, sodass Gewichte, Bias- und Requantisierungsparameter sowie Feature-Maps eigenständig zwischen hostseitigem Speicher und internen SRAM-Strukturen übertragen werden können. Durch die standardisierte AXI-Anbindung ist das Modul prinzipiell in RISC-V-basierte SoC-Architekturen integrierbar und kann dort als Coprozessor in eine entsprechende Prozessorplattform eingebunden werden.

Die Arbeit ist in mehrere Kapitel gegliedert, die den Weg von den Grundlagen über die Implementierung bis zur Validierung und Bewertung des entwickelten KI-Beschleunigers nachvollziehbar

strukturieren. Kapitel 2 stellt die fachlichen Grundlagen bereit; dazu gehören die wesentlichen Rechenoperationen von CNNs, Pooling und Aktivierungsfunktionen sowie die Einordnung systemischer Arrays und der verwendeten Quantisierungsarithmetik. In Kapitel 4 wird anschließend die Architektur des entwickelten KI-Beschleunigers beschrieben. Kapitel 5 dokumentiert die stufenweise Verifikation und weist die funktionale Korrektheit nach, beginnend bei Modultests über die Integration der drei Betriebsmodi bis zur End-to-End-Ausführung des vollständigen MobileNetV1-Datenpfads mit bitgenauem HW–SW-Vergleich. Darauf aufbauend analysiert Kapitel 6 die quantitativen Hardwarekennzahlen der Architektur, insbesondere Flächenbedarf, Timing und Performance, und ordnet die Ergebnisse im Vergleich zu CPU-basierten Referenzsystemen ein. Abschließend fasst Kapitel 7 die zentralen Erkenntnisse zusammen und diskutiert Grenzen sowie naheliegende Erweiterungen für eine breitere Modellunterstützung.

2 Grundlagen

2.1 Convolutional Neural Network

In dieser Arbeit steht die effiziente Inferenz im Edge-Umfeld im Fokus; entsprechend wird im Folgenden das *MobileNet*-Modell als Referenzarchitektur herangezogen, da es explizit für mobile und eingebettete Vision-Anwendungen entworfen wurde und durch die verwendeten Bausteine ein günstiges Verhältnis aus Genauigkeit, Latenz und Rechenaufwand ermöglicht.

In der Praxis ist MobileNet zudem ein häufiges Zielmodell für Edge-Deployments, etwa im Kontext von TensorFlow Lite bzw. spezialisierten Edge-Plattformen.

Abbildung 2.1 zeigt den prinzipiellen Aufbau: Auf eine initiale Faltung folgen wiederholte *depthwise separable* Blöcke, die eine räumliche *Depthwise*-Faltung (kanalweise 3×3) mit einer anschließenden 1×1 -*Pointwise*-Faltung zur Kanalmischung kombinieren; typischerweise werden diese durch Normalisierung und ReLU-Nichtlinearität ergänzt.

Die Auflösungsreduktion erfolgt an ausgewählten Stufen über Downsampling, bevor am Ende ein *global average pooling* die Feature-Maps verdichtet und eine abschließende Klassifikationsschicht die Ausgabe liefert [14].

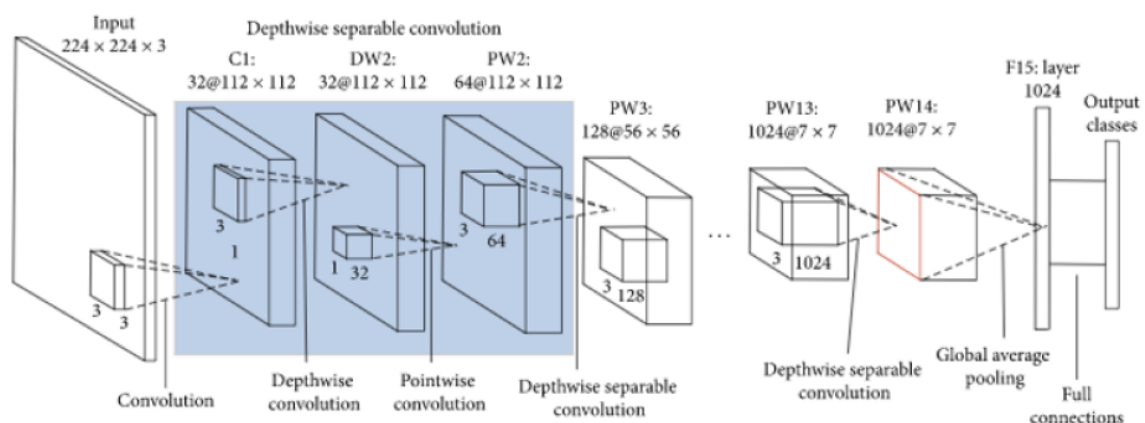


Abbildung 2.1: MobileNet-Architektur (nach [1]).

2.1.1 Convolution

CNN sind eine Klasse von Deep Neural Network (DNN) und der De-facto-Standard für viele Aufgaben der Computer Vision, z. B. Bildklassifikation, Objekterkennung und semantische Segmentierung [7, 8]. Zentrales Prinzip ist die Ausnutzung lokaler Bildstruktur durch *Faltungen* mit

geteilten Gewichten: kleine Filter (Kerne) werden über die Eingabe verschoben und erzeugen dabei *Feature-Maps*. Die Parameter werden datengetrieben mittels Gradientenverfahren und Backpropagation gelernt.

Abbildung 4.7 zeigt den Grundmechanismus: Die Eingabe ist die Input Feature Map (INFMAP) mit Input Feature Map Height (INFH) $\times$ Input Feature Map Width (INFW) und C_{in} Kanälen. Eine Faltungsschicht besitzt C_{out} Kerne der Größe Kernel Height (KH) $\times$ Kernel Width (KW); jeder Kern erstreckt sich über alle Eingangskanäle ($KH \times KW \times C_{in}$). Für jede Ausgabeposition wird ein lokales Eingabefenster mit dem Kern elementweise multipliziert und aufsummiert, typischerweise mit Bias. Das Fenster wird mit Schrittweite Schrittweite der Faltung (Stride) verschoben; optional wird eine Nullauffüllung Padding (Nullauffüllung) (P) verwendet. Die räumlichen Ausgabedimensionen Output Feature Map Height (OFH) und Output Feature Map Width (OFW) ergeben sich aus:

$$OFH = \frac{INFH + 2P - KH}{\text{Stride}} + 1 \quad (2.1)$$

$$OFW = \frac{INFW + 2P - KW}{\text{Stride}} + 1 \quad (2.2)$$

Durch das Stapeln mehrerer Faltungsschichten entstehen hierarchische Merkmalsrepräsentationen: frühe Schichten erfassen einfache Muster (z. B. Kanten), tiefere Schichten kombinieren diese zu komplexeren Strukturen.

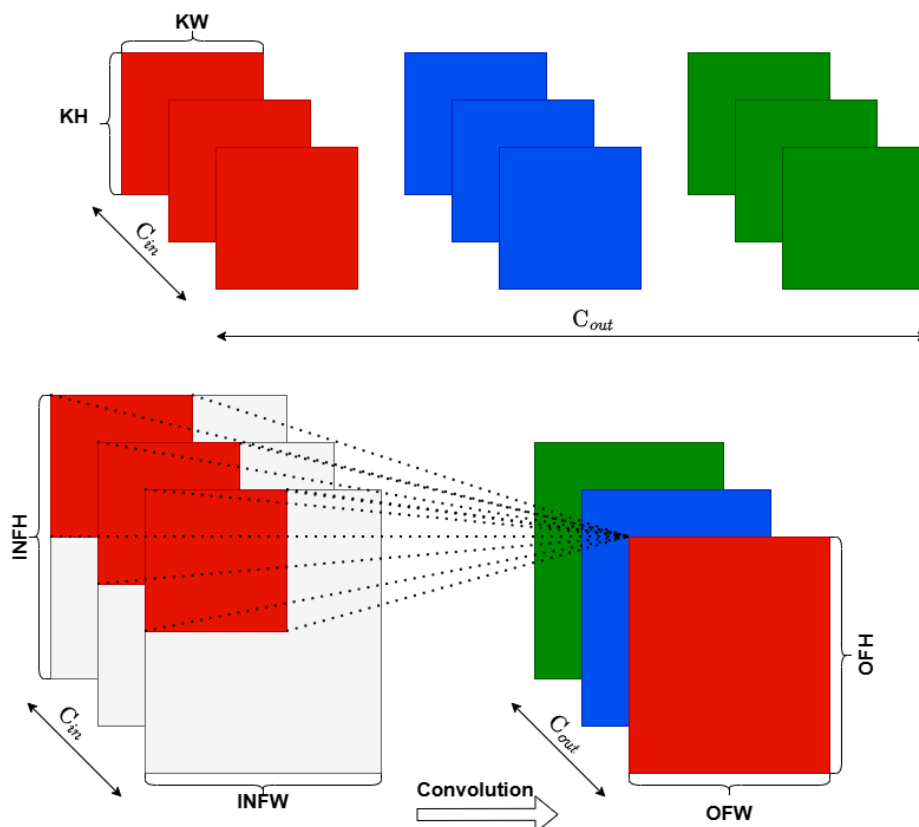


Abbildung 2.2: Schematische Darstellung einer Faltung: ein Kernel gleitet über die Eingabe-Feature-Maps und erzeugt eine Ausgabe-Feature-Map.

Der Rechenaufwand einer Standard-Convolution wird durch die Anzahl der *Multiply-Accumulates* (MACs) bestimmt.: Für jeden Ausgabepixel wird ein Skalarprodukt zwischen Kernel und Eingabe-

befenster berechnet. Für eine Eingabe mit C_{in} Kanälen und C_{out} Filtern der Größe $KH \times KW$ ergibt sich:

$$\#MACs = OFH \cdot OFW \cdot C_{\text{out}} \cdot (KH \cdot KW \cdot C_{\text{in}}). \quad (2.3)$$

Die Anzahl der Parameter (ohne Bias) beträgt entsprechend:

$$\#Weights = C_{\text{out}} \cdot KH \cdot KW \cdot C_{\text{in}}. \quad (2.4)$$

Am Beispiel aus Abbildung 2.3 mit Eingabe $5 \times 5 \times 3$, Kernel $3 \times 3 \times 3$ und Ausgabe $3 \times 3 \times 16$ gilt: $OFH \cdot OFW = 3 \cdot 3 = 9$, $KH \cdot KW \cdot C_{\text{in}} = 3 \cdot 3 \cdot 3 = 27$, $C_{\text{out}} = 16$. Damit ergeben sich $9 \cdot 27 \cdot 16 = 3888$ MACs und $16 \cdot 27 = 432$ Gewichte [2]. MobileNet wurde

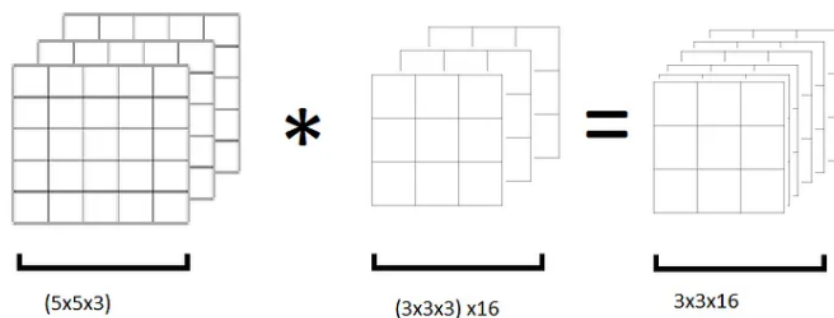


Abbildung 2.3: Beispiel einer Standard-Faltung: Eine Eingabe mit drei Kanälen wird mit 16 Filtern der Größe $3 \times 3 \times 3$ gefaltet und erzeugt eine Ausgabe mit 16 Kanälen. Nach [2].

für ressourcenbeschränkte Edge-Plattformen entwickelt und zielt darauf ab, die Rechenlast und Parameterzahl bei möglichst geringer Genauigkeitseinbuße zu reduzieren [14, 2]. Der zentrale Baustein ist die *depthwise separable convolution*, die eine Standard-Faltung in eine kanalweise *Depthwise*-Faltung und eine anschließende 1×1 -*Pointwise*-Faltung zur Kanalmischung zerlegt. Dadurch sinkt der Rechenaufwand im Vergleich zur Standard-Convolution deutlich, was MobileNet besonders geeignet für latenz- und energieeffiziente Inferenz im Edge-Umfeld macht [14, 2].

2.1.2 Depthwise Convolution

Abbildung 2.4 zeigt das Prinzip der *Depthwise Convolution*: Die räumliche Filterung erfolgt *kanalweise*. Für jeden Eingangskanal existiert ein eigener Kernel der Größe $KH \times KW$; der Kernel umfasst damit *genau einen* Kanal. Dadurch findet *keine Kanalmischung* statt: Jeder Eingangskanal wird separat gefaltet und erzeugt genau einen Ausgabekanal, sodass im Standardfall $C_{\text{out}} = C_{\text{in}}$ gilt. Im Unterschied zur Standard-Convolution, bei der ein Kernel die Form $KH \times KW \times C_{\text{in}}$ besitzt und ein Ausgabekanal aus der gewichteten Summe *aller* Eingangskanäle entsteht, hat ein Depthwise-Kernel nur die Größe $KH \times KW$. Die Kombination der Kanäle erfolgt erst in einem separaten Schritt, typischerweise durch eine nachgeschaltete 1×1 -*Pointwise*-Convolution [14].

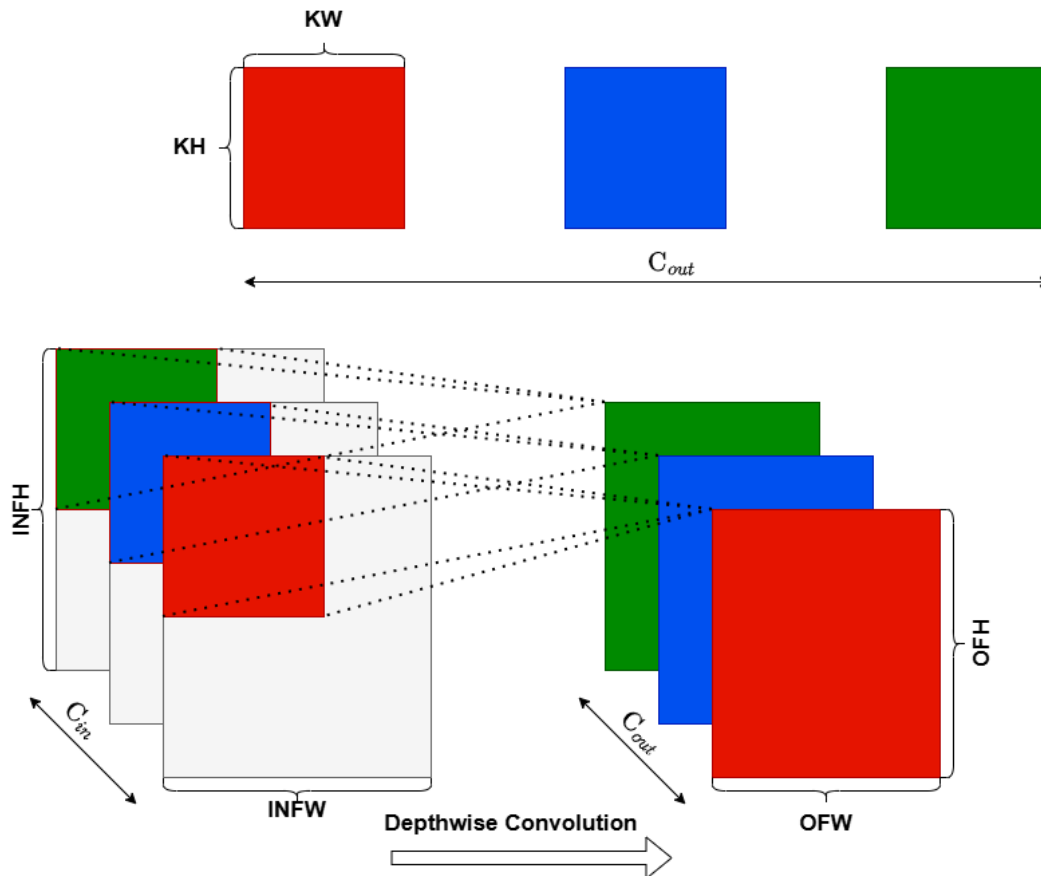


Abbildung 2.4: Prinzip der Depthwise-Convolution: kanalweise räumliche Faltung ohne Kanalmischung.

Bei der Depthwise-Convolution wird *pro Eingangskanal* ein eigener $KH \times KW$ -Kernel angewandt; ein Ausgabekanal hängt daher nur von *einem* Eingangskanal ab. Damit ergibt sich die Anzahl der MACs zu:

$$\#MACs_{DW} = OFH \cdot OFW \cdot C_{in} \cdot (KH \cdot KW). \quad (2.5)$$

Die Anzahl der Parameter (ohne Bias) ist entsprechend:

$$\#Weights_{DW} = C_{in} \cdot KH \cdot KW. \quad (2.6)$$

Am Beispiel aus Abbildung 2.5 mit Eingabe $5 \times 5 \times 3$, Kernen $(3 \times 3) \times 3$ und Ausgabe $3 \times 3 \times 3$ gilt: $OFH \cdot OFW = 3 \cdot 3 = 9$ und $KH \cdot KW = 3 \cdot 3 = 9$. Damit ergeben sich $9 \cdot 3 \cdot 9 = 243$ MACs und $3 \cdot 9 = 27$ Gewichte (zzgl. optional 3 Bias-Werte) [2].

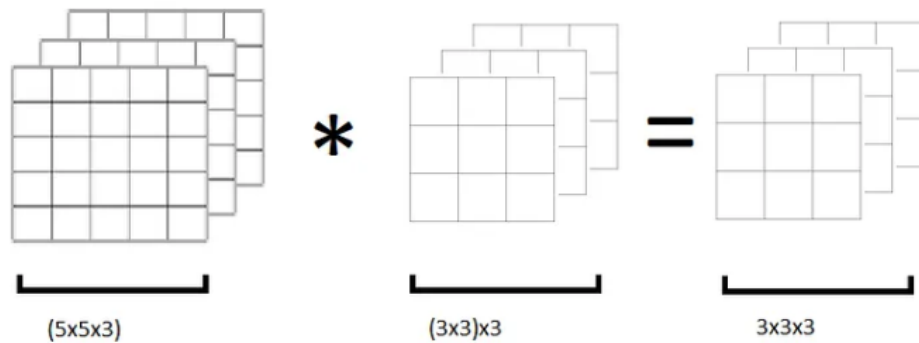


Abbildung 2.5: Beispiel einer Depthwise-Convolution: Jeder Eingangskanal wird separat mit einem 3×3 -Kernel gefaltet; es entsteht eine Ausgabe mit gleicher Kanalzahl. Nach [2].

2.1.3 Pointwise Convolution

Die *Pointwise Convolution* ist ein Spezialfall der Standard-Faltung mit $KH = KW = 1$. Wie in Abbildung 2.6 dargestellt, wird dabei an jeder räumlichen Position (H, W) ein Vektor aus C_{in} Eingangskanälen mit C_{out} verschiedenen $1 \times 1 \times C_{in}$ -Filtern verrechnet. Im Gegensatz zur Depthwise-Convolution (kanalweise, ohne Kanalmischung) übernimmt die Pointwise-Convolution somit gezielt die *Kanalmischung* und projiziert $C_{in} \rightarrow C_{out}$. In MobileNetV1 folgt sie typischerweise direkt auf die Depthwise-Convolution, um die dort erzeugten kanalweisen Merkmale effizient zu kombinieren [14].

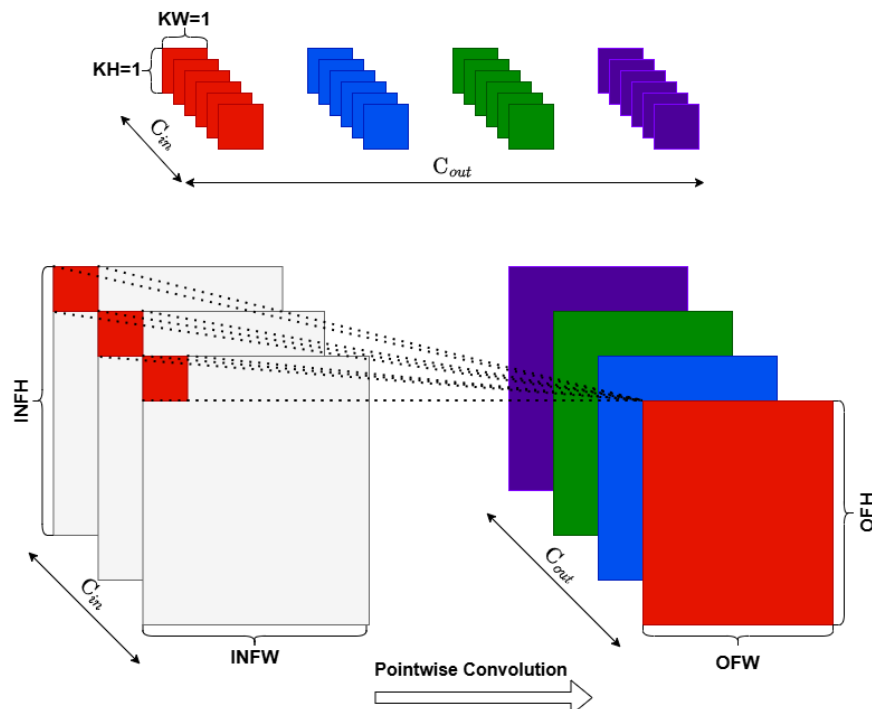


Abbildung 2.6: Prinzip der Pointwise-Convolution (1×1): pro Pixelposition werden die C_{in} Eingangskanäle linear zu C_{out} Ausgabekanälen kombiniert.

Analog zur Standard-Faltung ergibt sich der Aufwand über die Anzahl der MACs. Da $KH = KW = 1$ gilt:

$$\#MAC_{SPW} = OFH \cdot OFW \cdot C_{out} \cdot (1 \cdot 1 \cdot C_{in}) = OFH \cdot OFW \cdot C_{in} \cdot C_{out}. \quad (2.7)$$

Die Anzahl der Gewichte beträgt entsprechend:

$$\#Weights_{PW} = C_{out} \cdot (1 \cdot 1 \cdot C_{in}) = C_{in} \cdot C_{out}. \quad (2.8)$$

Am Beispiel aus Abbildung 2.7 mit Eingabe $3 \times 3 \times 3$ und $C_{out} = 16$ ($1 \times 1 \times 3$ -Filter, 16 Stück) gilt: $OFH \cdot OFW = 3 \cdot 3 = 9$ und $C_{in} \cdot C_{out} = 3 \cdot 16 = 48$. Damit ergeben sich $9 \cdot 48 = 432$ MACs und 48 Gewichte [2].

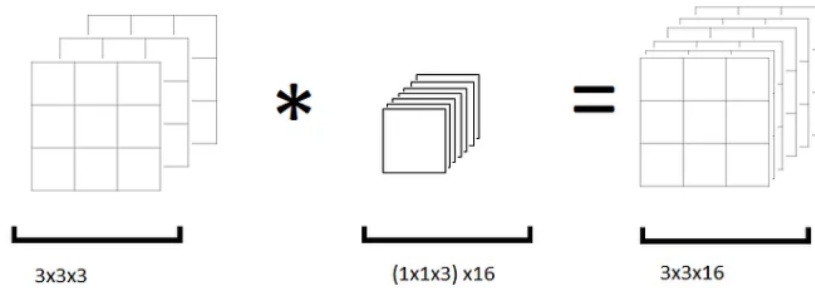


Abbildung 2.7: Beispiel einer Pointwise-Convolution: $3 \times 3 \times 3$ gefaltet mit $(1 \times 1 \times 3) \times 16$ ergibt $3 \times 3 \times 16$. Nach [2].

Gewinn von Depthwise + Pointwise gegenüber Standard-Convolution. Wie in den vorherigen Abschnitten anhand des Beispiels bereits numerisch gezeigt wurde (Depthwise: Gleichung 2.5, Pointwise: Gleichung 2.7, Standard: Gleichung 2.3), reduziert die *Depthwise-Separable-Convolution* den Rechenaufwand, indem sie die Standard-Faltung in eine kanalweise räumliche Filterung (Depthwise) und eine anschließende Kanalmischung (1×1 -Pointwise) zerlegt [14, 2]. Allgemein ergibt sich für gleiche $(OFH, OFW, C_{in}, C_{out}, KH, KW)$:

$$\#MAC_{SDW+PW} = \underbrace{OFH \cdot OFW \cdot C_{in} \cdot (KH \cdot KW)}_{\text{Depthwise}} + \underbrace{OFH \cdot OFW \cdot C_{in} \cdot C_{out}}_{\text{Pointwise}}, \quad (2.9)$$

und damit im Vergleich zur Standard-Convolution (Gleichung 2.3) der ideale Reduktionsfaktor:

$$\frac{\#MAC_{STD}}{\#MAC_{SDW+PW}} = \frac{KH \cdot KW \cdot C_{out}}{KH \cdot KW + C_{out}}. \quad (2.10)$$

Für das im Text bereits berechnete Beispiel ($KH = KW = 3$, $C_{out} = 16$) entspricht dies einer Reduktion um ca. 82.64% (von 3888 auf 675 MACs), was den zentralen Effizienzgewinn von MobileNetV1 erklärt [14, 2].

2.1.4 Pooling

Pooling reduziert die räumliche Auflösung der OFMAP kanalweise und verdichtet damit die Repräsentation. Übliche Varianten sind *Max*- und *Average*-Pooling; typisch sind kleine Fenster (2×2) mit Schrittweite 2 ohne Padding. Dies senkt Rechenaufwand und Parameterzahl, vergrößert den effektiven Rezeptivbereich und unterstützt eine gewisse Translationsinvarianz. Ein anschauliches Beispiel ist in Abbildung 2.8 dargestellt: Die OFMAP wird durch Max-Pooling zu einer kompakteren PFMAP verdichtet, die der nachfolgenden Schicht als Eingabe dient.

Die Ausgabedimensionen lassen sich analog zur Faltung bestimmen (hier meist ohne Padding):

$$\text{POH} = \left\lfloor \frac{\text{OFH} - \text{KH}}{\text{Stride}} \right\rfloor + 1, \quad (2.11)$$

$$\text{POW} = \left\lfloor \frac{\text{OFW} - \text{KW}}{\text{Stride}} \right\rfloor + 1. \quad (2.12)$$

In modernen Netzen wird Pooling teils durch *strided Convolution* ersetzt; beide Ansätze verfolgen dasselbe Ziel der kontrollierten Downsampling-Operation.

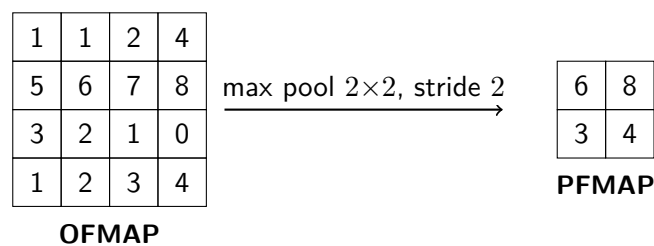


Abbildung 2.8: Beispiel für kanalweises Max-Pooling (2×2 , Schrittweite 2); die OFMAP wird zur PFMAP verdichtet.

2.1.5 Aktivierungsfunktionen

Aktivierungsfunktionen bringen die notwendige Nichtlinearität in DNN und beeinflussen damit Lernverhalten und Genauigkeit.

ReLU-Familie. Die Rectified Linear Unit (ReLU) mit $f(x) = \text{ReLU}(x) = \max(0, x)$ ist der Standard in verdeckten Schichten; Varianten wie *Leaky-ReLU* und *PReLU* halten Gradienten für negative Eingaben aufrecht und mildern „tote“ Neuronen.

Glatte Aktivierungen. Für bestimmte Ausgaben oder Module kommen häufig *Sigmoid* und *tanh* zum Einsatz. Ihre Definitionen sind in Gleichungen (2.13) und (2.14) angegeben.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (2.13)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1. \quad (2.14)$$

2.2 Systolisches Array

Im Laufe der Entwicklung von CNN-Beschleunigern wurden zahlreiche Hardware-Architekturen untersucht, um den hohen Rechen- und Speicherbedarf moderner Netze effizient zu bewältigen. Ein besonders geeigneter Ansatz sind SA, da sie stark datenparallele Operationen unterstützen und durch lokale Datenweitergabe eine hohe Datenwiederverwendung (*data reuse*) ermöglichen. Damit passen SA gut zu den regelmäßigen Berechnungsmustern der Faltung und können bei begrenzter Speicherbandbreite eine hohe Auslastung sowie eine gute Energieeffizienz erreichen [12].

Das Konzept systolischer Architekturen wurde von H.,T. Kung in seinem Beitrag *Why Systolic Architectures?* im Jahr 1982 vorgestellt und als Rechenmodell mit synchroner, „wellenartiger“ Datenbewegung zwischen benachbarten Verarbeitungselementen beschrieben [13]. Zu dieser Zeit setzten viele Arbeiten in der Praxis jedoch vorrangig auf generalistische Parallelplattformen wie Graphics Processing Unit (GPU)s bzw. später Multi-Core-Central Processing Unit (CPU)s, sodass systolische Arrays zunächst keine breite Verbreitung in der Deep-Learning-Praxis fanden; Vergleiche typischer Plattformen und deren Leistungsprofil (GPU/CPU/Tensor Processing Unit (TPU)) werden u. a. in [11] diskutiert.

In den letzten Jahren hat der erneute Aufschwung von KI- und Deep-Learning-Anwendungen systolischen Arrays neue Aufmerksamkeit verschafft. Einen wichtigen Impuls lieferte die erste TPU von Google, deren Rechenkern als systolisches Array für Matrixmultiplikationen ausgelegt ist und damit die Praxistauglichkeit von SA als zentrales Rechenelement für CNN-nahe Operationen demonstriert [10, 9]. Dabei werden Operanden aus dem Speicher in eine Kette bzw. ein Gitter aus Processing Element (PE)s eingespeist, lokal weitergereicht und über viele PEs hinweg wiederverwendet; die erforderlichen Speicherzugriffe (Input/Output (I/O)) pro Rechenoperation sinken, was die Auslastung und Energieeffizienz erhöht [10, 12]. In der Folge wurden vermehrt SA-basierte Beschleuniger sowie Verfahren zur Abbildung und Optimierung von CNN-Workloads auf systolischen Arrays publiziert [15, 16, 17, 18].

2.2.1 Datenfluss im systolischen Array

Input- und Weight-Reuse

Bevor die unterschiedlichen systolischen Datenflüsse betrachtet werden, wird zunächst der Rechenablauf der Faltung analysiert, um daraus wiederkehrende Muster für eine effiziente Beschleunigung abzuleiten. In Abbildung 2.9 sind beispielhaft drei Faltungskerne dargestellt; jeder Kernel besitzt drei Kanäle und wird über eine dreikanalige INFMAP mit Schrittweite eins verschoben. Für jede Fensterposition wird das lokale Eingabepatch kanalweise mit dem jeweiligen Kernel multipliziert und über alle Kanalbeiträge akkumuliert; die resultierende Summe bildet genau ein Ausgabeelement. Anschließend wird dasselbe Eingabepatch mit dem nächsten Kernel verarbeitet, wodurch an derselben Fensterposition ein weiterer Ausgabekanal entsteht. Insgesamt entspricht somit jeder Ausgabekanal einem Filter, das heißt die Anzahl der Ausgabekanäle entspricht C_{out} .

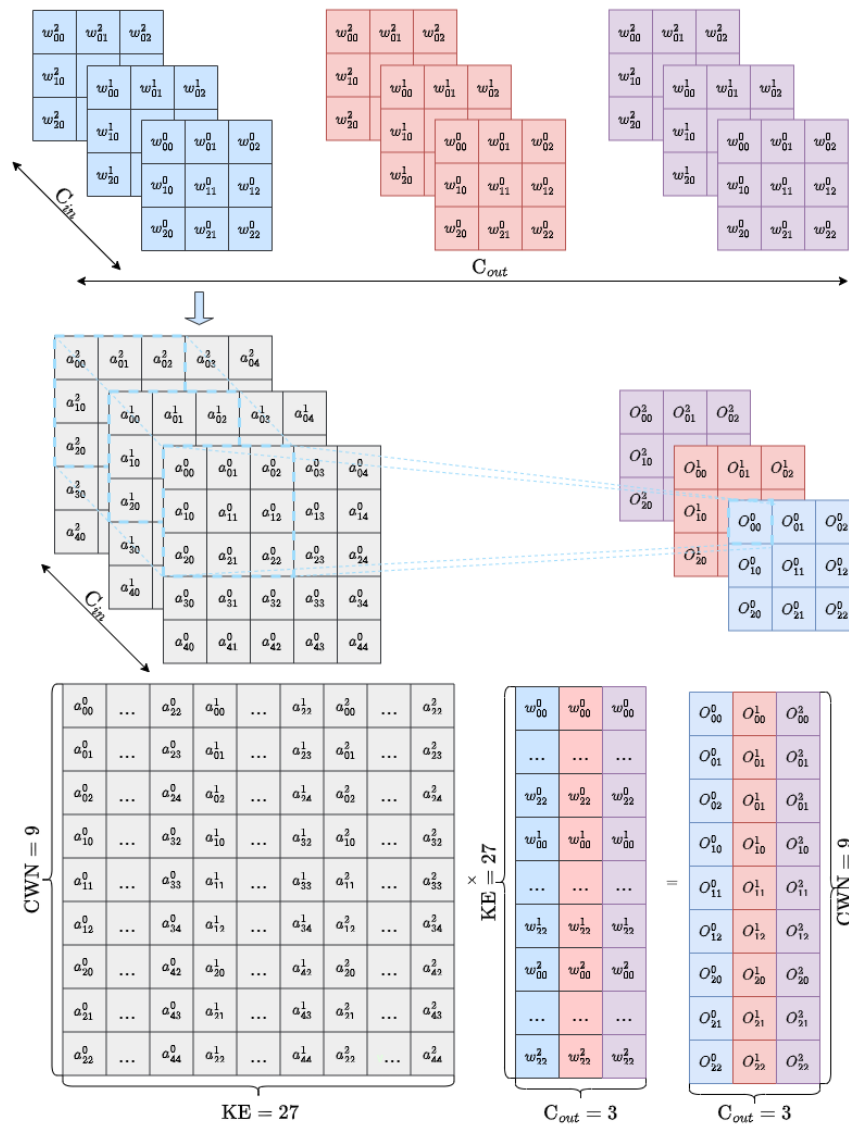


Abbildung 2.9: Abbildung der 2D-Faltung auf eine Matrizenmultiplikation: links das Aufrollen der Eingabe-Patches, in der Mitte das Stapeln der Filter, rechts die entstehenden Ausgabespalten.

Zur systematischen Beschreibung wird die Faltung in eine General Matrix–Matrix Multiplication (GEMM)-Form überführt. Dabei werden alle Fensterpositionen der INFMAP zu Zeilen einer Eingabematrix zusammengefasst; deren Anzahl wird mit CWN bezeichnet (*convolution windows*). Jedes aufgerollte Fenster enthält KE Werte (*kernel elements*), wobei:

$$KE = KH \cdot KW \cdot C_{in}. \quad (2.15)$$

Die Filtergewichte werden analog zu einer Gewichtsmatrix gestapelt; deren Spaltenzahl entspricht der Anzahl der Filter bzw. Ausgabekanäle C_{out} . Damit ergibt sich die Faltung als Matrizenmultiplikation der Form:

$$\underbrace{(CWN \times KE)}_{\text{aufgerollte Eingabe}} \cdot \underbrace{(KE \times C_{out})}_{\text{Gewichte}} = \underbrace{(CWN \times C_{out})}_{\text{Ausgabe}}. \quad (2.16)$$

Aus dieser Darstellung werden zwei zentrale Wiederverwendungsmechanismen sichtbar, die für die spätere SA-Implementierung wesentlich sind. Erstens tritt *Input Reuse* auf: Dieselben Eingabepixel werden in vielen überlappenden Fensterpositionen erneut verwendet, insbesondere bei kleinen Strides, da benachbarte Fenster große Teile ihrer Pixel teilen. Zweitens ergibt sich *Weight Reuse*: Ein Filter (bzw. seine Gewichte) wird über alle Fensterpositionen hinweg wiederholt eingesetzt. In Abbildung 2.9 bedeutet das konkret, dass das gleiche dreikanalige Eingabepatch nacheinander mit verschiedenen Filtern multipliziert wird, während ein gegebener Filter an vielen unterschiedlichen Fensterpositionen zur Anwendung kommt. Diese beiden Reuse-Eigenschaften lassen sich in systolischen Arrays gezielt ausnutzen, um Daten lokal weiterzureichen, Speicherzugriffe zu reduzieren und die zugrunde liegende Matrix-Matrix-Multiplikation hochgradig parallel abzubilden.

Output-Stationary (OS)

In der Literatur werden für systolische Arrays (SA) typischerweise drei grundlegende Datenflussmuster unterschieden: *Output-Stationary (OS)*, *Weight-Stationary (WS)* und *Input-Stationary (IS)* [19, 20, 21]. Im Folgenden wird OS betrachtet, da es die lokale Akkumulation von Teilsummen in den PEs direkt abbildet und damit besonders gut zur GEMM-Form der Faltung passt [19].

Beim OS-Datenfluss bleiben die (Teil-)Ausgaben (*partial sums*) innerhalb der PEs gespeichert, während Eingaben und Gewichte fortlaufend in das Array eingespeist und zwischen benachbarten PEs weitergereicht werden [19]. Abbildung 2.10 zeigt dieses Prinzip exemplarisch: Die Elemente eines aufgerollten Faltungsfensters werden von links in das Array gestreamt, während die zugehörigen Filtergewichte von oben zugeführt werden. Trifft in einem PE ein Eingabewert mit einem Gewicht zusammen, wird ein MAC-Schritt ausgeführt und das Ergebnis zur lokal gehaltenen Teilsumme addiert; die Ausgabe bleibt damit während der gesamten Akkumulation *stationär* im PE.

Dieses Mapping entspricht direkt der in Abbildung 2.9 eingeführten GEMM-Darstellung:

$$(CWN \times KE) \cdot (KE \times C_{out}) = (CWN \times C_{out}),$$

wobei jedes PE ein Element der Ergebnismatrix repräsentiert und über KE MACs akkumuliert.

Aus dem OS-Datenfluss ergeben sich die zentralen Wiederverwendungsmechanismen: (i) *Input Reuse* entlang einer Array-Zeile, da ein eingesteuertes Fenster-Element beim Weiterreichen nach rechts von mehreren PEs genutzt werden kann, um Beiträge zu unterschiedlichen Ausgabekanälen (C_{out}) zu erzeugen; und (ii) *Weight Reuse* entlang einer Array-Spalte, da ein Gewicht beim Weiterreichen nach unten für mehrere Fensterpositionen (unterschiedliche CWN-Zeilen) wiederverwendet wird [19]. Durch das stationäre Halten der Teilsummen reduziert OS insbesondere die Datenbewegung von Akkumulatorwerten und eignet sich daher gut als robuste Basisimplementierung für systolische GEMM-Beschleuniger [21].

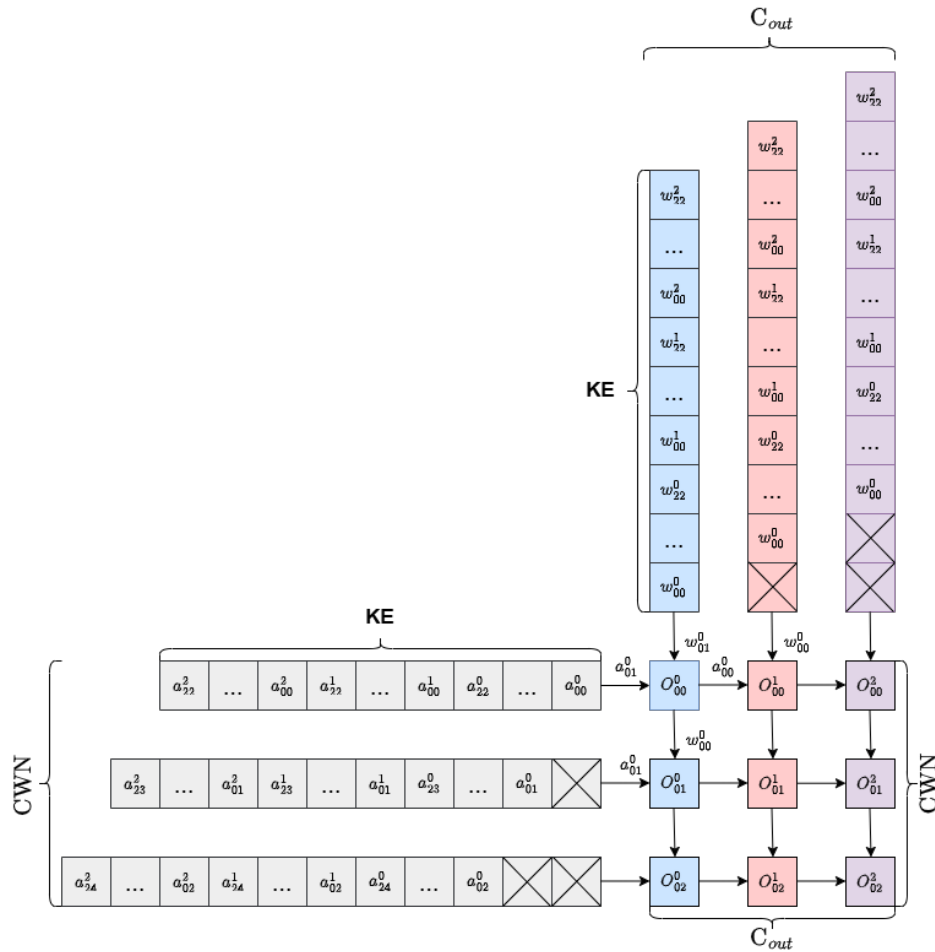


Abbildung 2.10: Output-Stationary-Datenfluss (OS): Teilsummen bleiben in den PEs, Eingaben werden von links und Gewichte von oben eingespeist und lokal akkumuliert.

2.3 Festkommazahlen und Quantisierung

Für die Inferenz auf Edge-Geräten sind Speicher- und Energiebudgets deutlich knapper als beim Training. Eine Reduktion der numerischen Präzision von 32-Bit-Gleitkommaformat (FP32) auf 8-Bit-Ganzzahlenformat (INT8) senkt Speicherbedarf, Speicherbandbreite und Rechenenergie spürbar—bei nur geringen Genauigkeitseinbußen nach geeigneter *Kalibrierung* [22].

Das INT8 Format kann gegenüber FP32 bis zu $4\times$ höhere Ausführungsgeschwindigkeit und bis zu $4\times$ geringeren Speicherbedarf erreichen sowie gegenüber 16-Bit-Gleitkommaformat (FP16) bis zu $2\times$ mehr Geschwindigkeit und Speichereffizienz [22], da ganzzahlige Multiply–Accumulate-Operationen sowie Transfers günstiger sind. Entsprechend ist INT8 in Edge-AI-Szenarien (Mobilgeräte, Drohnen, autonome Systeme) weit verbreitet.

Floating-Point (FP32/FP16) ist im Edge-Kontext nachteilig: breitere Datenpfade und aufwendige FPU-Operationen erhöhen Energie pro Operation und Bandbreitenbedarf und belasten die Taktfrequenz—ungünstig bei strengen Leistungs- und Thermikgrenzen [22].

2.4 Modell und Quantisierungsvarianten

Für die Evaluierung wurde **MobileNetV1** auf dem **EuroSAT**-Datensatz trainiert und anschließend mit TensorFlow Lite in mehrere Präzisions-/Speicherformate überführt. Die Konvertierung umfasste:

- **TFLite FP32** als Referenzinferenz,
- **Dynamic INT8**: nur Gewichte in INT8, Aktivierungen in Float,
- **Full INT8**: Gewichte und Aktivierungen in INT8 mit Kalibrierung über repräsentative Daten.
- **INT16x8**: Aktivierungen in 16-Bit-Ganzzahlenformat (INT16), Gewichte in INT8 mit Kalibrierung über repräsentative Daten.
- **Float16**: nur Gewichte in FP16, Aktivierungen in Float.

Ausgehend vom trainierten FP32-Modell erfolgt die Quantisierung mittels *Post-Training-Kalibrierung* (PTQ). Dabei werden mit einem repräsentativen Teildatensatz (`-calib-batches 100`) die Aktivierungsbereiche pro Tensor ermittelt (z. B. Min/Max). Aus diesen Statistiken berechnet der TFLite-Konverter pro Tensor die *Skalierung* s und den *Nullpunkt* z . Anschließend wird das Modell je nach Zielpräzision in ein kompaktes Flatbuffer-Format überführt (**Full INT8**, **INT16x8**).

Die Güte wird anschließend mit dem TFLite-Interpreter auf dem Validierungs-Split des **EuroSAT**-Datensatzes bestimmt. Pro Bild wird die Modellvorhersage (Top-1 via `arg max`) mit dem Ground-Truth-Label verglichen, woraus die Top-1-Genauigkeit resultiert. Beispielhafte Ergebnisse dieser Arbeit (Eingabe $224 \times 224 \times 3$):

Tabelle 2.1: TFLite-Varianten für MobileNetV1 auf EuroSAT: Genauigkeit und Modellgröße.

Variante	Aktivierungen / Gewichte	Kalibrierung	Größe	Top-1 (%)
TFLite FP32	FP32 / FP32	–	12.2 MB	96.48
Dynamic INT8	FP32 / INT8	–	3.2 MB	94.85
Full INT8	INT8 / INT8	repr. Daten (PTQ)	3.4 MB	82.43
INT16x8	INT16 / INT8	repr. Daten (PTQ)	3.5 MB	81.43
Float16	FP32 / FP16	–	6.1 MB	96.30

2.4.1 Trade-offs von Full-INT8 gegenüber FP32

Ausgehend vom FP32-Referenzmodell (*TFLite FP32*, 12,2 MB, 96,48 %) zeigt die *Full INT8*-Variante (3,4 MB, 82,43 %) den typischen Zielkonflikt aus Speicher-/Durchsatzgewinn und Genauigkeitsverlust:

- **Genauigkeitsverlust:** $96,48\% - 82,43\% = 14,05\text{ pp}$ (Top-1).
- **Speichergewinn:** $12,2\text{ MB} - 3,4\text{ MB} = 8,8\text{ MB}$ weniger; das entspricht einer relativen Reduktion um $\frac{8,8}{12,2} \approx 72,1\%$ bzw. einem Kompressionsfaktor von $\approx 3,59\times$.

Der drastisch reduzierte Speicherbedarf senkt DRAM-Verkehr, steigert die Inferenzrate auf eingebetteter Hardware und reduziert den Energieverbrauch pro MAC/Load/Store, geht jedoch

mit einem zweistelligen Verlust an Top-1-Genauigkeit einher. Die Eignung hängt somit vom Anwendungskontext (Echtzeitfähigkeit vs. Präzisionsanforderungen) ab.

3 Stand der Technik

Dieses Kapitel ordnet die vorliegende Arbeit in den aktuellen Forschungs- und Entwicklungsstand von Künstliche Intelligenz (KI)-Beschleunigern ein. Der Fokus liegt auf Beschleunigern für CNNs in eingebetteten Systemen bzw. SoCs, insbesondere auf systolisches Array (SA)s, Datenflusstategien, quantisierter Inferenz sowie der systemseitigen Datenanbindung über Standard-Interconnects und Direct Memory Access (DMA).

3.1 SoC-Integration

DL hat sich als dominierendes Verfahren für viele Wahrnehmungs- und Klassifikationsaufgaben etabliert, führt jedoch zu hohen Rechen- und Speicheranforderungen [7]. In Edge- und Embedded-Szenarien sind Energieeffizienz, deterministische Latenzen und begrenzte externe Speicherbandbreite zentrale Randbedingungen. Daher werden KI-Beschleuniger häufig als dedizierte Hardwareeinheiten in System-on-Chip (SoC)s integriert, während eine CPU die Orchestrierung, Konfiguration und Systemaufgaben übernimmt [9, 10].

Die Systemintegration folgt in vielen Designs einer Trennung zwischen Steuerpfad und Datenpfad. Konfiguration und Statusabfrage erfolgen über speichergemappede Registerzugriffe, während große Datenbewegungen über Burst-fähige Interconnects und DMA-Engines realisiert werden [4, 23, 24]. Lokale On-Chip-Speicher wie Random Access Memory (RAM)- bzw. SRAM-Bänke werden eingesetzt, um die Rechenpipeline mit gleichmäßiger Datenrate zu versorgen und Transfers mit der Verarbeitung zu überlappen.

3.2 Systolische Arrays und Datenfluss

SA-Architekturen zählen zu den etablierten Ansätzen, um Matrix- und Faltungsoperationen mit regelmäßiger, lokal gekoppelter Datenbewegung zu beschleunigen [13]. Ein SA besteht aus einer Gitterstruktur aus PEs, die Operanden taktsynchron weiterreichen und partielle Ergebnisse lokal akkumulieren. Diese Regularität führt zu kurzen lokalen Verbindungen, guter Skalierbarkeit und hoher Auslastung bei geeignetem Datenfluss [12]. In CNN-Beschleunigern sind SAs insbesondere für die Umsetzung GEMM-ähnlicher Rechenmuster verbreitet [10, 25].

Zur Maximierung der Datenwiederverwendung werden in der Literatur verschiedene Datenflussarten unterschieden, darunter *output-stationary*, *weight-stationary* und *input-stationary* [12, 19]. Beim *output-stationary* Datenfluss verbleiben partielle Summen lokal im PE, während Aktivierungen und Gewichte gestreamt werden. *Weight-stationary* hält Gewichte lokal und bewegt primär Aktivierungen bzw. partielle Summen. *Input-stationary* zielt auf eine Wiederverwendung der Eingaben über die räumliche Anordnung der Recheneinheit ab. Konfigurierbare Arrays adressieren

die Variabilität von CNN-Layern und die unterschiedliche Dominanz von Aktivierungs- bzw. Gewichtsbewegung [16].

3.3 Effiziente CNN-Operatoren und Quantisierung

Moderne effiziente CNNs-Architekturen wie MobileNets reduzieren Rechenaufwand und Parameterzahl häufig durch *depthwise separable convolutions*, also die Kombination aus depthwise Faltung und anschließender pointwise Faltung zur Kanalmischung [14]. Diese Layer-Typen verändern die Rechen- und Datenbewegungscharakteristik. Depthwise-Convolution erfordert effizientes kanalparalleles Streaming, während Pointwise-Convolution häufig eine starke Reduktion über Kanäle besitzt und damit Auslastung und Datenfluss im Array beeinflusst. Entsprechend werden in der Forschung Mapping- und Optimierungsverfahren untersucht, die solche Layer effizient auf systolischen SAs ausführen [18].

Für Embedded-Inferenz ist die Nutzung von Integer-Arithmetik, insbesondere INT8, weit verbreitet, da sie Speicherbedarf und Datenbewegung reduziert und die MAC-Einheit vereinfacht [9, 19]. Typischerweise wird $\text{INT8} \times \text{INT8}$ multipliziert, während die Akkumulation in einem größeren Wertebereich erfolgt. Anschließend wird requantisiert und in den Zielbereich gesättigt; nichtlineare Funktionen und optionale Operatoren wie Pooling sind gängige Bestandteile der Inferenzpipeline [26]. Neben klassischen Quantisierungsschemata existieren Arbeiten, die Quantisierung mit Sparsifizierung und Fine-Tuning kombinieren, um Speicherzugriffe und Rechenaufwand weiter zu reduzieren [27]. Für FPGA-basierte Systeme wurden zudem Design-Flows vorgestellt, die die Abbildung quantisierter CNNs auf ressourcenbeschränkte Plattformen unterstützen [28].

Da in vielen Beschleunigern nicht die Rechenleistung, sondern die Datenbewegung zum dominanten Engpass wird, stehen Speicherhierarchie und Transfermechanismen im Zentrum der Optimierung [12, 19]. Häufige Maßnahmen sind lokale On-Chip-RAM-Pufferung, Burst-Transfers über standardisierte Interconnects sowie DMA-Engines zur Entlastung des Hostprozessors und zur Überlappung von Transfer und Compute [4, 23, 24]. Im AMBA-Umfeld sind AXI-basierte Protokolle eine gängige Wahl; Steuerregister werden üblicherweise über AXI4-Lite eingebunden, während Datenbewegung über AXI4-Bursts erfolgt [4, 23].

3.4 Erweiterte Optimierungsansätze

Neben klassischen, dichten CNN-Workloads existieren Forschungsrichtungen, die auf zusätzliche Effizienzgewinne abzielen. Baek *et al.* schlagen eine Multi-Neural-Network-Acceleration-Architektur vor, die durch Parallelisierung und Scheduling eine bessere Auslastung von Rechenressourcen und Speicherbandbreite erreicht [29]. Samajdar *et al.* präsentieren *Self Adaptive Reconfigurable Arrays*, die Konfiguration und Mapping an den Workload anpassen, um Mapping-Effizienz und Datenreuse zu erhöhen [30]. Putic *et al.* untersuchen mit *DyHard-DNN* eine dynamische Re-Konfiguration während der Ausführung zur Optimierung von Performance und Energieeffizienz [31].

Für sparse Workloads adressieren Shomron *et al.* mit *SMT-SA* den PE-Leerlauf durch Nullen mittels paralleler Threads [32]. Wang *et al.* schlagen mit *SADD* einen systolischen Array-Beschleuniger mit dynamischem Dataflow für sparse GEMM vor [33]. Kung *et al.* zeigen mit *Packing Sparse CNNs* Methoden, um Sparse-Convolutionen effizienter auf systolische Arrays abzubilden [17]. Diese

Arbeiten verdeutlichen, dass Leistungsgewinne nicht nur über mehr Recheneinheiten, sondern auch über bessere Datenbereitstellung, flexible Mappings, reduzierte Datenbewegung und gezielte Unterstützung nicht-dichter Strukturen erreichbar sind.

4 Implementierung

4.1 Übersicht der Architektur

Die Abbildung 4.1 stellt dar, wie der Testbench die Rolle einer CPU übernimmt: Er schreibt die layerspezifischen Parameter (z. B. Eingabedimensionen, Kanalzahlen, Kernelgröße, Stride, Padding sowie Tiling- und Modus-Konfigurationen) mittels *AXI4-Lite*-Writes in definierte Control and Status Register (CSR). Diese CSR sind als *Memory-Mapped-Register* ausgeführt und damit über das AXI4-Lite-Interface adressierbar. Die Adressierung und Definition der CSR ist in einem gemeinsamen SystemVerilog-Package abgelegt. Durch den Import desselben Packages sowohl im Testbench als auch im Register-Transfer Level (RTL) wird sichergestellt, dass beide Seiten konsistent dieselben Registeradressen und Feldbelegungen verwenden. Die geschriebenen Werte werden im Registerfile als interne Register gespeichert und anschließend als dedizierte Ausgangssignale an die nachgelagerten Module weitergereicht. Damit werden die Inhalte der CSR unmittelbar zu Konfigurations-Ports, über die beispielsweise DMA-Controller-, Fetching- und Rechenmodule den aktuellen Layer deterministisch und reproduzierbar abarbeiten können.

Im Testbench-Aufbau liegen alle Modell- und Eingabedaten in einem read-only Host-Speicher (Read-Only Memory (ROM)). Für jede Layer werden die zugehörigen Datenblöcke (Gewichte, Bias-Werte sowie Quantisierungsparameter wie Shift- und Multiplikationsfaktoren) über definierte Offsets und Bytelängen im ROM adressiert. Die Übertragung dieser Daten in die internen On-Chip-Speicher erfolgt durch einen DMA-Controller, der als AXI-Master Lesezugriffe auf das ROM ausführt und die gelesenen Bytes in die entsprechenden lokalen SRAM-Bereiche schreibt. Zusätzlich wird für die erste Layer einmalig das Eingabebild per DMA in den IFMAP-Speicher geladen; nachfolgende Layer verwenden die jeweils zuvor berechneten Feature-Maps aus den internen Speicherbänken.

Das On-Chip-Speichersystem gliedert sich in Feature- und Parameter-Speicher. Das Feature-SRAM ist als gebankter Speicher für IFMAP/OFMAP umgesetzt und wird durch LANES_MAX unabhängige RAM-Bänke realisiert, sodass pro Takt ein Lane-Vektor aus LANES_MAX Pixelwerten parallel bereitgestellt werden kann. Dabei entspricht LANES_MAX der *Zeilenparallelität* des SA. Die Kanalwerte werden auf die Bänke verteilt und über einen Gruppenindex innerhalb der Bank adressiert, wodurch parallele Zugriffe auf mehrere Kanäle derselben Pixelposition möglich sind. Zur Überlappung von Lesen und Schreiben wird ein *Ping-Pong*-Prinzip verwendet, bei dem zwei Feature-Bänke abwechselnd als Lesequelle (IFMAP) bzw. Schreibziel (OFMAP) dienen (Bankauswahl über CSR). Ergänzend existieren separate Parameterspeicher: ein dedizierter Weight_SRAM für Filtergewichte sowie ein SRAM_Bias/Shift/Mult, das aus drei getrennten RAM-Blöcken für Bias-, Shift- und Multiplikationsfaktoren besteht, um Bias und Quantisierungsparameter synchron und unabhängig vom Weight-Streaming für die Akkumulation und Requantisierung im Rechenkern bereitzustellen.

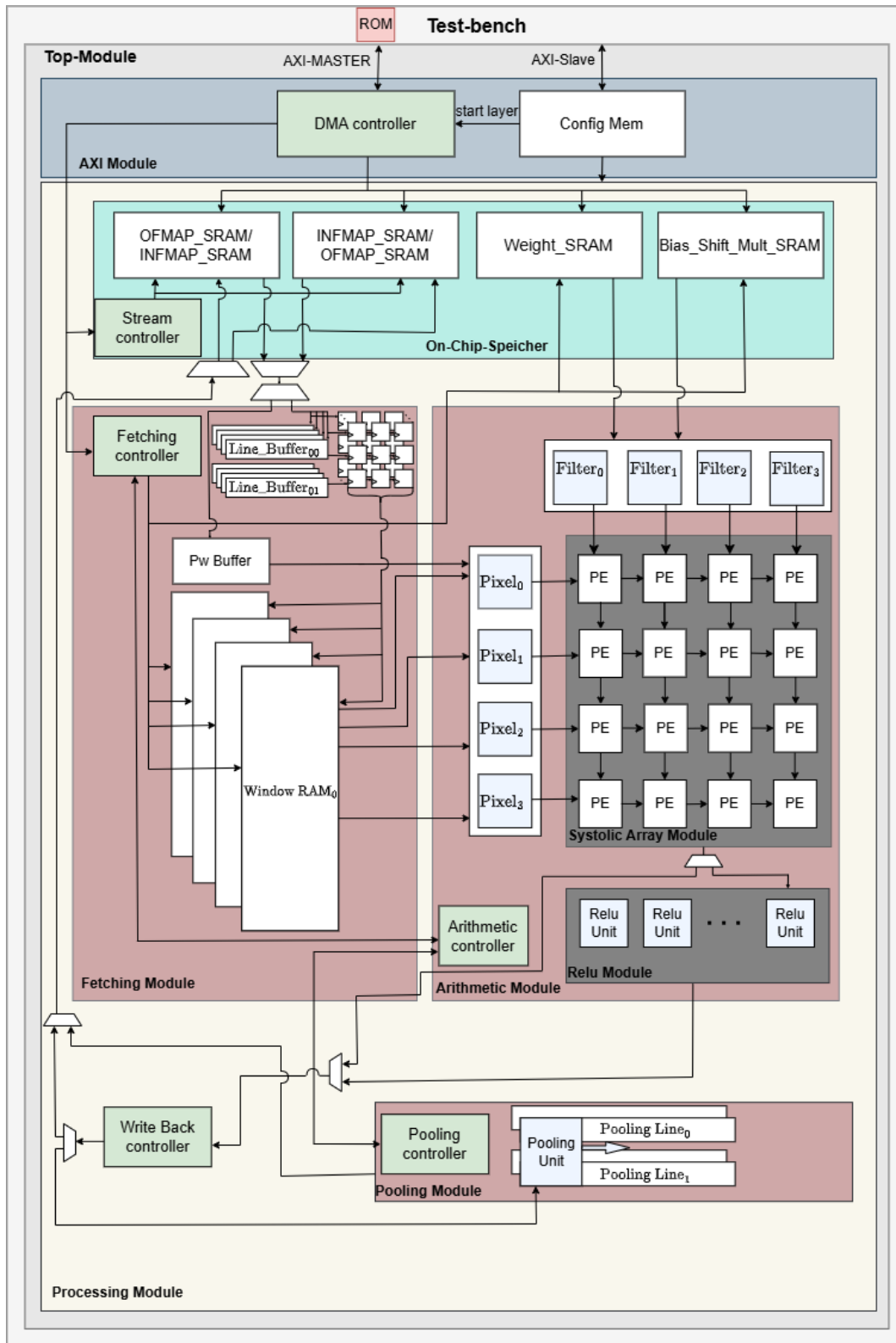


Abbildung 4.1: Systemübersicht des CNN-Beschleunigers im Testbench-Aufbau.

Nachdem der DMA-Controller die Schichtdaten in die lokalen On-Chip-SRAMs übertragen hat, wird das Streaming aus dem I/O-Feature-SRAM gestartet und die Aufbereitungsphase im `fetching_module` aktiviert. Ziel dieser Phase ist es, den Datenstrom für das systolische Array bereitzustellen. Bei Standard- und Depthwise-Convolution werden die gestreamten Pixel zunächst durch `line_buffer`-Strukturen geführt, um aus dem kontinuierlichen Stream räumliche $K \times K$ -Fenster zu rekonstruieren; dabei werden mehrere Line-Buffer parallel instanziiert, wobei ihre Anzahl der Lane- bzw. Zeilenparallelität des Arrays entspricht. Die erzeugten Fenster werden anschließend in `window_ram`-Blöcken zwischengespeichert, sodass sie taktgenau an den Rechenkern ausgegeben werden können. Auch hier existieren mehrere Window-RAMs parallel, deren Anzahl ebenfalls an die Zeilenparallelität des Arrays gekoppelt ist. Im Pointwise-Fall entfällt die räumliche Fensterbildung; stattdessen werden die Eingangskanäle einer Pixelposition in mehreren *Beats* gesammelt und in einem dedizierten Buffer gehalten, bis alle benötigten Kanalbeiträge für die nachfolgende Berechnung bereitstehen.

Die Verarbeitung erfolgt *gruppenweise* und *blockweise*. *Gruppenweise* bedeutet, dass die Eingangskanäle C_{in} in Kanalgruppen zerlegt werden, die jeweils als Lane-Vektor in das SA eingespeist werden und damit die parallele Array-Breite ausnutzen. *Blockweise* bezieht sich auf die Ausgabeseite: Da pro Rechendurchlauf nur COLS_MAX Spalten des Arrays aktiv genutzt werden können, wird die Anzahl der Filter bzw. Ausgabekanäle C_{out} in Filterblöcke der Größe COLS_MAX partitioniert. Für einen festen Filterblock werden zunächst alle Gruppen bzw. Fensterpositionen verarbeitet; anschließend wird zum nächsten Filterblock gewechselt, bis alle Ausgabekanäle berechnet sind. Zu Beginn jedes neuen Filterblocks werden die zugehörigen Bias-, Shift- und Multiplikationsparameter aus den entsprechenden RAM-Blöcken geladen und lokal in den PEs gespeichert, sodass Akkumulation und Requantisierung für diesen Block ohne zusätzliche externe Zugriffe erfolgen kann.

Die aktuelle Version des KI-Beschleunigers unterstützt neben der Faltungsberechnung sowohl ReLU als auch 2×2 -Pooling in Form von *Average*- und *Max*-Pooling. Nachdem eine Kachel im SA vollständig berechnet wurde, werden die resultierenden OFMAP-Werte unmittelbar an das ReLU-Modul weitergereicht. Dort führen parallel instanziierte ReLU_Units die Nichtlinearität aus; die Aktivierung kann über CSR ein- bzw. ausgeschaltet werden. Ist Pooling deaktiviert, werden die Ergebnisse des Arithmetic-Moduls anschließend über den Writeback-Controller direkt in das I/O-Feature-SRAM zurückgeschrieben.

Ist Pooling aktiviert, erfolgt die Rückschreibung nicht unmittelbar: Stattdessen werden die Ausgabewerte zunächst über den Writeback-Controller zeilenweise in `pooling_lines` zwischengespeichert. Da 2×2 -Pooling zwei benachbarte Zeilen benötigt, wird die Pooling-Operation erst ausgeführt, sobald die erforderlichen zwei Zeilen vollständig vorliegen. Der Pooling-Controller steuert anschließend die `pooling_unit`, die pro Kanal das 2×2 -Fenster gemäß gewähltem Modus (Average oder Max) reduziert, und schreibt die verdichteten Ergebnisse wieder in das I/O-Feature-SRAM zurück.

4.1.1 Zielplattform und Integrationskonzept (RISC-V SoC)

Die in dieser Arbeit entwickelte Architektur ist als KI-Beschleuniger konzipiert, der in eine RISC-V-basierte System-on-Chip (SoC)-Plattform integriert werden kann. Ziel ist eine enge Kopplung zwischen einem allgemeinen RISC-V-Core und einem spezialisierten Hardwaremodul zur Beschleunigung von Convolution-Operationen neuronaler Netze.

Im aktuellen Implementierungsstand erfolgt die Verifikation des Beschleunigers innerhalb einer dedizierten Testbench-Umgebung. Dabei übernimmt die Testbench die Rolle eines Host-Prozessors:

Sie initialisiert die Konfigurationsregister, überträgt Parameter und Eingangsdaten und startet die Verarbeitungseinheiten. Die Host-Interaktion wird somit funktional emuliert und erlaubt eine vollständige Validierung der Steuer- und Datenpfade des Beschleunigers.

In einer RISC-V-basierten Zielplattform würde diese Host-Rolle von einem RISC-V-Core (z. B. Ibex oder einer vergleichbaren Implementierung) übernommen. Die Kommunikation zwischen Core und Beschleuniger erfolgt über eine standardisierte AXI-Schnittstelle. Die Konfigurationsregister des Beschleunigers sind als Memory-Mapped-Register ausgeführt und können über ein AXI4-Lite-Interface adressiert werden. Der Prozessor setzt hierbei Betriebsparameter, Layer-Konfigurationen sowie Startsignale durch einfache Speicherzugriffe.

Für die Datenübertragung zwischen Hauptspeicher und internen Speichern des Beschleunigers ist ein DMA-Controller vorgesehen, der als AXI-Master agiert. Dieser lädt Gewichte, Bias-Werte, Requantisierungsparameter (Multiplier und Shift) sowie Eingabeaktivierungen in die jeweiligen SRAM-Strukturen des Beschleunigers. Nach Abschluss der Berechnung können die resultierenden Aktivierungen ebenfalls über den AXI-Interconnect in den Hauptspeicher zurückgeschrieben werden. Dieses Integrationsmuster entspricht der typischen Einbindung eines Coprozessors in eine RISC-V-basierte SoC-Architektur.

Es ist zu betonen, dass im Rahmen dieser Arbeit kein vollständiges RISC-V-Subsystem als zusammenhängendes RTL-SoC realisiert wurde. Stattdessen wird die Interaktion mit dem Host-Prozessor im Testbench-Setup modelliert. Die Architektur des Beschleunigers wurde jedoch so entworfen, dass sie über die implementierten AXI-Schnittstellen prinzipiell in eine RISC-V-basierte Edge-Plattform integriert werden kann. Somit stellt der entwickelte KI-Beschleuniger ein systemfähig ausgelegtes Hardwaremodul dar, dessen Integration in ein entsprechendes SoC-Konzept technisch vorbereitet ist.

4.2 AXI Module

4.2.1 Steuer- und Datenanbindung über AXI

KI-Beschleuniger sind spezialisierte Recheneinheiten, die typischerweise in System-on-Chip-Architekturen (SoC) integriert werden. Eine zentrale Designanforderung besteht in der effizienten und standardisierten Anbindung des Accelerators an andere SoC-Komponenten. In der Praxis wird hierfür häufig die AXI-Schnittstelle eingesetzt, die als Teil der von Arm standardisierten AMBA-Spezifikationsfamilie definiert ist [4]. Viele Prozessorkerne (z. B. ARM-Cores) sowie FPGA-basierte Systeme (z. B. von AMD/Xilinx oder Intel) unterstützen AXI-konforme Schnittstellen. Aus diesem Grund wird auch in der vorliegenden Arbeit ein AXI-basiertes Interface zur Anbindung und Steuerung des KI-Beschleunigers verwendet.

In diesem Kontext übernimmt das AXI-Modul in der vorliegenden Arbeit die Rolle der Kommunikationsschnittstelle zwischen dem Host (repräsentiert durch die Testbench) und dem KI-Beschleuniger. Es ist dafür zuständig, Konfigurations- und Steuerinformationen aus der Testbench in die Konfigurationsregister des Beschleunigers zu übertragen. Darüber hinaus koordiniert das AXI-Modul den Datentransfer vom hostseitigen ROM in die internen SRAMs des KI-Beschleunigers.

Für die Übertragung der Konfigurations- und Steuerinformationen in die Konfigurationsregister eignet sich das AXI4-Lite-Protokoll. Es ist speziell für speichergemappede Registerzugriffe ausgelegt

und weist eine geringe Protokollkomplexität auf. Dadurch ist AXI4-Lite besonders geeignet für den Zugriff auf Konfigurations- und Statusregister in Accelerator-Subsystemen [34].

Für die Übertragung großer Datenmengen, insbesondere von Feature-Maps und Gewichten, wird in der vorliegenden Arbeit ein AXI4-Master-Interface in Kombination mit einem DMA-Controller verwendet. Der DMA führt die blockweisen Transfers zwischen dem Host-Speicher und den internen SRAMs des KI-Beschleunigers weitgehend autonom aus und entkoppelt damit die Datenbewegung von der Rechenpipeline. Bei einer späteren Integration in ein SoC kann dieser Mechanismus genutzt werden, um den Host-Prozessor von Kopier- und Transferaufgaben zu entlasten und Datenbewegung mit anderen Softwareaktivitäten zu überlappen [23, 35, 36, 24].

4.2.2 Konfigurationsregister und AXI4-Lite Interface

Konfigurationsregister

Die Konfigurationsregister bilden die zentrale Schnittstelle zur Laufzeitsteuerung des KI-Beschleunigers. Über diese Register werden alle für den Betrieb relevanten Parameter, wie Dimensionsangaben, Betriebsmodi und Steuersignale, aus der Host-Seite in die Hardware übernommen.

Tabelle 4.1: CSR-Registerübersicht (Byte-adressiert)

Byte-Adresse	Register	Inhalt
0x00	CSR_CONTROL	Bit[0]: ctrl_relu_en (ReLU aktivieren) Bit[1]: ctrl_pool_en (Pooling aktivieren) Bit[2]: ctrl_dw_en (Depthwise-Modus aktivieren) Bit[3]: ctrl_pool_max_not_avg (1 = Max, 0 = Average) Bit[4]: ctrl_pw_en (Pointwise-Modus aktivieren) Bit[31:5]: Reserved
0x04	CSR_CFG_IN0	Bits[15:0]: cfg_in_h (INFH), Bits[31:16]: cfg_in_w (INFW)
0x08	CSR_CFG_IN1	Bits[15:0]: cfg_cin (C_{in}), Bits[31:16]: cfg_cout (C_{out})
0x0C	CSR_CFG_K	Bits[15:0]: cfg_kh (KH), Bits[31:16]: cfg_kw (KW)
0x10	CSR_CFG_STRIDE	Bits[15:0]: cfg_sh (Stride), Bits[31:16]: cfg_sw (Stride)
0x14	CSR_CFG_TILE	Bits[15:0]: cfg_tile_r, Bits[31:16]: cfg_tile_c
0x18	CSR_CFG_OUT0	Bits[15:0]: cfg_out_h, Bits[31:16]: cfg_out_w
0x1C	CSR_TOTAL_WINS	total_wins (32-bit, Anzahl Fenster)
0x20	CSR_TOTAL_GRPs	total_grps (32-bit, Anzahl Gruppen)
0x24	CSR_TOTAL_BLKs	total_blks (32-bit, Anzahl Blöcke / Filterblöcke)
0x28	CSR_CFG_NUM_CTX	num_ctx (32-bit, Kontextelemente pro Fenster)
0x2C	CSR_DMA_IN_SRC	Quelle: Startadresse INFMAP (Host/ROM)
0x30	CSR_DMA_IN_DST	Ziel: Startadresse Feature-SRAM (Write-Bereich)
0x34	CSR_DMA_IN_LEN	Transferlänge in Bytes (INFMAP)
0x38	CSR_DMA_IN_START	Startsignal DMA-IN

Byte-Adresse	Register	Inhalt
0x3C	CSR_DMA_WG_SRC	Quelle: Startadresse Gewichte (Host/ROM)
0x40	CSR_DMA_WG_DST	Ziel: Startadresse Weight-SRAM
0x44	CSR_DMA_WG_LEN	Transferlänge in Bytes (Gewichte)
0x48	CSR_DMA_WG_START	Startsignal DMA-WG
0x4C	CSR_DMA_OUT_SRC	Quelle: Startadresse OFMAP (Feature-SRAM, Read-Bereich)
0x50	CSR_DMA_OUT_DST	Ziel: Startadresse OFMAP (Host/DDR)
0x54	CSR_DMA_OUT_LEN	Transferlänge in Bytes (OFMAP)
0x58	CSR_DMA_OUT_START	Startsignal DMA-OUT
0x5C	CSR_DMA_BIAS_SRC	Quelle: Startadresse Bias-Daten (Host/ROM)
0x60	CSR_DMA_BIAS_DST	Ziel: Bias-Bereich im SRAM_BIAS_SHIFT_MULT
0x64	CSR_DMA_BIAS_LEN	Transferlänge in Bytes (Bias)
0x68	CSR_DMA_SHIFT_SRC	Quelle: Startadresse Shift-Daten (Host/ROM)
0x6C	CSR_DMA_SHIFT_DST	Ziel: Shift-Bereich im SRAM_BIAS_SHIFT_MULT
0x70	CSR_DMA_SHIFT_LEN	Transferlänge in Bytes (Shift)
0x74	CSR_WB_ENABLE	Bit[0]: wb_enable, Bit[31:1]: Reserved
0x78	CSR_BANK_CTRL	Bit[0]: cfg_read_bank_sel (0=A, 1=B) Bit[1]: cfg_write_bank_sel (0=A, 1=B) Bit[31:2]: Reserved
0x7C	CSR_CFG_PAD	Padding-Konfiguration: Bits[7:0]: cfg_pad_left Bits[15:8]: cfg_pad_right Bits[23:16]: cfg_pad_top Bits[31:24]: cfg_pad_bottom
0x80	CSR_PW_NEEDED_BEATS	Pointwise: benötigte Beats Bits[15:0]: cfg_needed_beats_pw Bits[31:16]: Reserved
0x84	CSR_DMA_MULT_SRC	Quelle: Startadresse Multiplier-Daten (Host/ROM)
0x88	CSR_DMA_MULT_DST	Ziel: Multiplier-Bereich im SRAM_BIAS_SHIFT_MULT
0x8C	CSR_DMA_MULT_LEN	Transferlänge in Bytes (Multiplikation)

Tabelle 4.1 fasst die speichergemappten Konfigurations- und Steuerregister des Beschleunigers zusammen. Über CSR_CONTROL wird das Laufzeitverhalten der Rechenpipeline festgelegt. Die einzelnen Bits werden im csr_regfile als Steuersignale ctrl_relu_en, ctrl_pool_en, ctrl_dw_en, ctrl_pw_en und ctrl_pool_max_not_avg ausgegeben und ermöglichen damit die Aktivierung von ReLU, Pooling sowie des Depthwise- und Pointwise-Modus.

Die Register CSR_CFG_IN0 und CSR_CFG_IN1 definieren die Eingangsdimensionen (INFH, INFW) sowie die Kanalanzahlen (C_{in} , C_{out}). Faltungsparameter werden über CSR_CFG_K (Kernelgrößen KH/KW) und CSR_CFG_STRIDE (Schrittweiten) konfiguriert. CSR_CFG_TILE definiert die Tile-Dimensionen der Verarbeitung und legt damit die Anzahl der Zeilen (cfg_tile_r) sowie die Anzahl der Spalten (cfg_tile_c) eines internen Verarbeitungstiles fest.

Die DMA-Register sind in Funktionsgruppen strukturiert: CSR_DMA_IN_* lädt die INFMAP aus dem ROM in das Feature-SRAM, CSR_DMA_WG_* überträgt die Gewichte in das Weight-SRAM und CSR_DMA_OUT_* schreibt die OFMAP zurück zum Host. Die Requantisierungsparameter werden blockweise in das gemeinsame SRAM_BIAS_SHIFT_MULT geladen: Bias-, Shift- und Multiplika-

tionsfaktoren werden dazu über `CSR_DMA_BIAS_*`, `CSR_DMA_SHIFT_*` sowie `CSR_DMA_MULT_*` in die jeweiligen Speicherbereiche übertragen. Die jeweiligen `*_LEN`-Register geben dabei die Transferlänge in Bytes für den zugehörigen Datentyp an (z. B. `INFMAP`, Gewichte, Bias, Shift oder Multiplier). Mit `CSR_WB_ENABLE` wird der Write-Back-Pfad aktiviert, während `CSR_BANK_CTRL` die Auswahl der Ping-Pong-Bänke für Lese- und Schreibzugriffe ermöglicht.

Zusätzlich zu den Basisparametern werden mit `CSR_CFG_OUT0`, `CSR_TOTAL_WINS`, `CSR_TOTAL_GRPs`, `CSR_TOTAL_BLKs` und `CSR_CFG_NUM_CTX` abgeleitete Laufzeitgrößen bereitgestellt, die der Steuerlogik (`fetching_module` und `processing_module`) zur Dimensionierung des Datenstroms und zur Schleifensteuerung dienen. Die Ausgabedimensionen ergeben sich unter Berücksichtigung des konfigurierten Paddings (`CSR_CFG_PAD0`) aus Eingangsmaßen, Kernelgrößen und Schrittweiten zu:

$$cfg_out_h = \left\lfloor \frac{cfg_in_h + cfg_pad_top + cfg_pad_bottom - cfg_kh}{cfg_sh} \right\rfloor + 1 \quad (4.1)$$

$$cfg_out_w = \left\lfloor \frac{cfg_in_w + cfg_pad_left + cfg_pad_right - cfg_kw}{cfg_sw} \right\rfloor + 1 \quad (4.2)$$

Aus `cfg_out_h` und `cfg_out_w` folgt die Gesamtanzahl der Fensterpositionen als:

$$total_wins = cfg_out_h \cdot cfg_out_w. \quad (4.3)$$

Die Fensterpositionen werden zeilenweise in Gruppen der Größe `cfg_tile_r` verarbeitet. Daraus ergibt sich die Anzahl der Gruppen zu:

$$total_grps = \left\lceil \frac{total_wins}{cfg_tile_r} \right\rceil. \quad (4.4)$$

Analog werden die Ausgabekanäle in Blöcke der Größe `cfg_tile_c` partitioniert, sodass:

$$total_blks = \left\lceil \frac{cfg_cout}{cfg_tile_c} \right\rceil. \quad (4.5)$$

Schließlich beschreibt `num_ctx` die Anzahl der Kontextelemente pro Fenster und damit die Länge des MAC-Datenstroms je Output-Position. Für eine normale Faltung gilt:

$$num_ctx = cfg_kh \cdot cfg_kw \cdot cfg_cin, \quad (4.6)$$

während im Depthwise-Modus die Kontextlänge auf die Kernelgröße reduziert wird:

$$num_ctx = cfg_kh \cdot cfg_kw. \quad (4.7)$$

Im Pointwise-Modus (1×1 -Faltung) entspricht die Kontextlänge direkt der Anzahl der Eingangskanäle, da pro Output-Position über alle `cfg_cin` Kanäle akkumuliert wird:

$$num_ctx = cfg_cin. \quad (4.8)$$

Darüber hinaus gibt `cfg_needed_beats_pw` an, in wie vielen *Beats* diese `cfg_cin` Eingangskanäle im Pointwise-Scan verarbeitet werden müssen. Da der Datenpfad pro Takt einen Vektor aus `LANES_MAX` Kanälen bereitstellt, ergibt sich die benötigte Beat-Anzahl als:

$$cfg_needed_beats_pw = \left\lceil \frac{cfg_cin}{LANES_MAX} \right\rceil. \quad (4.9)$$

Die genannten abgeleiteten Parameter werden softwareseitig aus den Basisregistern berechnet und in die CSR-Felder geschrieben, um im Beschleuniger zusätzliche Laufzeitdivisionen zu vermeiden und Zählergrenzen eindeutig festzulegen.

AXI4-Lite Interface

Der Zugriff auf die Konfigurationsregister erfolgt über ein AXI4-Lite-Interface (vgl. Abschnitt 4.2.1). In der vorliegenden Umgebung übernimmt die Testbench die Rolle des **AXI-Masters** und initiiert Lese- und Schreibtransaktionen, während das Modul `axi4_lite_slave` als **AXI-Slave** implementiert ist und die Registerzugriffe an das `csr_regfile` weiterleitet.

AXI4-Lite ist ein vereinfachtes, speichergemapptes Protokoll der AMBA-AXI-Familie, das für registerartige Zugriffe mit Einzeltransfers ausgelegt ist. Die Kommunikation erfolgt über getrennte Kanäle für Adresse, Daten und Antwort, wobei jede Übertragung über ein *Valid/Ready*-Handshake bestätigt wird. Die dabei verwendeten Kanäle und die zentralen Signale sind in Tabelle 4.2 zusammengefasst [4].

Schreibzugriff: Ein Schreibzugriff gilt als erfolgreich, wenn sowohl die Adresse als auch die zugehörigen Daten akzeptiert wurden. Dies ist der Fall, wenn auf dem Write-Address-Kanal `AWVALID` und `AWREADY` sowie auf dem Write-Data-Kanal `WVALID` und `WREADY` gleichzeitig aktiv sind (vgl. Tabelle 4.2). Der Abschluss der Transaktion wird anschließend über den Write-Response-Kanal signalisiert und ist erreicht, wenn `BVALID` und `BREADY` gleichzeitig gesetzt sind.

Lesezugriff: Ein Lesezugriff beginnt, wenn die Registeradresse auf dem Read-Address-Kanal übertragen und durch das Handshake `ARVALID` und `ARREADY` akzeptiert wurde (vgl. Tabelle 4.2). Die Registerdaten werden anschließend über den Read-Data-Kanal bereitgestellt. Der Zugriff ist abgeschlossen, sobald `RVALID` und `RREADY` gleichzeitig aktiv sind.

Tabelle 4.2: AXI4-Lite-Kanäle und zugehörige Signale [4]

Write Address	Write Data	Write Response	Read Address	Read Data
AWVALID	WVALID	BVALID	ARVALID	RVALID
AWREADY	WREADY	BREADY	ARREADY	RREADY
AWADDR	WDATA	BRESP	ARADDR	RDATA
AWPROT	WSTRB		ARPROT	RRESP

4.2.3 DMA-Controller

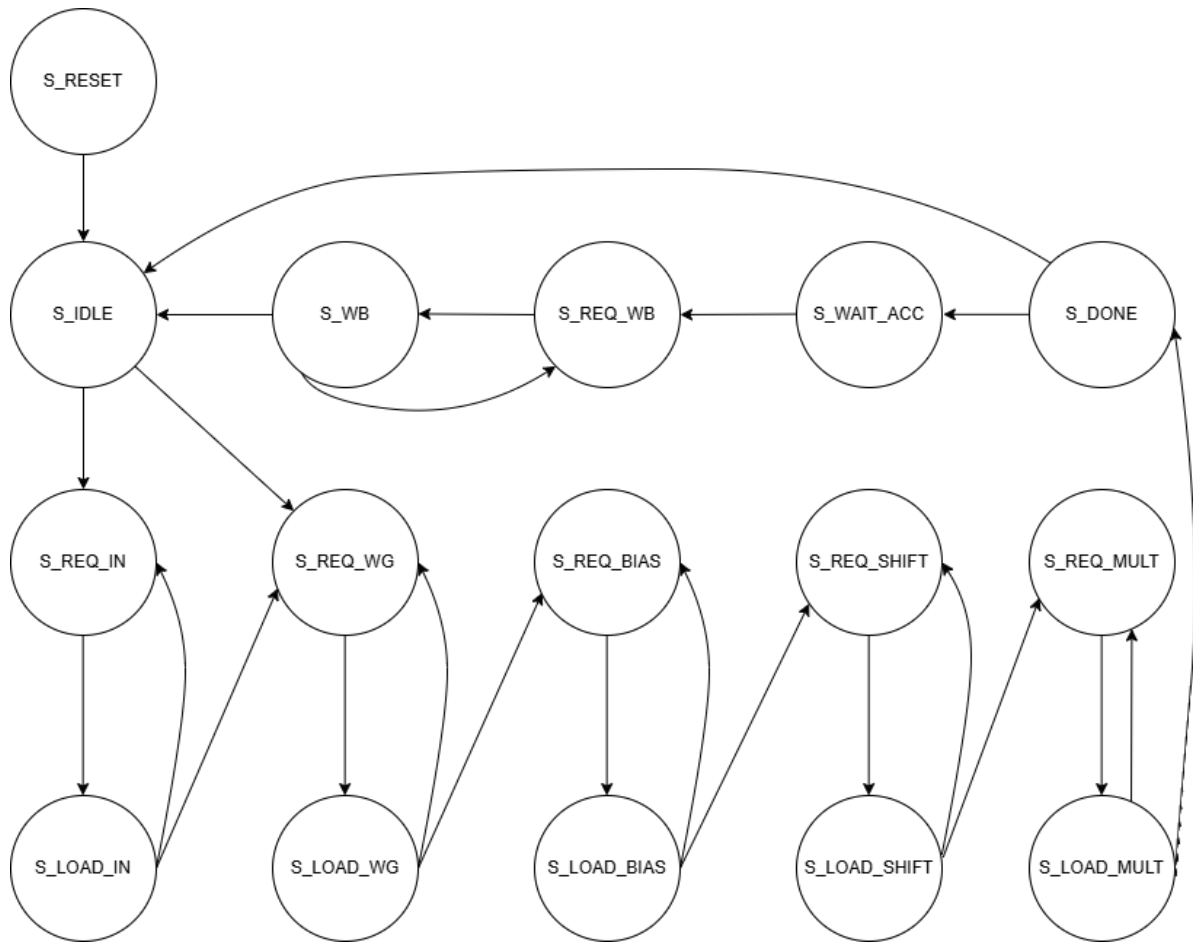


Abbildung 4.2: Zustandsdiagramm des DMA-Controllers mit sequenziellen Ladephasen (Input, Weights, Bias, Shift, Mult) sowie optionalem Write-Back.

Der *DMA-Controller* realisiert den Transfer der für einen Layer benötigten Daten (Input-Feature-Map, Gewichte, Bias, Shift und Multiplier) vom Host-Speicher in die lokalen SRAMs des Beschleunigers. Der Ablauf wird durch eine endliche Zustandsmaschine gesteuert, deren Zustände in Abbildung 4.2 dargestellt sind. Nach einem asynchronen Reset startet der DMA-Controller im Zustand **S_RESET**. Sobald **reset** deaktiviert wird, wechselt die FSM nach **S_IDLE** und wartet auf das Startsignal **start**. Dieses Startsignal wird durch einen Schreibzugriff über das AXI4-Lite-Interface auf das speichergemappede Register **CSR_DMA_IN_START** als *Write-One-Pulse* erzeugt (siehe Tabelle 4.1)

Nach **start** führt der DMA-Controller die Ladephasen in einer festen Reihenfolge aus. Für den ersten Layer erfolgt die Sequenz **S_REQ_IN** → **S_LOAD_IN** → **S_REQ_WG** → **S_LOAD_WG** → **S_REQ_BIAS** → **S_LOAD_BIAS** → **S_REQ_SHIFT** → **S_LOAD_SHIFT** → **S_REQ_MULT** → **S_LOAD_MULT**. Bei nachfolgenden Layern kann die Input-Phase entfallen, sodass die Sequenz direkt mit **S_REQ_WG** beginnt und anschließend Bias, Shift und Mult geladen werden (**S_REQ_WG** → **S_LOAD_WG** → **S_REQ_BIAS** → **S_LOAD_BIAS** → **S_REQ_SHIFT** → **S_LOAD_SHIFT** → **S_REQ_MULT** → **S_LOAD_MULT**). Die verwendeten AXI4-Master-Kanäle und Signale des DMA-Controllers sind in Tabelle 4.3 zusammengefasst.

AXI-Burst-Anforderung (Read-Transfers). In jedem `S_REQ_*`-Zustand initiiert der DMA-Controller einen AXI4 Read-Burst-Request über den Read-Address-Kanal (AR), indem `M_AXI_ARVALID` gesetzt und auf `M_AXI_ARREADY` gewartet wird. Die Burst-Länge wird durch `ARLEN` (Beats-1) bestimmt. Ein Burst kann maximal 256 Beats umfassen. Da im System `AXI_DATA_WIDTH=32` Bit verwendet wird, entspricht ein Beat 4 Byte, sodass pro maximalem Burst höchstens:

$$256 \cdot 4 \text{ Byte} = 1024 \text{ Byte}$$

übertragen werden. Nach erfolgreichem AR-Handshake wechselt der DMA-Controller in den zugehörigen `S_LOAD_*`-Zustand und empfängt die Datenwörter über den Read-Data-Kanal (`M_AXI_RDATA`).

Datenübernahme in SRAM. Die lokalen SRAM-Module sind byteadressiert und besitzen einen 8-bit Write-Port. Daher wird jedes empfangene 32-bit AXI-Datenwort zunächst in einem Hold-Register (`rdata_hold`) zwischengespeichert und anschließend in vier Bytes zerlegt. Diese Byte-Serialisierung erfolgt über `have_word` und `byte_idx`: Sobald ein AXI-Wort angenommen wurde (`RVALID` und `RREADY`), wird `rdata_hold` geladen, `have_word` gesetzt und `byte_idx` auf 0 initialisiert. Solange `have_word=1` ist, wird in jedem Takt genau ein Byte aus `rdata_hold[8*byte_idx +: 8]` an den entsprechenden SRAM-Write-Port ausgegeben und die Zieladresse `dst_cur` um 1 inkrementiert. Nach vier Bytes wird `have_word` zurückgesetzt, wodurch der DMA-Controller das nächste AXI-Wort annehmen kann. Der gleiche Mechanismus wird für Input, Weights, Bias, Shift und Multiplier verwendet; die Bytes werden jeweils in den zugehörigen Speicherbereich geschrieben.

Zähler für Burst und Gesamtlänge. Der Fortschritt innerhalb eines Bursts wird über `beat_cnt` verfolgt, welcher die empfangenen AXI-Beats bis `beats_in_burst` zählt. Parallel hält `bytes_rem` die verbleibenden Bytes der aktuellen Ladephase. Nach jedem vollständig serialisierten 32-bit Wort wird `bytes_rem` um 4 reduziert (bzw. auf 0 gesetzt, falls weniger als 4 Bytes verbleiben). Sobald alle Beats des aktuellen Bursts empfangen wurden und die Byte-Serialisierung abgeschlossen ist (Bedingung: `beat_cnt == beats_in_burst` und `have_word == 0`), gilt:

- Falls `bytes_rem > 0`: Rücksprung in den passenden `S_REQ_*`-Zustand, um einen weiteren Burst für die verbleibenden Daten anzufordern.
- Falls `bytes_rem == 0`: Übergang in die nächste Ladephase (`WG` → `BIAS` → `SHIFT` → `MULT`) bzw. nach Abschluss aller Ladephasen in `S_WAIT_FOR_ACC`.

Start des KI-Beschleunigers und optionales Write-Back. Nach Abschluss der Ladephasen wechselt der DMA-Controller in den Zustand `S_DONE` und erzeugt dort einen einzyklischen `done`-Puls. Dieser Puls wird in der Systemintegration als `ctrl_start_pulse` weiterverwendet und startet den KI-Beschleuniger. Der Zustand `S_DONE` dient somit als Synchronisationspunkt zwischen Datentransfer und Rechenstart.

Der weitere Ablauf nach `S_DONE` hängt von der Write-Back-Konfiguration ab. Ist der Write-Back-Pfad deaktiviert (`wb_enable=0` oder `wb_byte_len=0`), kehrt der DMA-Controller direkt aus `S_DONE` in den Ruhezustand `S_IDLE` zurück und es findet kein Rückschreiben statt. Ist der Write-Back-Pfad hingegen aktiviert (`wb_enable=1` und `wb_byte_len ≠ 0`), wechselt der DMA-Controller nach dem Startpuls aus `S_DONE` zunächst in den Zustand `S_WAIT_FOR_ACC` und wartet

auf das Signal `acc_layer_done`. Anschließend wird der Rücktransfer der Ergebnisse zum Host über `S_REQ_WB` und `S_WB` ausgeführt. Nach Abschluss des Write-Backs kehrt der DMA-Controller wieder in den Ruhezustand `S_IDLE` zurück.

Bias/Shift/Mult/Mult mit 32-bit Breite. Obwohl Bias, Shift und Multiplier logisch 32-bit Werte pro Filter bzw. Output-Kanal darstellen, erfolgt der DMA-Transfer dennoch byteweise. Der DMA-Controller schreibt hierfür vier aufeinanderfolgende Bytes pro Wert in die jeweiligen Speicherbereiche. Beim späteren Lesen setzt `sram_bias_shift` die 32-bit Werte wieder korrekt aus vier Bytes im *Little-Endian*-Format zusammen:

$$\text{value}[31:0] = \{\text{mem}[\text{base}+3], \text{mem}[\text{base}+2], \text{mem}[\text{base}+1], \text{mem}[\text{base}+0]\}.$$

Damit ist gewährleistet, dass Bias-, Shift- und Mult-Parameter im Rechenkern bitgenau zur Verfügung stehen, obwohl die physische Speicherbefüllung über einen 8-bit Write-Port erfolgt.

Für die Anbindung an das Systolic Array werden die Parameter blockweise bereitgestellt: Ein Lesezugriff liefert im Folgetakt einen Vektor aus `COLS_MAX` 32-bit Werten für Bias, Shift und Multiplier.

Tabelle 4.3: AXI4-Master-Schnittstelle des DMA-Controllers: verwendete Kanäle und Signale

Kanal	Signale (DMA)	Bedeutung im DMA-Controller
Read Address (AR)	M_AXI_ARADDR, M_AXI_ARLEN, M_AXI_ARSIZE, M_AXI_ARBURST, M_AXI_ARVALID, M_AXI_ARREADY	Startet einen Read-Burst. ARADDR ist die Quelladresse, ARLEN die Burst-Länge (Beats-1), ARSIZE=2 entspricht 4 Byte/Beat, ARBURST ist INCR. Handshake über ARVALID/ARREADY.
Read Data (R)	M_AXI_RDATA, M_AXI_RRESP, M_AXI_RVALID, M_AXI_RREADY, M_AXI_RLAST	Überträgt die gelesenen 32-bit Datenwörter. RVALID/RREADY bestätigt die Annahme eines Beats. RRESP signalisiert Fehler (ungleich OKAY). RLAST markiert das Ende eines Bursts.
Write Address (AW)	M_AXI_AWADDR, M_AXI_AWLEN, M_AXI_AWSIZE, M_AXI_AWBURST, M_AXI_AWVALID, M_AXI_AWREADY	Nur bei aktiviertem Write-Back: startet einen Write-Burst. AWADDR ist die Zieladresse, AWLEN die Burst-Länge (Beats-1), AWSIZE=2 entspricht 4 Byte/Beat, AWBURST ist INCR.
Write Data (W)	M_AXI_WDATA, M_AXI_WSTRB, M_AXI_WVALID, M_AXI_WREADY, M_AXI_WLAST	Nur bei Write-Back: sendet Datenwörter zum Host. WSTRB ist konstant auf "alle Bytes gültig" gesetzt. Handshake über WVALID/WREADY. WLAST markiert das Ende eines Bursts.
Write Response (B)	M_AXI_BVALID, M_AXI_BREADY, M_AXI_BRESP	Nur bei Write-Back: Abschlussmeldung einer Write-Transaktion. BRESP signalisiert Fehler (ungleich OKAY). Handshake über BVALID/BREADY.

4.3 Implementierung des Arithmetikmoduls

4.3.1 Arithmetikmodul: Überblick und Einordnung

Das Arithmetikmodul bildet den Kern der numerischen Verarbeitung innerhalb des Beschleunigers. Es übernimmt die Berechnung der Faltungsoperation auf Basis der vom Fetching-Pfad bereitgestellten Aktivierungs- und Gewichtsdaten und erzeugt daraus die Pixelwerte der Zwischenergebnisse bzw. Ausgabefeatures. Optional kann die Ausgabe im Anschluss durch eine ReLU-Stufe nichtlinear transformiert werden; weitere Nachverarbeitungsschritte wie Pooling werden übergeordnet gesteuert.

Strukturell ist das Arithmetikmodul als Pipeline aufgebaut, die aus drei logisch getrennten Teilbereichen besteht:

1. **Steuerung (`arithmetic_controller`):** Eine FSM koordiniert die Aktivierung der Rechenpipeline über `start_ctrl` und erzeugt die internen Enable-Signale für die Faltungsstufe und die ReLU-Stufe.
2. **Faltungsarithmetik (`Systolic Array Module`):** Dieses Modul instanziiert ein zweidimensionales Array aus *Processing Elements* (PEs). Im normalen Convolution-Modus wird das Array systolisch betrieben: Aktivierungen propagieren horizontal, Kernelwerte vertikal, und jedes PE akkumuliert eine MAC-Sequenz für einen Ausgabewert. Im Depthwise-Modus wird der Datenfluss angepasst, sodass nur eine Teilmenge der PEs (Diagonale) aktiv ist und die MAC-Sequenzlänge entsprechend reduziert wird.
3. **Nichtlinearität (`relu_array`):** Abhängig von `relu_on_cfg` werden die Pixelwerte nach der Arithmetik entweder unverändert durchgereicht oder elementweise mittels ReLU verarbeitet. Die endgültige Auswahl (mit/ohne ReLU) erfolgt im Arithmetikmodul durch eine kombinatorische Umschaltung der Arraysignale.

Die Schnittstelle des Arithmetikmoduls ist dabei konsequent auf die physische Maximalbreite des Arrays ausgelegt (`LANES_MAX` × `COLS_MAX`), während die tatsächlich genutzten Zeilen und Spalten über die Laufzeitparameter `cfg_tile_r` und `cfg_tile_c` aktiviert werden. Dadurch bleibt das Design parametrisierbar und kann für unterschiedliche Tile-Größen sowie Convolution- Varianten wiederverwendet werden.

4.3.2 Processing Element

Das *Processing Element* stellt die kleinste Recheneinheit des Systolic Arrays dar und führt die Multiply–Accumulate–Operation (MAC) aus. Jedes PE verarbeitet dabei eine Folge von Operandenpaaren (Eingabewert und Kernelgewicht) und akkumuliert deren Produkte zu einem Zwischenergebnis, das einem Ausgabepixel zugeordnet ist. Durch die regelmäßige Struktur kann eine Vielzahl identischer PEs zu einem zweidimensionalen Array kaskadiert werden, in dem Daten und Steuersignale takt synchron von PE zu PE weitergereicht werden (vgl. Abbildung 4.3).

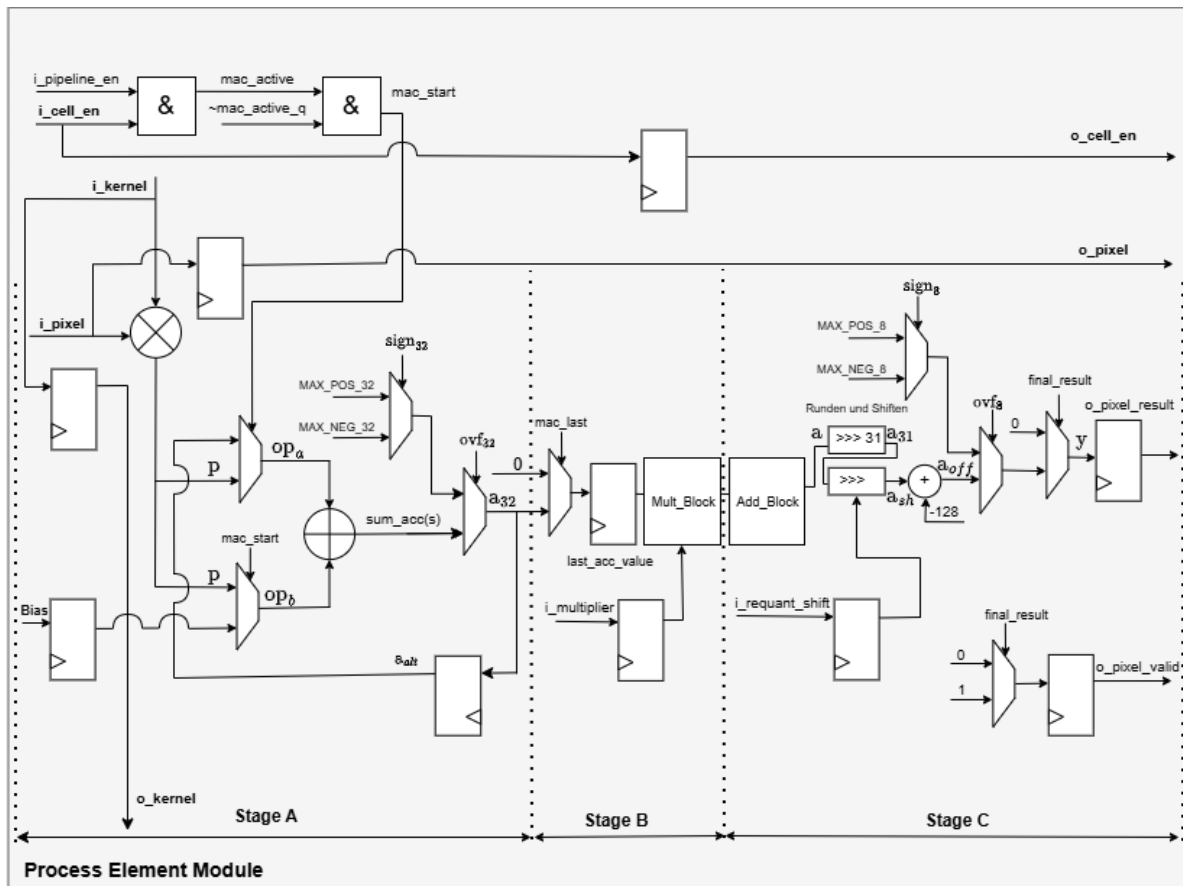


Abbildung 4.3: Processing Element: Datenpfad und Steuerlogik im PE.

Das PE hat in der Gesamtarchitektur drei zentrale Aufgaben:

1. **Lokale MAC-Berechnung:** Multiplikation von Aktivierungswert und Gewicht sowie Akkumulation im lokalen Register. Zu Beginn einer MAC-Sequenz wird zusätzlich der Bias-Term addiert; anschließend werden die weiteren Produkte auf den Akkumulator aufaddiert. Das Ergebnis wird auf 32 Bit gesättigt und am Ende eines Fensters als finales Akkumulatorresultat für die nachfolgende Pipeline-Stufe registriert.
2. **Multi-Block und Requantisierung:** Das registrierte 32-bit Akkumulatorergebnis wird über einen **Multi-Block** mit dem quantisierten Skalierungsfaktor multipliziert. Der Multi-Block ist als **pipelined 32×32 -Multiplikation** implementiert und zerlegt die Operation in vier partielle 17×17 -Produkte, die anschließend zu einem 64-bit Produkt zusammengesetzt werden. Zur Fixed-Point-Skalierung wird dieses Produkt **arithmetisch um 31 Bit nach rechts verschoben** ($\gg 31$); dies entspricht der in **TensorFlow Lite** verwendeten Quantisierungsarithmetik. Anschließend erfolgt die **Requantisierung** durch **Runden** und **arithmetisches Shiften** gemäß dem Requantierungs-Shift. Abschließend wird der Output-Offset (-128) addiert und das Ergebnis auf 8 Bit gesättigt (Clamp auf $[-128, 127]$), bevor der Pixelwert ausgegeben wird.
3. **Taktsynchrone Kopplung im systolischen Array:** Zur räumlich-korrekten Datenweitergabe werden Aktivierungswerte und Enable-Signale taktsynchron nach rechts weitergereicht, während die Gewichte nach unten propagiert werden. Dadurch bleibt die Zuordnung der Ope-

randen in der PE-Matrix erhalten und es entsteht das typische systolische Datenflussverhalten mit lokaler Berechnung und deterministischen Pipeline-Latenzen (Stage A/B/C).

Startdetektion und Aktivierung

Das PE wird über zwei Kontrollsignale aktiviert: das globale Pipeline-Enable $i_pipeline_en$ und das lokale Zellen-Enable i_cell_en . Daraus ergibt sich die effektive Aktivität als:

$$mac_active = i_pipeline_en \wedge i_cell_en. \quad (4.10)$$

Der Beginn einer neuen MAC-Sequenz wird durch Flankenerkennung realisiert:

$$mac_start = mac_active \wedge \neg mac_active_q, \quad (4.11)$$

wobei mac_active_q den registrierten Wert von mac_active aus dem vorherigen Takt darstellt. Damit ist mac_start genau einen Takt aktiv und markiert den ersten MAC-Schritt einer Sequenz.

Dreistufiger Datenpfad zur Reduktion des kritischen Pfades

Um den kritischen Pfad im PE zu verkürzen, ist der Rechenpfad in drei Pipeline-Stufen gegliedert. Ohne diese Aufteilung würde sich der kombinatorische Pfad über *Multiplikation* $\rightarrow$ *Addierer* $\rightarrow$ *Sättigung* $\rightarrow$ *Skalierungs-Multiplikation* $\rightarrow$ *Runden/Shift* $\rightarrow$ *8-bit Sättigung* erstrecken. Dies führt insbesondere bei höheren Taktraten zu einer kritischen Laufzeit.

Daher wird der Datenpfad in:

- **Stage A:** Produktbildung, MAC-Akkumulation (inkl. Bias-Initialisierung) und 32-bit Sättigung,
- **Stage B:** pipelined Skalierungs-Multiplikation (32×32) über partielle Produkte und Registrierung,
- **Stage C:** Requantisierung, Output-Offset und 8-bit Sättigung mit Ausgabe

unterteilt.

Stage A: MAC-Akkumulation und 32-bit Sättigung

In Stage A wird zunächst das Produkt aus Eingabewert und Kernelgewicht gebildet:

$$p = i_pixel \cdot i_kernel. \quad (4.12)$$

Wie in Abbildung 4.3 dargestellt, werden zwei Operandenpfade (op_a , op_b) in den Addierer sum_acc geführt. Der Steuereingang mac_start bestimmt die Initialisierung:

- **Starttakt** ($mac_start=1$): Bias-Initialisierung der Sequenz:

$$s = p + Bias.$$

- **Folgetakte:** Akkumulation über den bisherigen Akkumulatorwert:

$$s = a_{alt} + p.$$

Die Akkumulation erfolgt in einem erweiterten Akkumulator (33 bit), um Überläufe zuverlässig gegenüber dem 32-bit Zielbereich zu erkennen. Die Akkumulation erfolgt in ACC_W=33 bit, um Überläufe beim Übergang in den 32-bit Zielbereich sicher zu erkennen. Ein Überlauf liegt vor, wenn sich das erweiterte Vorzeichenbit vom MSB des 32-bit Ergebnisses unterscheidet:

$$\text{ovf}_{32} = s_{32} \oplus s_{31}.$$

Zur Unterscheidung zwischen positivem und negativem Überlauf wird das Vorzeichen des 33-bit Werts verwendet:

$$\text{sign}_{32} = s_{32}.$$

Mit:

$$\text{MAX_POS_32} = 2^{31} - 1, \quad \text{MIN_NEG_32} = -2^{31}$$

ergibt sich die Sättigung zu:

$$a_{32} = \begin{cases} \text{MAX_POS_32}, & (\text{ovf}_{32} = 1) \wedge (\text{sign}_{32} = 0), \\ \text{MIN_NEG_32}, & (\text{ovf}_{32} = 1) \wedge (\text{sign}_{32} = 1), \\ s_{31:0}, & (\text{ovf}_{32} = 0). \end{cases} \quad (4.13)$$

Das PE verarbeitet pro Aktivierung nicht nur einen einzelnen MAC-Schritt, sondern eine Sequenz mehrerer MAC-Operationen, die gemeinsam ein Ausgabepixel bilden. `mac_last` kennzeichnet dabei den letzten Schritt dieser Sequenz und damit den Zeitpunkt, an dem die vollständige Summe vorliegt.

Intern wird die Sequenzlänge zur Laufzeit über `i_kernel_elems` vorgegeben. Der Zähler `elem_cnt` zählt, wie viele MAC-Schritte innerhalb der aktuellen Sequenz bereits ausgeführt wurden. Der letzte Schritt ist erreicht, wenn:

$$\text{elem_cnt} = \text{i_kernel_elems} - 1. \quad (4.14)$$

In diesem Takt gilt `mac_last=1` und das PE erkennt, dass das aktuelle Akkumulationsergebnis vollständig ist. Nach Abschluss der Sequenz wird `elem_cnt` zurückgesetzt, sodass der nächste Pixel-Akkumulationslauf wieder eindeutig bei Null beginnen kann.

Für die Depthwise-Convolution beträgt die Sequenzlänge $\text{cfg_kh} \cdot \text{cfg_kw}$, da pro Ausgabepixel ausschließlich über die Kernelpositionen eines einzelnen Kanals akkumuliert wird. Für eine reguläre Convolution ergibt sich die Sequenzlänge zu $\text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin}$, weil zusätzlich über alle Eingangskanäle akkumuliert wird. Bei einer Pointwise-Convolution (1×1) entspricht die Sequenzlänge `cfg_cin`.

Sobald `mac_last` erreicht ist, wird der finale, bereits auf 32 bit saturierte Akkumulatorwert a_{32} in einem Register registriert:

$$\text{last_acc_value} \leftarrow a_{32}. \quad (4.15)$$

Damit steht der Endwert der Akkumulation im Folgetakt stabil zur Weiterverarbeitung in Stage B zur Verfügung.

Stage B: Mult-Block

In Stage B wird der in Stage A registrierte, 32-bit saturierte Akkumulatorwert `last_acc_value` als Operand_A mit dem pro Output-Kanal bereitgestellten Skalierungsfaktor `i_multiplier` als

Operand_B multipliziert. Die Multiplikation wird als pipelined Mult-Block realisiert und nutzt vier 17-bit Teilmultiplikatoren; der prinzipielle Aufbau ist in Abbildung 4.4 dargestellt.

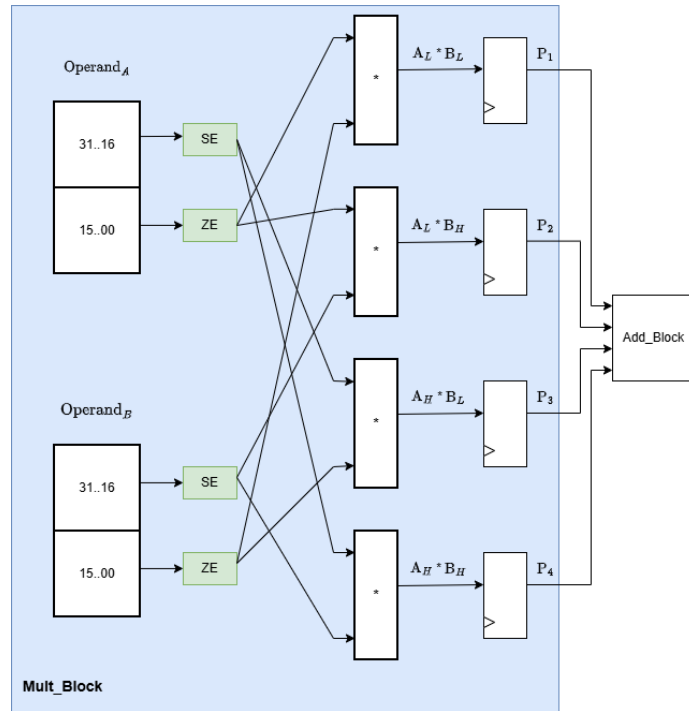


Abbildung 4.4: Mul-Block zur 32×32-Multiplikation: Zerlegung in Low-/High-Anteile, Bildung der vier Teilprodukte $P_1..P_4$ und anschließende Zusammenführung im Add-Block.

Zur Zerlegung werden beide Operanden in ein Low- und High-Wort aufgeteilt:

$$A = \{A_H, A_L\}, \quad B = \{B_H, B_L\}.$$

Die Low-Anteile A_L und B_L werden als *unsigned* 16-bit Werte interpretiert und per *Zero-Extension* (ZE) auf 17 bit erweitert. Dadurch gilt:

$$A_L, B_L \in \{0, \dots, 2^{16} - 1\}.$$

Die High-Anteile A_H und B_H entsprechen den oberen 16 bit der 32-bit Operanden und werden als *signed* 16-bit Werte interpretiert; sie werden per *Sign-Extension* (SE) auf 17 bit erweitert, um das Vorzeichen korrekt zu erhalten. Damit gilt:

$$A_H, B_H \in \{-2^{15}, \dots, 2^{15} - 1\}.$$

Aus diesen Anteilen entstehen die vier Teilprodukte (vgl. Abbildung 4.4):

$$P_1 = A_L \cdot B_L, \quad P_2 = A_L \cdot B_H, \quad P_3 = A_H \cdot B_L, \quad P_4 = A_H \cdot B_H.$$

Die vier Teilprodukte werden am Ausgang der Teilmultiplikatoren registriert ($P_1..P_4$) und anschließend im Add_Block zu einem 64-bit Produkt zusammengesetzt. Mit dem Eingang in den Add_Block beginnt die nachfolgende Pipeline-Stufe.

Stage C: Add-Block und Requantisierung

Stage C beginnt mit dem Add_Block, der die in Stage B registrierten Teilprodukte $P_1..P_4$ zu einem 64-bit Zwischenprodukt a zusammensetzt (vgl. Abbildung 4.5):

$$a = P_1 + (P_2 \ll 16) + (P_3 \ll 16) + (P_4 \ll 32). \quad (4.16)$$

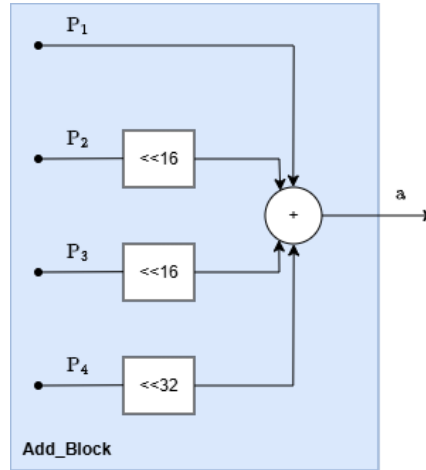


Abbildung 4.5: Add-Block: Zusammensetzen des 64-bit Zwischenprodukts a aus den registrierten Teilprodukten $P_1..P_4$.

Ausgehend von a wird zunächst der skaliert-gerundete 32-bit Zwischenwert gebildet, indem arithmetisch um 31 Bit nach rechts verschoben wird. Zur Rundung wird vor der Verschiebung ein Bias $\pm 2^{30}$ addiert:

$$a_{31} = \begin{cases} (a + 2^{30}) \ggg 31, & a \geq 0, \\ (a - 2^{30}) \ggg 31, & a < 0. \end{cases} \quad (4.17)$$

Der anschließende Requantisierungs-Shift wird durch $i_requant_shift = s$ vorgegeben. Für $s < 0$ erfolgt ein arithmetischer Right-Shift um $|s|$; für $s \geq 0$ bleibt der Wert unverändert. Damit wird direkt der requantisierte Zwischenwert a_{sh} aus a_{31} bestimmt. Zur korrekten Rundung wird vor der Verschiebung erneut ein Bias r addiert:

$$\text{right} = \begin{cases} 0, & s \geq 0, \\ |s|, & s < 0, \end{cases} \quad r = \begin{cases} 2^{\text{right}-1}, & \text{falls } a_{31} \geq 0, \\ 2^{\text{right}-1} - 1, & \text{falls } a_{31} < 0. \end{cases}$$

$$a_{sh} = \begin{cases} a_{31}, & \text{right} = 0, \\ (a_{31} + r) \ggg \text{right}, & \text{right} > 0. \end{cases} \quad (4.18)$$

Anschließend wird der konstante Output-Offset addiert:

$$a_{\text{off}} = a_{sh} - 128. \quad (4.19)$$

Das Ergebnis wird abschließend auf den 8-bit Ausgabebereich gesättigt. Die Sättigungsgrenzen lauten:

$$\text{MAX_POS_8} = 127, \quad \text{MIN_NEG_8} = -128.$$

Analog zur 32-bit Sättigung kann dies über eine Sign-Extension-Prüfung formuliert werden: Für einen korrekt darstellbaren 8-bit Wert müssen alle Bits oberhalb von Bit 7 einer Sign-Extension von Bit 7 entsprechen. Formal:

$$\text{ovf}_8 = \bigvee_{i=8}^{31} ((a_{\text{off}})_i \oplus (a_{\text{off}})_7).$$

Damit ergibt sich die Ausgabe:

$$y = \begin{cases} \text{MAX_POS_8}, & (\text{ovf}_8 = 1) \wedge ((a_{\text{off}})_{31} = 0), \\ \text{MIN_NEG_8}, & (\text{ovf}_8 = 1) \wedge ((a_{\text{off}})_{31} = 1), \\ (a_{\text{off}})_{7:0}, & \text{ovf}_8 = 0. \end{cases} \quad (4.20)$$

Das Resultat wird als `o_pixel_result` ausgegeben.

Durch die Pipeline entsteht eine definierte Latenz zwischen dem Abschluss der Akkumulation in Stage A und der Ausgabe des finalen 8-bit Pixels. Da die nachfolgende Verarbeitung in zwei weiteren Stufen organisiert ist (Stage B: Mul-Block, Stage C: Add-Block/Requantisierung), ergibt sich insgesamt eine **Zwei-Takt-Latenz** vom Zeitpunkt `mac_last` bis zur gültigen Ausgabe. Zur zeitlichen Kennzeichnung der Gültigkeit wird ein internes Flag `final_result` verwendet. Ein gesetztes `final_result` signalisiert, dass der zugehörige Pixelwert am Ausgang der Pipeline gültig ist.

In dem Takt, in dem `final_result` anliegt, wird:

- der berechnete Pixelwert auf `o_pixel_result` übernommen und
- `o_pixel_valid` für genau einen Takt gesetzt.

Bypass-/Pipeline-Register zur Array-Kopplung

Zur Kopplung im Systolic Array werden ausgewählte Eingangsgrößen taktsynchron über Pipeline-Register an benachbarte PEs weitergereicht (vgl. Abbildung 4.3). Dabei propagieren der Eingabepixelstrom `i_pixel` sowie das lokale Enable `i_cell_en` in Richtung des rechtsliegenden PEs (horizontale Weitergabe). Der Kernelstrom `i_kernel` wird hingegen in Richtung des darunterliegenden PEs weitergereicht (vertikale Weitergabe).

Entsprechend stellt das PE registrierte Ausgänge für die horizontale und vertikale Kopplung bereit, sodass die Operanden im Array taktsynchron verschoben werden. Dadurch entsteht eine deterministische Pipeline über die PE-Nachbarschaften, die das systolische Rechenprinzip ermöglicht.

Reset-Verhalten

Bei aktivem `reset` werden Akkumulator, Zähler, `final_result`, Valid-Flag sowie alle Pipeline-Register deterministisch auf Null gesetzt. Dadurch ist ein definierter Startzustand gewährleistet, der nachfolgende MAC-Sequenzen eindeutig und reproduzierbar starten lässt.

4.3.3 Arithmetic Controller

Der Arithmetic Controller bildet die zentrale Laufzeitsteuerung der Rechenpipeline des Beschleunigers. Er koordiniert den Start der Arithmetikphase, überwacht Abschlussbedingungen der Pipeline und stellt die zeitlich korrekte Aktivierung der Recheneinheit sicher. Die zugehörige Steuerlogik ist als endlicher Zustandsautomat in Abbildung 4.6 dargestellt.

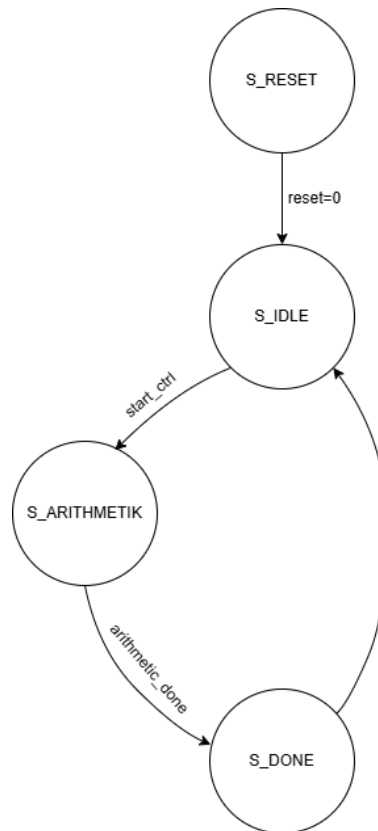


Abbildung 4.6: Zustandsautomat des Arithmetic Controllers zur Laufzeitsteuerung der Rechenpipeline.

Konfigurationsschnittstelle Die Aktivierung optionaler Funktionsblöcke erfolgt über das CSR_CONTROL-Register. Dabei steuert `ctrl_relu_en` (bit0) die ReLU-Aktivierung, während `ctrl_dw_en` (bit2) und `ctrl_pw_en` (bit4) den Betriebsmodus (Convolution vs. Depthwise vs. Pointwise) der Recheneinheit vorgeben. Der Start der Berechnung wird in der Integration als Puls `start_ctrl` (vom Fetching-Modul) bereitgestellt; ein explizites CSR-start ist in der hier verwendeten CSR-Map nicht vorgesehen.

Start- und Timing-Verhalten. Der Ausgang `arithmetic_en` ist bewusst als *1-Takt-Startpuls* ausgeführt, der im letzten `S_IDLE`-Zyklus erzeugt wird, wenn ein effektiver Start erkannt wird. Dadurch kann das `arithmetic_module` intern in einen „running“-Zustand wechseln, während der erste tatsächliche MAC-Schritt im Systolic Array erst im darauffolgenden Takt erfolgt. Diese Definition ist wesentlich, damit die Datenanlieferung aus dem Fetcher zeitlich exakt zur ersten `mac_start`-Aktivierung der Processing Elements passt.

Zustände und Übergänge. Nach einem Reset befindet sich der Controller in S_RESET und wechselt unmittelbar in S_IDLE, sobald reset deassertiert ist. In S_IDLE wartet die FSM auf einen Startpuls (start_ctrl). Bei erkannter Startflanke erfolgt der Übergang nach S_ARITHMETIC, und gleichzeitig wird arithmetic_en für genau einen Takt gesetzt.

Im Zustand S_ARITHMETIC läuft die eigentliche Faltungsberechnung: je nach dw_en werden Standard-Convolution oder Depthwise-Operationen ausgeführt. Falls ctrl_relu_en=1 ist, wird die ReLU-Verarbeitung parallel bzw. unmittelbar nach der Erzeugung gültiger Ausgabepixel angestoßen. Der Zustand S_ARITHMETIC wird verlassen, sobald die Recheneinheit arithmetic_done=1 signalisiert. Anschließend wird S_DONE erreicht und die FSM kehrt danach nach S_IDLE zurück, um auf den nächsten Start zu warten.

Write-Back und Done-Semantik. Das arithmetic_done-Ereignis markiert in der vorliegenden Implementierung ausschließlich das Ende der *Rechenphase* und zeigt an, dass das Ergebnisraster `pix_accel[r][c]` für das aktuelle Tile stabil anliegt. Der eigentliche Rückschreibvorgang in die als Schreibbank konfigurierte Feature-SRAM wird im `processing_module` realisiert und durch eine separate Write-Back-Controller-FSM ausgeführt.

Daher wird der Fetching-Pfad nicht durch arithmetic_done, sondern durch das vom `processing_module` zurückgeführte Signal `acc_done` synchronisiert. Dieses Signal entspricht dem Puls `store_done_pulse` und signalisiert, dass alle relevanten Tile-Elemente vollständig geschrieben wurden bzw. zumindest vollständig aus dem Rechenraster übernommen wurden. Erst nach `acc_done` darf der Fetcher die nächste Gruppe starten, wodurch eine konsistente Übergabe zwischen Streaming, Rechenpipeline und Rückschreibpfad gewährleistet ist.

4.3.4 Systolic Array Module

Das Modul `Systolic Array Module` bildet den Rechenkern des Systolic Arrays. Es steuert die zeitliche Einspeisung von Aktivierungen (`i_act_left_array`) und Gewichten (`i_wgt_top_array`) in das PE-Gitter, erzeugt die lokalen Enable-Signale für die PEs und liefert die Pixelwerte inklusive valid-Kennzeichnung.

Convolution-Modus

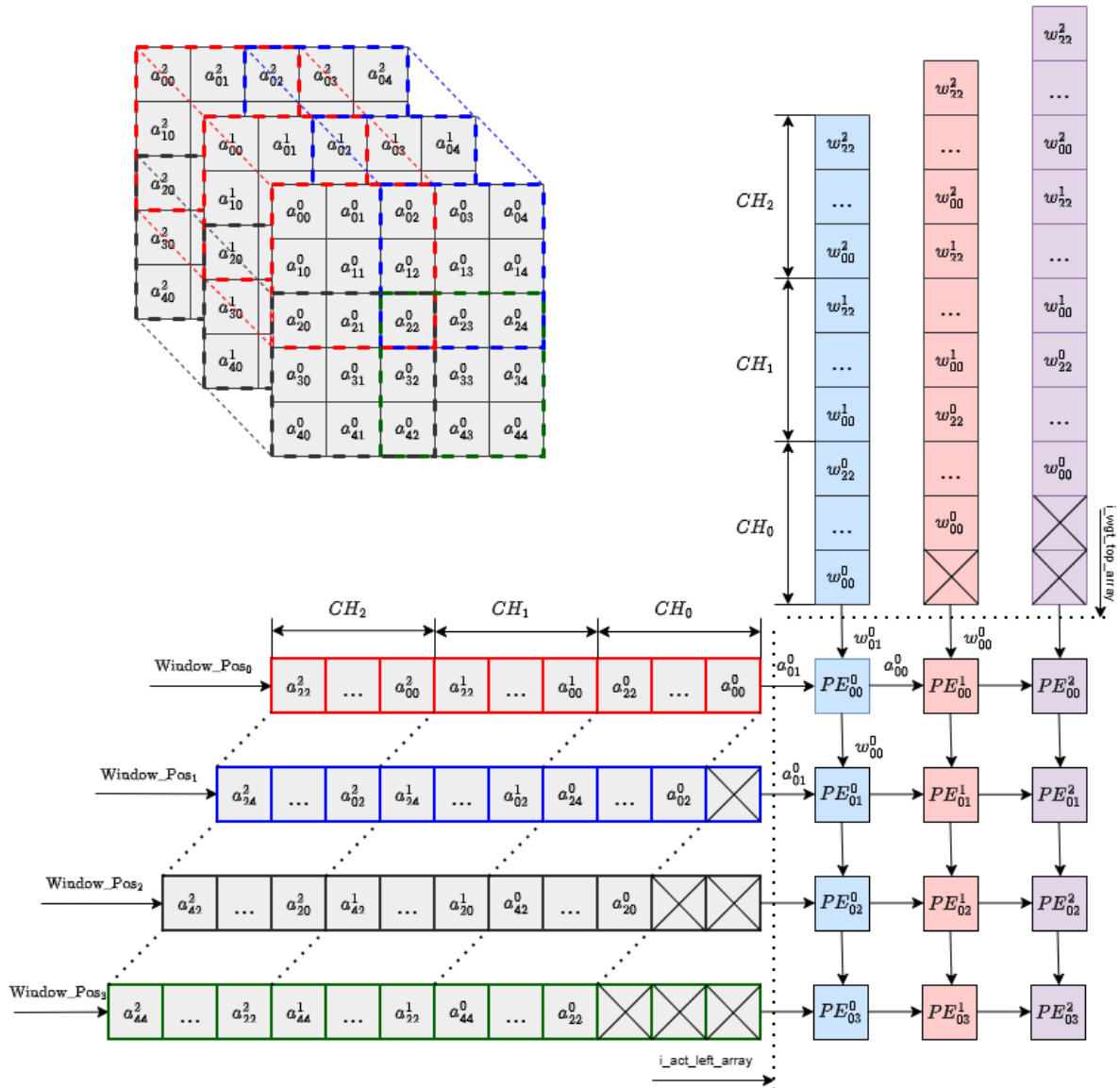


Abbildung 4.7: Datenfluss im Convolution-Modus (*output-stationary*): Aktivierungen (von links) und Gewichte (von oben) strömen durch das Array, während die partiellen Summen im jeweiligen PE verbleiben und über K_{elems} MAC-Schritte akkumuliert werden; die farbigen Zeilen zeigen aufeinanderfolgende Fensterpositionen $Window_Pos_i$ über alle Kanäle.

Aktivierung über Konfigurationsregister. Der Convolution-Modus ist aktiv, wenn $ctrl_dw_en=0$. Die Geometrie und Sequenzlänge werden zur Laufzeit über folgende CSR-Felder vorgegeben: cfg_tile_r und cfg_tile_c (aktive PE-Kachel innerhalb des physisch maximalen Arrays), cfg_kh , cfg_kw sowie cfg_cin . Daraus ergibt sich die Anzahl Kernel-Elemente pro Ausgabepixel zu:

$$K_{\text{elems}} = cfg_kh \cdot cfg_kw \cdot cfg_cin. \quad (4.21)$$

Output-stationary Verarbeitung und Laufzeit. Im output-stationary Datenfluss berechnet jedes PE ein fest zugeordnetes Ausgabepixel: Aktivierungen (`i_act_left_array`) und Gewichte (`i_wgt_top_array`) werden taktsynchron durch das Array gestreamt, während der Akkumulator im PE die partiellen Summen über mehrere Takte aufsummiert (vgl. Unterabschnitt 4.3.2). Die Gesamtdauer setzt sich dabei aus zwei Anteilen zusammen:

- **Pipeline-Fill-Zeit:** $(\text{cfg_tile_r} - 1) + (\text{cfg_tile_c} - 1)$ Takte, bis die Operanden das rechte untere PE der aktiven Kachel erreichen (diagonale Datenwelle).
- **MAC-Sequenzdauer:** K_{elems} Takte für die Akkumulation der $\text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin}$ Kernel-Elemente im Output-Akkumulator des PEs.

Durch das Pipelining im Processing Element (Stage A/Stage B/Stage C mit `final_result`) entsteht zusätzlich eine definierte Zwei-Takt-Latenz zwischen dem letzten MAC-Schritt (`mac_last`) und der Ausgabe des requantisierten Pixels. Daher müssen zur Laufzeitberechnung zwei zusätzliche Takte berücksichtigt werden.

Im Modul wird diese Zeit als:

$$\text{max_cycles_cur} = (\text{cfg_tile_r} - 1) + (\text{cfg_tile_c} - 1) + K_{\text{elems}} + 2 \quad (4.22)$$

implementiert. Die Terme $(\text{cfg_tile_r} - 1)$ und $(\text{cfg_tile_c} - 1)$ modellieren die diagonale Ausbreitung der Datenwelle im Array, während K_{elems} die Anzahl der MAC-Schritte beschreibt, in denen der im PE liegende Output-Akkumulator gefüllt wird. Der zusätzliche Term $+2$ entspricht den beiden abschließenden Pipeline-Stufen (Stage B/Stage C) im PE. Das Signal `running` ist über genau diese Zyklusanzahl aktiv; anschließend wird `o_busy_arithmetic` deassertiert.

Synchronisation der Einspeisung Damit Aktivierungen und Gewichte in jedem PE im richtigen Takt zusammentreffen, werden die externen Arrays vor der Einspeisung gezielt verzögert:

- **Aktivierungen:** `act_pipe[rr][...]` bildet eine Verzögerungskette pro Zeile. In die linke Arraykante wird zeilenabhängig versetzt eingespeist: `cw_bus[gr][0] = act_pipe[gr][gr]`.
- **Gewichte:** `wgt_pipe[cc][...]` bildet eine Verzögerungskette pro Spalte. In die obere Arraykante wird spaltenabhängig versetzt eingespeist: `kernel_bus[0][gc] = wgt_pipe[gc][gc]`.

Diese diagonale Verzögerung stellt sicher, dass die Operanden an jeder PE-Position gleichzeitig ankommen und dort unmittelbar in den lokalen Output-Akkumulator eingehen.

Aktive Kachel und Deaktivierung nicht genutzter PEs. Für jedes PE (r, c) wird zunächst geprüft, ob es innerhalb der konfigurierten Kachel liegt:

$$\text{pe_active}(r, c) \Leftrightarrow (\text{cfg_tile_r} > r) \wedge (\text{cfg_tile_c} > c).$$

Liegt ein PE außerhalb der aktiven Kachel, werden (i) sein lokales Enable auf 0 gesetzt und (ii) seine Weitergabe-Busse auf 0 gelegt. Dadurch bleiben ungenutzte PEs inaktiv und beeinflussen weder den Datenfluss noch die Akkumulation der aktiven PEs.

Enable-Wavefront im Convolution-Modus. Die Aktivierung der PEs erfolgt als Wavefront von links nach rechts:

- **Linke Kante** ($c = 0$): `cell_en_bus[r][0]` wird für genau K_{elems} Takte aktiv, beginnend ab `cycle_cnt` $\geq r$. Damit startet jede Zeile zeitversetzt, passend zur Pipeline-Ausbreitung.
- **Innere Spalten** ($c > 0$): das Enable wird von links nach rechts weitergereicht: `cell_en_bus[r][c] = cell_en_pe[r][c-1]`.

Damit wird pro Ausgabepixel eine zusammenhängende Aktivierungsphase erzeugt, in der das jeweilige PE seinen lokalen Output-Akkumulator über K_{elems} MAC-Schritte befüllt.

Datenstrom der Fensterpositionen (`Window_Posi`). Der Fetcher liefert im Convolution-Modus die Aktivierungswerte fensterpositionsweise: Für eine feste Fensterposition `Window_Posi` werden die zugehörigen Werte über alle Kanäle $c = 0, \dots, \text{cfg_cin} - 1$ nacheinander in das Array eingespeist. Danach folgt die nächste Fensterposition `Window_Posi+1`. Die farbigen Zeilen in Abbildung 4.7 illustrieren dieses Prinzip exemplarisch (z. B. `Window_Pos0` rot, `Window_Pos1` blau, `Window_Pos2` schwarz, `Window_Pos3` grün). Pro Takt gilt dabei:

- von **links** kommt ein Aktivierungs-Array (`i_act_left_array`) für die Zeilen,
- von **oben** kommt ein Gewichts-Array (`i_wgt_top_array`) für die Spalten,

wodurch sich der in Abbildung 4.7 gezeigte *output-stationary* Datenfluss ergibt: die Operanden strömen, das Output (partielle Summe) bleibt lokal im PE.

Valid- und Done-Erzeugung. Im Convolution-Modus wird das done-Ereignis über die Flanke des `valid`-Signals des rechten unteren PEs der aktiven Kachel detektiert:

$$(r, c) = (\text{cfg_tile_r} - 1, \text{cfg_tile_c} - 1).$$

Sobald dieses PE ein neues gültiges Pixel meldet, erzeugt das Modul einen einzyklischen `o_done_arithmetic`-Puls.

Pointwise-Modus

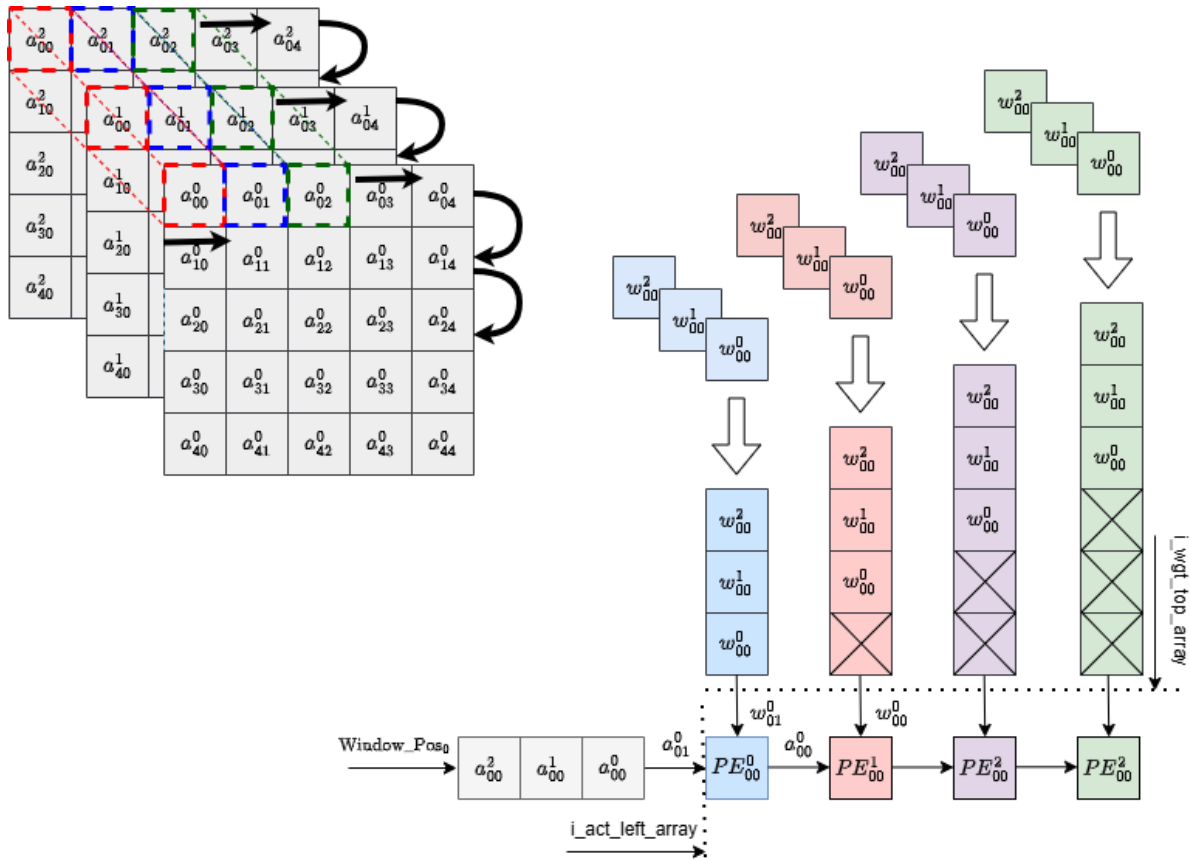


Abbildung 4.8: Datenfluss im Pointwise-Modus (1×1): Für eine feste Pixelposition (x, y) werden die Kanalwerte seriell eingespeist, während pro Takt ein Gewichtsvektor für cfg_tile_c Ausgabekanäle von oben in das Array gestreamt wird. In der aktuellen Implementierung ist $\text{cfg_tile_r} = 1$, sodass ausschließlich die oberste PE-Zeile aktiv ist.

Aktivierung und Laufzeitparameter. Der Pointwise-Modus ist aktiv, wenn das Steuersignal $\text{ctrl_pw_en}=1$ gesetzt ist. Da es sich um eine 1×1-Operation handelt, gilt zur Laufzeit $\text{cfg_kh} = 1$ und $\text{cfg_kw} = 1$. Die Sequenzlänge pro Ausgabepixel wird durch die Anzahl Eingabekanäle bestimmt und ist in der Implementierung als:

$$K_{\text{elems}}^{(\text{PW})} = \text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin} = \text{cfg_cin} \quad (4.23)$$

realisiert. Weiterhin definiert cfg_tile_c die Anzahl parallel berechneter Ausgabekanäle (aktive Spalten des Arrays).

Im Pointwise-Modus entspricht eine Fensterposition Window_Pos_i einer festen räumlichen Position (x, y) im Feature-Map. Für dieses (x, y) werden die Aktivierungen über alle Kanäle $c = 0, \dots, \text{cfg_cin} - 1$ *seriell* verarbeitet: Der Fetcher stellt pro Takt genau einen Kanalwert bereit und speist ihn in die linke Arraykante ein. In der aktuellen Implementierung wird hierfür ausschließlich die erste Zeile genutzt, d.h. $\text{cfg_tile_r} = 1$ und nur $\text{i_act_left_array}[0]$ ist aktiv.

Analog zum Convolution-Modus werden die Gewichte als Spaltenvektor von oben eingespeist: Pro Takt wird ein Gewichtspaket bestehend aus cfg_tile_c Werten in i_wgt_top_array bereitgestellt. Die interne, spaltenabhängige Verzögerungskette sorgt weiterhin für den systolischen Versatz der Gewichte, sodass Aktivierung und Gewicht an der aktiven PE-Position taktsynchron zusammentreffen.

Da $\text{cfg_tile_r} = 1$ gilt, reduziert sich die aktive Kachel auf eine einzelne Zeile:

$$\text{pe_active}(r, c) \Leftrightarrow (r = 0) \wedge (\text{cfg_tile_c} > c).$$

Somit wird pro Takt ein einzelner Aktivierungswert (aktueller Kanal c) in die Arrayzeile eingespeist, während gleichzeitig die zugehörigen Gewichte für mehrere Ausgabekanäle von oben anliegen. Über $K_{\text{elems}}^{(\text{PW})} = \text{cfg_cin}$ Takte akkumuliert jedes aktive PE seine partielle Summe für genau *einen* Ausgabekanal der festen Position (x, y) .

Im Vergleich zur 3×3 -Convolution entfällt im Pointwise-Modus die räumliche Wiederverwendung von Aktivierungen über überlappende Fenster, da keine Nachbarschaftswerte verarbeitet werden. Die aktuelle Implementierung nutzt daher primär *kanalbezogene* Wiederverwendung: Ein Kanalwert der festen Position (x, y) trägt zur Berechnung mehrerer Ausgabekanäle (mehrere Spalten) bei, während die Parallelität über mehrere PE-Zeilen (mehrere gleichzeitige Fensterpositionen) aufgrund von $\text{cfg_tile_r} = 1$ nicht ausgeschöpft wird. Gewichte werden zwar über viele Pixelpositionen wiederverwendet, jedoch in dieser Realisierung weiterhin gestreamt und nicht dauerhaft im Array gehalten.

Laufzeit und Done-Erzeugung. Die Laufzeit im Array setzt sich analog zum Convolution-Modus aus Pipeline-Fill und MAC-Sequenz zusammen. Da $\text{cfg_tile_r} = 1$, reduziert sich die Fill-Zeit auf $(\text{cfg_tile_c} - 1)$ Takte; hinzu kommt die Sequenzdauer cfg_cin sowie die zusätzliche Zwei-Takt-Latenz der PE-Pipeline:

$$\text{max_cycles_cur}^{(\text{PW})} = (\text{cfg_tile_c} - 1) + \text{cfg_cin} + 2. \quad (4.24)$$

Das done-Ereignis wird über die Flanke des `valid`-Signals des rechten Endes der aktiven Zeile detektiert, das heißt für $(r, c) = (0, \text{cfg_tile_c} - 1)$, und als einzyklischer `o_done_arithmetic`-Puls ausgegeben.

Depthwise-Modus

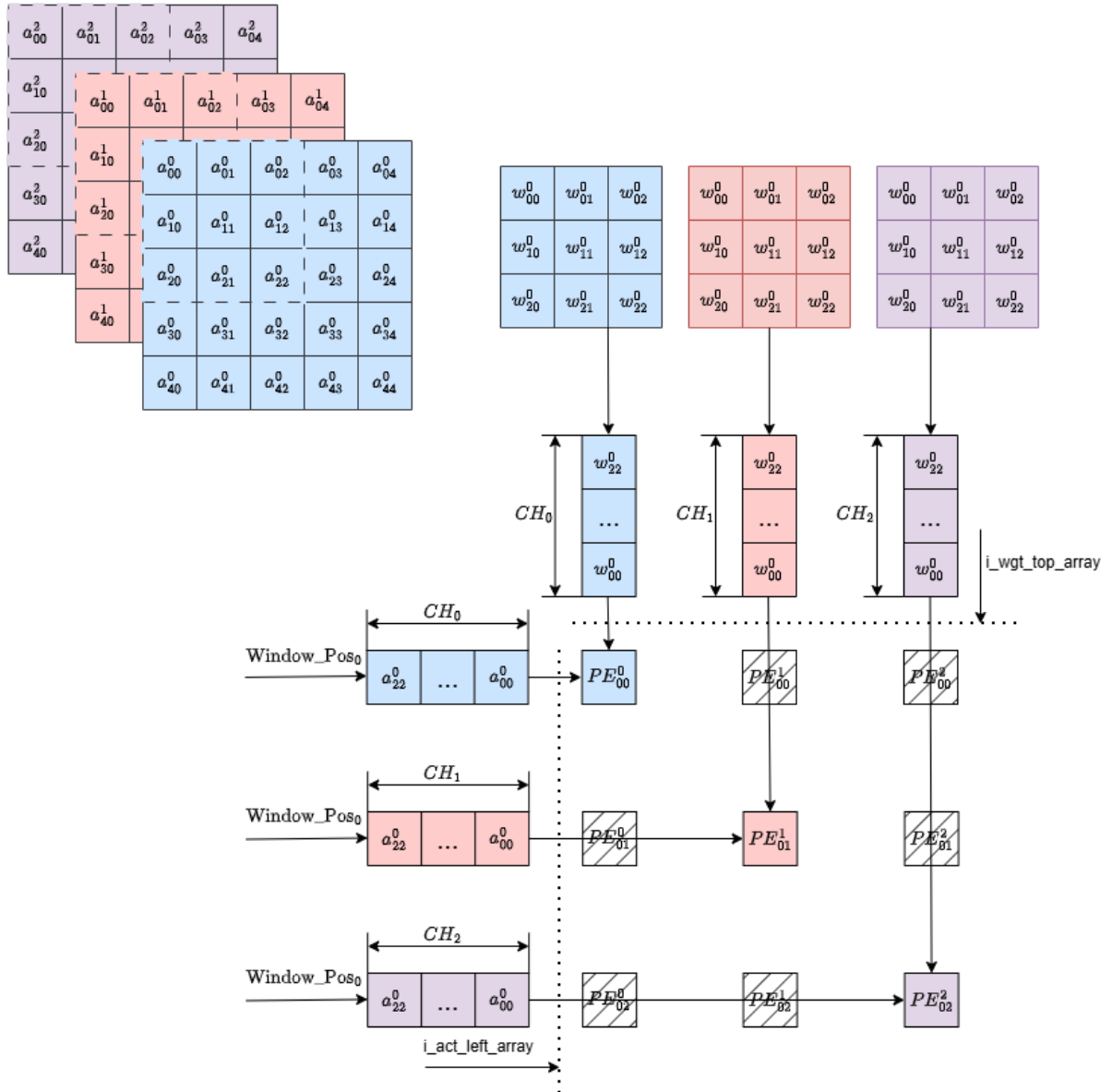


Abbildung 4.9: Datenfluss im Depthwise-Modus (output-stationary): pro Kanal wird ein separater Faltungsfilter verwendet. Die Aktivierungen werden zeilenweise (links) und die zugehörigen Kanal-Gewichte spaltenweise (oben) eingespeist; aktiv sind ausschließlich die diagonalen PEs $PE_{r,r}$.

Im Depthwise-Modus (`ctrl_dw_en=1`) wird *kanalweise* gefaltet: Für jeden Eingabekanal existiert genau ein Filter und es findet *keine Kanalakkumulation* statt. Entsprechend reduziert sich die MAC-Sequenzlänge pro Ausgabewert auf

$$K_{\text{elems,dw}} = \text{cfg_kh} \cdot \text{cfg_kw}. \quad (4.25)$$

Kachelgeometrie und Kanaluordnung. Für die in Abbildung 4.9 dargestellte Zuordnung wird eine quadratische Kachel verwendet, sodass pro Kanal genau ein diagonales PE arbeitet ($\text{cfg_tile_r} = \text{cfg_tile_c}$). Allgemein nutzt das Modul im Depthwise-Modus die Diagonale PEs die übrigen PEs bleiben inaktiv.

Einspeisung durch den Fetcher. Der Fetcher stellt im Depthwise-Modus die Operanden so bereit, dass *in einem Takt* dieselbe Fensterposition Window_Pos_i für alle Kanäle parallel anliegt:

- von **links** kommt ein Aktivierungs-Array i_act_left_array , bei dem Zeile r den Wert des Kanals CH_r für Window_Pos_i trägt,
- von **oben** kommt ein Gewichts-Array i_wgt_top_array , bei dem Spalte c das Gewicht des Kanals CH_c für dieselbe Fensterposition bereitstellt.

Damit erhält das diagonale PE $\text{PE}_{r,r}$ in jedem Takt genau das Paar ($\text{Aktivierung}_{CH_r}, \text{Gewicht}_{CH_r}$) für die aktuelle Fensterposition (vgl. Abbildung 4.9).

Aktive PEs: nur die Diagonale. Im Depthwise-Modus werden ausschließlich diagonale Zellen aktiviert. Dies wird im Modul über is_diag und das lokale Enable umgesetzt:

- **Linke Kante** ($c = 0$): nur $\text{PE}_{0,0}$ wird gestartet ($\text{cell_en_bus}[0][0] = \text{running}$).
- **Innere Spalten** ($c > 0$): nur diagonale PEs werden aktiviert ($\text{cell_en_bus}[r][c] = \text{running} \ \&\& \ (r==c)$).

Alle nicht-diagonalen PEs bekommen $\text{i_cell_en}=0$ und arbeiten damit nicht mit.

Deaktivierung der Datenweitergabe. Im Unterschied zum Convolution-Modus findet im Depthwise-Modus *kein systolisches Durchreichen* über das Array statt. Das wird im Code explizit erzwungen, indem bei $\text{con_mode_dw}=1$ die Weitergabe-Busse genullt werden:

$$\text{cw_bus}[r][c+1] = 0, \quad \text{kernel_bus}[r+1][c] = 0.$$

Zusätzlich werden die PE-Eingänge so gewählt, dass nur die Diagonale echte Operanden sieht:

$$\text{PE}_{r,r} : \text{pe_cw_in} = \text{i_act_left_array}[r], \quad \text{pe_kernel_in} = \text{i_wgt_top_array}[r],$$

während alle anderen PEs Nullwerte erhalten. Dadurch entstehen unabhängige, kanalweise MAC-Ketten ausschließlich auf der Diagonale.

Laufzeit und Done-Kriterium im Depthwise-Modus. Im Depthwise-Modus wird keine systolische Wavefront über Reihen und Spalten propagiert; die Laufzeit entspricht daher direkt der MAC-Sequenzlänge:

$$\text{max_cycles_cur} = \text{cfg_kh} \cdot \text{cfg_kw} + 2. \quad (4.26)$$

Der zusätzliche Term $+2$ entspricht den zwei abschließenden Pipeline-Stufen (Stage B/Stage C) im PE. Das Signal running bleibt genau über diese Anzahl an Takten aktiv. Der Abschluss der Berechnung ist damit äquivalent dazu, dass der Zähler cycle_cnt das Ende der Sequenz erreicht, dies entspricht:

$$\text{cycle_cnt} = \text{max_cycles_cur} - 1.$$

In diesem Zeitpunkt liegt das letzte MAC-Teilergebnis an, und das Modul erzeugt den einzyklischen Puls `o_done_arithmetic`.

4.3.5 ReLU-Module

Die Aktivierungsstufe realisiert eine elementweise ReLU-Funktion, welche negative Werte auf null abbildet und nicht-negative Werte unverändert durchreicht. Auf Skalarebene wird dies durch das Modul ReLU umgesetzt. Die Eingabe `data_in` liegt als vorzeichenbehafteter Vektor der Breite `W` vor. Die Entscheidung erfolgt über das Vorzeichenbit (MSB) `data_in[W-1]`: Ist dieses Bit gesetzt, wird `data_out` auf '0' gesetzt, andernfalls wird `data_in` direkt übernommen. Funktional entspricht dies:

$$\text{data_out} = \max(0, \text{data_in}).$$

Hardwareseitig wird die ReLU durch einen 2:1-Multiplexer realisiert: Ein Pfad führt den arithmetisch berechneten Wert, der zweite Pfad eine konstante Null. Das Vorzeichenbit steuert die Auswahl. Abbildung 4.10 zeigt den Aufbau.

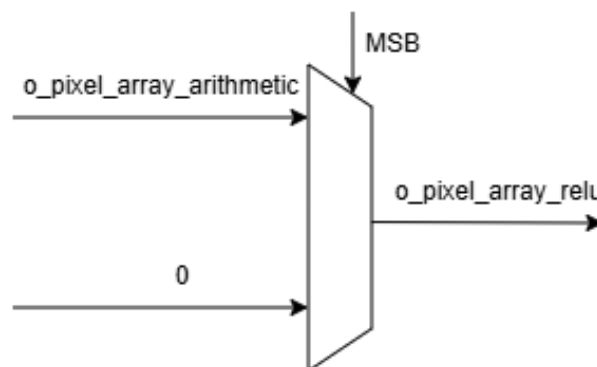


Abbildung 4.10: MUX-basierte Realisierung der ReLU-Aktivierung: negative Werte werden auf 0 abgebildet, nicht-negative Werte unverändert durchgereicht.

Für die kachelbasierte Verarbeitung wird das Skalar-Modul in `relu_array` zu einem zweidimensionalen Array erweitert. Dabei instanziiert `relu_array` für jede Kachelposition (r, c) eine eigene ReLU-Unit, sodass alle Werte der Kachel parallel rectifiziert werden.

4.4 On-Chip-Speicher

4.4.1 I/O-SRAM

Die I/O-SRAM dient als lokaler Zwischenspeicher für Eingangs- und Ausgangs-Feature-Maps und bildet die Schnittstelle zwischen dem DMA-basierten Import und dem internen Streaming-Pfad. Aus Sicht des `fetching_module` ist entscheidend: Pro Takt steht ein komplettes Lane-Array (kanalparallel) bereit, während der DMA die Daten seriell als Byte-Stream anliefert.

Layout, Banking und DMA-Schreiblogik. Die I/O-SRAM ist als *lane-bankierte* Struktur umgesetzt und besteht physisch aus `LANES_MAX` unabhängigen SRAM-Bänken (d. h. `LANES_MAX`

parallel instanziierten RAMs, jeweils eine Bank pro Lane). Jede Bank adressiert Pixel flach und speichert mehrere Kanäle in aufeinanderfolgenden Segmenten (*Kanalgruppen*). Durch diese Banking-Struktur kann der Fetch-Pfad pro Takt ein lane-paralleles Wort aus LANES_MAX Bytes bereitstellen (ein Byte pro Bank), während zusätzliche Kanäle über den Gruppenindex innerhalb derselben Bank selektiert werden.

In der aktuellen Implementierung liefert der DMA pro Takt genau ein Byte (`w_flat_en`, `w_flat_data`) im HWC-Stream: Der Kanalindex läuft am schnellsten, danach folgt x und nach Zeilenende y . Diese Reihenfolge wird direkt durch die Zähler `wr_ch`, `wr_x` und `wr_y` erzeugt und in der Testbench über die ersten Transfers nachvollzogen.

Für das Ablegen in die lane-bankierte SRAM wird der Kanalindex auf Bank und Kanalgruppe abgebildet:

$$\text{bank_idx} = \text{wr_ch} \bmod \text{LANES_MAX}, \quad \text{group} = \left\lfloor \frac{\text{wr_ch}}{\text{LANES_MAX}} \right\rfloor.$$

Da LANES_MAX als Zweierpotenz ausgelegt ist (z. B. 4 oder 8), werden diese Operationen in Hardware nicht über einen Divider realisiert, sondern als reine Bitoperationen:

$$\text{bank_idx} \equiv \text{wr_ch}[\$clog2(\text{LANES_MAX}) - 1 : 0], \quad \text{group} \equiv \text{wr_ch} \gg \$clog2(\text{LANES_MAX}).$$

Die Pixeladresse innerhalb eines Bildes ergibt sich in der verwendeten Zeilenadressierung zu:

$$\text{pix_addr}(x, y) = y \cdot \text{IMG_W_MAX} + x.$$

Die effektive Adresse innerhalb der selektierten Bank wird gebildet, indem die Kanalgruppe als Basis vorangestellt wird:

$$\text{addr_bank} = \text{group} \cdot (\text{IMG_W_MAX} \cdot \text{IMG_H_MAX}) + \text{pix_addr}(x, y).$$

Damit speichert jede Lane-Bank mehrere Kanäle in getrennten *Kanalgruppen*-Segmenten hintereinander. Praktisch ist das die entscheidende Eigenschaft für den späteren Read-Pfad: Für ein fixes Pixel (x, y) kann der Fetcher kanalparallel lesen (über alle Banken), während unterschiedliche Kanalgruppen über den group-Offset selektiert werden.

Der Speicher ist als LANES_MAX-fach replizierter 1R1W-RAM realisiert (synchroner Write- und Read-Port) mit einer Read-Latenz von einem Takt. Diese 1-Takt-Latenz ist in der Fetch-Logik berücksichtigt, indem Adress- und Valid-Signale entsprechend gepipelined werden.

Der folgende Pseudocode fasst die DMA-Schreibsequenz in der implementierten Zählerreihenfolge kompakt zusammen:

on each DMA byte when `w_flat_en`:

```
// pixel address (row base + x)
pix_addr = wr_y * IMG_W_MAX + wr_x

// channel mapping (LANES_MAX is power-of-two)
bank_idx = wr_ch % LANES_MAX           // == wr_ch[log2(LANES_MAX)-1:0]
group    = wr_ch / LANES_MAX           // == wr_ch >> log2(LANES_MAX)

addr_bank = group * (IMG_W_MAX*IMG_H_MAX) + pix_addr
```

```

write bank[bank_idx][addr_bank] = w_flat_data

// increment in HWC order: ch -> x -> y
if (wr_ch == cfg_cin-1):
    wr_ch = 0
    if (wr_x == cfg_img_w-1):
        wr_x = 0
        wr_y = (wr_y == cfg_img_h-1) ? 0 : (wr_y + 1)
    else:
        wr_x = wr_x + 1
else:
    wr_ch = wr_ch + 1

```

Padding. Padding wird in der Feature-SRAM nicht als separater Speicherbereich abgelegt, sondern direkt in den *Streaming-Ablauf* integriert. Die Randbreiten werden über die CSR-Felder `cfg_pad_left`, `cfg_pad_right`, `cfg_pad_top` und `cfg_pad_bottom` konfiguriert (vgl. Tabelle 4.1, CSR_CFG_PAD bei 0x7C). Beim start-Ereignis werden diese Werte einmalig in Laufzeitregister übernommen (`run_pad_left`, `run_pad_right`, `run_pad_top`, `run_pad_bottom`), damit der Stream während eines Frames mit stabilen Parametern arbeitet.

Ausgehend von der Originalbildgröße `cfg_img_w/cfg_img_h` werden anschließend die effektiven Stream-Abmessungen (inkl. Padding) berechnet und als `run_w` bzw. `run_h` gespeichert:

$$\text{run_w} = \text{cfg_img_w} + \text{cfg_pad_left} + \text{cfg_pad_right}, \quad (4.27)$$

$$\text{run_h} = \text{cfg_img_h} + \text{cfg_pad_top} + \text{cfg_pad_bottom}. \quad (4.28)$$

Damit traversiert `S_STREAM` nicht nur den Originalbereich, sondern das komplette gepaddete Raster:

$$(x, y) \in [0, \text{run_w} - 1] \times [0, \text{run_h} - 1].$$

Der Vorteil dieser Wahl ist praktisch: Die nachfolgenden Module sehen immer ein konsistentes, rechteckiges Raster (inkl. Rand), ohne dass zusätzliche Spezialfälle außerhalb des Fetchers benötigt werden.

Pro Stream-Koordinate wird zuerst entschieden, ob das aktuelle (x, y) im Originalbild liegt oder im Padding-Bereich. Dafür werden die beiden Bedingungen verwendet:

$$\text{in_img_x} : x \geq \text{cfg_pad_left} \wedge x < \text{cfg_pad_left} + \text{cfg_img_w}, \quad (4.29)$$

$$\text{in_img_y} : y \geq \text{cfg_pad_top} \wedge y < \text{cfg_pad_top} + \text{cfg_img_h}, \quad (4.30)$$

wobei $\text{in_img_xy} = \text{in_img_x} \wedge \text{in_img_y}$ die kombinierte Entscheidung ist.

Falls $\text{in_img_xy} = 1$, wird die Speicheradresse durch Rückabbildung auf die Originalkoordinaten gebildet. Die Offset-Korrektur entspricht genau der „Verschiebung“ durch das Padding:

$$\text{mem_x} = x - \text{cfg_pad_left}, \quad (4.31)$$

$$\text{mem_y} = y - \text{cfg_pad_top}, \quad (4.32)$$

und daraus folgt die linearisierte Pixeladresse:

$$\text{mem_pix_addr} = \text{mem_y} \cdot \text{IMG_W_MAX} + \text{mem_x}.$$

Mit dieser Adresse werden dann die Lane-Bänke wie gewohnt gelesen.

Liegt (x, y) dagegen im Padding-Bereich ($\text{in_img_xy} = 0$), werden die SRAM-Daten bewusst *nicht* als gültig verwendet. Stattdessen werden alle Ausgabekanäle deterministisch auf Null gesetzt ($\text{px_out_lane}[:] = 0$). Damit ist das Padding vollständig im Stream enthalten und bleibt für den restlichen Datenpfad transparent: Es gibt weder „halbgültige“ Fenster noch undefinierte Randzugriffe, sondern einen eindeutig definierten Null-Rand.

Streaming Das Streaming-Lesen wird durch den in Abbildung 4.11 gezeigten Zustandsautomaten umgesetzt.

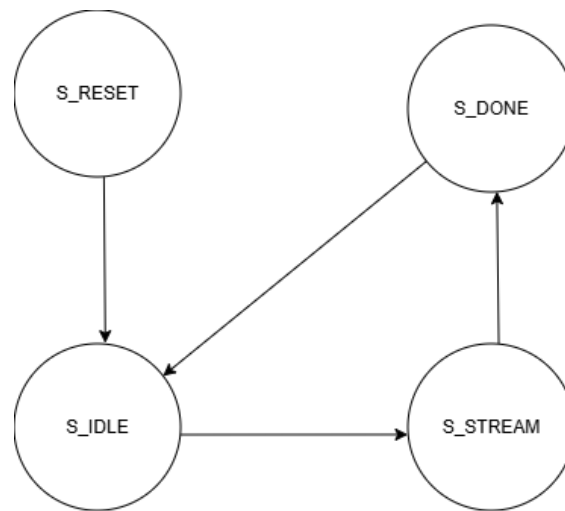


Abbildung 4.11: Zustandsautomat der Feature-I/O-SRAM für das Streaming-Lesen: S_IDLE wartet auf start. In S_STREAM werden die (gepaddingten) Bildkoordinaten (x, y) traversiert und Read-Anforderungen an die synchronen Kanalbänke gestellt; die gelesenen Werte stehen aufgrund der 1-Takt-Read-Latenz im Folgetakt zur Verfügung und werden als lane-paralleles Pixelwort px_out_lane ausgegeben. Die Übernahme durch den nachfolgenden Pfad wird über das Handshake $\text{px_valid}/\text{px_ready}$ gesteuert. In S_DONE wird done_frame gesetzt und anschließend nach S_IDLE zurückgekehrt.

Der Ablauf ist so gewählt, dass zwei Punkte sauber zusammenpassen: Erstens besitzt jede Bank eine synchrone Read-Latenz von einem Takt, das heißt eine Read-Anforderung liefert die Daten erst im Folgetakt. Zweitens soll der Pixelstrom gegenüber dem nachfolgenden Pfad entkoppelt bleiben. Genau dafür wird das Handshake $\text{px_valid}/\text{px_ready}$ verwendet. Die I/O-SRAM ist dabei die Quelle: Sie setzt px_valid , sobald ein gültiges px_out_lane anliegt; der Fetch-Pfad quittiert über px_ready , ob dieses Wort in diesem Takt übernommen werden kann. Nur bei $\text{px_valid} = 1 \wedge \text{px_ready} = 1$ wird das Ausgangswort tatsächlich konsumiert und der Stream fortgeschaltet.

- **S_RESET:** Nach einem asynchronen Reset verbleibt der Automat in S_RESET. In diesem Zustand werden interne Register und Statussignale auf definierte Startwerte gesetzt. Sobald $\text{reset}=0$ ist, erfolgt der Übergang nach S_IDLE.
- **S_IDLE:** Im Leerlauf werden keine gültigen Daten ausgegeben ($\text{px_valid}=0$). Zusätzlich ist $\text{pending}=0$, und die Koordinatenzähler (x, y) stehen auf Null. Mit $\text{start}=1$ wird ein

neuer Streaming-Durchlauf initialisiert und der Automat wechselt nach S_STREAM. Dasselbe start-Ereignis stößt im nachfolgenden `fetching_module` den Übergang nach S_SCAN an.

- **S_STREAM (Non-Pointwise-Modus):** In diesem Modus liefert die Feature-SRAM für jede Position (x, y) des gepaddeten Rasters genau *ein* lane-paralleles Pixelwort `px_out_lane`. Die Übergabe erfolgt ausschließlich über das Handshake. Zur Abkürzung wird das Übernahmeereignis als

$$\text{out_accept} = \text{px_valid} \wedge \text{px_ready}$$

bezeichnet.

Das Auslösen neuer Bank-Reads wird über `issue_read` gesteuert. Der Kern ist: Ein Read wird nur dann gestartet, wenn keine offene Read-Anforderung mehr aussteht (`pending=0`) und gleichzeitig das Ausgangsregister entweder leer ist (`px_valid=0`, zu Beginn) oder das zuletzt ausgegebene Wort gerade in diesem Takt übernommen wurde (`out_accept=1`). Damit ergibt sich ein einfaches, robustes Verhalten: Bei `px_ready=1` kann praktisch in jedem Takt ein neues Wort vorbereitet werden; bei `px_ready=0` bleibt das aktuell gültige Wort stabil stehen und es werden keine weiteren Reads „nachgeschoben“.

Die 1-Takt-Read-Latenz der Banken wird über `pending` modelliert. Beim Auslösen einer Anforderung (`issue_read=1`) wird `pending ← 1` gesetzt und die zugehörigen Lane-/Bank-Zuordnungen werden zwischengespeichert. Im Folgetakt (solange `pending=1`) liegen die Bankdaten an `bank_rdata[*]` an und werden zu einem Zwischenwort zusammengefügt. Sobald das Ausgangsregister frei ist ($\neg \text{px_valid}$ oder `out_accept`), wird `px_out_lane` aktualisiert, `px_valid` gesetzt und `pending` wieder gelöscht.

Die Rasterkoordinaten werden in diesem Modus bereits beim Auslösen der Read-Anforderung fortgeschaltet: x zählt hoch; am Zeilenende wird $x \leftarrow 0$ und $y \leftarrow y + 1$ gesetzt. Dadurch ist die nächste Read-Adresse bereits vorbereitet, während die aktuelle Read im Folgetakt ausgegeben wird. Das Frame-Ende (`done_frame`) wird erst nach dem `out_accept` des letzten ausgegebenen Wortes signalisiert, sodass das letzte Pixel garantiert bestätigt wurde. Bei `px_ready=0` bleibt `px_valid=1` stabil und es werden keine weiteren Reads ausgelöst, bis das Wort übernommen wurde.

- **S_STREAM (Pointwise-Modus):** Im Pointwise-Betrieb werden die Kanäle eines Pixels in *mehreren Beats* ausgegeben. Pro Beat entsteht genau ein lane-paralleles Wort `px_out_lane`. Wie viele Beats pro Pixel nötig sind, wird über `cfg_needed_beats_pw` vorgegeben.

Auch hier wird wegen der synchronen Read-Latenz nur dann ein neuer Read gestartet, wenn `pending=0` ist. Anders als im Non-Pointwise-Modus wird `issue_read` jedoch bewusst strikter getaktet: Es wird nur dann ein Read ausgelöst, wenn aktuell *kein* gültiges Ausgangswort anliegt (`px_valid=0`). Damit wird verhindert, dass ein Beat „vorausgelesen“ wird, während der vorherige Beat noch nicht übernommen wurde. Das Verhalten bei Backpressure ist dadurch eindeutig: Wenn `px_ready=0`, steht `px_valid=1` stabil und der gesamte Beat-Stream hält an, bis das Wort bestätigt ist.

Die `pending`-Behandlung bleibt identisch: Beim Auslösen der Read-Anforderung wird `pending ← 1` gesetzt und die Zuordnungen werden zwischengespeichert; im Folgetakt werden `bank_rdata[*]` zu `px_out_lane` zusammengesetzt, `px_valid` wird gesetzt und `pending` wieder gelöscht.

Für den Beat-Ablauf existiert der Zähler `pw_beat_idx`. Pro Beat wird die effektive Kanalbasis um `LANES_MAX` verschoben, sodass in Beat b die Kanalgruppe ab `cfg_ch_base + b ·`

LANES_MAX gelesen und lane-parallel ausgegeben wird. Wichtig ist hier die Koordinatenfortschaltung: Im Pointwise-Modus wird (x, y) *nicht* bei `issue_read` weitergezählt, sondern erst dann, wenn ein Beat wirklich übernommen wurde (`out_accept=1`) *und* dieser Beat zugleich der letzte Beat der aktuellen Pixelposition ist. Konkret: Nach jedem `out_accept` wird `pw_beat_idx` erhöht. Sobald der letzte Beat bestätigt wurde, wird `pw_beat_idx` auf 0 zurückgesetzt und anschließend (x, y) zur nächsten Pixelposition fortgeschaltet. Dadurch ist garantiert, dass ein Pixel erst dann „abgeschlossen“ ist, wenn alle seine Beats übertragen wurden.

Analog dazu wird `done_frame` erst dann erzeugt, wenn für die letzte Rasterposition *und* den letzten Beat das Ausgangswort per `out_accept` bestätigt wurde. Bei `px_ready=0` bleibt `px_valid=1` stabil und weder Beat- noch Koordinatenfortschaltung erfolgt, bis das aktuelle Wort übernommen wurde.

- **S_DONE:** Nachdem das letzte Pixel des Frames ausgegeben und per Handshake bestätigt wurde, setzt das Modul `done_frame` und kehrt anschließend nach `S_IDLE` zurück.

Blockweises Streaming über Lane-Arrays. Da pro Takt nur LANES_MAX Werte ausgegeben werden können, wird der Kanalraum im Non-Pointwise-Betrieb in Blöcke zerlegt und als lane-paralleles Pixelwort `px_out_lane` gestreamt. Die Blockbreite wird durch `cfg_tile_r` festgelegt; der Startkanal des aktuellen Blocks wird über `cfg_ch_base` vorgegeben. Die Auswahl und Fortschaltung von `cfg_ch_base` übernimmt der übergeordnete Fetching-Controller (vgl. `S_SCAN` in Unterabschnitt 4.5.3).

Für eine feste Pixelposition (x, y) werden damit die Kanäle `cfg_ch_base` bis `cfg_ch_base + cfg_tile_r - 1` adressiert und auf die ersten Lanes abgebildet: `px_out_lane[0]` enthält den Wert von Kanal `cfg_ch_base`, `px_out_lane[1]` den Wert von `cfg_ch_base+1` usw. Die Kanal-zu-Bank-Zuordnung und die Gruppenadressierung innerhalb der lane-bankierten SRAM entsprechen dabei exakt dem Layout aus Unterabschnitt 4.4.1. Da die Bank-RAMs synchron lesen, werden beim Auslösen einer Read-Anforderung die relevanten Indizes (Bank/Group/Lane) intern zwischengespeichert, sodass im Folgetakt die passenden `bank_rdata`-Werte pro Lane selektiert werden können.

Lane-Klemmung und Nullauffüllung. Zum start-Zeitpunkt wird `cfg_tile_r` auf den zulässigen Bereich begrenzt, das heißt $\text{cfg_tile_r} \leq \text{LANES_MAX}$. Zusätzlich werden Lanes nur dann als aktiv betrachtet, wenn der zugehörige Kanalindex innerhalb `cfg_cin` liegt. Ist ein Block am Kanalende unvollständig (z. B. $\text{cfg_cin} < \text{cfg_tile_r}$ oder `cfg_ch_base` nahe `cfg_cin`), werden die verbleibenden Lane-Positionen deterministisch auf Null gesetzt. Dadurch werden keine alten oder undefinierten Werte weitergereicht, und der nachfolgende Pfad bleibt deterministisch.

Beispiel: Konvolution. Für eine frühe Konvolutionsschicht mit $\text{cfg_cin} = 3$ und $\text{cfg_tile_r} = 4$ ergibt sich bei `cfg_ch_base = 0` ein unvollständiger Block: Die ersten drei Lanes enthalten die drei Kanäle, die vierte Lane wird auf Null gesetzt:

$$(\text{px_out_lane}[0..3]) = (\text{ch}_0, \text{ch}_1, \text{ch}_2, 0).$$

Im Fetching-/Arithmetic-Pfad wird dabei nur die tatsächlich aktive Kanalzahl verwendet; Null-Lanes werden weder gezählt noch akkumuliert.

Beispiel: Depthwise-Konvolution. Im Depthwise-Fall (z. B. $\text{cfg_cin} = 32$) wird der Kanalraum in Blöcke der Größe cfg_tile_r partitioniert. Für $\text{cfg_tile_r} = 4$ entstehen $32/4 = 8$ Blöcke. Für Block $b \in \{0, \dots, 7\}$ setzt der Controller $\text{cfg_ch_base} = 4 \cdot b$. Die I/O-SRAM streamt dann pro Pixelposition die vier Kanäle dieses Blocks parallel:

$$(\text{px_out_lane}[0..3]) = (\text{ch}_{4b}, \text{ch}_{4b+1}, \text{ch}_{4b+2}, \text{ch}_{4b+3}).$$

Nach Abschluss der Verarbeitung für Block b wird cfg_ch_base auf den nächsten Block erhöht und der Streaming-Durchlauf erneut gestartet. Daraus ergibt sich der erwartete Ablauf „scan-compute-next-block“, ohne dass mehr als LANES_MAX Werte pro Takt transportiert werden müssen.

OFMAP-Rückschreiben. Neben dem DMA-Write unterstützt die I/O-SRAM ein optionales Rückschreiben von Ergebnissen über we_ofmap mit expliziten Koordinaten (w_x, w_y) und Kanal w_ch . Die Ablage verwendet dieselbe flache Adressbildung wie beim Lesen, das heißt die räumliche Adresse wird aus (w_x, w_y) gebildet und der Kanal wird über das bekannte Banking-/Grouping-Schema auf die Lane-Bänke abgebildet. Damit bleibt die räumliche Interpretation der Daten über Einlesen, Verarbeitung und Rückschreiben konsistent.

4.4.2 Weight-SRAM

Die Weight-SRAM speichert quantisierte Faltungsgewichte als linearen Byte-Stream, der *softwareseitig* in der Zielordnung $[c][h][w][k]$ erzeugt wird (vgl. Abbildung 4.12). Dabei ist der Output-Kanalindex k die schnellste Dimension, sodass für ein festes Kontext-Tupel (c, h, w) die Werte $w_{hw,c}^0, w_{hw,c}^1, \dots$ im Speicher unmittelbar aufeinander folgen. Dieses Layout passt direkt zur spaltenparallelen Verarbeitung im Systolic Array: Der Fetcher kann so für einen Kontext die Gewichte mehrerer Spalten als zusammenhängenden Block lesen.

Die Reorganisation erfolgt im Software-Tool durch ein Flattening in genau dieser Reihenfolge:

```
flat = []
for c in 0 .. cfg_cin-1:
    for h in 0 .. cfg_kh-1:
        for w in 0 .. cfg_kw-1:
            for k in 0 .. cfg_cout-1:
                flat.append( W[k][c][h][w] )    // int8
```

Der DMA schreibt anschließend byteadressiert in die Weight-SRAM (vgl. Unterabschnitt 4.2.3). Das `fetching_module` liest ebenfalls byteadressiert ab einer Startadresse `sram_raddr` und nutzt pro Zugriff die ersten `cfg_tile_c` Bytes als aktuelles Output-Kanal-Tile (bis maximal `cfg_cout=COLS_MAX`). Der Read-Pfad ist synchron mit einer Latenz von einem Takt: Im Takt des Read-Issue wird die Adresse angelegt, im Folgetakt sind die Daten gültig und werden über `sram_rvalid` signalisiert; dadurch lässt sich der Weight-Fetch taktsynchron mit dem Fenster-/Kontext-Stream im `fetching_module` koppeln.

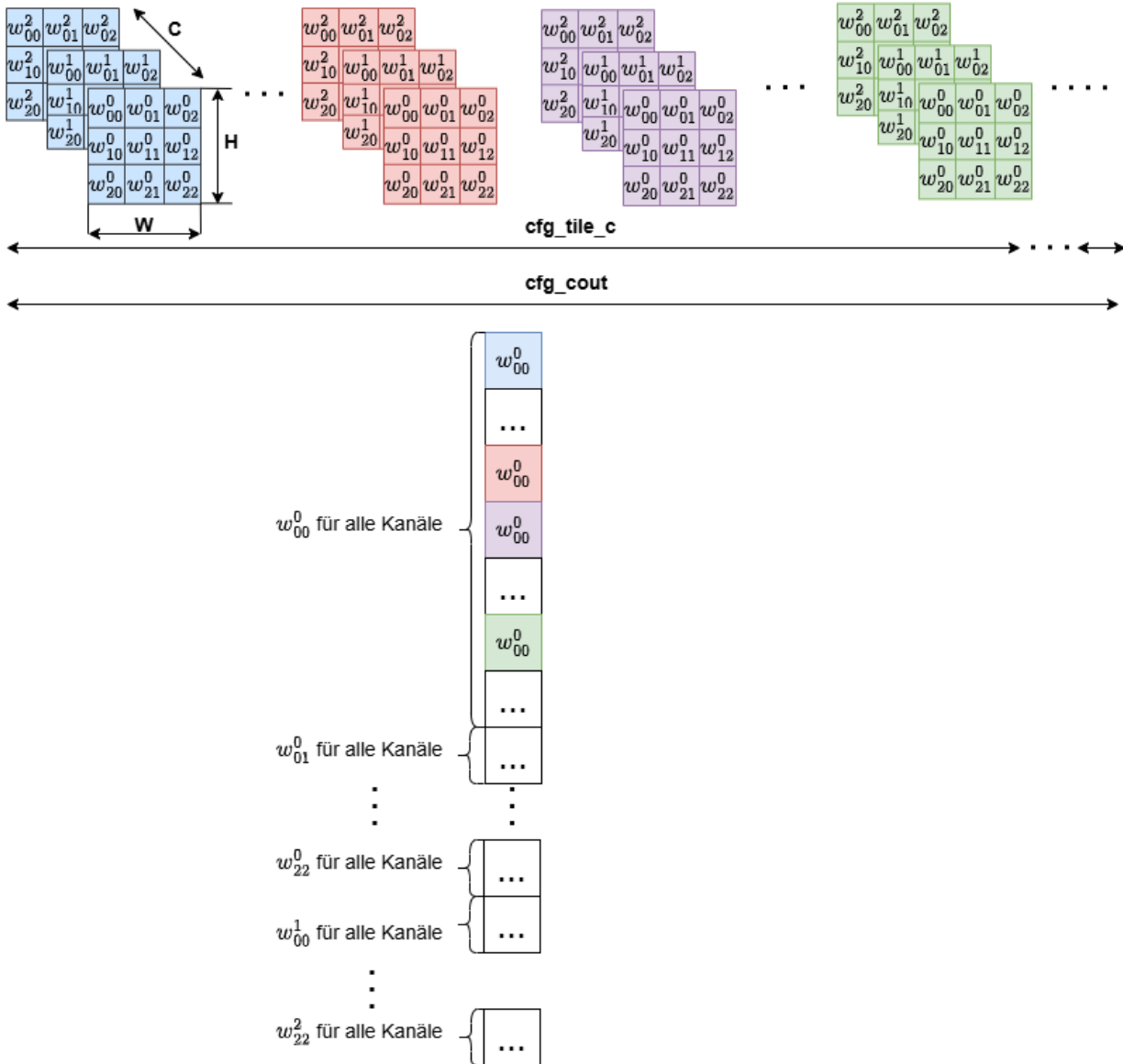


Abbildung 4.12: Softwareseitig erzeugtes Weight-Layout in der Weight-SRAM (Zielordnung $[c][h][w][k]$ mit k als schnellster Dimension): Für ein fixes (c, h, w) liegen die Gewichte über die Output-Kanäle k direkt hintereinander. Pro Fetch werden cfg_tile_c aufeinanderfolgende Output-Kanal-Gewichte als Tile genutzt.

4.4.3 Bias/Shift/Mult-SRAM

Die Bias/Shift/Mult-SRAM speichert die `int32`-Biaswerte sowie die zugehörigen Requantisierungs-Shiftwerte pro Output-Kanal. Analog zur Weight-SRAM werden beide Datenströme *softwareseitig* in ein für den Hardwarezugriff geeignetes Format gebracht und anschließend durch den DMA *byteadressiert* in die jeweiligen Speicher geschrieben.

Der DMA überträgt pro Takt genau ein Byte. Bias, Shift und Multiplier werden dabei als fortlaufender Byte-Stream abgelegt, wobei jeweils ein 32-bit Wert aus vier aufeinanderfolgenden Bytes besteht (*little-endian*). Die Byteadresse entspricht somit direkt dem Index im erzeugten Stream. Intern werden die Bytes zu einem breiten Speicherwort zusammengefasst, sodass ein

späterer Read-Zugriff spaltenparallel erfolgen kann (vgl. Unterabschnitt 4.2.3). Die Implementierung verwendet dazu drei getrennte RAM-Instanzen: eine RAM für Bias, eine RAM für Shift und eine RAM für Multiplier, jeweils mit 1R1W-Port und synchroner 1-Takt-Read-Latenz.

Für den spaltenparallelen Betrieb des Systolic Arrays werden Bias, Shift und Multiplier nicht kanalweise einzeln gelesen, sondern *blockweise*: Ein Block entspricht einem Output-Kanal-Tile der Breite `cfg_tile_c`. Der Fetcher adressiert die Bias/Shift/Mult-SRAM deshalb über den Blockindex `blk_idx`; pro Lesezugriff liefert der Speicher im Folgetakt (synchrone 1-Takt-Latenz) genau die `cfg_tile_c` Werte für Bias, Shift und Multiplier, passend zu den gleichzeitig gefetchten `cfg_tile_c` Gewichten. Dadurch sind Bias/Shift/Multiplier zeitlich und strukturell direkt an das Weight-Tiling gekoppelt:

$$\text{blk_idx} \Rightarrow \{\text{Bias}_0, \dots, \text{Bias}_{\text{cfg_tile_c}-1}\} \text{ und } \{\text{Shift}_0, \dots, \text{Shift}_{\text{cfg_tile_c}-1}\} \\ \text{und } \{\text{Mult}_0, \dots, \text{Mult}_{\text{cfg_tile_c}-1}\}.$$

Damit erhält jede Spalte des Arrays pro Tile exakt die zugehörigen Bias-, Shift- und Multiplier-Werte, ohne zusätzliche Umformatierung im Hardwarepfad.

4.5 Fetching Module

4.5.1 Fetching Module: Überblick und Einordnung

Das `fetching_module` bildet den Datenbereitstellungspfad des Beschleunigers. Es steht zwischen den lokalen Speichern (IO-SRAM als Quelle des Pixelstreams, Weight-SRAM sowie Bias/Shift/Mult-SRAM) und dem Rechenkern des Arithmetikmoduls (vgl. Unterabschnitt 4.3.1). Seine Aufgabe besteht darin, die benötigten Operanden *taktgenau* bereitzustellen: Eingabeaktivierungen (aus dem Feature-/IO-SRAM), die zugehörigen Gewichte (aus dem Weight-SRAM) sowie Bias- und Requantisierungsparameter (aus dem Bias/Shift/Mult-SRAM).

Im Sinne des Architekturgrundsatzes „*ein Array pro Takt*“ liefert das Fetching-Modul dem Systolic Array in jedem Zyklus ein Aktivierungs-Array `ifmap_array` (`cfg_tile_r` Lanes) und ein Gewichts-Array `weight_array` (`cfg_tile_c` Spalten). Damit wird sichergestellt, dass die Operanden in der MAC-Pipeline zeitlich ausgerichtet ankommen und die Verarbeitung ohne zusätzliche Stalls erfolgen kann.

Funktionale Teilaufgaben. Der Ablauf lässt sich in drei Schritte einteilen: (i) Erzeugen und Zwischenspeichern von Fenstern aus dem Pixelstream, (ii) blockweises Vorladen von Bias/Shift/Mult sowie kontextweises Anfordern der Gewichte und (iii) Ausgabe eines taktgenauen Operandenstroms (Aktivierungen/Gewichte) an das Systolic Array.

4.5.2 Line Buffer und Fenstersammlung

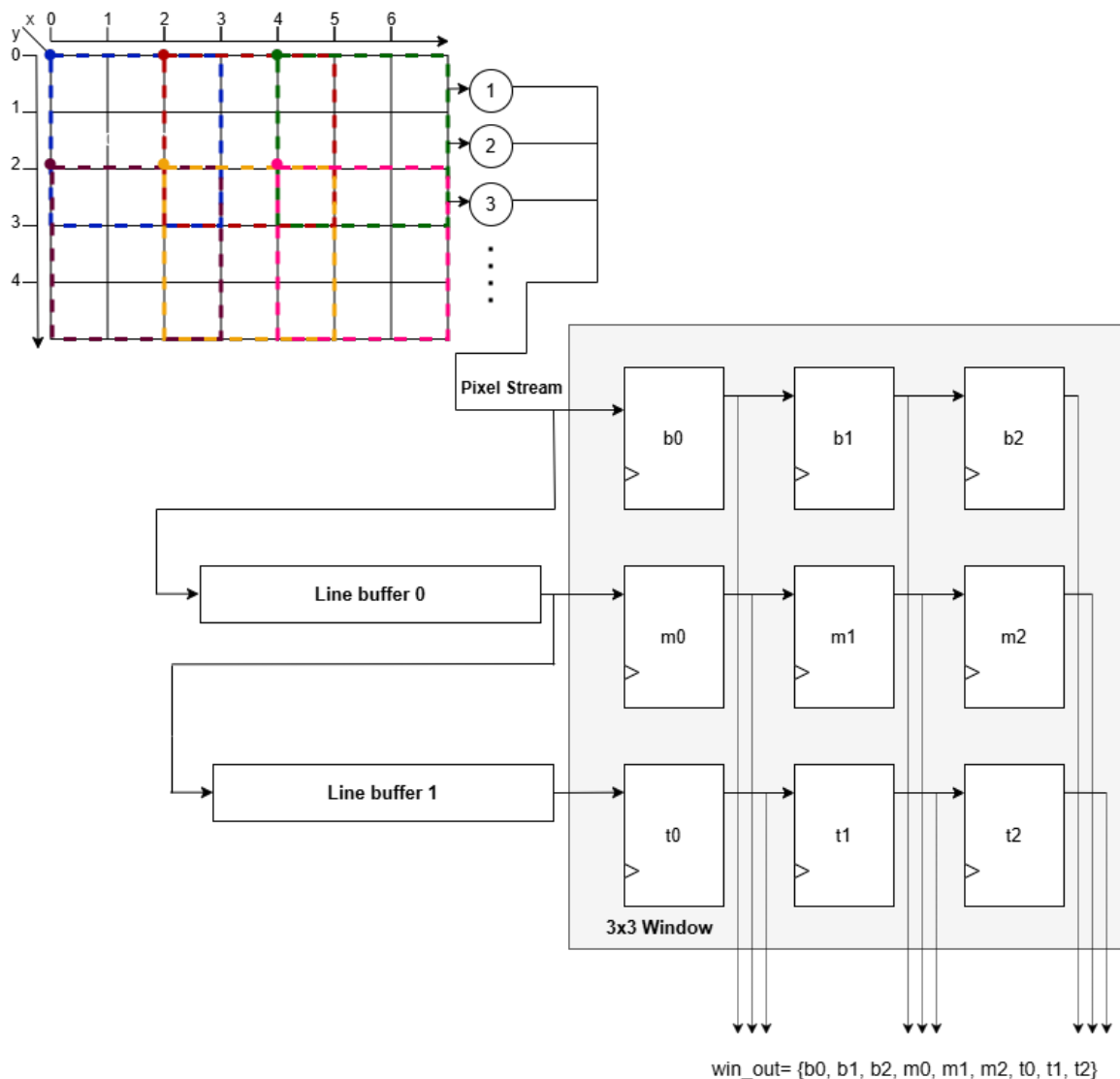


Abbildung 4.13: Prinzip der Line-Buffer-Pipeline zur 3×3 -Fensterbildung aus einem Pixelstream: zwei Zeilenpuffer halten vorherige Bildzeilen, während Schieberegister die letzten Spaltenwerte der drei beteiligten Zeilen speichern [3].

Die Aufgabe der Line-Buffer-Struktur besteht darin, aus einem seriell eintreffenden Pixelstream lokale 3×3 -Fenster zu rekonstruieren, ohne das gesamte Bild zwischenspeichern. Dieses Vorgehen ist typisch für streambasierte Bildverarbeitung auf SRAM-basierten FPGAs und bildet eine zentrale Grundlage für lokale Operatoren wie die Faltung (vgl. [3]). In der vorliegenden Architektur wird die Fensterbildung im `fetching_module` genutzt, um Faltungsfenster deterministisch zu identifizieren und anschließend in einem lokalen *Window-SRAM* zu sammeln.

Koordinatenbezug. Der Pixelstream wird zeilenweise geliefert. Im Folgenden werden die Bildkoordinaten konsequent als (x, y) beschrieben, wobei x die Spalte und y die Zeile bezeichnet.

Streaming im Line Buffer. Wie in Abbildung 4.13 dargestellt, wird der Pixelstrom zeilenweise in die Pipeline eingespeist. Zwei Zeilenpuffer speichern die vorherigen Bildzeilen (`linebuf0`, `linebuf1`). Zusätzlich halten Schieberegister die letzten drei Pixelwerte der drei beteiligten Zeilen. Damit kann nach jedem gültigen Pixel-Takt ein vollständiger 3×3 -Vektor $\{b0, b1, b2, m0, m1, m2, t0, t1, t2\}$ gebildet werden.

Wann wird ein Fenster gültig? Ein 3×3 -Fenster ist genau dann vollständig, wenn das aktuell eintreffende Pixel die *rechte untere Ecke* des Fensters bildet. Für `WIN_SIZE = 3` bedeutet dies:

$$y \geq 2 \quad \wedge \quad x \geq 2.$$

Zusätzlich sollen Fenster nicht an jeder Pixelposition ausgegeben werden, sondern nur mit der gewünschten Schrittweite *Stride*. Dafür verwendet das Modul die beiden Phasenzähler `x_phase` und `y_phase` als einfache *Stride-Takte*: `x_phase` zählt horizontal zwischen zwei Fenstern hoch und wird nach *Stride* Pixeln wieder auf 0 gesetzt; `y_phase` macht dasselbe vertikal, wird jedoch nur beim Zeilenwechsel aktualisiert. Ein Fenster wird daher nur dann als gültig markiert, wenn es sowohl horizontal als auch vertikal genau auf einer *Stride*-Position liegt, das heißt:

$$(\text{x_phase} = 0) \wedge (\text{y_phase} = 0).$$

Damit wird die *Stride*-Bedingung umgesetzt, ohne zur Laufzeit Modulo-Operationen zu benötigen.

Fenstersammlung im Fetching-Modul (SCAN-Zustand). Während die Pixel aus dem Feature-/IO-SRAM gestreamt werden, befindet sich das Fetching-Modul im Zustand `S_SCAN`. Jedes Mal, wenn der Line-Buffer ein gültiges Fenster signalisiert (`win_valid`), wird dieses Fenster zunächst in ein internes Window-RAM geschrieben.

Pseudocode zur Phasenlogik. Der folgende Pseudocode fasst die Phasensteuerung zusammen:

```
for each incoming pixel at (x,y):
```

```
    // Fenster ist vollstaendig erst ab (x>=2,y>=2)
    if (y >= WIN_SIZE-1) and (x >= WIN_SIZE-1):
        if (x_phase == 0) and (y_phase == 0):
            win_valid = 1
        else:
            win_valid = 0

    // x_phase: horizontal zaehlen
    if (x is end_of_row):
        x_phase = 0
    else if (x <= WIN_SIZE-1):
        x_phase = 0
    else:
        x_phase = (x_phase == stride-1) ? 0 : (x_phase + 1)

    // y_phase: nur beim Zeilenwechsel aktualisieren
    if (x is end_of_row):
```

```

if (y <= WIN_SIZE-1):
    y_phase = 0
else:
    y_phase = (y_phase == stride-1) ? 0 : (y_phase + 1)

```

4.5.3 Fetching-Controller

Der `fetching_module`-Controller steuert den vollständigen Datenstrom zwischen den On-Chip-Speichern (IFMAP-Stream/Window-RAM, Weight-SRAM sowie Bias/Shift/Mult-SRAM) und dem Rechenkern. Dazu organisiert er (i) das zeilenweise Sammeln der Sliding-Window-Positionen, (ii) die Gruppierung der Fenster in `cfg_tile_r`-große Gruppen (Lane-Tiling) und (iii) die Blockbildung der Output-Kanäle in `cfg_tile_c`-große Blöcke (Spalten-Tiling). Die hierfür benötigten Grenzen und Schleifenlängen werden vollständig über CSR-Parameter vorgegeben. Die abgeleiteten Größen $\{cfg_out_h, cfg_out_w, total_wins, total_grps, total_blks, num_ctx\}$ aus Unterunterabschnitt 4.2.2 werden direkt als CSR-Felder bereitgestellt. Damit kann der Controller seine Zählergrenzen ohne Laufzeitdivisionen festlegen und die Ausgabe exakt über `total_wins` Fensterpositionen, `total_grps` Gruppen, `total_blks` Blöcke sowie `num_ctx` Kontextelemente pro Fenster takt synchron abarbeiten.

Beispiel (Layer 0, 128×128). Für $cfg_in_h = cfg_in_w = 128$, $cfg_cin = 3$, $cfg_cout = 32$, $cfg_kh = cfg_kw = 3$ und $cfg_sh = cfg_sw = 2$ sowie $cfg_pad_left = 0$, $cfg_pad_right = 1$, $cfg_pad_top = 0$, $cfg_pad_bottom = 1$ folgt aus Gleichung 4.1 und Gleichung 4.2:

$$\begin{aligned}
 cfg_out_h &= \left\lfloor \frac{128 + 0 + 1 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{126}{2} \right\rfloor + 1 = 63 + 1 = 64, \\
 cfg_out_w &= \left\lfloor \frac{128 + 0 + 1 - 3}{2} \right\rfloor + 1 = 64.
 \end{aligned}$$

Damit ergibt sich nach Gleichung 4.3 direkt:

$$total_wins = 64 \cdot 64 = 4096.$$

Für $cfg_tile_r = 4$ gilt nach Gleichung 4.4:

$$total_grps = \left\lceil \frac{4096}{4} \right\rceil = 1024.$$

Für $cfg_cout = 32$ und $cfg_tile_c = 4$ ergibt sich nach Gleichung 4.5:

$$total_blks = \left\lceil \frac{32}{4} \right\rceil = 8.$$

Die Kontextlänge ist bei normaler Faltung nach Gleichung 4.6:

$$num_ctx = 3 \cdot 3 \cdot 3 = 27.$$

Im Depthwise-Fall (bei gleicher Kernelgröße) gilt nach Gleichung 4.7:

$$num_ctx = 3 \cdot 3 = 9.$$

Im Pointwise-Modus (1×1) ergibt sich nach Gleichung 4.8:

$$num_ctx = cfg_cin = 3,$$

und die benötigten Beats nach Gleichung 4.9 bei $LANES_MAX = 4$ zu:

$$cfg_needed_beats_pw = \left\lceil \frac{3}{4} \right\rceil = 1.$$

Zustände des Fetching-Controllers. Der Fetching-Controller ist als endlicher Zustandsautomat implementiert und steuert die zeitliche Abfolge des gesamten Fetch-Ablaufs innerhalb eines Frames (vgl. Abbildung 4.14).

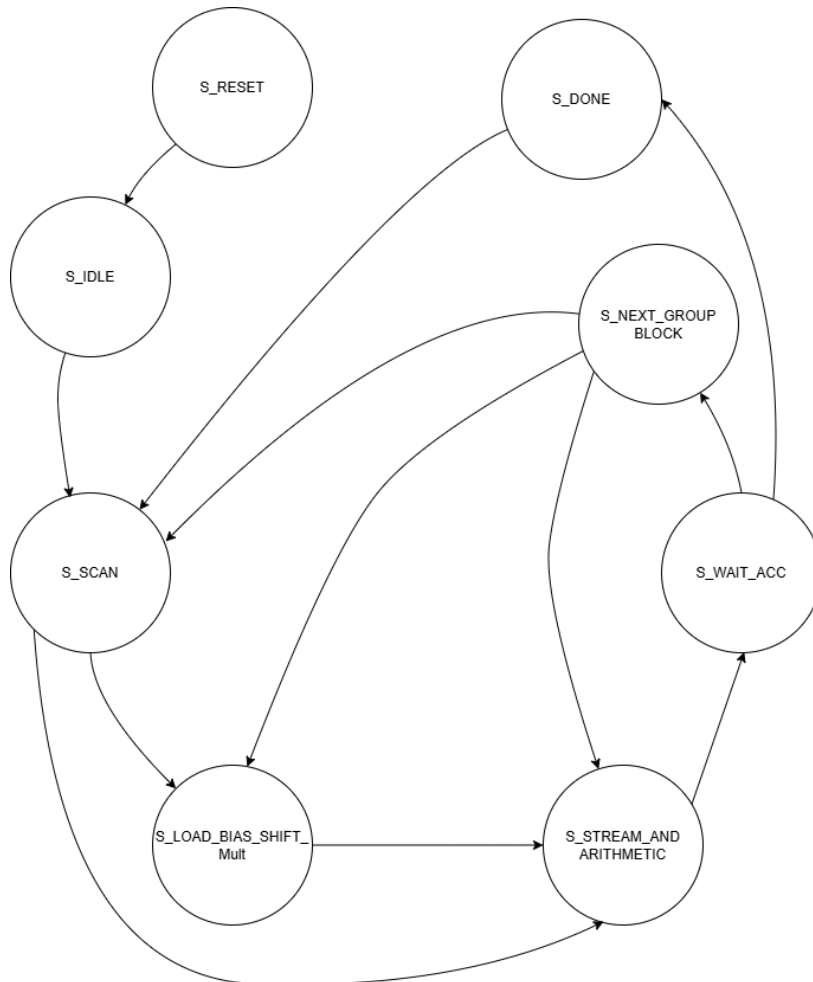


Abbildung 4.14: Zustandsautomat des `fetching_module`. Der Controller trennt die Erzeugung der Fenster (Scan-Phase) von der nachfolgenden, takt synchronen Streaming-Phase, in der pro Kontext `cfg_tile_r` Aktivierungen und `cfg_tile_c` Gewichte (inkl. Bias/Shift/Mult/Mult pro Block) bereitgestellt werden.

Im Folgenden werden die Zustände beschrieben.

S_RESET: Nach einem Reset befindet sich die FSM in S_RESET. In diesem Zustand werden interne Register, Statusflags und Zählerstände zurückgesetzt.

S_IDLE: Nach Abschluss der Initialisierung wechselt die FSM in den Leerlaufzustand S_IDLE. In S_IDLE ist der Controller bereit, löst jedoch keinerlei Speicherzugriffe oder Datenübertragungen aus. Der gesamte Fetch-Pfad bleibt inaktiv, bis ein Startsignal anliegt. Dieses start-Signal wird vom DMA-Controller erzeugt, sobald die vorausgehende Übertragung (DMA-Transfer der benötigten Feature-/Weight-/Bias/Shift/Mult-Daten in die lokalen SRAMs) abgeschlossen ist. Sobald start=1 wird, verlässt die FSM S_IDLE und wechselt in den Zustand S_SCAN, um die Fensterpositionen des Frames zu erfassen und in das Window-RAM zu schreiben.

S_SCAN: Im Zustand S_SCAN sammelt das `fetching_module` alle gültigen Sliding-Windows eines Frames und legt sie in einem lokalen Window-RAM ab. Der Pixelstrom stammt dabei aus der I/O-SRAM, die während des Scan-Vorgangs im Zustand S_STREAM arbeitet und pro Takt ein kanalparalleles Pixelwort bereitstellt (vgl. Unterabschnitt 4.4.1 sowie Abbildung 4.11). Das Fetching-Modul nimmt Pixel ausschließlich in S_SCAN an: `px_ready` ist standardmäßig 0 und wird nur in S_SCAN auf 1 gesetzt. Ein gültiger Pixeltransfer kann daher konsistent als:

$$px_get = px_valid \wedge px_ready$$

definiert werden. Damit werden die Line-Buffer nur dann getaktet, wenn die Scan-Phase aktiv ist.

Abbildung 4.15 illustriert die Scan-Vorgehensweise an einem Beispiel. Konzeptionell gilt: Die Anzahl der parallel verarbeiteten Kanäle entspricht `cfg_tile_r`, jedoch ist die Hardware *physisch* durch die feste maximale Lane-Anzahl LANES_MAX begrenzt. In der Implementierung werden daher genau LANES_MAX Line-Buffer-Instanzen und LANES_MAX Window-RAM-Instanzen erzeugt. Der Laufzeitparameter `cfg_tile_r` wird zur Sicherheit auf LANES_MAX geklemmt, es gilt stets:

$$cfg_tile_r \leq LANES_MAX.$$

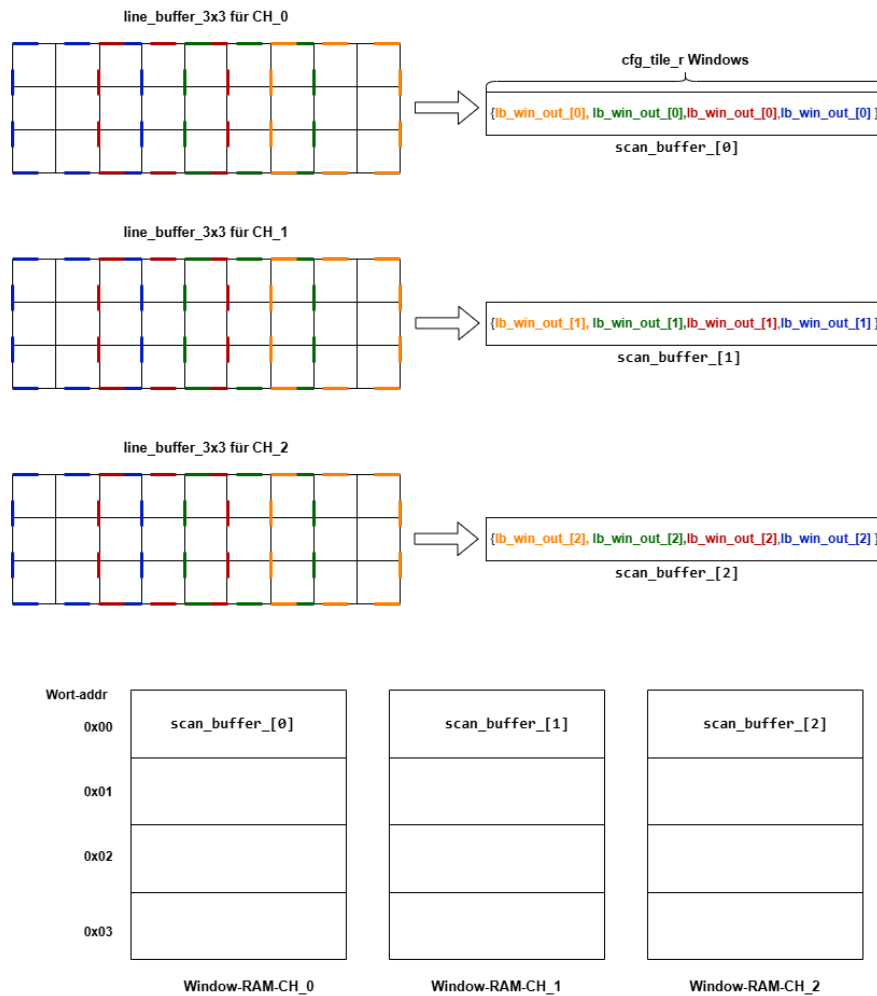


Abbildung 4.15: Scan-Vorgehensweise im Zustand S_SCAN: kanalsynchrone Fensterausgabe über mehrere Line-Buffer, Packen von `cfg_tile_r` Fensterpositionen pro Wort und Schreiben der gepackten Wörter in wortadressierte Window-RAM-Instanzen. Beispiel: `cfg_cin = 3`, `cfg_tile_r = 4`; die vierte Instanz bleibt ungenutzt, da nur `ch < cfg_cin` beschrieben wird und Pack-Puffer vorab deterministisch auf 0 gesetzt werden.

Für den Scan-Vorgang bedeutet das: Pro Takt existieren maximal LANES_MAX parallele Datenpfade, und `cfg_tile_r` bestimmt, wie viele davon in diesem Run als „Lane-Breite“ genutzt werden. Im gezeigten Beispiel ist `cfg_cin = 3` und `cfg_tile_r = 4`; folglich werden vier Line-Buffer-/Window-RAM-Instanzen instanziiert (`ch = 0, 1, 2, 3`), obwohl logisch nur drei Eingangskanäle verwendet werden (`ch = 0, 1, 2`). Die vierte Instanz (`ch = 3`) bleibt ungenutzt, ohne die Funktionalität zu beeinträchtigen: Beim Übernehmen der Fenster werden nur die Instanzen `ch < cfg_cin` beschrieben; nicht benötigte Instanzen behalten Nullwerte, da die zugehörigen Pack-Puffer vor dem Schreiben deterministisch auf 0 gesetzt werden. Dabei bestimmt `cfg_tile_r` die Anzahl der parallel geführten Instanzen, während die tatsächliche Kanalanzahl durch `cfg_cin` begrenzt ist.

Während des Streams erhalten alle Line-Buffer gleichzeitig ihre kanalbezogenen Pixelwerte aus dem kanalparallelen Wort `px_in_lane` (im I/O-SRAM: `px_out_lane`), das von der I/O-SRAM im Streaming-Zustand S_STREAM bereitgestellt wird (vgl. Unterabschnitt 4.4.1 sowie Abbildung 4.11). Sobald ein 3×3 -Fenster an einer Position gültig wird, wird es in allen aktiven Kanälen gleichzeitig

erkannt. Das bedeutet: Wenn im Bild die blauen Fenster in Abbildung 4.15 gültig werden, erzeugt `line_buffer_3x3` für CH_0 das blaue Fenster in `lb_win_out[0]`, für CH_1 analog in `lb_win_out[1]` und für CH_2 in `lb_win_out[2]`. Dasselbe passiert synchron für die roten, grünen und orangefarbenen Fensterpositionen. Die Fensterausgabe ist damit kanalsynchron.

Die übernommenen Fenster werden anschließend in Gruppen der Größe `cfg_tile_r` zu einem Speicherwort gepackt. Dazu existiert pro instanziiertem Kanal ein Pack-Puffer `scan_buffer[ch]` mit $\text{PACKBITS} = \text{cfg_tile_r} \cdot \text{WINBITS}$. Ein interner Zähler `scan_window` läuft von 0 bis `cfg_tile_r - 1` und bestimmt, an welcher Lane-Position das aktuell erkannte Fenster im Pack-Puffer abgelegt wird:

$$\text{scan_buffer}[ch] \left[(\text{scan_window} \cdot \text{WINBITS}) + : \text{WINBITS} \right] \leftarrow \text{lb_win_out}[ch].$$

Dabei beschreibt `WINBITS` die Bitbreite eines einzelnen Fensters. Für ein 3×3 -Fenster (`WIN_SIZE = 3`) und Pixelbreite `PXW = 8` gilt:

$$\text{WINBITS} = \text{WIN_SIZE}^2 \cdot \text{PXW} = 3^2 \cdot 8 = 72 \text{ Bit}.$$

Damit entspricht ein Pack-Wort bei `cfg_tile_r = 4` einer Breite von $\text{PACKBITS} = 4 \cdot 72 = 288$ Bit. Für dieses Beispiel mit `cfg_cin = 3` werden die Fenster pro `scan_window`-Schritt wie folgt in die drei Puffer geschrieben:

$$\begin{aligned} \text{scan_buffer}[0] \left[(\text{scan_window} \cdot 72) + : 72 \right] &\leftarrow \text{lb_win_out}[0], \\ \text{scan_buffer}[1] \left[(\text{scan_window} \cdot 72) + : 72 \right] &\leftarrow \text{lb_win_out}[1], \\ \text{scan_buffer}[2] \left[(\text{scan_window} \cdot 72) + : 72 \right] &\leftarrow \text{lb_win_out}[2]. \end{aligned}$$

Für `cfg_tile_r = 4` entsteht pro Kanalinstanz ein gepacktes Wort aus vier aufeinanderfolgenden Fenstern, die in den vier Lane-Slots (Fenster-Slots) des Pack-Puffers abgelegt werden, also

$$\left\{ \begin{array}{l} \text{lb_win_out}[ch]_{\text{scan_window}=0}, \\ \text{lb_win_out}[ch]_{\text{scan_window}=1}, \\ \text{lb_win_out}[ch]_{\text{scan_window}=2}, \\ \text{lb_win_out}[ch]_{\text{scan_window}=3} \end{array} \right\}.$$

Sobald `scan_window` das Ende erreicht, wird der Inhalt des Pack-Puffers in das zugehörige Window-RAM geschrieben. Anschließend wird der Pack-Puffer für die nächste Gruppe deterministisch zurückgesetzt und die Wortadresse `scan_grp_addr` um eins inkrementiert. Die Window-RAMs sind *wortadressiert*: Eine Adresse `scan_grp_addr = 0, 1, 2, \dots` referenziert jeweils genau ein vollständiges Pack-Wort; dies entspricht einer Gruppe aus `cfg_tile_r` aufeinanderfolgenden Fensterpositionen pro Kanalinstanz. Die maximale Anzahl der zu schreibenden Wörter wird beim `start`-Ereignis als Scan-Limit gelatcht. Im Convolution-Fall entspricht dieses Limit direkt `total_grps`.

Eine Restbelegung des letzten Gruppenworts ist nur dann erforderlich, wenn `total_wins` nicht durch `cfg_tile_r` teilbar ist und am Ende des Frames keine vollständig gefüllte Gruppe mehr entsteht. In diesem Fall enthält das letzte Pack-Wort weniger als `cfg_tile_r` reale Fenster. Die verbleibenden Lane-Slots werden automatisch mit Nullwerten aufgefüllt, weil der Pack-Puffer zu Beginn jeder neuen Gruppe deterministisch auf 0 initialisiert wird und ausschließlich an Positionen

überschrieben wird, für die tatsächlich ein gültiges Fenster erkannt wurde. Beim abschließenden Commit wird somit ein Wort geschrieben, in dem nicht belegte Slots weiterhin 0 enthalten.

Für den Depthwise-Fall bleibt das Grundprinzip der Scan-Phase identisch, jedoch erfolgt die Verarbeitung *blockweise über Kanalgruppen*: Die logischen Kanäle werden in Blöcke der Breite `cfg_tile_r` partitioniert und über eine Basis `blk_ch_base` jeweils auf die instanziierten Line-Buffer-/Window-RAM-Kanäle gemappt. Nach einem Blockwechsel kann daher ein erneuter Scan für den nächsten Kanalblock erforderlich sein. Diese Blockbildung ist auch dann sinnvoll, wenn `cfg_cin` groß ist, weil die Hardware pro Takt durch `LANES_MAX` Lanes begrenzt ist und daher nicht alle Kanäle gleichzeitig als getrennte physische Pfade geführt werden können.

Sobald der Scan abgeschlossen ist, werden keine weiteren Pack-Wörter mehr in das Window-RAM geschrieben. In diesem Moment setzt der Controller die gruppen- und kontextbezogenen Zähler für die anschließende Streaming-Phase deterministisch zurück und wechselt nach `S_LOAD_BIAS_SHIFT_Mult`. Damit ist gewährleistet, dass die nachfolgende Verarbeitung für den aktuellen Output-Block stets bei Gruppe 0 und Kontext 0 startet, während die im Scan erzeugten Fenster bereits vollständig im Window-RAM abgelegt sind.

Ist der Pointwise-Modus aktiviert (`ctrl_pw_en=1`), ist `S_SCAN` *keine* Scan-Phase für Sliding-Windows. In diesem Modus werden weder Line-Buffer noch Window-RAMs verwendet. Stattdessen dient `S_SCAN` als *Einsammelzustand* für die kanalparallelen Eingangsdaten eines festen Pixels (x, y) in mehreren *Beats*.

Da der Datenpfad pro Takt nur `LANES_MAX` Kanäle aufnehmen kann, werden die `cfg_cin` Kanäle in `cfg_needed_beats_pw` Beats partitioniert (vgl. Gleichung 4.9). Während `S_SCAN` hält der Fetcher den Eingabestrom genau so lange offen, bis alle Beats des aktuellen Pixels übernommen wurden. Dazu wird:

$$px_ready = \neg pw_buf_full$$

gesetzt, sodass ein gültiger Transfer wieder als:

$$px_get = px_valid \wedge px_ready$$

definiert werden kann. Bei jedem `px_get` wird das anliegende Lane-Wort `px_in_lane` *explizit* in den Pointwise-Puffer geschrieben:

$$pw_buffer[pw_beat][0..LANES_MAX-1] \leftarrow px_in_lane[0..LANES_MAX-1].$$

Der Beat-Zähler `pw_beat` wird inkrementiert, bis der letzte Beat `pw_beat = cfg_needed_beats_pw - 1` erreicht ist. Dann wird `pw_buf_full` gesetzt und die FSM verzweigt abhängig von `bias_loaded`:

- falls `bias_loaded=0`: nächster Zustand `S_LOAD_BIAS_SHIFT_Mult`,
- sonst: nächster Zustand `S_STREAM_AND_ARITHMETIC`.

Abbildung 4.16 zeigt den Ablauf für `LANES_MAX = 4` und `cfg_cin = 8`. In diesem Fall gilt `cfg_needed_beats_pw = \lceil 8/4 \rceil = 2`: Es werden zwei Beats übernommen, wobei Beat 0 die Kanäle $c = 0..3$ und Beat 1 die Kanäle $c = 4..7$ des *gleichen* Pixels (x, y) in `pw_buffer` ablegt. Erst nach dem zweiten Beat wird in die nachfolgende Pointwise-Streaming-Phase übergegangen.

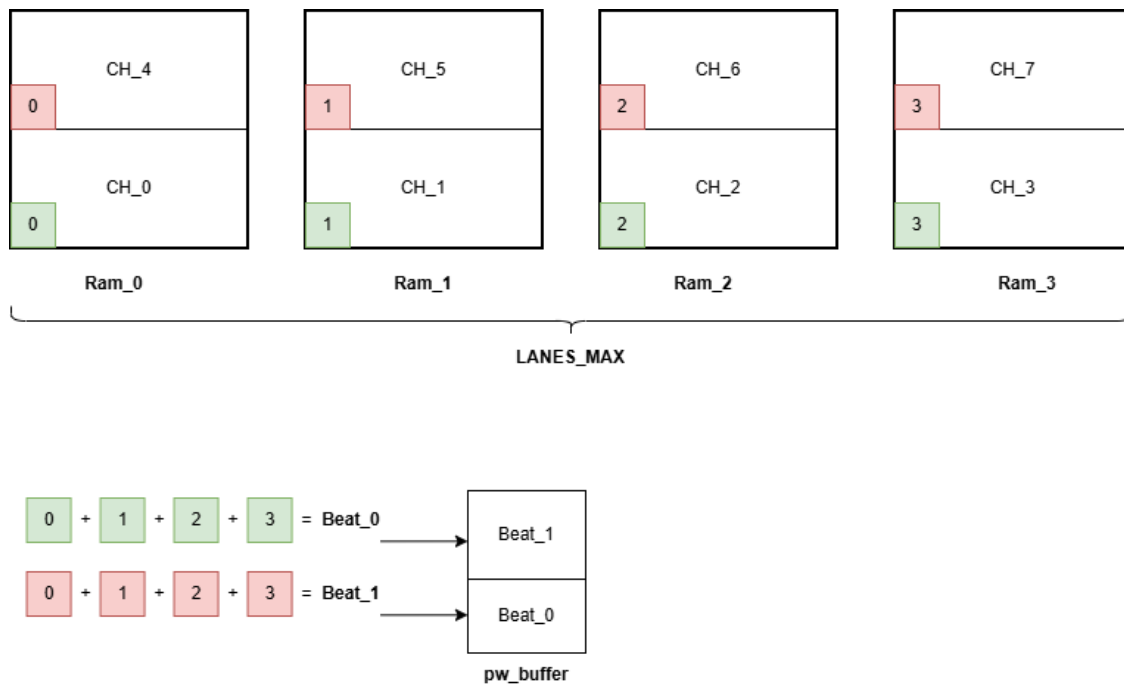


Abbildung 4.16: Pointwise-Datenerfassung in S_SCAN für $\text{LANES_MAX} = 4$ und $\text{cfg_cin} = 8$: zwei Beats werden nacheinander aufgenommen und im `pw_buffer` als `Beat_0` (Kanäle $c = 0..3$) und `Beat_1` (Kanäle $c = 4..7$) abgelegt.

S_LOAD_BIAS_SHIFT_Mult

Im Zustand `S_LOAD_BIAS_SHIFT_Mult` lädt das `fetching_module` die Biaswerte, die Requantisierungs-Shiftfaktoren sowie die zugehörigen Multiplikationsfaktoren für den aktuell zu verarbeitenden Output-Block `blk_idx`. Ein Block umfasst `cfg_tile_c` Output-Kanäle (Spalten-Tile-Breite des Arrays). Die zugrundeliegende Speicherorganisation, das DMA-Schreibformat (byteadressiert, little-endian) sowie die blockweise Bereitstellung der Werte sind in Unterabschnitt 4.4.3 beschrieben.

Zur Laufzeit adressiert der Fetcher die Bias/Shift/Mult-SRAM blockweise über den Blockindex:

$$\text{bs_raddr} = \text{blk_idx}.$$

Der Controller setzt `bs_ren = 1` und wartet auf `bs_rvalid`. Sobald `bs_rvalid = 1` anliegt, stehen die `cfg_tile_c` Werte für Bias, Shift und den Multiplikationsfaktor als Spaltenvektoren `bs_bias_cols[0..cfg_tile_c - 1]`, `bs_shift_cols[0..cfg_tile_c - 1]` sowie `bs_mult_cols[0..cfg_tile_c - 1]` bereit und können unmittelbar den `cfg_tile_c` parallelen Array-Spalten zugeordnet werden. Anschließend setzt der Controller `bias_loaded = 1`, deaktiviert `bs_ren` und wechselt in den Zustand `S_STREAM_AND_ARITHMETIC`.

Zusätzlich wird beim Eintritt in die Streaming-Phase der Weight-Read-Pointer auf den Blockstart gesetzt:

$$\text{raddr_pointer} = \text{base_addr} + \text{blk_idx},$$

sodass die erste Gewichts-anforderung in `S_STREAM_AND_ARITHMETIC` konsistent zum geladenen Bias/Shift/Mult-Block passt.

S_STREAM_AND_ARITHMETIC

Im Zustand `S_STREAM_AND_ARITHMETIC` beginnt der Fetcher mit dem synchronisierten Streaming von (i) Gewichten aus der Weight-SRAM und (ii) Eingabedaten aus den Window-RAMs in das Systolic Array. Ziel ist, pro Takt genau ein Gewichtsarray der Breite `cfg_tile_c` bereitzustellen und gleichzeitig die dazu passenden `cfg_tile_r` Eingabedaten bereitzustellen.

Der Einstieg in `S_STREAM_AND_ARITHMETIC` ist an den zuvor geladenen Bias/Shift/Mult-Block gekoppelt: Beim Übergang aus `S_LOAD_BIAS_SHIFT_Mult` wird der Weight-Read-Pointer auf den *Blockstart* initialisiert. Dieser Blockstart bildet die Grundlage für das blockweise Gewichtestreaming über `blk_idx` und ist in Abbildung 4.17 schematisch dargestellt.

Solange noch Kontextschritte ausstehen, d. h. solange `ctx_issue < cfg_num_ctx` gilt, erzeugt der Fetcher in jedem Takt einen Read-Request an die Weight-SRAM. Dieses *Issue* bezeichnet dabei genau den Takt, in dem der Controller die Leseanforderung *ausgibt*, also `sram_re = 1` setzt und `sram_raddr` auf den gewünschten Zugriff zeigt:

$$\text{sram_re} = 1, \quad \text{sram_raddr} = \text{raddr_pointer} \ll \log_2(\text{cfg_tile_c}).$$

Dabei ist `raddr_pointer` eine *Wortadresse*. Die Linksverschiebung realisiert die Umrechnung in eine *Byteadresse*: Ein Weight-Array besteht aus `cfg_tile_c` Bytes, daher entspricht ein Schritt $\Delta \text{raddr_pointer} = 1$ einem Byte-Offset von `cfg_tile_c`. Die zugehörige SRAM-Organisation (Blockstruktur über `cfg_cout` und Kontextfortschritt mit Sprungweite `total_blks`) ist in Abbildung 4.17 visualisiert.

Nach jedem Issue wird der Pointer so weitergeschaltet, dass für denselben Block der nächste Kontext angefordert wird:

$$\text{raddr_pointer} \leftarrow \text{raddr_pointer} + \text{total_blks}.$$

Damit springt der Zugriff in der Weight-SRAM immer in Schritten von `total_blks` durch die Speicherstruktur. Erst wenn später `blk_idx` inkrementiert wird, wird `raddr_pointer` erneut auf den neuen Blockstart gesetzt und der Stream wechselt konsistent auf den nächsten Bias/Shift/Mult-Block.

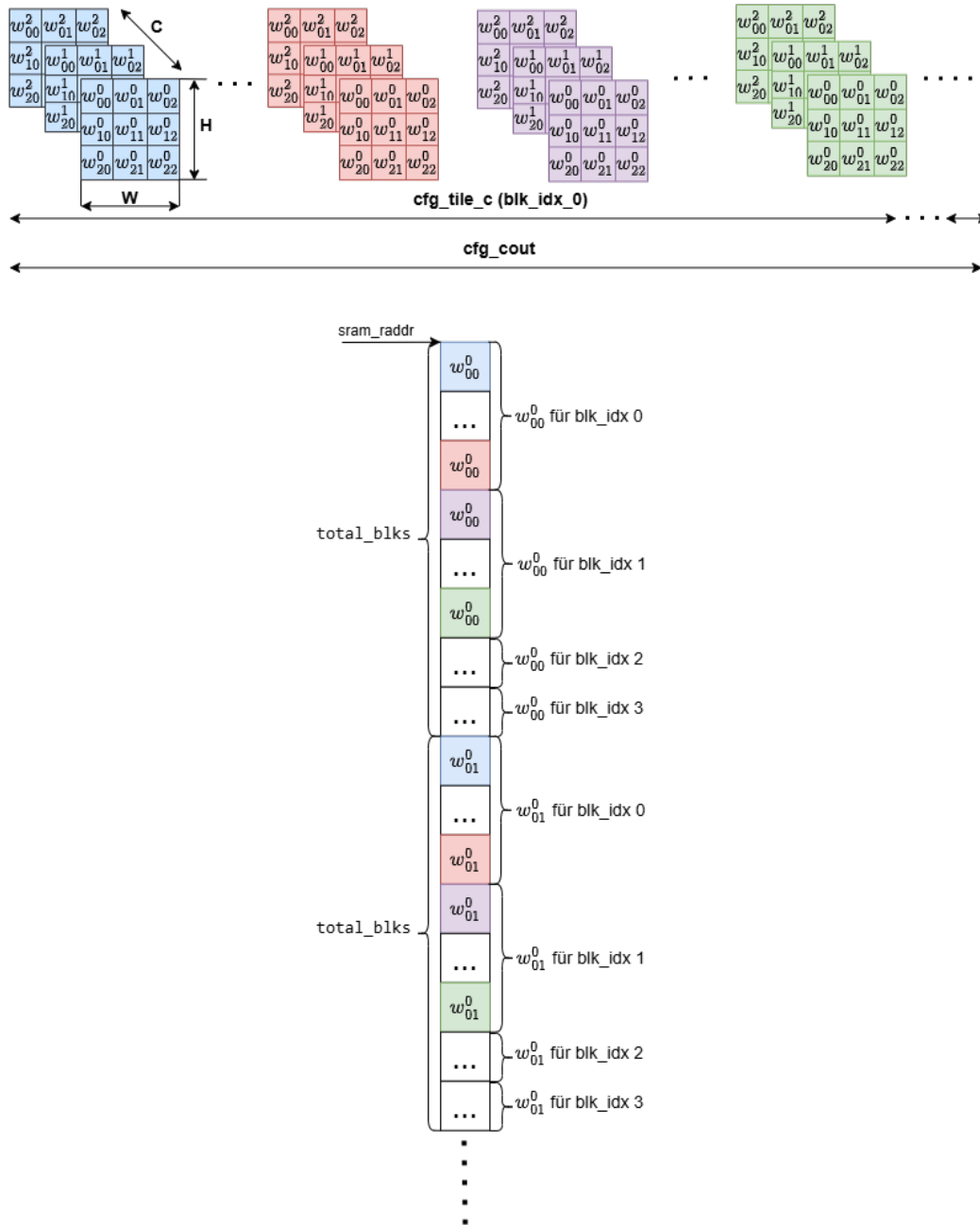


Abbildung 4.17: Gewichtestreaming: Initialisierung $\text{raddr_pointer} = \text{base_addr} + \text{blk_idx}$ und Kontextschritte mit Sprungweite total_blks .

Die Weight-SRAM ist synchron. Das Signal sram_rvalid markiert daher den Takt, in dem das Gewichtsarray $\text{weight_array}[0..\text{cfg_tile_c} - 1]$ tatsächlich gültig ist. Aus diesem Grund wird der gesamte *Ausgabezeitpunkt* des Fetchers an sram_rvalid ausgerichtet; die zeitliche Beziehung zwischen *Issue* ($\text{sram_re/sram_raddr}$) und *Valid* (sram_rvalid) folgt der in Abbildung 4.17 gezeigten blockweisen SRAM-Abfolge.

Convolution-Modus ($\text{ctrl_dw_en} = 0$ und $\text{ctrl_pw_en} = 0$). Im Convolution-Modus wird pro Takt genau *ein* Window-RAM gelesen: Es wird die Instanz chan_sel_i aktiviert und mit der Wortadresse:

$$\text{cv_word_addr} = \text{grp_idx}$$

adressiert. Für eine feste Gruppe bleibt `cv_word_addr` konstant, während innerhalb dieses Worts zuerst alle Pixelpositionen eines Fensters (9 Pixel bei 3×3) und anschließend die Kanäle durchlaufen werden.

Im Ausgabetakt (`sram_rvalid = 1`) wird `stream_valid = 1` gesetzt. Aus dem gelesenen gepackten Wort wird genau ein Byte pro Lane extrahiert. Dazu wird zunächst die Bitposition innerhalb eines Fensters über `pix_sel` bestimmt:

$$\text{bit_index} = (\text{pix_sel}) \ll \text{PX_SHIFT}, \quad \text{PX_SHIFT} = \log_2(\text{PXW}), \quad \text{PXW} = 8 \text{ Bit.}$$

Der Shift $\ll \text{PX_SHIFT}$ liefert den Bit-Offset zur gewünschten Byteposition. Da $\text{PXW} = 8$ gilt, ist jedes b_i genau ein Byte. Anschließend werden die `cfg_tile_r` Lanes aus dem *gleichen* Window-Wort gelesen:

$$\text{ifmap_array}[\text{lane}] = \text{win_rdata_ch}[\text{chan_sel}] \left[(\text{lane} \cdot \text{WINBITS}) + \text{bit_index} + : 8 \right].$$

Damit liefert ein einzelnes gepacktes Window-Wort gleichzeitig `cfg_tile_r` Bytes: je Lane das Byte derselben Pixelposition, jedoch aus den `cfg_tile_r` Fenstern, die in diesem Wort gepackt sind (vgl. Darstellung mit $b_0 \dots b_8$ pro Fenster in Abbildung 4.18).

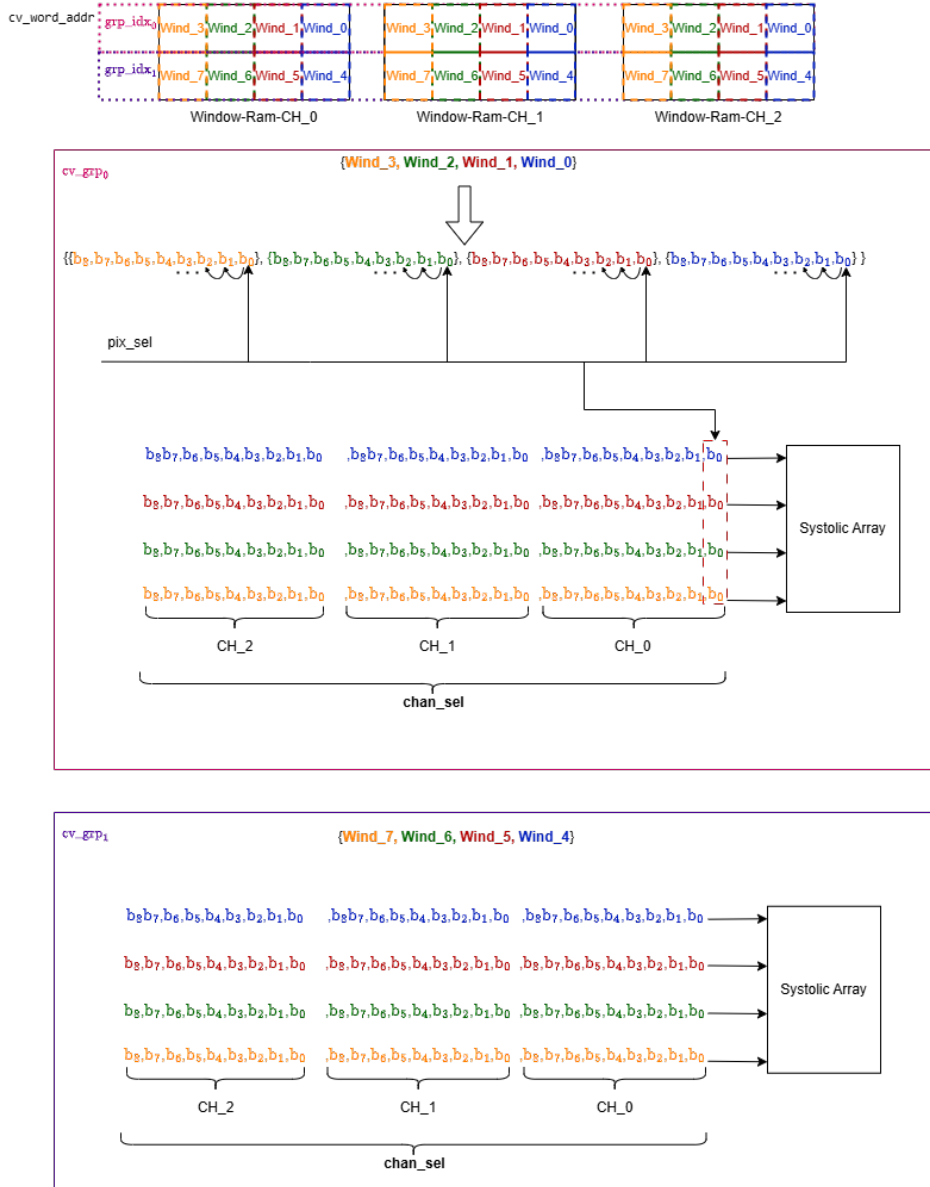


Abbildung 4.18: Convolution: Lesen eines gepackten Window-Ram-Worts pro Kanal ($cv_word_addr = grp_idx$) und Byte-Auswahl über pix_sel / $chan_sel$.

Beispiel ($cfg_tile_r = 4$, 3×3 -Fenster, $grp_idx = 0$, $chan_sel = 0$). Ein 3×3 -Fenster hat $NUM_PIX = 9$ Pixel, damit $WINBITS = 9 \cdot 8 = 72$ Bit pro Fenster. Das gelesene Gruppenwort $win_rdata_ch[0]$ enthält bei $cfg_tile_r = 4$ genau vier Fenster ($Wind_0, Wind_1, Wind_2, Wind_3$), jeweils 72 Bit breit, hintereinander gepackt:

$$win_rdata_ch[0] = \{ \underbrace{Wind_3}_{[4 \cdot 72 - 1 : 3 \cdot 72]}, \underbrace{Wind_2}_{[3 \cdot 72 - 1 : 2 \cdot 72]}, \underbrace{Wind_1}_{[2 \cdot 72 - 1 : 1 \cdot 72]}, \underbrace{Wind_0}_{[1 \cdot 72 - 1 : 0 \cdot 72]} \}.$$

Für $pix_sel = 0$ gilt:

$$bit_index = (0) \ll 3 = 0,$$

somit wird pro Fenster das Byte b_0 adressiert. Damit ergeben sich die vier gleichzeitig ausgegebenen Bytes:

$$\begin{aligned}\text{ifmap_array}[0] &= \text{win_rdata_ch}[0] \left[(0 \cdot 72) + 0 + : 8 \right] = b_0(\text{Wind}_0), \\ \text{ifmap_array}[1] &= \text{win_rdata_ch}[0] \left[(1 \cdot 72) + 0 + : 8 \right] = b_0(\text{Wind}_1), \\ \text{ifmap_array}[2] &= \text{win_rdata_ch}[0] \left[(2 \cdot 72) + 0 + : 8 \right] = b_0(\text{Wind}_2), \\ \text{ifmap_array}[3] &= \text{win_rdata_ch}[0] \left[(3 \cdot 72) + 0 + : 8 \right] = b_0(\text{Wind}_3).\end{aligned}$$

Im nächsten gültigen Ausgabetakts wird pix_sel inkrementiert, $\text{pix_sel} = 1$ mit $\text{bit_index} = 1 \ll 3 = 8$, und es werden analog die Bytes $b_1(\text{Wind}_0.. \text{Wind}_3)$ gelesen, bis $\text{pix_sel} = 8$ erreicht ist. Danach wird pix_sel auf 0 zurückgesetzt und der Kanalindex chan_sel inkrementiert, sodass im *gleichen* cv_word_addr das nächste Kanalwort gelesen wird. Dieser Ablauf wiederholt sich, bis alle aktiven Kanäle dieses Gruppenworts verarbeitet sind.

Der Gruppenwechsel (Inkrement von grp_idx und damit nächste cv_word_addr) erfolgt anschließend im Zustand $\text{S_NEXT_GROUP_BLOCK}$. Da Convolution im Rechenmodul gepipelined ist (Unterabschnitt 4.3.4), startet der Fetcher den Arithmetic-Controller erst dann, wenn das *erste* gültige Weight-Array tatsächlich anliegt:

$$\text{start_acc} = 1 \quad \text{bei} \quad (\text{sram_rvalid} = 1) \wedge (\text{ctx_out} = 0).$$

So ist garantiert, dass die Pipeline mit konsistenten Daten gefüllt wird (erste Inputs und erste Gewichte).

Depthwise-Fall ($\text{ctrl_dw_en} = 1$). Im Depthwise-Modus wird eine Gruppe als *eine* Fensterposition interpretiert, die gleichzeitig über cfg_tile_r Kanäle verarbeitet wird (vgl. Abbildung 4.19). Deshalb werden in jedem Issue alle instanziierten Window-RAM-Kanäle parallel aktiviert:

$$\forall \text{ch} \in \{0, \dots, \text{cfg_tile_r} - 1\} : \quad \text{win_ren_ch}[\text{ch}] = 1, \quad \text{win_raddr_ch}[\text{ch}] = \text{dw_word_addr}.$$

Die Auswahl, *welches* Fenster innerhalb dieses gelesenen Worts verwendet wird, erfolgt über dw_lane_sel . Im Ausgabetakts wird dann pro Lane ein Byte aus dem jeweils passenden Kanalwort extrahiert:

$$\text{ifmap_array}[\text{lane}] = \text{win_rdata_ch}[\text{lane}] \left[(\text{dw_lane_sel} \cdot \text{WINBITS}) + \text{bit_index} + : 8 \right].$$

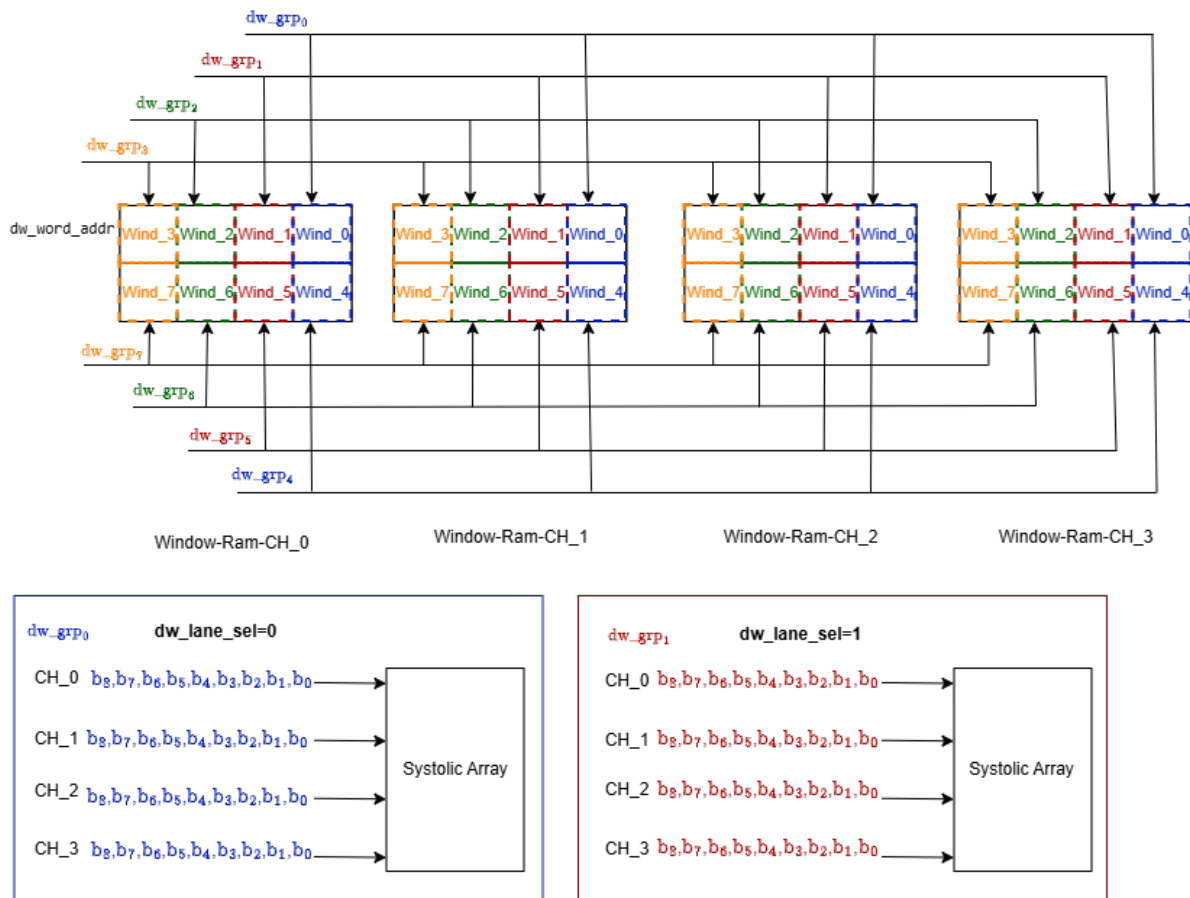


Abbildung 4.19: Depthwise: Pro Gruppe wird dieselbe Fensterposition über alle Kanäle gelesen; Fensterwahl im gepackten Wort über `dw_lane_sel` und Wortwahl über `dw_word_addr`.

Intuitiv (vgl. Abbildung 4.19): Für `dw_grp0` wird in allen Kanalinstanzen das „blaue“ Fenster gewählt (`dw_lane_sel = 0`), für `dw_grp1` das „rote“ Fenster (`dw_lane_sel = 1`), usw. Sobald `dw_lane_sel = cfg_tile_r - 1` erreicht ist, wird `dw_lane_sel` auf 0 zurückgesetzt und `dw_word_addr` inkrementiert, um das nächste gepackte Wort zu lesen. Damit zählt der Gruppenindex im Depthwise-Fall effektiv Fensterpositionen (insgesamt `total_wins` Positionen).

Für Depthwise wird der Arithmetic-Controller bereits beim Issue gestartet (`start_acc` bei `sram_re = 1`), da der Depthwise-Rechenpfad in dieser Architektur ohne die gleiche Pipeline-Vorlaufbedingung wie Convolution betrieben wird: das erste gültige Weight-Array kommt im Folgetakt und der Controller befindet sich dann bereits im aktiven Zustand.

Pointwise-Modus (1×1): „Gruppe“ als Pixel (x, y) über alle Kanäle Im Pointwise-Modus ist die semantische Bedeutung von *Gruppe* gegenüber Convolution/Depthwise verschoben: Eine Gruppe entspricht genau *einem* Arbeitspixel an einer festen Bildposition (x, y) und umfasst dessen komplette Kanalachse $\{0, \dots, \text{cfg_cin} - 1\}$. Während die Gewichtsseite in `S_STREAM_AND_ARITHMETIC analog` zum Convolution-Pfad arbeitet, liegt die Besonderheit ausschließlich auf der Aktivierungsseite: Die `cfg_cin` Kanäle des Pixels werden in `cfg_needed_beats_pw` Beats partitioniert und in `S_SCAN` vollständig in den Pointwise-Puffer `pw_buffer` übernommen.

Im Ausgabetak (sram_rvalid = 1) wird anschließend genau *ein* Kanalwert dieses gepufferten Pixels in das Array eingespeist, während die Gewichte weiterhin spaltenparallel gestreamt werden. Der Ablauf ist in Abbildung 4.20 illustriert: Für LANES_MAX = 4 und cfg_cin = 8 wird das Arbeitspixel in zwei Beats in pw_buffer abgelegt (Beat 0 enthält $c = 0..3$, Beat 1 enthält $c = 4..7$). In der nachfolgenden Streaming-Phase wird dann pro gültigem Gewichtsvektor genau ein Aktivierungswert aus dem Puffer ausgewählt und auf ifmap_array[0] gelegt; alle weiteren Lanes bleiben in diesem Modus ungenutzt und werden auf 0 gesetzt. Formal gilt:

$$\text{ifmap_array}[0] \leftarrow \text{pw_buffer}[\text{act_beat}][\text{act_lane}], \quad \text{ifmap_array}[\text{lane} > 0] \leftarrow 0.$$

Dabei werden die Indizes aus dem Kontextzähler abgeleitet, d. h. es gilt $\text{act_beat} = \lfloor \text{ctx} / \text{LANES_MAX} \rfloor$ und $\text{act_lane} = \text{ctx} \bmod \text{LANES_MAX}$. Damit wird die Kanalachse des Pixels seriell abgearbeitet, während die Spaltenparallelität des Arrays vollständig erhalten bleibt.

Die Gruppe gilt als vollständig verarbeitet, sobald genau cfg_num_ctx gültige (Activation,Weight)-Paare ausgegeben wurden; die Detektion erfolgt *analog* zu den anderen Modi über ctx_out in gültigen Ausgabetakten. Im Pointwise-Modus werden (x, y) im Fetcher *nicht* als explizite Zähler geführt. Der Positionsfortschritt ist implizit durch den Eingabestrom definiert: Das nächste Pixel (x, y) wird genau dann übernommen, wenn die FSM nach abgeschlossener Gruppe den Eingabestrom kurz freigibt ($\text{px_ready} = 1$) und dadurch das I/O-SRAM zum nächsten Pixel weiterschaltet.

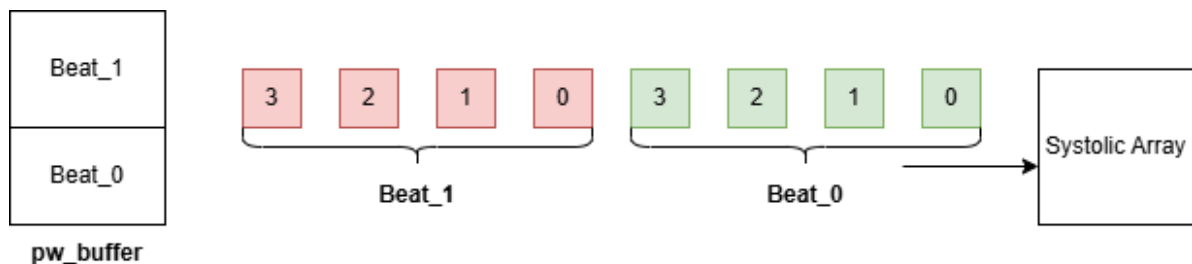


Abbildung 4.20: Pointwise-Streaming aus dem Puffer pw_buffer am Beispiel LANES_MAX = 4 und $\text{cfg_cin} = 8$: Das Arbeitspixel wird in zwei Beats gepuffert (Beat 0: $c = 0..3$, Beat 1: $c = 4..7$) und anschließend kanalweise seriell in $\text{ifmap_array}[0]$ ausgegeben.

Der Fetcher verbleibt so lange in S_STREAM_AND_ARITHMETIC, bis die aktuelle Gruppe vollständig gestreamt wurde. Als Abbruchkriterium wird ein Puls group_done_pulse erzeugt, sobald der letzte Kontextschritt der Gruppe im *Ausgabetak* (also bei gültigen Gewichten) erreicht ist. Formal entspricht das:

$$\text{group_done_pulse} = (\text{sram_rvalid} = 1) \wedge (\text{ctx_out} = \text{cfg_num_ctx} - 1).$$

Dabei zählt ctx_out die tatsächlich *ausgegebenen* Kontextschritte (gebunden an sram_rvalid). Sobald $\text{ctx_out} = \text{cfg_num_ctx} - 1$ erreicht ist, wurden genau cfg_num_ctx Weight-Arrays für diese Gruppe in das Array gestreamt. In diesem Moment wird group_done_pulse aktiv und der Controller wechselt in den Wartezustand:

$$\text{group_done_pulse} = 1 \Rightarrow \text{state} \leftarrow \text{S_WAIT_ACC}.$$

Im Zustand `S_WAIT_ACC` werden keine weiteren Daten gestreamt.

`S_WAIT_ACC`

Nach dem vollständigen Streaming der aktuellen Gruppe verlässt der Fetcher den Zustand `S_STREAM_AND_ARITHMETIC` über `group_done_pulse` und wechselt in `S_WAIT_ACC`. In diesem Zustand wird die nächste Gruppe bewusst *nicht* sofort gestartet, sondern der Fetcher wartet auf das Abschlusssignal:

$$\text{acc_done} = 1,$$

welches aus dem Processing-/Writeback-Pfad kommt. Erst wenn `acc_done` aktiv ist, sind die Ausgabewerte der aktuellen Gruppe vollständig übernommen und die internen Zählerstände konsistent.

Erzeugung von `acc_done` im Processing-Modul. Im Processing-Modul wird `acc_done` als `acc_done_grp` an den Fetcher zurückgeführt und entspricht direkt dem Puls `store_done_pulse`. Dieser Puls entsteht im Write-Back-Controller, sobald alle gültigen Ausgabewerte der aktuellen Kachel vollständig über die internen Zähler `st_lane` und `st_col` durchlaufen wurden. Damit markiert `store_done_pulse` das Ende des „Writeback-Zyklus“ dieser Gruppe (bei deaktiviertem Pooling: direktes Schreiben in die Feature-Bank; bei aktiviertem Pooling: vollständige Übergabe der Kachelwerte in den Pooling-Pfad). Anschaulich verhindert diese Wartephase, dass die nächste Gruppe startet, während die vorherige Kachel noch nicht vollständig aus dem Accelerator übernommen wurde.

Sonderfall Pooling (`cfg_pool_enable=1`). Bei aktiviertem Pooling kennzeichnet `acc_done` bzw. `store_done_pulse` *zunächst nur* den Abschluss der Store-Phase des aktuellen Tiles: Die Kachelwerte wurden vollständig in die internen Pooling-Zeilenspuffer (`pool_line0` und `pool_line1`) übernommen. Die eigentliche Pooling-Operation ist damit noch nicht zwingend abgeschlossen.

Sobald ein vollständiger Stripe aus zwei aufeinanderfolgenden OFMAP-Zeilen verfügbar ist, wird `start_pool` ausgelöst und das separate `pooling_module` wechselt in seinen Laufzustand (`pool_busy=1`). Das Pooling-Modul liest dann deterministisch die 2×2 -Fenster aus `pool_line0/1` und schreibt die gepoolten Ergebnisse direkt über den gemeinsamen Write-Port in die durch `cfg_write_bank_sel` selektierte Write-Bank zurück. Der Abschluss dieser Pooling-Phase wird über `pooling_done` signalisiert (und `pool_busy` fällt wieder auf 0).

Im Pooling-Fall darf der Fetcher *nicht in jedem Fall* sofort nach `store_done_pulse` zur nächsten Gruppe übergehen. Es wird unterschieden:

- **Kein Stripe-Abschluss:** Wenn durch das aktuelle Tile noch kein vollständiger Stripe entsteht, wird `acc_done` direkt aus `store_done_pulse` gebildet. Der Fetcher kann ohne Stall fortfahren, während die Lines durch nachfolgende Gruppen weiter gefüllt werden.
- **Stripe-Abschluss:** Wenn das aktuelle Tile den Stripe vervollständigt und `start_pool` auslöst, muss der Fetcher warten, damit die Zeilenspuffer während des Poolings nicht überschrieben werden. In diesem Fall wird `acc_done` erst nach Abschluss des Poolings gegeben, d. h. aus `pooling_done` abgeleitet.

Sobald `acc_done=1` wird abhängig von Block- und Gruppenende verzweigt:

- **Letzte Gruppe und letzter Block:** Falls `last_group` und `last_block` aktiv sind, wechselt der Fetcher in `S_DONE`.

- **Sonst:** In allen anderen Fällen folgt der Übergang nach S_NEXT_GROUP_BLOCK, wo die Indizes (grp_idx, blk_idx sowie dw_lane_sel/dw_word_addr) für die nächste Gruppe bzw. den nächsten Block vorbereitet werden.

Damit stellt S_WAIT_ACC die notwendige Synchronisationsbarriere zwischen „Streaming“ und „Writeback/Capture“ dar und garantiert einen konsistenten Übergang zwischen aufeinanderfolgenden Gruppen.

S_NEXT_GROUP_BLOCK

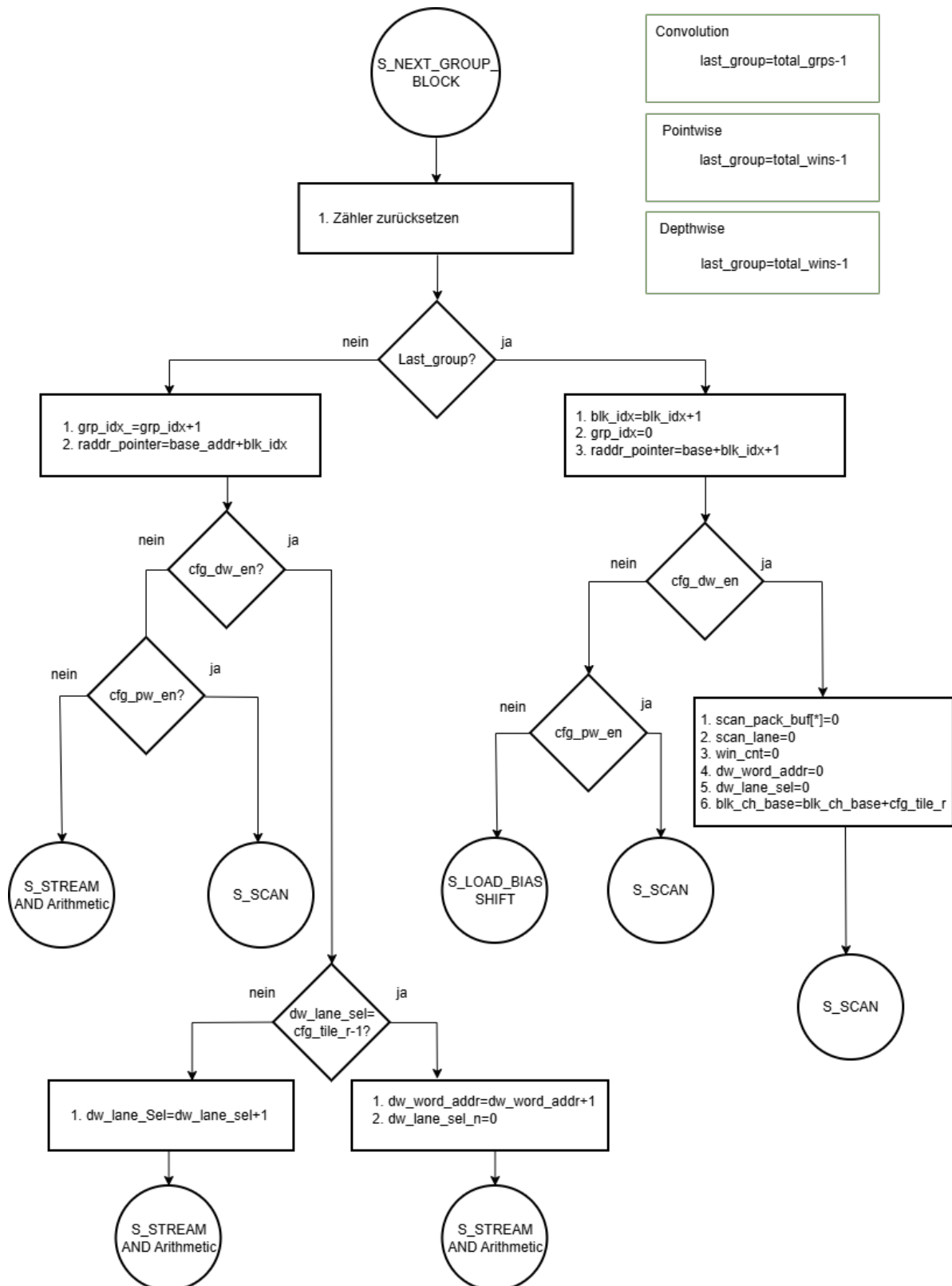


Abbildung 4.21: Übergangslogik für Gruppen- und Blockwechsel. Links: nächstes Gruppenwort innerhalb desselben Blocks. Rechts: Blockwechsel (inkl. Pointer-/Index-Umschaltung) mit anschließendem Bias/Shift/Mult-Laden (Convolution), neuem Scan (Depthwise) bzw. Pixel-Fortschritt (Pointwise).

Der Zustand `S_NEXT_GROUP_BLOCK` ist die Umschaltstelle zwischen zwei Streaming-Phasen. Hier werden die gruppeninternen Zähler zurückgesetzt und anschließend entweder (i) die nächste Gruppe im *gleichen Block* gestartet (linker Pfad in Abbildung 4.21) oder (ii) ein Blockwechsel durchgeführt (rechter Pfad).

Beim Eintritt werden alle Zähler, die den Ablauf *innerhalb* einer Gruppe steuern, auf definierte Startwerte gesetzt, sodass der folgende Durchlauf stets mit einem sauberen Kontext beginnt. Dazu zählen insbesondere:

$$\text{ctx_out} = 0, \quad \text{pix_sel} = 0, \quad \text{chan_sel} = 0,$$

sowie der Kanalindex `pw_ch_sel = 0`. Zusätzlich werden im Pointwise-Pfad der Beat-Zähler und das Full-Flag des Puffers zurückgesetzt (`pw_beat = 0`, `pw_buf_full = 0`), sodass im folgenden `S_SCAN` wieder neue Beats des nächsten Pixels eingesammelt werden können.

Ob links (nächste Gruppe) oder rechts (Blockwechsel) verzweigt wird, hängt von `last_group` ab. Die Bedeutung von „Gruppe“ ist dabei modusabhängig (vgl. Abbildung 4.21):

- **Convolution** (`cfg_dw_en = 0`, `cfg_pw_en = 0`): $\text{last_group} \iff \text{grp_idx} = \text{total_grps} - 1$.
- **Depthwise** (`cfg_dw_en = 1`): $\text{last_group} \iff \text{grp_idx} = \text{total_wins} - 1$.
- **Pointwise** (`cfg_pw_en = 1`, `cfg_dw_en = 0`): $\text{last_group} \iff \text{grp_idx} = \text{total_wins} - 1$, wobei eine Gruppe *ein Pixel* (x, y) *über alle Kanäle* repräsentiert (vgl. Pointwise-Skizze in Abbildung 4.20).

Fall A: `last_group = 0` — **nächste Gruppe innerhalb desselben Blocks.** Ist die aktuelle Gruppe *nicht* die letzte Gruppe, bleibt `blk_idx` unverändert und es wird nur der Gruppenindex fortgeschaltet:

$$\text{grp_idx} \leftarrow \text{grp_idx} + 1.$$

Der Folgezustand hängt vom Modus ab:

- **Convolution** (`cfg_dw_en = 0`, `cfg_pw_en = 0`): Übergang nach

$$\text{state} \leftarrow \text{S_STREAM_AND_ARITHMETIC},$$
 d. h. es wird unmittelbar die nächste Gruppe im selben Block gestreamt.
- **Depthwise** (`cfg_dw_en = 1`): ebenfalls Übergang nach `S_STREAM_AND_ARITHMETIC`, wobei zusätzlich die Fensterposition innerhalb des gepackten Wortes weitergeschaltet wird (wie in Abbildung 4.21 unten dargestellt):

$$\begin{aligned} \text{wenn } \text{dw_lane_sel} = \text{cfg_tile_r} - 1: \quad & \text{dw_lane_sel} \leftarrow 0, \\ & \text{dw_word_addr} \leftarrow \text{dw_word_addr} + 1, \\ \text{sonst:} \quad & \text{dw_lane_sel} \leftarrow \text{dw_lane_sel} + 1. \end{aligned}$$

- **Pointwise** (`cfg_pw_en = 1`, `cfg_dw_en = 0`): hier entspricht die „nächste Gruppe“ dem *nächsten Pixel*. Deshalb wird das Streaming nicht direkt aus `S_NEXT_GROUP_BLOCK` heraus neu gestartet, sondern in den Einsammelzustand `S_SCAN` zurückgewechselt:

$$\text{state} \leftarrow \text{S_SCAN}.$$

Das Weiterschalten des Pixels erfolgt anschließend in `S_SCAN` über das reguläre Handshake, d. h. `px_ready = ¬pw_buf_full` (vgl. Abbildung 4.16).

Fall B: last_group = 1 — Blockwechsel. Ist die aktuelle Gruppe die letzte Gruppe im Block, wird auf den nächsten Block umgeschaltet. Dazu gilt konzeptionell:

$$\text{blk_idx} \leftarrow \text{blk_idx} + 1, \quad \text{grp_idx} \leftarrow 0, \quad \text{bias_loaded} \leftarrow 0.$$

Zusätzlich wird die Weight-Adressierung auf den Start des *neuen* Blocks vorpositioniert:

$$\text{raddr_pointer} \leftarrow \text{base_addr} + (\text{blk_idx} + 1).$$

Der Folgezustand ist wieder modusabhängig:

- **Convolution** ($\text{cfg_dw_en} = 0, \text{cfg_pw_en} = 0$): Übergang nach

$$\text{state} \leftarrow \text{S_LOAD_BIAS_SHIFT_Mult},$$

damit für den neuen Block zunächst Bias/Shift/Mult geladen werden.

- **Depthwise** ($\text{cfg_dw_en} = 1$): Übergang nach

$$\text{state} \leftarrow \text{S_SCAN},$$

da für den nächsten Block die Eingabedaten blockweise neu in die Window-RAMs gepackt werden. Dabei wird die Kanalbasis fortgeschaltet und Scan-/Depthwise-Pointer werden zurückgesetzt:

$$\text{blk_ch_base} \leftarrow \text{blk_ch_base} + \text{cfg_tile_r}, \quad \text{dw_word_addr} \leftarrow 0, \quad \text{dw_lane_sel} \leftarrow 0,$$

sowie die Scan-Puffer/Indizes (gemäß Abbildung 4.21) auf Startwerte.

- **Pointwise** ($\text{cfg_pw_en} = 1, \text{cfg_dw_en} = 0$): auch hier wird für den neuen Block ein neuer Pixel-Durchlauf gestartet. Daher wird in S_SCAN gewechselt; beim Blockwechsel wird zusätzlich der Feature-Bank-Eingabestrom über einen ifmap_restart-Puls neu gestartet:

$$\text{state} \leftarrow \text{S_SCAN}.$$

In S_SCAN werden anschließend die Beats des nächsten Pixels wieder vollständig in den Pointwise-Puffer übernommen (vgl. Abbildung 4.16).

Kurzfassung.

- $\text{last_group} = 0$: grp_idx wird erhöht. Convolution/Depthwise gehen direkt zurück in S_STREAM_AND_ARITHMETIC; Pointwise wechselt nach S_SCAN, und das nächste Pixel wird dort über $\text{px_ready} = \neg \text{pw_buf_full}$ übernommen.
- $\text{last_group} = 1$: blk_idx wird erhöht, grp_idx auf 0 gesetzt, Bias/Shift/Mult blockweise neu initialisiert, und raddr_pointer auf $\text{base_addr} + (\text{blk_idx} + 1)$ vorpositioniert. Convolution geht nach S_LOAD_BIAS_SHIFT_Mult, Depthwise nach S_SCAN (Re-Pack/Pointer-Reset), Pointwise nach S_SCAN und triggert beim Blockwechsel zusätzlich einen ifmap_restart-Puls.

S_DONE

Der Zustand S_DONE markiert den Abschluss der gesamten Verarbeitung eines Frames. Er wird ausschließlich aus S_WAIT_ACC erreicht, sobald $\text{acc_done}=1$ gilt *und* gleichzeitig $\text{last_group}=1$ sowie $\text{last_block}=1$ sind. Damit ist garantiert, dass sowohl die letzte Gruppe des letzten Blocks fertig berechnet als auch vollständig zurückgeschrieben wurde.

In S_DONE wird entschieden, ob unmittelbar ein neuer Durchlauf gestartet wird oder ob der Fetcher in den Leerlauf zurückkehrt.

4.6 Processing Module

Das *Processing Module* bildet die zentrale Integrations- und Steuerinstanz der vorgestellten Beschleunigerpipeline. Es verbindet das *Fetching Module* mit dem *Arithmetic Module* und stellt damit den durchgängigen Datenpfad von den Eingangs-Feature-Maps bis zur Ausgabe inklusive optionalem Pooling bereit. Darüber hinaus kapselt es die Speicherperipherie des Systems, indem es die notwendigen SRAM-Komponenten instanziiert und über definierte Handshake-Signale koordiniert.

In den folgenden Unterabschnitten wird das Processing Module detailliert: Zunächst werden die Speicherinstanzen (Feature-Banks, Weight-SRAM, Bias/Shift/Mult-SRAM) und deren Schnittstellen beschrieben. Anschließend folgt die Datenfluss- und Steuerlogik (Start/Restart, Ping-Pong-Select, Handshake zum Fetcher und zum Arithmetic Module). Abschließend werden das Capture/Store-Konzept inklusive Depthwise-Spezifika sowie die optionale Pooling-Pipeline und die Done-Bedingungen auf Systemebene erläutert.

4.6.1 Daten- und Speicherpfad im Processing Module

Das *Processing Module* kapselt die im System verwendeten On-Chip-Speicher und bindet sie so ein, dass ein durchgängiger Datenpfad vom DMA-Import bis zum spaltenparallelen Rechenpfad entsteht. Die grundlegenden Konzepte der Speicherorganisation wurden bereits in Unterabschnitt 4.4.1 beschrieben und werden im Folgenden nur noch in ihrer Rolle innerhalb des *Processing Module* eingeordnet.

Ping-Pong-Anbindung der I/O-SRAM. Zur Entkopplung von Einlesen und Verarbeiten instanziiert das *Processing Module* zwei identische I/O-SRAM-Bänke (Bank A/B) in der in Unterabschnitt 4.4.1 erläuterten WHC-Struktur. Welche Bank als *Lesequelle* für den Fetch-Stream verwendet wird, legt `cfg_read_bank_sel` fest; analog bestimmt `cfg_write_bank_sel` die *Schreibbank* für OFMAP-Rückschreiben.

Wie in Unterabschnitt 4.4.1 gezeigt, sind die Feature-Bänke physisch auf eine maximale Bankanzahl (`LANES_MAX`) ausgelegt, d. h. es werden bis zu `LANES_MAX` RAM-Bänke instanziiert.

Weight-SRAM Die Weight-SRAM wird als separater Speicherblock instanziiert und vom DMA byteadressiert befüllt. Das softwareseitig erzeugte Layout ermöglicht dem `fetching_module` das Lesen zusammenhängender Gewichtstiles pro Kontext (vgl. Abbildung 4.12 und Unterabschnitt 4.4.2). Im *Processing Module* wird dieser Speicher lediglich angebunden (Read-Handshake `w_ren`, `w_raddr`, `w_rvalid`), sodass der Fetcher die Gewichtsdaten taktsynchron zum Fenster-/Kontextstrom abrufen kann.

Bias/Shift/Mult-SRAM Analog werden Bias, Shift und der Multiplier in separaten Speicherinstanzen geführt, die vom DMA byteadressiert mit Little-Endian-int32-Streams beschrieben werden. Für den Rechenpfad werden Bias/Shift/Mult jedoch blockweise gelesen: Der Fetcher adressiert über den Blockindex (`blk_idx`) und erhält im Folgetakt bis zu `cfg_tile_c` Werte pro Zugriff, wodurch Bias/Shift/Mult zeitlich direkt an das Weight-Tiling gekoppelt sind. Das *Proces-*

sing Module übernimmt hierbei die reine Speicheranbindung und übergibt die spaltenparallelen Vektoren (Bias, Shift und Multiplier) an den nachfolgenden Pfad.

Einordnung in den Gesamtfluss. Zusammengefasst stellt das *Processing Module* die in Abschnitt 4.4 eingeführten Speicherbausteine so bereit, dass (i) die I/O-SRAM den kanalparallelen Pixelstream für den Fetcher liefert, (ii) Gewichte aus der Weight-SRAM als zusammenhängende Output-Kanal-Tiles gelesen werden, und (iii) Bias/Shift/Mult blockweise passend zum Tile bereitstehen.

4.6.2 Datenfluss und Rückschreiben

Nachdem die Speicherinstanzen eingeordnet wurden, wird im Folgenden der Datenfluss im *Processing Module* betrachtet. Der zentrale Punkt ist dabei das Rückschreiben der vom *Arithmetic Module* erzeugten Ergebnisse in die als Write-Bank konfigurierte I/O-SRAM. Abhängig vom Laufzeitbit `ctrl_pool_enable` wird der Ergebnisstrom entweder direkt zurückgeschrieben (`ctrl_pool_enable=0`) oder zunächst über eine Pooling-Zwischenstufe geführt (`ctrl_pool_enable=1`). In diesem Abschnitt wird zunächst der Fall ohne Pooling beschrieben.

Das *Arithmetic Module* liefert seine Ergebnisse als zweidimensionales Pixelraster `pix_accel[r][c]`. Zur Laufzeit werden die tatsächlich verwendeten Dimensionen durch `cfg_tile_r` (Zeilen) und `cfg_tile_c` (Spalten) vorgegeben und zusätzlich auf die physische Obergrenze geklemmt. Damit gilt:

$$\text{tile_r_use} = \min(\text{cfg_tile_r}, \text{LANES_MAX}), \quad \text{tile_c_use} = \min(\text{cfg_tile_c}, \text{COLS_MAX}).$$

Im Depthwise-Fall wird in der Implementierung `tile_c_use = tile_r_use` gesetzt, da die Belegung diagonal gekoppelt ist.

Direktes Rückschreiben aus `pix_accel`. Das *Processing Module* schreibt die vom *Arithmetic Module* berechneten Ergebnisse ohne Zwischenspeicher unmittelbar in die als Write-Bank konfigurierte Feature-SRAM zurück. Das *Arithmetic Module* stellt dazu neben dem Ergebnisraster `pix_accel[r][c]` ein internes Abschluss-Signal `done_acc_int` bereit. Aus diesem wird ein Flankenpuls `done_acc_pulse` erzeugt, der den Beginn des Rückschreibvorgangs markiert und die Store-FSM startet.

Da das Rückschreiben sequentiell über mehrere Takte erfolgt, werden beim Auftreten von `done_acc_pulse` die relevanten Tile-Basiswerte gelatcht, um eine konsistente Adressierung über die gesamte Store-Phase sicherzustellen. Hierzu gehören die räumlichen Offsets `cap_base_x` und `cap_base_y` sowie die Kanalbasis `cap_blk_base_co`. Diese gelatchten Offsets definieren eindeutig, an welcher OFMAP-Position (x, y) und in welchem Kanalbereich das fertiggestellte Tile abgelegt wird.

Die Write-Back-FSM iteriert anschließend deterministisch über die Tile-Indizes r und c und schreibt pro Takt genau einen Wert aus `pix_accel` als Byte in die Feature-SRAM. Ein Write wird nur dann ausgeführt, wenn die betrachtete Zelle innerhalb der aktiven Tile-Dimensionen liegt, die berechneten Koordinaten innerhalb der OFMAP-Grenzen liegen und der resultierende Kanalindex

kleiner als `cfg_cout` ist. Dadurch bleibt das Rückschreiben auch in Randbereichen deterministisch und konsistent zur verwendeten WHC-Adressierung.

Write-Back-Controller im Processing Module. Das byteweise Rückschreiben wird durch die in Abbildung 4.22 dargestellten Write-Back-Controller gesteuert. Semantisch entspricht `S_IDLE` dem Warten auf `done_acc_pulse` und `S_WRITEBACK` dem sequentiellen Durchlaufen der Tile-Indizes r, c . Pro Takt wird höchstens ein Byte-Write erzeugt.

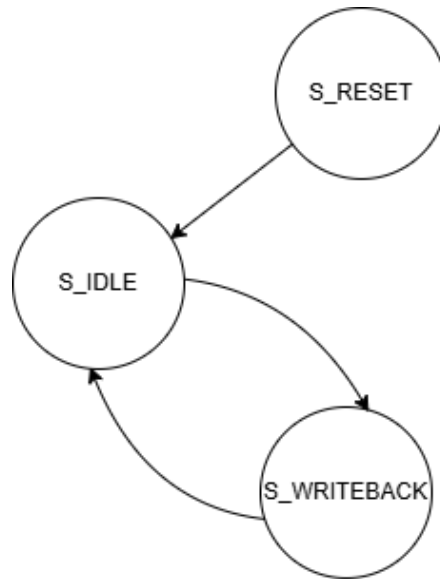


Abbildung 4.22: Write-Back-FSM für das Rückschreiben ohne Pooling: Nach `done_acc_pulse` iteriert die FSM deterministisch über das Tile und schreibt die Ergebnisse byteweise in die I/O-SRAM.

S_WRITEBACK. Im Zustand `S_WRITEBACK` schreibt der Write-Back-Controller das zuletzt fertiggestellte Ergebnis-Tile sequentiell in die als Write-Bank konfigurierte Feature-SRAM zurück. Dazu werden zwei Zähler r und c verwendet. Pro Takt wird genau ein Element `pix_accel[r][c]` betrachtet und als Byte geschrieben. Die Iteration erfolgt deterministisch über ein `cfg_tile_r × cfg_tile_c`-Raster: c wird inkrementiert; bei $c = \text{cfg_tile_c} - 1$ wird $c \leftarrow 0$ gesetzt und anschließend r inkrementiert. Nach dem letzten Paar $(r, c) = (\text{cfg_tile_r} - 1, \text{cfg_tile_c} - 1)$ ist der Write-Back des Tiles abgeschlossen und die FSM kehrt nach `S_IDLE` zurück.

Gelatchte Basen und Adressbildung (innerhalb einer Gruppe). Zum Eintritt in `S_WRITEBACK` werden die Basen des fertigen Tiles gelatcht: `base_x`, `base_y` und `base_ch`. Innerhalb *dieser* Gruppe werden daraus pro Takt die Zielindizes gebildet als:

$$x = \text{base_x} + r, \quad y = \text{base_y}, \quad ch = \text{base_ch} + c.$$

Damit erfolgt der *Kanalwechsel innerhalb einer Gruppe* automatisch über c : Für festes r werden nacheinander die Kanäle $ch = \text{base_ch} + 0, \dots, \text{base_ch} + \text{cfg_tile_c} - 1$ geschrieben. Danach wird c auf 0 gesetzt und r weitergeschaltet (nächste x -Position).

Gültigkeitsbedingungen (Write-Enable). Ein Byte-Write wird nur ausgeführt, wenn (i) $r < \text{cfg_tile_r}$ und $c < \text{cfg_tile_c}$, (ii) die berechnete OFMAP-Position (x, y) innerhalb der

Bildgrenzen liegt und (iii) $ch < \text{cfg_cout}$ gilt. Werden Bedingungen verletzt (Randbereiche), wird kein Write erzeugt.

Depthwise-Spezialfall (nur Diagonale). Im Depthwise-Modus sind innerhalb des zweidimensionalen Rasters typischerweise nur Diagonalelemente gültig. Zusätzlich zu den obigen Bedingungen wird daher das Write-Enable so gegated, dass nur für $r = c$ geschrieben wird. Die FSM iteriert weiterhin über alle (r, c) , Off-Diagonalelemente werden jedoch übersprungen (kein Write), sodass effektiv nur $\text{pix_accel}[0][0]$, $\text{pix_accel}[1][1]$, ... zurückgeschrieben werden.

4.6.3 Pooling Module

Im Folgenden wird der Datenfluss für den Fall $\text{cfg_pool_enable}=1$ beschrieben. Während ohne Pooling ($\text{cfg_pool_enable}=0$) die Ergebnisse des *Arithmetic Module* direkt über den Write-Back-Controller byteweise in die als Write-Bank konfigurierte Feature-SRAM zurückgeschrieben werden (vgl. Unterabschnitt 4.6.2), wird bei aktiviertem Pooling der Ergebnisstrom zunächst über eine Pooling-Zwischenstufe geführt. Dabei bleibt die in Unterabschnitt 4.6.2 eingeführte *Gruppe/Block-Adressierung* (Basen, Gruppenfortschaltung, Blockwechsel) unverändert. Lediglich die *Senke* des Write-Back-Controllers ändert sich: Statt direkt in die Feature-SRAM zu schreiben, werden die Werte zunächst in zeilenweise Puffer (pool_line0/1) eingetragen, anschließend im *Pooling Module* zu 2×2 -Werten reduziert und erst danach in die Feature-SRAM zurückgeschrieben.

Aktivierungsbedingung und Modus. Pooling wird ausschließlich im **Convolution**-Fall verwendet und im **Depthwise**-Fall deaktiviert. Dies wird zur Laufzeit durch

$$\text{pool_active} = \text{cfg_pool_enable} \wedge \neg \text{cfg_dw_enable}$$

realisiert. Damit gilt: Sobald $\text{cfg_dw_enable}=1$ ist, wird der direkte Write-Back-Pfad gemäß Unterabschnitt 4.6.2 genutzt.

Zwei Zeilenpuffer pro Kanal innerhalb eines Blocks. Für Pooling existieren zwei zeilenweise Puffer:

- $\text{pool_line0}[c][x]$: Puffer für eine *gerade* OFMAP-Zeile,
- $\text{pool_line1}[c][x]$: Puffer für die *folgende ungerade* OFMAP-Zeile.

Dabei ist $c \in [0, \text{cfg_tile_c} - 1]$ der Kanal-Offset innerhalb des aktuellen Blocks. Der *absolute* OFMAP-Kanal wird — exakt wie im direkten Rückschreiben — über:

$$ch = \text{base_ch} + c$$

gebildet. Die Bedeutung von base_ch als Kanalbasis (Block) und die Interpretation von c als Offset innerhalb dieses Blocks entspricht damit unmittelbar den Begriffen *Block* und *Gruppe* aus Unterabschnitt 4.6.2.

Der Write-Back-Controller iteriert wie in Unterabschnitt 4.6.2 deterministisch über die Tile-Indizes r, c . Pro Takt wird höchstens ein Element $\text{pix_accel}[r][c]$ verarbeitet. Die räumliche und kanalbezogene Zuordnung ist im Convolution-Fall *identisch* zur Adressbildung im direkten Rückschreiben:

$$x = \text{base_x} + r, \quad y = \text{base_y}, \quad ch = \text{base_ch} + c.$$

Wie im direkten Pfad wird ein Eintrag nur dann übernommen, wenn (i) $r < \text{cfg_tile_r}$ und $c < \text{cfg_tile_c}$, (ii) (x, y) innerhalb der OFMAP-Grenzen liegt ($x < \text{cfg_out_w}$, $y < \text{cfg_out_h}$), und (iii) $ch < \text{cfg_cout}$ gilt. Anstelle eines SRAM-Writes wird der Wert jedoch in die Pooling-Zeilenspuffer geschrieben. Ob in `pool_line0` oder `pool_line1` geschrieben wird, hängt von der Parität der Zielzeile ab:

$$\begin{cases} y \text{ gerade} & \Rightarrow \text{pool_line0}[c][x] \leftarrow \text{pix_accel}[r][c], \\ y \text{ ungerade} & \Rightarrow \text{pool_line1}[c][x] \leftarrow \text{pix_accel}[r][c]. \end{cases}$$

Damit fungiert der Write-Back-Controller im Pooling-Fall als deterministischer *Zeilenpuffer-Füller*. Das resultierende Füllmuster (tile-/gruppenweise Befüllung, sowie Stripe-Bildung aus zwei Zeilen) ist in Abbildung 4.23 visualisiert.

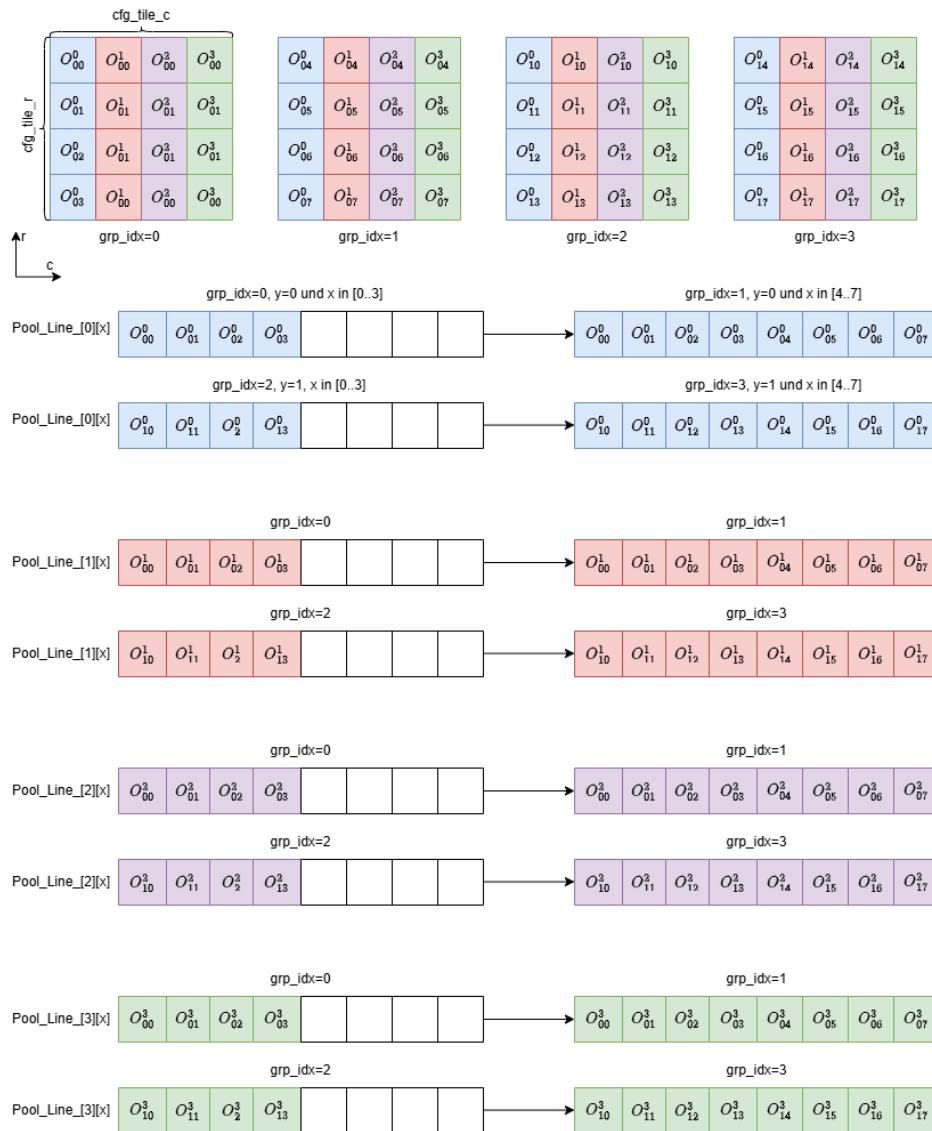


Abbildung 4.23: Übersicht über das Befüllen der Pooling-Lines pro Gruppe und die Bildung eines Pooling-Stripes aus zwei OFMAP-Zeilen. Pro Kanaloffset c existieren zwei Lines (`pool_line0` für die gerade Zeile, `pool_line1` für die folgende ungerade Zeile).

Für die nächste Gruppe derselben OFMAP-Zeile ($\text{grp_idx} = 1$, zweites Tile von links in Abbildung 4.23) bleibt $\text{base_y} = 0$ unverändert, aber die x-Basis wird wie im Gruppenscan fortgeschaltet:

$$\text{base_x} \leftarrow \text{base_x} + \text{cfg_tile_r} = 0 + 4 = 4.$$

Damit bleibt die Taktstruktur identisch, jedoch verschiebt sich die Zieladresse um +4 in x-Richtung: Die Writes gehen nun nach $\text{pool_line0}[c][4], \dots, \text{pool_line0}[c][7]$. Auf diese Weise füllen die Gruppen $\text{grp_idx} = 0$ und $\text{grp_idx} = 1$ gemeinsam die komplette Zeile $y = 0$ in pool_line0 .

Wechselt der Scan anschließend zur nächsten OFMAP-Zeile, gilt $\text{base_y} = 1$ (ungerade). Die Adressbildung bleibt weiterhin unverändert ($x = \text{base_x} + r$, $y = \text{base_y}$, $ch = \text{base_ch} + c$), jedoch wechselt die Senke aufgrund der Parität von y auf pool_line1 :

$$y \text{ ungerade} \Rightarrow \text{pool_line1}[c][x] \leftarrow \text{pix_accel}[r][c].$$

Sobald dadurch zwei aufeinanderfolgende, vollständig gefüllte Zeilen vorliegen (y gerade in pool_line0 und $y + 1$ ungerade in pool_line1), bilden sie den Stripe aus Abbildung 4.23. Erst dann kann das *Pooling Module* die 2×2 -Fenster deterministisch aus beiden Lines auslesen und die gepoolten Ergebnisse in die Feature-SRAM zurückschreiben.

Pooling (2×2 , Stride 2) benötigt zwei vollständig gefüllte, aufeinanderfolgende OFMAP-Zeilen: eine gerade Zeile y und die direkt folgende ungerade Zeile $y + 1$. Das Ende einer OFMAP-Zeile wird *nur am Tile-Ende* erkannt (store_done_pulse), d. h. genau dann, wenn der Write-Back-Controller das aktuelle Tile vollständig in die Lines eingetragen hat (vgl. Ablauf in Abbildung 4.23).

Eine Zeile gilt als vollständig, wenn die letzte durch das Tile abgedeckte x-Position das rechte Ende erreicht:

$$\text{row_end_tile} \iff (x_{\text{last}} = \text{cfg_out_w} - 1), \quad \text{last_row_is_odd} = y_{\text{last}}[0].$$

Damit ergeben sich zwei Fälle:

- **Gerade Zeile vollständig:** Falls $\text{row_end_tile} = 1$ und $\text{last_row_is_odd} = 0$, wird even_row_ready gesetzt und even_row_y speichert den Index dieser geraden Zeile. Damit ist $\text{pool_line0}[*][0..\text{cfg_out_w}-1]$ vollständig gültig.
- **Ungerade Zeile vollständig + passende gerade Zeile vorhanden:** Falls anschließend $\text{row_end_tile} = 1$ und $\text{last_row_is_odd} = 1$, wird Pooling nur gestartet, wenn die unmittelbar vorhergehende gerade Zeile vorliegt:

$$\text{even_row_ready} = 1 \wedge \text{even_row_y} = (y_{\text{last}} - 1).$$

Erst dann bilden pool_line0 (Zeile $y_{\text{last}} - 1$) und pool_line1 (Zeile y_{last}) einen vollständigen Stripe, und es wird start_pool erzeugt.

Das *Pooling Module* besteht aus (i) einer reinen *Pooling-Unit* (kombinatorisch: Max- bzw. Average-Pooling über vier Werte) und (ii) einem *Pooling-Controller* mit eigener FSM, der die Sliding-Iteration über einen Stripe steuert. Abbildung 4.24 visualisiert das Sliding-Prinzip: Für einen festen Kanaloffset c wird sukzessive ein 2×2 -Fenster aus den beiden Zeilenpuffern $\text{pool_line0}/1$ ausgelesen, in der Pooling-Unit reduziert und anschließend als ein Ausgabewert in die Write-Bank geschrieben. Dabei wird x fortgeschaltet, bis die Stripe-Breite abgedeckt ist; erst danach wechselt der Controller zum nächsten Kanaloffset c .

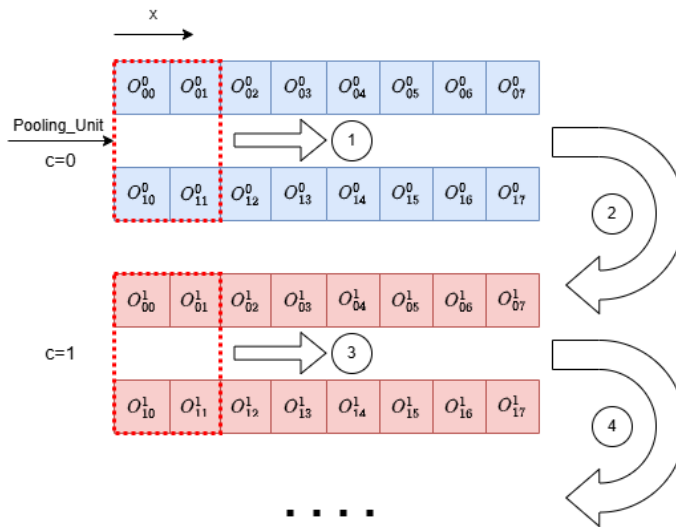


Abbildung 4.24: Sliding der Pooling-Unit über einen Stripe: Für einen Kanaloffset c wird ein 2×2 -Fenster aus `pool_line0/1` gelesen, gepoolt und ausgegeben; danach wird x fortgeschaltet. Nach Abschluss der Stripe-Breite wird zum nächsten Kanaloffset c gewechselt.

Der Pooling-Controller nutzt hierzu eine einfache Zustandsmaschine. Nach `reset` wird der Startzustand `S_RESET` verlassen und in `S_IDLE` übergegangen. In `S_IDLE` wartet der Controller auf `start_pool`. Sobald `start_pool` gesetzt ist, werden die internen Laufzeitparameter aus den CSRs abgeleitet und die internen Indizes initialisiert:

$$\text{pool_w_rt} = \left\lfloor \frac{\text{cfg_out_w}}{\text{POOL_STRIDE}} \right\rfloor, \quad \text{pool_ch_rt} = \text{cfg_tile_c}, \quad y_{\text{out}} = \left\lfloor \frac{\text{stripe_y}}{\text{POOL_STRIDE}} \right\rfloor.$$

Dabei ist `stripe_y` der (ungerade) Zeilenindex des abgeschlossenen Stripes (d.h. der Index der zweiten Zeile des Zeilenpaares). Die Größe y_{out} bezeichnet die *Ausgabezeile* im gepoolten Feature-Map: Da Pooling mit Stride `POOL_STRIDE` die räumliche Auflösung reduziert, werden jeweils `POOL_STRIDE` Eingabezeilen auf genau eine Ausgabezeile abgebildet. Somit schreibt der Stripe aus den Eingabezeilen $(\text{stripe_y} - 1, \text{stripe_y})$ deterministisch in die Ausgabezeile y_{out} .

Anschließend wechselt die FSM nach `S_POOLING`. In `S_POOLING` iteriert der Controller deterministisch zuerst über $x = 0..\text{pool_w_rt} - 1$ und danach über $c = 0..\text{pool_ch_rt} - 1$. Pro Takt wird genau ein 2×2 -Fenster aus `pool_line0/1` für den aktuellen Kanaloffset c ausgewählt, in der Pooling-Unit (Max/Average) verarbeitet (vgl. Listing 4.1) und als ein gepoolter Wert über `of_we/of_ch/of_x/of_y/of_wdata` ausgegeben. Der gepoolte Wert wird im selben Takt direkt in die Feature-SRAM zurückgeschrieben; die konkrete Write-Bank wird dabei durch den globalen Write-Port-Multiplexer im *Processing Module* über `cfg_write_bank_sel` selektiert, während das Pooling-Modul lediglich die standardisierten Write-Signale liefert. Die Koordinaten innerhalb der Bank ergeben sich eindeutig aus der Controller-Iteration: `of_x = x` adressiert die *gepoolte* x-Position (Sliding-Zähler, $0..\text{pool_w_rt} - 1$), `of_y = y_{\text{out}}` ist für den gesamten Stripe konstant, und der Kanal folgt konsistent zur Block-/Gruppen-Notation aus Unterabschnitt 4.6.2 als `of_ch = base_ch + c`. Nach Abschluss der letzten Kombination (c, x) wird `pooling_done` erzeugt und die FSM kehrt nach `S_IDLE` zurück, um auf den nächsten Stripe zu warten. Der FSM-Ablauf ist in Abbildung 4.25 dargestellt.

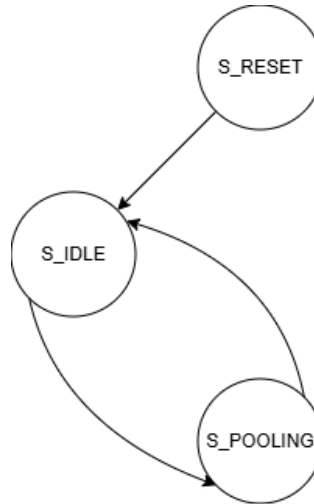


Abbildung 4.25: Pooling-Controller-FSM: S_RESET → S_IDLE (Warten auf start_pool) → S_POOLING (Stripe verarbeiten) → S_IDLE.

Pseudo-Code: Pooling-Unit (Max/Average über 4 Werte). Die Pooling-Unit berechnet pro Takt genau einen gepoolten Wert aus vier Eingängen $v_{00}, v_{01}, v_{10}, v_{11}$. Dabei entspricht `cfg_pool_max_not_avg` dem Laufzeitbit `use_max`.

```

function pooled_val = pool_unit(v00,v01,v10,v11, use_max):
    if use_max == 1:
        return max( max(v00,v01), max(v10,v11) )
    else:
        # Average-Pooling (arithmetisch, signed)
        return (v00 + v01 + v10 + v11) >>> 2
  
```

Listing 4.1: Pseudo-Code der Pooling-Unit (kombinatorisch).

Rückschreiben der gepoolten Werte in die Write-Bank (Adressierung wie zuvor). Die gepoolten Ausgaben werden in S_POOLING pro Takt direkt in die durch `cfg_write_bank_sel` ausgewählte Write-Bank geschrieben (über den gemeinsamen Write-Port bzw. I/O-SRAM-Multiplexer). Die Adressbildung bleibt konsistent zur Notation aus Unterabschnitt 4.6.2:

$$ch = \text{base_ch} + c, \quad x_{\text{out}} = x, \quad y_{\text{out}} = \left\lfloor \frac{\text{stripe_y}}{\text{POOL_STRIDE}} \right\rfloor.$$

Damit ist die Kanalzuordnung identisch zum Store-Pfad ($\text{base_ch} + c$), während die räumliche Auflösung durch Pooling reduziert wird ($x_{\text{out}} \in [0, \lfloor \text{cfg_out_w}/\text{POOL_STRIDE} \rfloor - 1]$, $y_{\text{out}} \in [0, \lfloor \text{cfg_out_h}/\text{POOL_STRIDE} \rfloor - 1]$). Pro Takt wird höchstens ein Write erzeugt (`of_we=1`); die Gültigkeit wird durch die Laufzeitgrenzen ($x < \text{pool_w_rt}$, $c < \text{cfg_tile_c}$, $ch < \text{cfg_cout}$) sowie die Stripe-Startbedingung abgesichert. Nach `pooling_done` kehrt der Controller nach S_IDLE zurück und wartet auf den nächsten `start_pool`-Impuls.

5 Validierung

5.1 Validierungsstrategie

Die Validierung des KI-Beschleunigers erfolgt stufenweise und kombiniert *Modultests*, einen *Integrationstest* sowie eine abschließende *End-to-End*-Validierung des vollständigen MobileNetV1-Datenpfads. Ziel ist es, zunächst die funktionale Korrektheit einzelner Bausteine (Recheneinheit, Aktivierung, Pooling) isoliert nachzuweisen, anschließend das Zusammenspiel der zentralen Datenpfade in einer mehrschichtigen Inferenzkette zu prüfen und zuletzt die Gesamtfunktionalität über alle 27 Layer bis einschließlich Global Average Pooling zu bestätigen.

Im ersten Schritt werden die Kernmodule des Datenpfads mit dedizierten Testbenches verifiziert (Abschnitt 5.2). Darauf aufbauend wird in einem Integrationstest die korrekte Ausführung der drei Betriebsmodi anhand der ersten drei Layer (Convolution → Depthwise → Pointwise) nachgewiesen (Abschnitt 5.3). Abschließend wird der komplette MobileNetV1-Datenpfad mittels einer End-to-End-Ausführung validiert (Abschnitt 5.4). Um die Parametrisierbarkeit der Architektur abzusichern, wird diese Gesamtvalidierung für zwei Konfigurationen des systolischen Arrays (4×4 und 8×8) durchgeführt. Die Korrektheit wird durch bitgenaue HW-SW-Vergleiche der pro Layer erzeugten Featuremap-Dumps im HWC-Layout belegt.

5.2 Validierung der Teilfunktionalitäten

In diesem Abschnitt werden die Modultests beschrieben, die einzelne Funktionsblöcke des Beschleunigers isoliert verifizieren. Verwendet wurden die folgenden Testbenches:

- `processing_element_module_tb.sv`
- `relu_module_tb.sv`
- `pooling_module_tb.sv`

processing_element_module_tb.sv Diese Testbench validiert das `processing_element_module` als kleinste Recheneinheit des systolischen Arrays. Getestet wird der vollständige Datenpfad einer *Multiply-Accumulate*-Berechnung (MAC) über eine feste Kernel-Länge von $KE = 5$. Pro Takt wird jeweils ein Eingabewert der Feature-Map zusammen mit dem zugehörigen Gewichtswert des Filters eingespeist. Die Recheneinheit multipliziert beide Werte, akkumuliert die Produkte über alle KE Schritte und addiert zusätzlich einen Bias-Term.

Die Wahl von $KE = 5$ ist eine **gezielt kompakte Testlänge** für die isolierte Verifikation des PE-Datenpfads. Mit $KE = 5$ werden alle relevanten arithmetischen Mechanismen des Moduls vollständig abgedeckt: (i) wiederholte Produktbildung und Akkumulation über mehrere Takte, (ii)

Bias-Addition zu Beginn der Sequenz, sowie (iii) korrekte Behandlung von Vorzeichenwechseln und Zwischenergebnissen im Akkumulator.

Im Fokus stehen dabei insbesondere die folgenden arithmetischen Prüfpunkte:

- **Sättigung im Akkumulatorraum:** Korrektes Verhalten bei Über-/Unterlauf durch Begrenzung auf den zulässigen 32-Bit-Bereich.
- **Requantisierung:** Abbildung des akkumulierten Ergebnisses in den 8-Bit-Ausgabebereich mittels arithmetischem Rechts-Shift mit Rundung.
- **Sättigung auf `int8`:** Begrenzung der Ausgabe auf den Wertebereich $[-128, 127]$.

Die Testbench umfasst mehrere vordefinierte Testfälle, um Normalbetrieb und Randbedingungen gezielt abzudecken:

- **Normalfall:** Gemischte positive/negative Eingabe- und Gewichtswerte mit moderatem Bias zur Prüfung der regulären MAC-Akkumulation.
- **Positiver Grenzfall:** Sehr großer positiver Bias (und große Produkte), um eine Sättigung im positiven 32-Bit-Bereich auszulösen und anschließend eine Ausgabe nahe $+127$ zu erzwingen.
- **Negativer Grenzfall:** Sehr großer negativer Bias (und große negative Beiträge), um eine Sättigung im negativen 32-Bit-Bereich auszulösen und anschließend eine Ausgabe nahe -128 zu erzwingen.
- **Gezielter Mittelwert:** Nullprodukte und ein so gewählter Bias, dass nach Requantisierung ein definierter mittlerer `int8`-Wert entsteht (z. B. $+50$).
- **Maximalwerte `int8`:** Bias-Wahl zur Erzeugung von $+127$ bzw. -128 nach Requantisierung inklusive finaler `int8`-Sättigung.

Für den Soll-Ist-Vergleich wird innerhalb der Testbench eine bitgenaue Referenzberechnung ausgeführt. Diese Referenz modelliert denselben Ablauf wie die Hardware (Produktbildung, Akkumulation über KE, Sättigung im Akkumulatorraum, Rundungs-Shift zur Skalierung und Sättigung auf `int8`). Das erwartete Ergebnis wird mit dem simulierten Ausgangswert verglichen, sobald die Ausgabe als gültig markiert ist.

relu_module_tb.sv Diese Testbench validiert die ReLU-Funktionalität im Zusammenhang mit dem Top-Level-Datenpfad, d. h. sie prüft, ob nach Abschluss der Faltungsakkumulation negative Ergebnisse zuverlässig auf Null gesetzt werden, während nichtnegative Werte unverändert passieren. Dazu wird ein kleines, vollständig kontrollierbares Szenario aufgebaut: Es werden mehrere $3 \times 3 \times 3$ -Fenster (RGB) über vier räumliche Positionen verarbeitet (2×2 Fensterpositionen), und es werden gleichzeitig zwei Filterspalten berechnet. Die Gewichte sind so gewählt, dass eine Filterspalte eine ausschließlich positive Summe erzeugt, während die zweite Filterspalte gezielt eine negative Summe erzeugt. Dadurch lässt sich das erwartete Verhalten der ReLU-Ausgabe eindeutig überprüfen.

Die Validierung erfolgt zweistufig:

- **Vergleich vor der Aktivierung (Faltungsausgabe):** Nach Abschluss der reinen Akkumulation werden die resultierenden Faltungswerte mit einer in der Testbench berechneten Referenzmatrix verglichen. Dadurch wird sichergestellt, dass die vorherige Arithmetik (Multiplikation, Summation, Bias-Addition) korrekt arbeitet.

- **Vergleich nach der Aktivierung (ReLU-Ausgabe):** Anschließend wird geprüft, ob die Aktivierungsfunktion die zuvor berechneten Werte korrekt abbildet, d. h. $y = \max(0, x)$. Negative Faltungswerte müssen exakt zu 0 werden, während positive Werte unverändert bleiben.

Die Referenzwerte werden direkt in der Testbench aus den definierten Bilddaten, den Filtergewichten und den Bias-Terms berechnet. Für jede Fensterposition wird die vollständige $3 \times 3 \times 3$ -Summe gebildet; daraus entstehen die erwarteten Faltungsausgaben sowie die daraus abgeleiteten ReLU-Ergebnisse. Der Soll-Ist-Vergleich erfolgt elementweise über alle betrachteten Fensterpositionen und beide Filterspalten. Die Simulation gilt als bestanden, wenn sowohl die Faltungsausgabe als auch die ReLU-Ausgabe vollständig mit den Referenzwerten übereinstimmen und die Abschluss-Signalisierung korrekt erfolgt.

pooling_module_tb.sv Diese Testbench validiert das `pooling_module` und prüft die korrekte kanalweise Verdichtung der Feature-Maps nach der Faltungsberechnung. Im Fokus steht ein 2×2 -Pooling-Fenster mit Schrittweite 2, wobei beide in der Hardware vorgesehenen Varianten abgedeckt werden: *Max-Pooling* und *Average-Pooling*. Damit wird sichergestellt, dass die Pooling-Logik funktional korrekt arbeitet und die resultierende Ausgabentensorform den Erwartungen entspricht.

Nach Abschluss der Simulation wird die resultierende Feature-Map aus der **I/O-SRAM** ausgelesen und als Binärdatei im **HWC-Layout** gespeichert. Dieser Dump dient als Referenzausgabe der Hardware und wird anschließend softwareseitig verifiziert.

Zur bitgenauen Ergebnisvalidierung werden zwei Python-Skripte verwendet:

- **validate_avgpool.py:** Bildet das 2×2 *Average-Pooling* mit Stride 2 softwareseitig nach und vergleicht die resultierenden Werte elementweise mit dem aus der I/O-SRAM gedumpten HWC-Binärfile.
- **validate_maxpool.py:** Bildet das 2×2 *Max-Pooling* mit Stride 2 nach und führt ebenfalls einen elementweisen Vergleich gegen den HWC-Dump der Hardwareausgabe durch.

Die Simulation gilt als bestanden, wenn die Pooling-Ausgaben vollständig mit den Software-Referenzen übereinstimmen; damit ist die korrekte Funktion des Pooling-Blocks unabhängig von später beschriebenen Systemaspekten nachgewiesen.

5.3 Validierung der drei Betriebsmodi

In diesem Abschnitt wird die Systemvalidierung der drei Betriebsmodi beschrieben, bei der mehrere Architekturbausteine zusammenwirken und die ersten drei Layer als zusammenhängende Inferenzkette ausgeführt werden (Layer1: Convolution, Layer2: Depthwise, Layer3: Pointwise). Verwendet wurde die folgende Testbench:

- `first_three_layer.sv`

first_three_layer.sv Die Testbench `first_three_layer.sv` verknüpft die ersten drei Schichten des Beschleunigers zu einem zusammenhängenden Inferenzablauf und übernimmt dabei die Rolle einer „CPU“: Sie stellt die benötigten Modelldaten bereit, initialisiert die Speicherbereiche

und orchestriert die Ausführung der Layer in der richtigen Reihenfolge (Layer1: Convolution, Layer2: Depthwise, Layer3: Pointwise).

Für diesen Ablauf werden sämtliche Eingabe- und Parameterdaten der drei Layer im Testbench-Setup vorbereitet:

- **Eingabebild (Inference-Input):** Das gepaddete Eingabebild wird als Binärdatei bereitgestellt und dient als Ausgangsdatenbasis für die Berechnung der ersten Schicht.
- **Gewichte pro Layer:** Die Filtergewichte der Layer1–Layer3 werden in einer *kombinierten* Weight-Datei zusammengeführt. Jeder Layer belegt darin einen eigenen, fest definierten Speicherbereich.
- **Bias pro Layer:** Analog dazu werden die Bias-Vektoren aller drei Layer in einer gemeinsamen Bias-Datei gespeichert, ebenfalls mit separaten Bereichen pro Layer.
- **Requant-Shift-Faktoren pro Layer:** Die pro Output-Kanal benötigten Shift-Werte für die Requantisierung werden in einer gemeinsamen Shift-Datei abgelegt, wiederum mit festen Offsets je Layer.
- **Requant-Multiplier pro Layer:** Zusätzlich werden die pro Output-Kanal benötigten Multiplikationsfaktoren in einer gemeinsamen Mult-Datei gespeichert, ebenfalls mit festen Offsets je Layer.

Die Testbench emuliert diese Datenbereitstellung über ein read-only **ROM**. Dazu werden die einzelnen Dateien im ROM hinterlegt und über definierte Basisadressen im Adressraum abgebildet:

- **Basisadressen:** IMG_BASE, WGT_BASE, BIAS_BASE, SHIFT_BASE und MULT_BASE markieren die Startadressen der jeweiligen Datenregionen im ROM.
- **Layer-spezifische Offsets:** Innerhalb der kombinierten Weight-/Bias-/Shift-/Mult-Dateien besitzt jeder Layer einen festen Offset (L1_*_OFF, L2_*_OFF, L3_*_OFF), der den Beginn der zugehörigen Daten beschreibt.
- **Layer-spezifische Datenlängen:** Zusätzlich sind die exakten Byte-Längen pro Layer (L1_*_BYTES, L2_*_BYTES, L3_*_BYTES) bekannt und dienen als Grundlage für die korrekte Übertragung der Datenblöcke.

Diese Offsets und Byte-Längen sind insbesondere für den späteren Ablauf entscheidend, da sie dem **DMA-Controller** erlauben, die benötigten Daten der jeweiligen Schicht aus dem ROM gezielt und vollständig in die internen SRAM-Strukturen des Beschleunigers zu übertragen. Dadurch kann jeder Layer mit den zugehörigen Gewichten, Bias-, Shift- und Multiplier-Parametern ausgeführt werden, ohne dass die Testbench selbst die Rechenschritte implementieren muss.

AXI-Schnittstellen, CSR-Programmierung und Laufzeitmessung in first_three_layer.sv

Für die Systemvalidierung nutzt `first_three_layer.sv` drei zentrale Schnittstellenblöcke, die die bereits in Unterabschnitt 4.2.1 beschriebenen Konzepte (AXI4-Lite zur Steuerung sowie AXI4-Master für Datentransfers) in einer vollständigen Ausführungssequenz zusammenführen. Die Testbench übernimmt dabei die Rolle einer „CPU“: Sie programmiert die CSR, stößt die DMA-Transfers an, wartet auf den Abschluss eines Layer-Laufs und extrahiert die Ergebnisse.

- **AXI4-Lite (Testbench → DUT):** Die Signale S_AW*, S_W*, S_B*, S_AR* und S_R* bilden die AXI4-Lite-Verbindung zur Registerschnittstelle des DUT. Über diese Schnittstelle werden die in Tabelle 4.1 definierten Registerfelder beschrieben, insbesondere

CSR_CONTROL (Betriebsmodusbits), die Konfigurationsregister CSR_CFG_* sowie die DMA-Register CSR_DMA_*_*.

Die Testbench kapselt die AXI4-Lite-Schreibtransaktionen in der Task `axi_lite_write_byteaddr`. Diese implementiert die notwendigen Handshakes auf AW- und W-Kanal (Warten auf AWREADY bzw. WREADY) und schließt die Transaktion erst nach Empfang der Write-Response über den B-Kanal ab. Dadurch wird sichergestellt, dass CSR-Schreibzugriffe deterministisch und *byte-adressiert* erfolgen, wie es die Registerabbildung in Tabelle 4.1 voraussetzt.

Für die Programmierung des Modusregisters CSR_CONTROL wird eine Hilfsfunktion `pack_control_ext` verwendet. Diese kodiert die einzelnen Laufzeitmodi in ein 32-bit Steuerwort (Bitbelegung gemäß Tabelle 4.1):

- Bit0: `ctrl_relu_en`
- Bit1: `ctrl_pool_en`
- Bit2: `ctrl_dw_en`
- Bit3: `ctrl_pool_max_not_avg`
- Bit4: `ctrl_pw_en`

Damit kann die Testbench pro Layer gezielt zwischen normaler Faltung, Depthwise- und Pointwise-Modus sowie optionalen Post-Operationen umschalten, ohne die Bitfelder manuell setzen zu müssen.

- **AXI4-Master Read (DUT → ROM):** Die Signale `M_AR*` und `M_R*` bilden die AXI4-Master-Leseverbindung, über die der DMA-Controller die im ROM abgelegten Datenblöcke (Input, Weights, Bias, Shift, Mult) abrufen.
- **AXI4-Master Write (optional, DUT → Testbench-Stub):** Zusätzlich sind mit `M_AW*`, `M_W*` und `M_B*` die AXI4-Schreibkanäle vorhanden, die den optionalen Write-Back-Pfad abdecken (siehe Unterabschnitt 4.2.1).

Simulationsausgabe der Testbench `first_three_layer.sv` Der Ablauf der Testbench wird über einen formatierten Log-Auszug im Dokument dokumentiert. Listing 5.1 zeigt die zentrale Konsolenausgabe der 3-Layer-Kette (Conv → Depthwise → Pointwise) inklusive Bankwahl, Modusbits, Padding (LRTB), ROM-Offsets/Byte-Längen (Weights/Bias/Shift/Mult), Zyklusmessung und erzeugten HWC-Dumps.

Listing 5.1: Konsolenlog der 3-Layer-Ausführung in `first_three_layer.sv` (Conv -> DW -> PW).

```
=====
# -----
# LAYER 1 CONFIG:
#   banks      : read=0 write=1
#   mode       : dw=0 pw=0 pool=0
#   IN         : 128x128  cin=3  cout=32
#   K/STRIDE   : K=3x3  S=2x2
#   PAD LRTB   : 0/1/0/1
#   TILE c/r   : tile_c=4 tile_r=4
#   OUT        : 64x64
#   totals     : wins=4096 grps=1024 blks=8  num_ctx=27
#   pw_beats   : needed=1 (LANES_MAX=4, BEATS_MAX=256)
```

```

# ROM offs : w=0 b=0 s=0 m=0
# ROM bytes : w=864 b=128 s=128 m=128
# -----
# TB: LAYER 1 DONE | cycles=587998
# Wrote OFMAP (HWC) to ../layer_1_ofmap_64x64x32_hwc.bin (bank=1, out=64x64, ch=32)
# -----
# LAYER 2 CONFIG:
# banks      : read=1 write=0
# mode       : dw=1 pw=0 pool=0
# IN         : 64x64 cin=32 cout=32
# K/STRIDE   : K=3x3 S=1x1
# PAD LRTB   : 1/1/1/1
# TILE c/r   : tile_c=4 tile_r=4
# OUT        : 64x64
# totals     : wins=4096 grps=4096 blks=8 num_ctx=9
# pw_beats   : needed=1 (LANES_MAX=4, BEATS_MAX=256)
# ROM offs   : w=864 b=128 s=128 m=128
# ROM bytes  : w=288 b=128 s=128 m=128
# -----
# TB: LAYER 2 DONE | cycles=824266
# Wrote OFMAP (HWC) to ../layer_2_ofmap_64x64x32_hwc.bin (bank=0, out=64x64, ch=32)
# -----
# LAYER 3 CONFIG:
# banks      : read=0 write=1
# mode       : dw=0 pw=1 pool=0
# IN         : 64x64 cin=32 cout=64
# K/STRIDE   : K=1x1 S=1x1
# PAD LRTB   : 0/0/0/0
# TILE c/r   : tile_c=4 tile_r=1
# OUT        : 64x64
# totals     : wins=4096 grps=4096 blks=16 num_ctx=32
# pw_beats   : needed=8 (LANES_MAX=4, BEATS_MAX=256)
# ROM offs   : w=1152 b=256 s=256 m=256
# ROM bytes  : w=2048 b=256 s=256 m=256
# -----
=====

```

Interpretation des Log-Auszuges Der Log-Auszug in Listing 5.1 belegt die sequentielle Ausführung der drei Layer. Pro Layer werden (i) Bankwahl, Modusbits und Padding (LRTB) ausgegeben, (ii) die ROM-Offets sowie Byte-Längen innerhalb der kombinierten ROM-Dateien (Weights/-Bias/Shift/Mult) ausgewiesen, (iii) die Laufzeit in Zyklen gemessen und (iv) die resultierende Output-Featuremap als Binärdump im HWC-Layout exportiert.

Software-Validierung der Layer-Dumps (Unit-Tests, HW–SW Vergleich) Die durch `first_three_layer.sv` erzeugten Featuremap-Dumps:

- `layer_1_ofmap_64x64x32_hwc.bin`
- `layer_2_ofmap_64x64x32_hwc.bin`
- `layer_3_ofmap_64x64x64_hwc.bin`

werden anschließend softwareseitig mittels *Unit-Tests* bitgenau gegen eine Referenzimplementierung in Python validiert. Jeder Unit-Test bildet exakt einen Layer in Software nach und vergleicht die

resultierende Output-Featuremap elementweise mit dem Dump aus der Simulation. Die Python-Modelle replizieren dabei explizit die RTL-Arithmetik (Sättigung im `int32`-Akkumulator, Rundung beim arithmetischen Rechts-Shift sowie Sättigung auf `int8`) und berücksichtigen zusätzlich die pro Output-Kanal geladenen Requant-Parameter (Bias, Shift und Multiplier), sodass der Vergleich als *bitgenauer Korrektheitsnachweis* der Hardwareausgabe dient.

- **Unit-Test Layer1 (Convolution, 3×3 , Stride 2, Padding 0/1/0/1):** Der Test lädt das gepaddete Eingabebild im HWC-Layout sowie den Hardware-Dump `layer_1_ofmap_64x64x32_hwc.bin`. Die Gewichte sowie die Requant-Parameter (Bias, Shift, Multiplier) werden aus den kombinierten ROM-Dateien anhand der im Testbench-Setup ausgegebenen Offsets und Byte-Längen extrahiert (ROM offs/bytes für w, b, s, m). Anschließend wird die Faltung für alle Output-Kanäle in Software berechnet und pro Output-Element mit der RTL-identischen Requantisierung (Multiplier $\rightarrow$ Shift $\rightarrow$ Sättigung) in den `int8`-Bereich abgebildet. Abschließend erfolgt ein elementweiser Vergleich zwischen Software-Output und Hardware-Dump.
- **Unit-Test Layer2 (Depthwise, 3×3 , Stride 1, Padding 1/1/1/1):** Als Input wird der Dump des vorherigen Layers geladen (`layer_1_ofmap_64x64x32_hwc.bin`), während der Hardware-Output aus `layer_2_ofmap_64x64x32_hwc.bin` stammt. Gewichte sowie Bias/Shift/Multiplier werden wieder layer-spezifisch aus dem ROM extrahiert. Die Weight-Rekonstruktion ordnet jedem Kanal c genau ein 3×3 -Filter zu ($C_{in} = C_{out} = 32$). Danach wird die Depthwise-Faltung im SAME-Padding softwareseitig berechnet und mit der gleichen RTL-Requantisierung (inkl. Multiplier, Shift-Klemmung und Rundung) in `int8` überführt. Der HW–SW Vergleich erfolgt elementweise für alle (y, x, c) .
- **Unit-Test Layer3 (Pointwise, 1×1 , Stride 1, Padding 0/0/0/0):** Als Input dient der Dump des Depthwise-Layers `layer_2_ofmap_64x64x32_hwc.bin`, während der Hardware-Output aus `layer_3_ofmap_64x64x64_hwc.bin` stammt. Die Parameter (Weights/Bias/Shift/Multiplier) werden konsistent zu den im Log ausgewiesenen ROM-Segmenten geladen. Anschließend wird die 1×1 -Faltung softwareseitig nachgebildet und die vollständige Output-Featuremap elementweise gegen den Hardware-Dump verglichen.

Durch diese Unit-Tests wird die korrekte Ausführung der drei Betriebsmodi zusätzlich abgesichert: Jeder Layer verwendet als Eingang genau den zuvor erzeugten Hardware-Dump, und die zugehörigen Parameter (Weights, Bias, Shift, Multiplier) werden konsistent zu den im ROM abgelegten Segmenten (Offsets/Byte-Längen) geladen.

Testbedingungen: Das aktuelle Weight-Fetching unterstützt keine Maskierung für teilweise belegte Spaltenkacheln. Daher müssen die Tests mit vollständig genutzter Array-Breite ausgeführt werden, d.h. `cfg_tile_c = COLS_MAX`. Zudem ist `COLS_MAX` als gerader Wert (Vielfaches von 2, beginnend bei 4) zu wählen.

Zusätzlich ergeben sich Einschränkungen für `cfg_tile_r` in Abhängigkeit vom Betriebsmodus und der Array-Variante. Bei der Standard-Convolution im ersten Layer gilt `cfg_cin = 3`; daher muss `cfg_tile_r` mindestens 3 sein, damit alle drei Eingangskanäle gleichzeitig verarbeitet werden. Für die 4×4 -Variante ist damit `cfg_tile_r` $\in \{3, 4\}$ zulässig, für die 8×8 -Variante `cfg_tile_r` $\in [3, 8]$. In diesem Fall werden entsprechend `cfg_tile_r` Line-/Window-Pfade (Line Buffers und Window-RAMs) genutzt; bei `cfg_tile_r = 3` werden beispielsweise drei parallele Window-RAMs verwendet, was für die drei Kanäle ausreichend ist. Eine Konfiguration mit `cfg_tile_r < cfg_cin` (z.B. `cfg_tile_r = 2`) wird in der aktuellen Implementierung nicht

unterstützt, da hierbei eine Verarbeitung in mehreren Teilgruppen mit späterer Partial-Summen-Akkumulation erforderlich wäre.

Für Depthwise-Convolution gelten folgende Randbedingungen: Da jedem Eingangskanal genau ein eigener Filter zugeordnet ist, werden die Lane- und Spaltenparallelität vollständig ausgenutzt; entsprechend wird $\text{cfg_tile_r} = \text{LANES_MAX}$ und $\text{cfg_tile_c} = \text{COLS_MAX}$ (jeweils 4 bzw. 8) verwendet. Im Pointwise-Modus wird in der aktuellen Implementierung jeweils nur eine feste räumliche Position (x, y) pro Ausgabeoperation in das Array eingespeist; daher ist $\text{cfg_tile_r} = 1$ gesetzt.

In der Validierung wurden sowohl *symmetrisches Padding* (pad_{LRTB} links/rechts und oben/unten gleich, z. B. 1/1/1/1) als auch *asymmetrisches Padding* (unterschiedliche Werte je Seite, z. B. 0/1/0/1) in Kombination mit verschiedenen Strides betrachtet. Getestet wurden die Konfigurationen: (i) $S = 1 \times 1$ mit $\text{pad}_{LRTB} = 1/1/1/1$, (ii) $S = 2 \times 2$ mit $\text{pad}_{LRTB} = 0/1/0/1$ sowie (iii) $S = 1 \times 1$ mit $\text{pad}_{LRTB} = 0/0/0/0$ (kein Padding für Pointwise 1×1).

Zur zusätzlichen Robustheitsprüfung wurden vier weitere Kombinationen aus Stride und Padding (L/R/T/B) getestet, die nicht Teil der MobileNetV1-Konfiguration sind. Die resultierenden OFMAPs wurden jeweils mit dem Python-Validierungsskript gegen die HW-Ausgaben bitgenau verglichen; alle vier Tests stimmen vollständig überein. Damit ist der KI-Beschleuniger auch fuer diese zusätzlichen Kombinationen verifiziert.

Tabelle 5.1: Zusätzlich getestete Stride-/Padding-Kombinationen (außerhalb MobileNetV1).

Stride S	Padding (L/R/T/B)
$S = 3$	2/2/2/2
$S = 3$	2/3/2/3
$S = 4$	2/2/2/2
$S = 4$	3/2/3/2

5.4 Gesamtvalidierung des MobileNetV1-Datenpfads

In diesem Abschnitt wird die **Gesamtfunktionalität des implementierten MobileNetV1-Datenpfads** durch eine End-to-End-Ausführung über **alle 27 Layer bis einschließlich Global Average Pooling** validiert. **Zur Absicherung der Parametrisierbarkeit** wurde die End-to-End-Validierung dabei für zwei Konfigurationen des systolischen Arrays durchgeführt, nämlich für eine 4×4 - sowie eine 8×8 -Variante. Damit wird gezeigt, dass die Korrektheit der Datenpfade und der **bitgenauen RTL-Arithmetik** unabhängig von der gewählten Arraygröße erhalten bleibt.

Testbench: `ki_beschleuniger_tb.sv` Die Testbench `ki_beschleuniger_tb.sv` orchestriert die komplette Ausführung der 27 Layer. Dazu initialisiert sie die Speicherbereiche, programmiert die Konfigurationsregister, startet die DMA-Transfers aus einem read-only ROM und misst die Laufzeit pro Layer. Nach Abschluss jedes Layers wird die resultierende Output-Featuremap als **Binärdump im HWC-Layout** aus den Feature-Bänken exportiert.

Die Testbench arbeitet dabei mit **kombinierten ROM-Dateien** für die Modellparameter. Pro Layer werden über feste Offsets und Byte-Längen die benötigten Parametersegmente geladen:

- **Weights** (rom_weights_all.bin)
- **Bias** (rom_bias_all.bin)
- **Requant-Shift** (rom_shift_all.bin)
- **Requant-Multiplier** (rom_mult_all.bin)

Damit ist sichergestellt, dass jede Schicht mit konsistenten Gewichten sowie den zugehörigen Requant-Parametern (Bias, Shift, Multiplier) ausgeführt wird.

Software-Validierung: Die durch `ki_beschleuniger_tb.sv` erzeugten Layer-Dumps werden anschließend mit dem Python-Skript `validate_mobilenet_v1.py` validiert. Das Skript bildet den gesamten Netzwerkpfad **layerweise** nach und vergleicht nach jedem Layer die berechnete Software-Featuremap elementweise mit dem entsprechenden Hardware-Dump.

Die Referenzimplementierung repliziert explizit die **RTL-Arithmetik** der Processing-Elemente, inklusive:

- Akkumulation mit definierter Wortbreite und Sättigungsverhalten,
- Requantisierung unter Verwendung von **Multiplier** und **Shift**,
- Ausgabequantisierung auf `int8` sowie
- Berücksichtigung des pro Layer gesetzten **Padding** und der Stride-Konfiguration.

Als Ergebnis erzeugt `validate_mobilenet_v1.py` eine Zusammenfassung und liefert damit einen **bitgenauen Korrektheitsnachweis** der Hardwareausgabe für die vollständige 27-Layer-Kette.

6 Evaluation

Nachdem in Kapitel 5 die funktionale Korrektheit des KI-Beschleunigers nachgewiesen wurde, erfolgt in diesem Kapitel die quantitative Bewertung der Architektur. Ziel der Evaluation ist es, die hardwarebezogenen Kenngrößen des Beschleunigers systematisch zu analysieren. Im Fokus stehen dabei insbesondere *Fläche*, *Timing* und *Performance*, da diese Metriken den zentralen Zielkonflikt zwischen Ressourceneinsatz und erreichbarem Durchsatz abbilden und somit eine Einordnung der Architektur hinsichtlich Effizienz und Skalierbarkeit ermöglichen. Darüber hinaus wird untersucht, wie sich unterschiedliche Array-Konfigurationen auf die Ressourcennutzung, das Timingverhalten und die resultierende Rechenleistung auswirken.

6.1 Front-End-Synthese

Die Synthese der Hardware-Implementierung wurde mit *Synopsys Design Compiler* (Version W-2024.09-SP1) in der ASAP7 7 nm Standard-Cell-Technologie durchgeführt. Als Bibliotheken wurden `asap7sc7p5t_LVT/RVT/SLVT` aus der ASAP7 Standard-Cell-Library im *TT*-Corner verwendet; die Operating-Condition ist `PVT_0P7V_25C` bei einem Clock-Constraint von $T_{\text{clk}} = 600$ ps.

Die ausgewiesene Zellfläche (*Total cell area*) wird in μm^2 berichtet und zusätzlich in mm^2 umgerechnet. Dedizierte Memory-/SRAM-IP-Makros sind in dieser Synthese nicht enthalten: Die im KI-Beschleuniger verwendeten SRAM-Module sind als Black-Boxes modelliert. Somit beziehen sich die angegebenen Flächen- und Timingwerte ausschließlich auf die Standard-Cell-Logik (ohne On-Chip-Speicher-Makros).

Die Syntheseresultate werden für das parametrisierbare Systolic-Array in zwei Array-Konfigurationen (4×4 und 8×8) dargestellt.

6.1.1 Konfigurationen

Für die nachfolgenden Syntheseläufe wird ausschließlich **MobileNetV1** mit $\alpha = 1$ betrachtet. Die Eingabegröße beträgt 128×128 ; in der Testbench wird vor dem ersten Layer ein Padding angewendet, sodass intern mit 130×130 gearbeitet wird.

Tabelle 6.1: Parameterübersicht für die Synthese (128×128). In den Spalten der Array-Größen ändern sich ausschließlich LANES_MAX, COLS_MAX sowie die daraus abgeleiteten Maximalwerte; alle übrigen Parameter bleiben konstant.

Parameter	4×4	8×8
LANES_MAX	4	8
COLS_MAX	4	8
IMG_W_MAX	130	130
IMG_H_MAX	128	128
COUT_MAX	1024	1024
CIN_MAX	1024	1024
K_MAX	3	3
STRIDE_MAX	2	2
OUT_W_MAX	64	64
OUT_H_MAX	64	64
TOTAL_WINS_MAX	4096	4096
TOTAL_GRPS_MAX	4096	4096
TOTAL_BLKES_MAX	256	128
NUM_CTX_MAX	1024	1024
BEATS_MAX	256	128

6.1.2 Flächenergebnisse

Wie in Tabelle 6.2 dargestellt, wurden die Syntheseeergebnisse für zwei Systolic-Array-Konfigurationen (4×4 und 8×8) hinsichtlich *Total cell area* ausgewertet.

Tabelle 6.2: Flächenergebnisse für verschiedene Array-Konfigurationen. Die Fläche ist als *Total cell area* in μm^2 und in mm^2 angegeben.

Konfiguration	Area [μm^2]	Area [mm^2]
4×4	19725.136	0.019725
8×8	49578.692	0.049579

Aus den Synthesewerten ergibt sich eine Skalierung von:

$$\frac{A_{8 \times 8}}{A_{4 \times 4}} \approx \frac{49578.692}{19725.136} \approx 2.51.$$

Damit steigt die Zellfläche bei größerer Array-Konfiguration deutlich, jedoch nicht proportional zur reinen Anzahl der Processing-Elemente $R \cdot C$ (hier: $16 \rightarrow 64$, also $\times 4$). Im Folgenden wird daher untersucht, welche Modulanteile mit LANES_MAX/COLS_MAX skalieren.

Beitrag des Processing Module: Ein wesentlicher skalierender Anteil im *Processing Module* entsteht im Pooling-Datenfluss. Wie in Unterabschnitt 4.6.3 beschrieben, werden bei aktiviertem Pooling die Ergebnisse des *Arithmetic Module* zunächst in zwei zeilenweise Puffer `pool_line0/1` geschrieben und erst anschließend stripeweise zu 2×2 -Werten reduziert (vgl. Abbildung 4.23).

Für jeden lokalen Kanaloffset c innerhalb eines Blocks müssen dabei zwei vollständige Zeilen vorgehalten werden. Die Anzahl der parallel geführten Pooling-Lines ist damit:

$$N_{\text{pool_lines}} = 2 \cdot \text{COLS_MAX}, \quad \Delta N_{\text{pool_lines}} = 2 \cdot (\Delta \text{COLS_MAX}).$$

Daraus folgt ein näherungsweise linearer Flächenanteil der Pufferstruktur:

$$A_{\text{pool_lines}} \propto 2 \cdot \text{COLS_MAX} \cdot \text{IMG_W_MAX}.$$

Für die in dieser Arbeit ausgewerteten Synthesen wurde der Pooling-Zeilenspeicher bereits hardwareseitig reserviert, wobei im hier betrachteten Aufbau eine feste Pufferbreite von zwei Elementen verwendet wurde ($\text{POOL_W_MAX}=2$, entsprechend einer 2×2 -Pooling-Reduktion). Die Puffer werden dabei als `pool_line0` und `pool_line1` realisiert und besitzen die Dimension $[0:\text{COLS_MAX}-1][0:\text{POOL_W_MAX}-1]$. Damit skaliert der Speicher- und Kontrollaufwand der Pooling-Logik direkt mit COLS_MAX und wirkt sich insbesondere bei größeren Array-Konfigurationen sichtbar auf den Flächenbedarf aus.

Konkret entstehen bei $4 \times 4 \rightarrow 8 \times 8$ acht zusätzliche Pooling-Lines ($\Delta N = 2 \cdot (8 - 4) = 8$). Dieser skalierende Speicher- und Kontrollanteil trägt wesentlich zum Anstieg der Flächenwerte über die Array-Konfigurationen hinweg bei.

Beitrag des *Fetching Module*: Ein weiterer skalierender Anteil entsteht im *Fetching Module*. Die Line-Buffer werden dort nicht zur Laufzeit erzeugt, sondern *physisch* gemäß LANES_MAX instanziiert:

$$N_{\text{LB}} = \text{LANES_MAX}.$$

Damit führt eine Vergrößerung der Array-Konfiguration direkt zu mehr Line-Buffer-Instanzen. Für $4 \times 4 \rightarrow 8 \times 8$ steigt LANES_MAX von 4 auf 8, d. h. es werden **vier zusätzliche** Line-Buffer instanziiert. Da pro Lane Fenster-Generierung (z. B. 3×3) inklusive Register- und Steuerlogik benötigt wird, wächst der entsprechende Flächenanteil mit der Lane-Zahl.

Beitrag des *Systolic-Array-Pfads*: Der *Systolic-Array*-Pfad prägt den Flächenbedarf maßgeblich. Das Array wird als zweidimensionale Struktur mit $R = \text{LANES_MAX}$ Zeilen und $C = \text{COLS_MAX}$ Spalten instanziiert; pro Gitterpunkt (r, c) existiert ein `process_element_module`. Damit gilt für die Anzahl physisch instanziierten Processing-Elements:

$$N_{\text{PE}} = \text{LANES_MAX} \cdot \text{COLS_MAX}.$$

Innerhalb eines PEs entsteht der Speicherbedarf im Wesentlichen durch Register, die (i) den lokalen Zustand halten und (ii) die Pipeline-Stufen voneinander entkoppeln. Diese Register sind nicht nur für die Datenwerte relevant, sondern auch für die zugehörigen Steuersignale (Enable/Valid) sowie für pro-Spalte Parameter (Bias, Shift, Multiplier). Im Folgenden wird der registerbasierte Speicherbedarf innerhalb eines Processing Elements stufenweise betrachtet:

- **Stage A:** Der Speicherbedarf wird hier durch den lokalen Akkumulatorzustand bestimmt. Dazu gehören das Akkumulatorregister sowie der Sequenzzähler zur Bestimmung des letzten MAC-Schritts. Zusätzlich werden Durchreichregister für die systolische Kopplung gehalten, um den Datenfluss taktsynchron weiterzugeben.

- **Stage B:** In dieser Stufe entsteht Registerbedarf durch die Zwischenspeicherung der Multiplikationszerlegung. Insbesondere werden die Teilprodukte $P_1..P_4$ am Ausgang der Teilmultiplikatoren registriert, sodass das resultierende 64-bit Produkt in einem nachgelagerten Addierpfad stabil aufgebaut werden kann.
- **Stage C:** Der Speicherbedarf ergibt sich hier aus den Registern, die die Ergebnisse des Addierpfads sowie die Validierung bis zur finalen Ausgabe tragen.

6.1.3 Timing-Ergebnisse

Wie in Tabelle 6.3 dargestellt, wurden die Syntheseergebnisse für zwei Systolic-Array-Konfigurationen (4×4 und 8×8) hinsichtlich der kritischen Pfadverzögerung ausgewertet.

Tabelle 6.3: Timing-Ergebnisse für verschiedene Array-Konfigurationen.

Konfiguration	Timing [ps]
4×4	580.52
8×8	584.82

Die Synthese wurde mit einem Clock-Constraint von $T_{\text{clk}} = 600$ ps durchgeführt. Die zugehörige Ziel-Taktfrequenz beträgt:

$$f_{\text{clk}} = \frac{1}{T_{\text{clk}}} = \frac{1}{600 \text{ ps}} \approx 1.67 \text{ GHz}.$$

Die in Tabelle 6.3 ausgewiesenen Worst-Case-Timingwerte liegen für beide betrachteten Array-Konfigurationen unterhalb dieses Zielwerts (4×4 : 580.52 ps; 8×8 : 584.82 ps). Daraus ergibt sich die entsprechende maximal erreichbare Frequenz (Worst Case) zu:

$$f_{\text{max},4 \times 4} = \frac{1}{580.52 \text{ ps}} \approx 1.72 \text{ GHz}, \quad f_{\text{max},8 \times 8} = \frac{1}{584.82 \text{ ps}} \approx 1.71 \text{ GHz}.$$

Damit erfüllt die Standard-Cell-Logik der untersuchten Varianten das Timingziel von 600 ps.

Die Timing-Analyse zeigt für beide Array-Konfigurationen sehr ähnliche Worst-Case-Verzögerungen (4×4 : 580.52 ps, 8×8 : 584.82 ps). Der kritische Pfad liegt in Stage B des *Processing Elements*, da dort die 32×32 -Multiplikation über **vier 17×17 -Partialprodukte** realisiert und anschließend zu einem Gesamtergebnis zusammengeführt wird. Die nahezu konstante Verzögerung trotz größerem Array deutet darauf hin, dass nicht das globale Routing, sondern die lokale PE-Arithmetik die maximal erreichbare Frequenz begrenzt. Der geringe Anstieg bei 8×8 ist plausibel durch erhöhte Fanout- und Verteilnetze globaler Steuersignale.

6.2 Performanceanalyse

6.2.1 MAC Throughput

Im Folgenden wird der mittlere MAC-Throughput betrachtet. Die Durchsatzmetrik wird *layerweise* mit den jeweils tatsächlich anfallenden Sequenzlängen K_{elems} ausgewertet. Diese ergeben sich

direkt aus den zur Laufzeit konfigurierten Parametern des jeweiligen Layer-Typs ($K_{\text{elems}} = \text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin}$ im Convolution-Layer, $K_{\text{elems}} = \text{cfg_cin}$ im Pointwise-Layer und $K_{\text{elems}} = \text{cfg_kh} \cdot \text{cfg_kw}$ im Depthwise-Layer). Die Umrechnung von Zyklen in physikalische Zeit erfolgt über die Taktfrequenz:

$$f_{\text{clk}} = \frac{1}{T_{\text{clk}}}. \quad (6.1)$$

Durchschnittlicher MAC-Throughput. Während eines Runs führt jedes *aktive* PE pro Takt einen MAC aus. Der Peak-MAC-Throughput beträgt damit:

$$\text{MAC}_{\text{peak}} = N_{\text{PE,active}}, \quad (6.2)$$

wobei $N_{\text{PE,active}}$ modusabhängig ist:

- **Convolution:** $N_{\text{PE,active}} = \text{cfg_tile_r} \cdot \text{cfg_tile_c}$,
- **Pointwise:** $N_{\text{PE,active}} = \text{cfg_tile_c}$,
- **Depthwise:** $N_{\text{PE,active}} = \text{cfg_tile_r}$.

Der über den gesamten Run gemittelte MAC-Throughput ergibt sich aus der Gesamtzahl an MACs ($N_{\text{PE,active}} \cdot K_{\text{elems}}$) dividiert durch die Run-Zyklen:

$$\text{MAC}_{\text{avg}} = \frac{N_{\text{PE,active}} \cdot K_{\text{elems}}}{\text{max_cycles_cur}} \quad [\text{MAC/Zyklus}], \quad (6.3)$$

und zu $\text{MAC/s} = f_{\text{clk}} \cdot \text{MAC}_{\text{avg}}$.

Die folgenden Tabellen geben die resultierenden MAC-Durchsatzwerte für die betrachteten MobileNet-Layer an. Tabelle 6.4 zeigt die Ergebnisse für die 4×4 -Konfiguration, während Tabelle 6.5 die entsprechenden Werte für die 8×8 -Konfiguration zusammenfasst.

Dabei werden ausschließlich Convolution-, Depthwise- und Pointwise-Layer berücksichtigt. Die Werte basieren auf den modusabhängigen Laufzeitmodellen, d. h. der jeweiligen Definition von max_cycles_cur im Convolution-, Pointwise- und Depthwise-Modus (Gleichung 4.22, Gleichung 4.24, Gleichung 4.26), sowie auf den layerabhängigen Sequenzlängen K_{elems} und den zugehörigen Synthese-Taktperioden der 4×4 - bzw. 8×8 -Konfiguration.

Abbildung 6.1 zeigt den mittleren MAC-Throughput pro Layer für 4×4 und 8×8 im direkten Vergleich. Im Convolution-Layer (L1) fällt der Zugewinn des 8×8 -Arrays besonders deutlich aus, da hier die Anzahl aktiver PEs quadratisch mit der Kachelgröße skaliert ($N_{\text{PE,active}} = 64$ statt 16) und sich der größere Parallelitätsgrad direkt in einem höheren MAC/s niederschlägt. In Depthwise-Layern ist der Anstieg geringer, weil im Depthwise-Modus ausschließlich die Diagonale aktiv ist und damit nur $N_{\text{PE,active}} = \text{cfg_tile_r}$ PEs beitragen (linearer Skalierungseffekt). In Pointwise-Layern wird MAC/s primär durch die Kanalakkumulation bestimmt ($K_{\text{elems}} = \text{cfg_cin}$); der Wechsel von 4×4 auf 8×8 erhöht zwar die parallel verarbeiteten Ausgabekanäle, jedoch bleibt der Durchsatzanstieg gegenüber Convolution begrenzt, da die Sequenzlänge mit cfg_cin wächst.

Tabelle 6.4: Layerweiser MAC-Durchsatz für die 4×4 -Konfiguration.

Layer	Typ	K_{elems}	max_cycles	MAC _{avg}	MAC/s [GMAC/s]
L1	Conv 3×3	27	35	12,3429	21,26
L2	Depthwise 3×3	9	11	3,2727	5,64
L3	Pointwise 1×1	32	37	3,4595	5,96
L4	Depthwise 3×3	9	11	3,2727	5,64
L5	Pointwise 1×1	64	69	3,7101	6,39
L6	Depthwise 3×3	9	11	3,2727	5,64
L7	Pointwise 1×1	128	133	3,8496	6,63
L8	Depthwise 3×3	9	11	3,2727	5,64
L9	Pointwise 1×1	128	133	3,8496	6,63
L10	Depthwise 3×3	9	11	3,2727	5,64
L11	Pointwise 1×1	256	261	3,9234	6,76
L12	Depthwise 3×3	9	11	3,2727	5,64
L13	Pointwise 1×1	256	261	3,9234	6,76
L14	Depthwise 3×3	9	11	3,2727	5,64
L15	Pointwise 1×1	512	517	3,9613	6,82
L16	Depthwise 3×3	9	11	3,2727	5,64
L17	Pointwise 1×1	512	517	3,9613	6,82
L18	Depthwise 3×3	9	11	3,2727	5,64
L19	Pointwise 1×1	512	517	3,9613	6,82
L20	Depthwise 3×3	9	11	3,2727	5,64
L21	Pointwise 1×1	512	517	3,9613	6,82
L22	Depthwise 3×3	9	11	3,2727	5,64
L23	Pointwise 1×1	512	517	3,9613	6,82
L24	Depthwise 3×3	9	11	3,2727	5,64
L25	Pointwise 1×1	512	517	3,9613	6,82
L26	Depthwise 3×3	9	11	3,2727	5,64
L27	Pointwise 1×1	1024	1029	3,9806	6,86
Mittelwert über alle Layer				3.9019	6.72

Tabelle 6.5: Layerweiser MAC-Durchsatz für die 8×8 -Konfiguration.

Layer	Typ	K_{elems}	max_cycles	MAC _{avg}	MAC/s [GMAC/s]
L1	Conv 3×3	27	43	40,1860	68,71
L2	Depthwise 3×3	9	11	6,5455	11,20
L3	Pointwise 1×1	32	41	6,2439	10,68
L4	Depthwise 3×3	9	11	6,5455	11,20
L5	Pointwise 1×1	64	73	7,0137	12,00
L6	Depthwise 3×3	9	11	6,5455	11,20
L7	Pointwise 1×1	128	137	7,4745	12,79
L8	Depthwise 3×3	9	11	6,5455	11,20
L9	Pointwise 1×1	128	137	7,4745	12,79
L10	Depthwise 3×3	9	11	6,5455	11,20
L11	Pointwise 1×1	256	265	7,7283	13,22
L12	Depthwise 3×3	9	11	6,5455	11,20
L13	Pointwise 1×1	256	265	7,7283	13,22
L14	Depthwise 3×3	9	11	6,5455	11,20
L15	Pointwise 1×1	512	521	7,8628	13,45
L16	Depthwise 3×3	9	11	6,5455	11,20
L17	Pointwise 1×1	512	521	7,8628	13,45
L18	Depthwise 3×3	9	11	6,5455	11,20
L19	Pointwise 1×1	512	521	7,8628	13,45
L20	Depthwise 3×3	9	11	6,5455	11,20
L21	Pointwise 1×1	512	521	7,8628	13,45
L22	Depthwise 3×3	9	11	6,5455	11,20
L23	Pointwise 1×1	512	521	7,8628	13,45
L24	Depthwise 3×3	9	11	6,5455	11,20
L25	Pointwise 1×1	512	521	7,8628	13,45
L26	Depthwise 3×3	9	11	6,5455	11,20
L27	Pointwise 1×1	1024	1033	7,9322	13,57
Mittelwert über alle Layer				8.2978	14.19

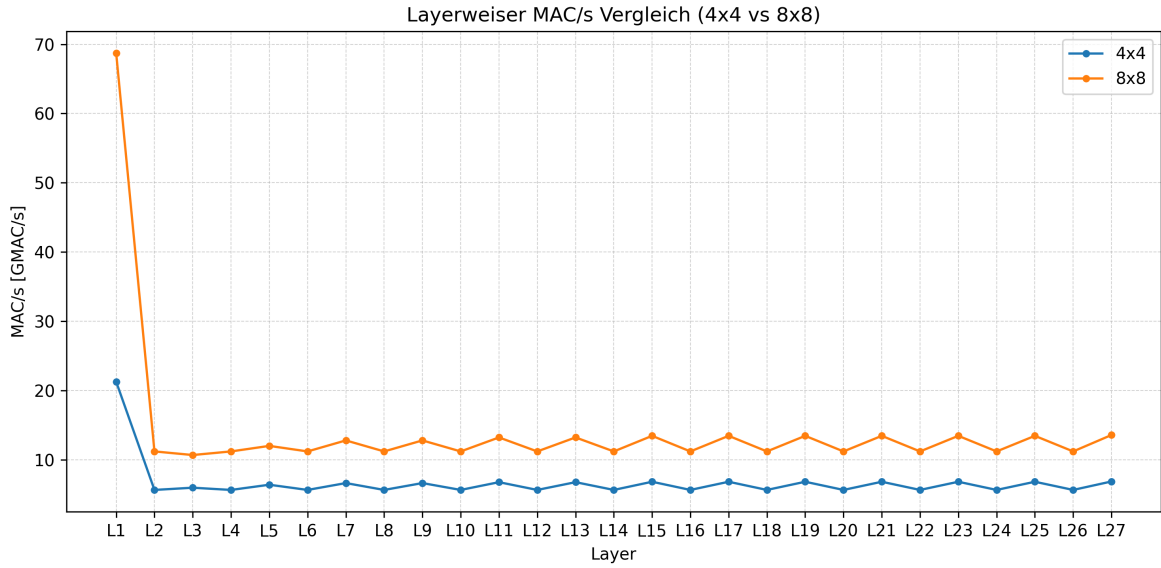


Abbildung 6.1: Layerweiser Vergleich des mittleren MAC-Throughputs (MAC/s) für 4×4 und 8×8 .

6.2.2 Utilization

Neben dem reinen Durchsatz ist die *Utilization* eine zentrale Kenngröße, um zu bewerten, wie effizient die im SA physisch vorhandenen Rechenressourcen tatsächlich genutzt werden. Während der MAC-Throughput den absolut erreichten Rechendurchsatz (MAC/s) angibt, setzt die Utilization die *tatsächlich ausgeführten MAC-Operationen* ins Verhältnis zur *theoretisch maximal möglichen MAC-Kapazität* des Arrays innerhalb der gemessenen Laufzeit. Dadurch lassen sich Overheads sowie ungenutzte Parallelität sichtbar machen und zwischen unterschiedlichen Array-Konfigurationen vergleichbar quantifizieren.

Für einen Layer mit gemessener Laufzeit cycles und einer aktiven PE-Anzahl $N_{\text{PE,active}}$ ergibt sich die maximal mögliche Anzahl an MACs zu $\text{cycles} \cdot N_{\text{PE,active}}$. Die tatsächlich anfallenden MACs werden aus der Layer-Geometrie und den Kernel-/Kanalparametern bestimmt. Damit ist die Utilization definiert als:

$$U = \frac{\text{MAC}_{\text{real}}}{\text{cycles} \cdot N_{\text{PE,active}}} \in [0, 1]. \quad (6.4)$$

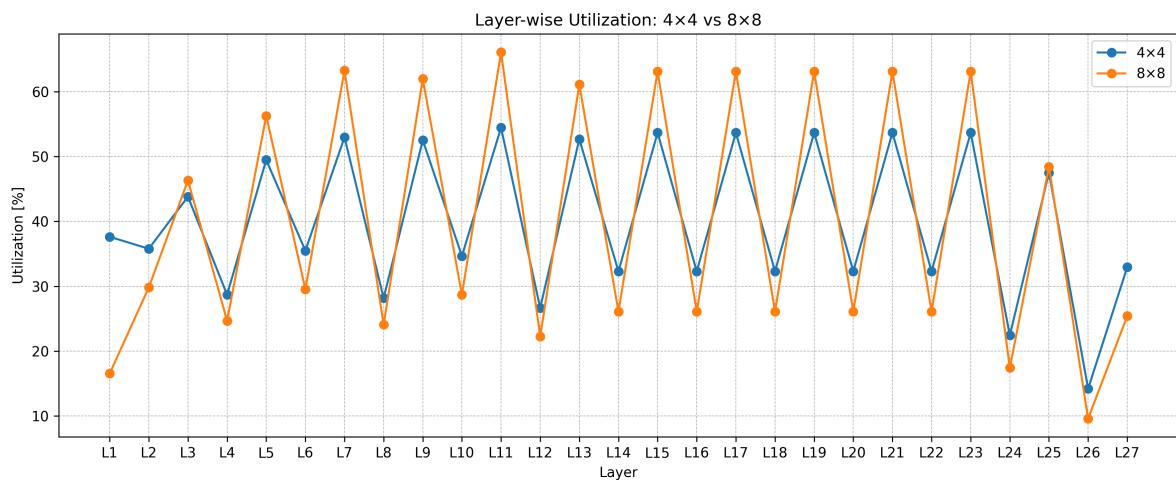
Die für die Utilization benötigte reale MAC-Anzahl MAC_{real} ergibt sich direkt aus den in den Grundlagen hergeleiteten Rechenaufwandsformeln. Die räumlichen Ausgabedimensionen OFH und OFW folgen aus Gleichung 2.1 und Gleichung 2.2. Abhängig vom Layer-Typ wird MAC_{real} wie folgt berechnet:

$$\text{MAC}_{\text{real}} = \begin{cases} \# \text{MACs} & \text{für Standard-Convolution gemäß Gleichung 2.3,} \\ \# \text{MAC}_{\text{SDW}} & \text{für Depthwise-Convolution gemäß Gleichung 2.5,} \\ \# \text{MAC}_{\text{SPW}} & \text{für Pointwise-Convolution gemäß Gleichung 2.7.} \end{cases} \quad (6.5)$$

Tabelle 6.6 fasst die *layerweise* End-to-End-Utilization für die 4×4 - und 8×8 -Konfiguration zusammen. Die zugehörige grafische Gegenüberstellung der Werte ist in Abbildung 6.2 dargestellt.

Tabelle 6.6: Layerweise Utilization für 4×4 und 8×8 .

Layer	Util 4×4 [%]	Util 8×8 [%]
L1	37.62	16.55
L2	35.78	29.82
L3	43.80	46.30
L4	28.70	24.68
L5	49.47	56.26
L6	35.48	29.54
L7	52.96	63.25
L8	28.17	24.09
L9	52.51	61.98
L10	34.66	28.68
L11	54.44	66.06
L12	26.65	22.27
L13	52.66	61.09
L14	32.30	26.08
L15	53.66	63.12
L16	32.30	26.08
L17	53.66	63.12
L18	32.30	26.08
L19	53.66	63.12
L20	32.30	26.08
L21	53.66	63.12
L22	32.30	26.08
L23	53.66	63.12
L24	22.45	17.42
L25	47.51	48.41
L26	14.21	9.61
L27	32.94	25.40

Abbildung 6.2: Layerweiser Vergleich für 4×4 und 8×8 .

L1 (Standard-Convolution): Warum ist $U_{4 \times 4} > U_{8 \times 8}$? Der erste Layer (L1) ist eine Standard-Convolution mit $\text{cfg_cin} = 3$ (RGB) und $\text{KE} = \text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin} = 27$. Obwohl die 8×8 -Variante mehr physische Parallelität bereitstellt ($\text{LANES_MAX} = \text{COLS_MAX} = 8$ statt 4), ist die *effektiv nutzbare Zeilenparallelität* in L1 durch die geringe Kanalzahl begrenzt: Bei nur drei Eingangskanälen können im Fetching-/Linebuffer-Pfad nur drei Kanalpfade (Line-Buffer/Window-RAM) gleichzeitig mit gültigen Daten versorgt werden. Damit bleiben in der 8×8 -Konfiguration zusätzliche Zeilenpfade ungenutzt, ohne dass sich die tatsächlich anfallende Rechenarbeit (MAC_{real}) entsprechend erhöht.

Zusätzlich fällt in L1 ein *Scan-Overhead* an, der in 4×4 und 8×8 *identisch* bleibt. Dieser Overhead entsteht in der sequentiellen *Scan-Phase*, in der die Eingabe-INFMAP adressiert und durch die Line-Buffer geschoben wird, um die $K \times K$ -Fenster zu rekonstruieren. Da diese Scan-Phase primär von der räumlichen Fensteranzahl $\text{OFH} \cdot \text{OFW}$ sowie Stride/Padding abhängt und bei $\text{cfg_cin} = 3$ nicht mit der Arrayfläche skaliert, bleibt ihr Zyklusanteil für beide Konfigurationen identisch. Dadurch wächst bei 8×8 der Normalisierungsfaktor der Utilization ($N_{\text{PE,active}} = \text{cfg_tile_r} \cdot \text{cfg_tile_c} = 64$) stärker als der Scan-Overhead abnimmt, was $U_{8 \times 8}$ gegenüber $U_{4 \times 4}$ reduziert.

Würde derselbe Layer eine höhere Kanalzahl besitzen (z. B. $\text{cfg_cin} = 16$), ergäben sich zugleich mehr *Kontextelemente* pro Output ($\text{KE} = \text{cfg_kh} \cdot \text{cfg_kw} \cdot \text{cfg_cin}$). Damit können die zusätzlichen Line-Buffer-/Window-RAM-Pfade der 8×8 -Variante besser mit gültigen Daten versorgt werden, und die Rechenphase pro Fenster wird länger, d. h. es werden mehr MAC-Schritte pro Fenster ausgeführt. Dadurch steigt die „Work-per-Overhead“-Relation: Der Scan-Overhead pro Fenster bleibt ähnlich, während die nutzbare Rechenarbeit pro Fenster mit KE zunimmt. Entsprechend ist für größere cfg_cin bzw. größere KE eine höhere Utilization der 8×8 -Variante zu erwarten.

Depthwise-Layer: Warum ist $U_{4 \times 4} > U_{8 \times 8}$? Bei der Depthwise-Convolution ist die Anzahl aktiver Processing-Elements durch den Modus fest vorgegeben: es ist lediglich die Diagonale aktiv. Damit arbeiten in der 4×4 -Variante effektiv 4 PEs und in der 8×8 -Variante 8 PEs parallel.

Der entscheidende Effekt für die Utilization in den Depthwise-Layern ist die kurze MAC-Sequenz pro Output: es gilt $\text{KE} = \text{cfg_kh} \cdot \text{cfg_kw} = 3 \cdot 3 = 9$. Damit ist die Multiply-Accumulate-Phase pro Fenster sehr kurz und nach wenigen Takten ist die Faltung pro Kanal abgeschlossen und es folgen erneut Scan-/Adressierungs- sowie Writeback-Anteile. Diese Fixkosten lassen sich bei $\text{KE} = 9$ nur begrenzt amortisieren. Da bei 8×8 der Normalisierungsfaktor der Utilization ($N_{\text{PE,active}} = 8$) doppelt so groß ist wie bei 4×4 , während sich der nicht-rechenlastige Anteil pro Fenster nicht im gleichen Maß reduziert, ergibt sich insgesamt eine geringere Utilization für 8×8 in den Depthwise-Layern.

Pointwise-Convolution (PW): Für die Pointwise-Layer L_i mit $i \in \{3, 5, 7, \dots, 23\}$ zeigt Tabelle 6.6, dass die End-to-End-Utilization in der 8×8 -Konfiguration durchgängig *höher* ist als in der 4×4 -Konfiguration. In L_{25} liegen die Utilization-Werte *sehr nahe beieinander*. Im letzten Pointwise-Layer L_{27} kehrt sich der Trend um: hier gilt $U_{4 \times 4} > U_{8 \times 8}$.

Der zentrale Grund für $U_{8 \times 8} > U_{4 \times 4}$ in den meisten PW-Layern liegt in der Struktur des Pointwise-Modus: Es werden *keine* $K \times K$ -Fenster aufgebaut, sondern pro räumlicher Position (x, y) ausschließlich die *Kanalvektoren* verarbeitet. Die Scan-Phase besteht daher im Wesentlichen darin,

die notwendigen Kanalwerte in *Beats* zu laden, bis alle $K_{\text{elems}} = \text{cfg_cin}$ Kontexte abgedeckt sind. Durch die größere Lane-Breite reduziert sich dabei die Anzahl der benötigten Beats: Bei 8×8 werden pro Beat 8 Kanäle erfasst, bei 4×4 nur 4. Beispielsweise gilt bei $\text{cfg_cin} = 32$: 8×8 benötigt $32/8 = 4$ Beats, 4×4 dagegen $32/4 = 8$ Beats. Da Pointwise-Layer typischerweise *große Kontextlängen* besitzen (32, 64, 128, ..., 512), ist die Rechenphase pro Pixelposition entsprechend lang, sodass die zusätzlich verfügbaren aktiven PEs (Pointwise: $N_{\text{PE,active}} = \text{cfg_tile_c}$, also 8 statt 4) effizient genutzt werden können. Mit abnehmender räumlicher Größe $\text{OFH} \cdot \text{OFW}$ gibt es weniger Pixelpositionen, für die der Pointwise-Ablauf wiederholt wird. Anschaulich: Bei großen Feature-Maps (z. B. 8×8 mit 64 Positionen) wird die Rechenpipeline lange „durchgehend“ beschäftigt; bei sehr kleinen Feature-Maps (z. B. 2×2 mit nur 4 Positionen) ist der Layer schnell beendet, und konstante Zusatzzyklen machen einen größeren Anteil an *cycles* aus. Dadurch sinkt die Utilization, weil

$$U = \frac{\text{MAC}_{\text{real}}}{\text{cycles} \cdot N_{\text{PE,active}}}$$

bei kleiner werdendem $\text{MAC}_{\text{real}} \propto (\text{OFH} \cdot \text{OFW})$ stärker abnimmt, während *cycles* nicht im selben Verhältnis schrumpft.

Im letzten Pointwise-Layer ist die räumliche Dimension extrem klein (2×2), d. h. es existieren nur 4 Pixelpositionen. In dieser Situation dominiert der konstante Layer-Overhead besonders stark. Gleichzeitig ist bei 8×8 der Normalisierungsfaktor $\text{cycles} \cdot N_{\text{PE,active}}$ größer, da im Pointwise-Modus $N_{\text{PE,active}} = \text{cfg_tile_c}$ gilt (8 statt 4). Wenn die Nutzarbeit aufgrund der nur vier Positionen nicht ausreicht, um diese Parallelität auszulasten, kann $U_{8 \times 8}$ trotz höherer theoretischer Parallelität unter $U_{4 \times 4}$ fallen.

6.3 Vergleich mit Referenzsystem

Bevor Speedup und Flächenbedarf verglichen werden, wird zunächst die erreichbare Taktfrequenz der jeweiligen Konfiguration betrachtet. Verglichen werden dabei (i) der in dieser Arbeit entwickelte KI-Beschleuniger in zwei Array-Varianten (4×4 und 8×8) sowie (ii) zwei Referenzsysteme auf CPU-Seite: ein skalare Ibex-Implementierung und ein Ibex-Vektor-Design mit unterschiedlichen VLEN-Konfigurationen. Dieser Vergleich ist notwendig, um den Nutzen der Spezialisierung quantitativ einzuordnen: Während Ibex als allgemeiner RISC-V-Core eine flexible Ausführung ermöglicht, zielt der KI-Beschleuniger auf maximalen Durchsatz für Faltungsoperationen durch parallele MAC-Ausführung und lokale Datenweitergabe im systolischen Array. Da die Ausführungszeit neben der Zyklusanzahl maßgeblich von der Taktperiode abhängt, wird der *kritische Pfad* aus der Timing-Analyse herangezogen, weil er die minimale Taktperiode und damit die maximal mögliche Frequenz begrenzt. Abbildung 6.3 zeigt die Auswertung grafisch; die zugehörigen Zahlenwerte sind in Tabelle 6.7 zusammengefasst.

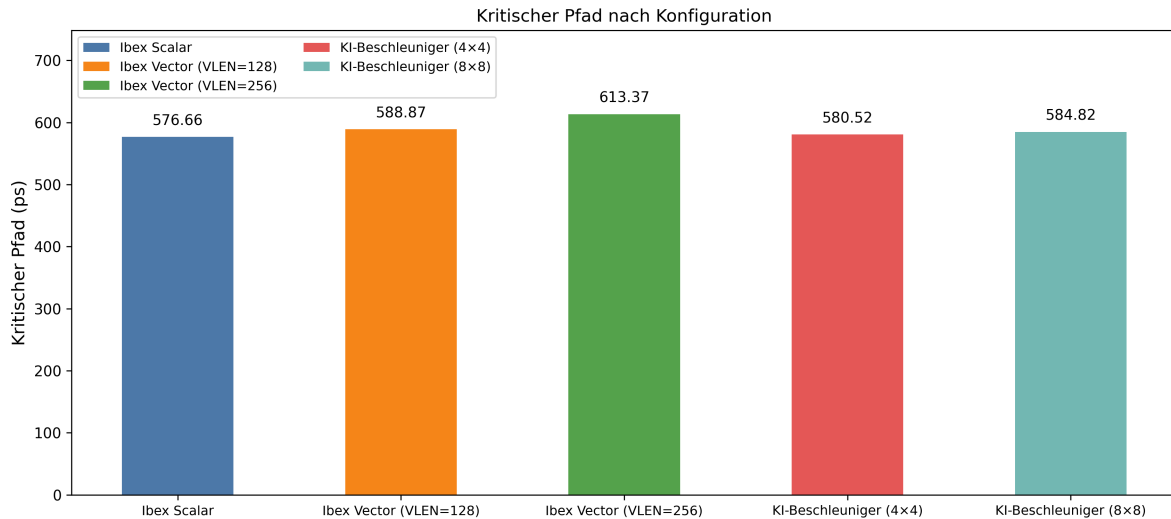


Abbildung 6.3: Kritischer Pfad der untersuchten Konfigurationen.

Tabelle 6.7: Kritischer Pfad, minimale Taktperiode und maximale Taktfrequenz.

Konfiguration	Kritischer Pfad (ps)	$T_{\min}$ (ps)	$f_{\max}$ (GHz)
Ibex Scalar	576.66	576.66	1.734
Ibex Vector (VLEN=128)	588.87	588.87	1.698
Ibex Vector (VLEN=256)	613.37	613.37	1.630
KI-Beschleuniger (4x4)	580.52	580.52	1.723
KI-Beschleuniger (8x8)	584.82	584.82	1.710

Die physikalische Ausführungszeit eines vollständigen Laufs wird über die Zyklusanzahl *Cycles* bestimmt:

$$t = \text{Cycles} \cdot T_{\min}. \quad (6.6)$$

Abbildung 6.4 zeigt den Vergleich der Ausführungszeiten grafisch; die zugehörigen Zahlenwerte sind in Tabelle 6.8 zusammengefasst.

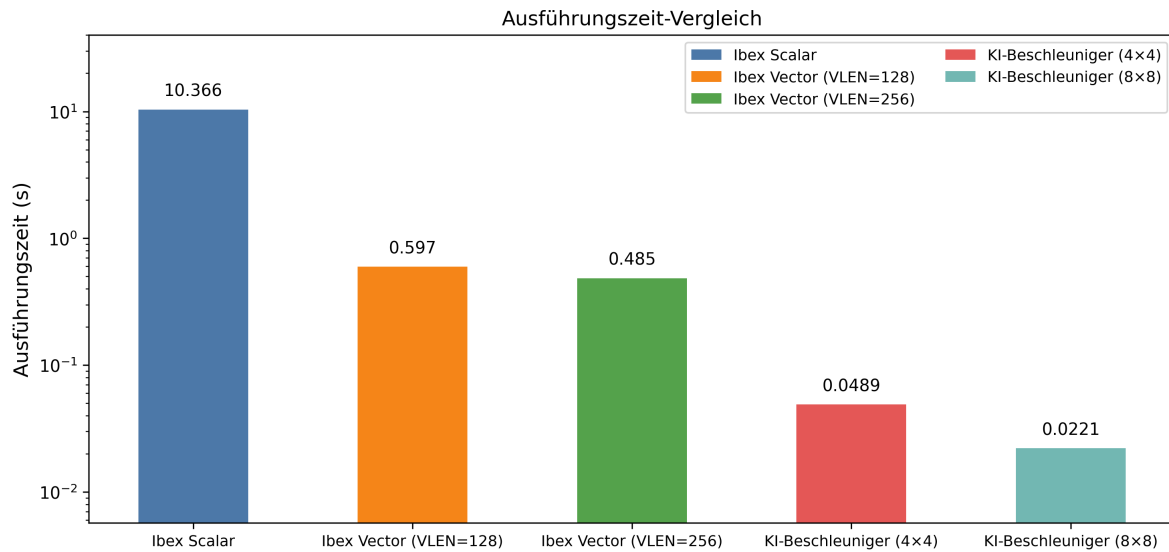


Abbildung 6.4: Ausführungszeit der MobileNetV1-Inferenz für Ibex (Scalar/Vektor) und den KI-Beschleuniger (4×4 / 8×8).

Tabelle 6.8: Ausführungszeit der MobileNetV1-Inferenz.

Konfiguration	Ausführungszeit t (s)
Ibex Scalar	10.366
Ibex Vector (VLEN=128)	0.597
Ibex Vector (VLEN=256)	0.485
KI-Beschleuniger (4×4)	0.0489
KI-Beschleuniger (8×8)	0.0221

Auf Basis von Gleichung 6.6 wird der Speedup als Verhältnis der Ausführungszeiten definiert:

$$\text{Speedup}_t = \frac{t_{\text{Basis}}}{t_{\text{neu}}}.$$

Die resultierenden Speedups und die prozentuale Zeitverkürzung sind in Tabelle 6.9 angegeben.

Tabelle 6.9: Speedup und Zeitverkürzung.

Vergleich (neu vs. Basis)	Speedup	Zeitverkürzung (%)
KI-Beschleuniger (4×4) vs. Ibex Scalar	212.07	99.53
KI-Beschleuniger (4×4) vs. Ibex Vector (VLEN=128)	12.22	91.82
KI-Beschleuniger (4×4) vs. Ibex Vector (VLEN=256)	9.92	89.92
KI-Beschleuniger (8×8) vs. Ibex Scalar	468.92	99.79
KI-Beschleuniger (8×8) vs. Ibex Vector (VLEN=128)	27.01	96.30
KI-Beschleuniger (8×8) vs. Ibex Vector (VLEN=256)	21.94	95.44
KI-Beschleuniger (8×8) vs. KI-Beschleuniger (4×4)	2.21	54.77

Nach dem Laufzeitvergleich wird der Ressourcenbedarf der untersuchten Konfigurationen betrachtet. Abbildung 6.5 zeigt die synthetisierte Zellfläche (in μm^2) für Ibex (Skalar/Vektor) sowie den KI-Beschleuniger in den beiden Array-Größen. Die exakten Werte sind in Tabelle 6.10 angegeben.

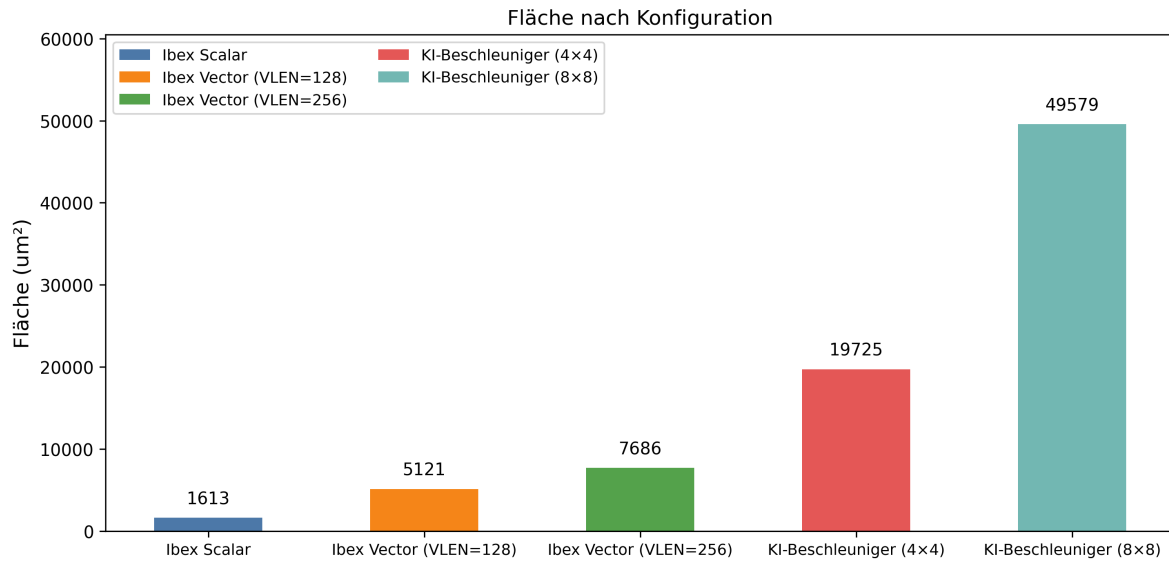


Abbildung 6.5: Flächenbedarf der untersuchten Konfigurationen.

Tabelle 6.10: Flächenbedarf der untersuchten Konfigurationen (KI-Beschleuniger ohne RAM-Makros)

Konfiguration	Fläche (μm^2)
Ibex Scalar	1613
Ibex Vector (VLEN=128)	5121
Ibex Vector (VLEN=256)	7686
KI-Beschleuniger (4x4)	19725.136
KI-Beschleuniger (8x8)	49578.692

Zur kompakten Quantifizierung des Flächenmehraufwands wird ein Skalierungsfaktor definiert als:

$$\alpha_A = \frac{A_{\text{neu}}}{A_{\text{Basis}}}. \quad (6.7)$$

Tabelle 6.11 fasst die resultierenden Flächenfaktoren für relevante Vergleiche zusammen.

Tabelle 6.11: Skalierungsfaktor α_A relativ zu verschiedenen Baselines (KI-Beschleuniger ohne RAM-Makros).

Vergleich (neu vs. Basis)	α_A ($\times$)
KI-Beschleuniger (4×4) vs. Ibex Scalar	12.23
KI-Beschleuniger (4×4) vs. Ibex Vector (VLEN=128)	3.85
KI-Beschleuniger (4×4) vs. Ibex Vector (VLEN=256)	2.57
KI-Beschleuniger (8×8) vs. Ibex Scalar	30.74
KI-Beschleuniger (8×8) vs. Ibex Vector (VLEN=128)	9.68
KI-Beschleuniger (8×8) vs. Ibex Vector (VLEN=256)	6.45
KI-Beschleuniger (8×8) vs. KI-Beschleuniger (4×4)	2.51

Beim Übergang von 4×4 auf 8×8 steigt die Anzahl physisch instanziierten Processing Elements von $N_{PE} = 16$ auf $N_{PE} = 64$ (Faktor $\times 4$), der gemessene Speedup der gesamten MobileNetV1-Inferenz beträgt jedoch nur 2.21. Dieser Unterschied ist konsistent mit der in Abbildung 6.1 und Abbildung 6.2 gezeigten *layerweisen Auslastung*: MobileNetV1 besteht überwiegend aus Depthwise- und Pointwise-Layern, die das Array nur *teilweise* aktivieren. Während im Convolution-Modus das Array vollständig genutzt wird, ist im Depthwise-Modus lediglich die Diagonale aktiv (d. h. 25 % bei 4×4 und 12.5 % bei 8×8) und im Pointwise-Modus nur eine Zeile (ebenfalls 25 % bzw. 12.5 %). Damit führt die Vergrößerung des Arrays zwar zu mehr *Peak*-Parallelität, der *effektive* Gesamtspeedup wird jedoch durch die dominierenden Layer-Typen mit reduzierter PE-Auslastung begrenzt. Zusätzlich zeigt die Laufzeitverteilung, dass der größte Anteil der Zyklen in Pointwise-Layern anfällt. Tabelle 6.12 fasst die Zyklusanteile nach Layer-Typ zusammen.

Tabelle 6.12: Zyklusanteile nach Layer-Typ für MobileNetV1.

Layer-Typ	Cycles [Mio]	Anteil Cycles [%]
Standard Conv	388.25	2.15
Pointwise Conv	16911.90	94.00
Depthwise Conv	676.15	3.76
Summe	17976.30	100.00

Im Vergleich zu den CPU-basierten Ibex-Varianten zeigt der KI-Beschleuniger einen deutlich höheren Speedup. Ibex ist als *Allzweckprozessor* für kontrollflusslastige, heterogene Aufgaben ausgelegt, während der vorgestellte KI-Beschleuniger einen *spezialisierten Datenpfad* für CNN-Operationen bereitstellt. Die Spezialisierung verlagert Aufwand von Software in Hardware (Datenflüsse, parallele MAC-Ausführung und lokale Datenweitergabe im SA) und führt damit zu deutlich kürzeren Ausführungszeiten. Dieser Gewinn wird jedoch typischerweise mit einem höheren Flächenbedarf erkauft, da zusätzliche Parallelität und Datenpfadlogik implementiert werden müssen. Insgesamt zeigt der Vergleich damit den klassischen Zielkonflikt: Spezialisierung kostet Fläche und Designkomplexität, spart aber Laufzeit und kann die Energieeffizienz erhöhen.

6.4 Diskussion

Die Ergebnisse der Front-End-Synthese und der Performanceanalyse zeigen, dass der KI-Beschleuniger die angestrebte Taktperiode in beiden Array-Varianten einhalten kann und damit die funktionale Auslegung auch unter einem strikten Timing-Constraint praktisch nutzbar bleibt. Gleichzeitig wird deutlich, dass die Fläche mit der Arraygröße erwartungsgemäß ansteigt, jedoch nicht proportional zur reinen PE-Anzahl. Diese Beobachtung ist konsistent mit den identifizierten Skalierungsanteilen in Fetching-, Buffer- und Kontrolllogik, die nicht in gleicher Weise wie die PE-Matrix vervierfacht werden, sondern je nach Modul näherungsweise linear mit LANES_MAX bzw. COLS_MAX wachsen.

Der Vergleich der beiden Array-Konfigurationen zeigt außerdem, dass der End-to-End-Speedup beim Übergang von 4×4 auf 8×8 deutlich kleiner ausfällt als der Faktor der physisch instanziierten Processing Elements. Der Hauptgrund liegt in der Arbeitslastverteilung von MobileNetV1: Der größte Zyklusanteil entsteht in Pointwise-Layern, während Depthwise-Layer zwar häufig auftreten, jedoch nur eine kurze Kontextlänge besitzen. In beiden Modi wird das Array strukturell nur teilweise aktiviert (Pointwise: spaltenparallel mit geringer Zeilenaktivität, Depthwise: diagonale Aktivität). Damit erhöht ein größeres Array zwar die Peak-Parallelität, der Gesamtdurchsatz wird aber durch die dominierenden Layer-Typen und deren begrenzte Auslastung geprägt. Die Utilization-Werte stützen diese Einordnung und machen sichtbar, in welchen Layern die zusätzliche Hardware parallel arbeiten kann und in welchen Fällen Overhead-Anteile relativ stärker ins Gewicht fallen.

Im Vergleich zu den Referenzsystemen liefert der Beschleuniger einen klaren Laufzeitvorteil, was die zentrale Zielsetzung der Arbeit unterstützt: die Beschleunigung von CNN-Inferenz durch einen spezialisierten Datenpfad. Der Vergleich ist insbesondere deshalb aussagekräftig, weil er zwei grundsätzlich unterschiedliche Ansätze gegenüberstellt: Ein allgemeiner RISC-V-Core bietet Flexibilität und ist für heterogene, kontrollflusslastige Aufgaben ausgelegt, während der KI-Beschleuniger den Rechenfluss der Faltungsoperationen in Hardware abbildet und durch lokale Datenweitergabe im systolischen Array sowie parallele MAC-Ausführung einen deutlich höheren Durchsatz erreicht. Gleichzeitig wird der klassische Zielkonflikt sichtbar: Die erzielte Beschleunigung geht mit einem höheren Logikflächenbedarf einher, wobei in dieser Arbeit die SRAM-Makros nicht in der Flächenangabe enthalten sind und somit die reine Standard-Cell-Logik bewertet wird.

Abschließend wird der Anwendungsbereich der aktuellen Implementierung über MobileNetV1 hinaus eingeordnet. Im Fokus stehen dabei CNN-Architekturen, deren Datenfluss vom MobileNetV1-typischen Muster abweicht. Anhand zweier etablierter Modelle (VGG16 und LeNet-5) wird diskutiert, welche Einschränkungen sich hinsichtlich Layer-Typen, Kernelgrößen und Downsampling-Strukturen ergeben und welche Erweiterungen für eine breitere Modellunterstützung naheliegend sind.

Als Referenz werden zwei etablierte CNN-Modelle betrachtet: **VGG16** als tiefes, klassisches Convolutional Network mit wiederholten 3×3 -Convolutions und nachgelagerten Fully-Connected-Layern [37], sowie **LeNet-5** als frühe CNN-Architektur mit größeren Convolution-Kernen und vollständig verbundenen Schichten [8]. Zur Orientierung werden beide Modellarchitekturen zunächst tabellarisch zusammengefasst.

Tabelle 6.13: VGG16-Architektur (Input $224 \times 224 \times 3$) [5].

Stufe	Operation	Feature Maps	Output Size	Kernel	Stride	Akt.
Input	Image	3	$224 \times 224 \times 3$	–	–	–
1	2× Conv	64	$224 \times 224 \times 64$	3×3	1	ReLU
2	MaxPool	64	$112 \times 112 \times 64$	2×2	2	–
3	2× Conv	128	$112 \times 112 \times 128$	3×3	1	ReLU
4	MaxPool	128	$56 \times 56 \times 128$	2×2	2	–
5	2× Conv	256	$56 \times 56 \times 256$	3×3	1	ReLU
6	MaxPool	256	$28 \times 28 \times 256$	2×2	2	–
7	3× Conv	512	$28 \times 28 \times 512$	3×3	1	ReLU
8	MaxPool	512	$14 \times 14 \times 512$	2×2	2	–
9	3× Conv	512	$14 \times 14 \times 512$	3×3	1	ReLU
10	MaxPool	512	$7 \times 7 \times 512$	2×2	2	–
11	FC (Flatten)	–	25088	–	–	ReLU
12	FC	–	4096	–	–	ReLU
13	FC	–	4096	–	–	ReLU
Output	FC (Classifier)	–	1000	–	–	Softmax

Tabelle 6.14: LeNet-5-Architektur (Input $32 \times 32 \times 1$) [6].

Layer	Operation	Feature Maps	Output Size	Kernel	Stride	Akt.
Input	Image	1	$32 \times 32 \times 1$	–	–	–
1	Convolution	6	$28 \times 28 \times 6$	5×5	1	tanh
2	Avg Pooling	6	$14 \times 14 \times 6$	2×2	2	tanh
3	Convolution	16	$10 \times 10 \times 16$	5×5	1	tanh
4	Avg Pooling	16	$5 \times 5 \times 16$	2×2	2	tanh
5	Convolution	120	$1 \times 1 \times 120$	5×5	1	tanh
6	FC	–	84	–	–	tanh
Output	FC (Classifier)	–	10	–	–	softmax

Aktivierungsfunktionen. In der aktuellen Implementierung ist ausschließlich die Aktivierungsfunktion **ReLU** als Hardwarepfad vorgesehen. Andere in klassischen CNNs häufig genutzte Aktivierungen wie **tanh** (z. B. in LeNet-5) sowie **Softmax** als Klassifikationsausgabe (z. B. in VGG16 und LeNet-5) sind derzeit *nicht* als dedizierte Hardware-Operationen integriert. Für eine Inferenz solcher Modelle müssten diese Funktionen entweder außerhalb des Beschleunigers (auf dem Host/Prozessor) ausgeführt oder durch zusätzliche Rechenpfade im Design ergänzt werden.

Kernelgröße und Window-Packung. In der aktuellen Implementierung ist die Breite eines in die Window-RAMs geschriebenen Datenworts durch:

$$\text{PACKBITS} = \text{LANES_MAX} \cdot \text{K_MAX} \cdot \text{PXW}$$

gegeben. Damit hängt die pro Takt bzw. pro Word-Adresse abgelegte Fensterrepräsentation direkt von der maximal unterstützten Kernelgröße K_MAX ab. In den beiden synthetisierten Konfigurationen (4×4 und 8×8) wurde $K_MAX = 3$ gewählt. Diese Parametrierung ist ausreichend für MobileNetV1 und VGG16. Für LeNet-5 hingegen werden in den frühen Convolution-Stufen klassische 5×5 -Kerne verwendet. Um diese Layer ohne Zerlegung oder Mehrfachdurchläufe abzubilden, muss K_MAX auf 5 erhöht werden. Nur dann können die durch den Line-Buffer generierten 5-Pixel-Fenster pro Lane vollständig in einem gepackten Datenwort abgelegt werden, sodass die entsprechenden Window-Daten pro Adresse der Window-RAMs konsistent und ohne zusätzliche Fragmentierung gespeichert werden.

Kanalblocking bei Convolution ($C_{in} > LANES_MAX$). In der aktuellen Implementierung tritt der Fall $C_{in} > LANES_MAX$ im betrachteten MobileNetV1-Setup praktisch nicht auf, da lediglich im ersten Layer eine reguläre Convolution ausgeführt wird und dort $C_{in} = 3$ gilt. Für andere Netze (z. b. VGG16) ist C_{in} jedoch größer, sodass die Eingabekanäle *blockweise* verarbeitet werden müssen.

Im Gegensatz zur Depthwise-Convolution werden bei einer regulären Convolution die Beiträge *über alle Eingabekanäle* aufsummiert. Wird C_{in} daher in Blöcke $b = 0, \dots, B - 1$ aufgeteilt, entsteht nach jedem Block zunächst nur eine **Partialsommen**-Ausgabe (nicht das finale Output-Featuremap). Um das korrekte Endergebnis zu erhalten, müssen die Partialsommen aller Blöcke pro Output-Element akkumuliert werden:

$$y = \sum_{b=0}^{B-1} y_b.$$

Abbildung 6.6 skizziert ein mögliches Erweiterungskonzept: Ein *Partialsommen-Controller* schreibt die Blockergebnisse zunächst in eine *Partialsommen_SRAM*. Nach Abschluss aller Kanalblöcke liest ein *Akkumulator* diese Werte aus, summiert sie über b auf und übergibt das finale Resultat an den *Write-Back-Controller*, der die fertigen Output-Featuremaps in die *I/O_SRAM* zurückschreibt. Damit wäre eine korrekte Convolution-Ausführung auch für $C_{in} > LANES_MAX$ möglich.

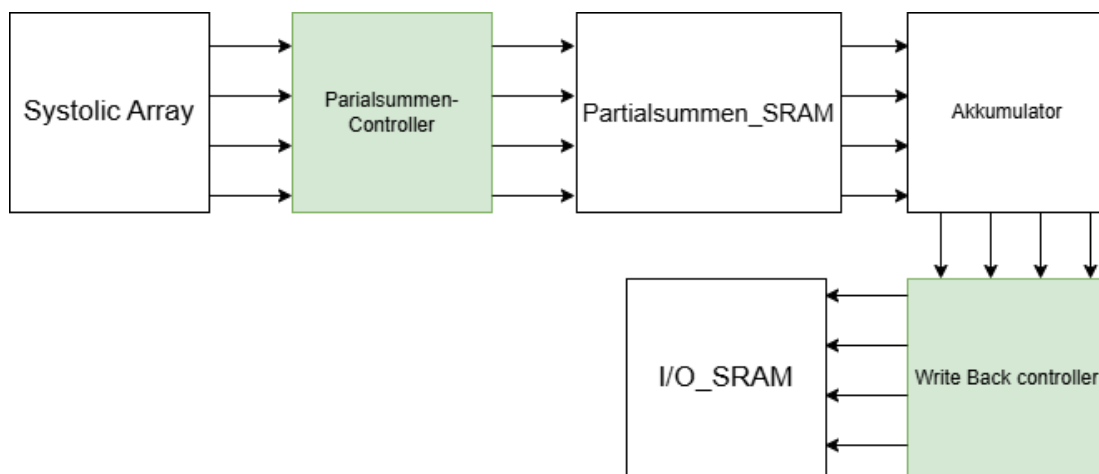


Abbildung 6.6: Konzept zur Blockverarbeitung bei *Convolution* mit $C_{in} > LANES_MAX$: Zwischenspeicherung und Akkumulation von Partialsommen über mehrere Kanalblöcke.

7 Zusammenfassung und Ausblick

Die in dieser Arbeit entwickelte Architektur ist als KI-Beschleuniger konzipiert, der in eine RISC-V-basierte System-on-Chip-(SoC)-Plattform integrierbar ist. Ziel ist es, Faltungsoperationen und verwandte CNN-Primitive durch einen spezialisierten Datenpfad hardwareseitig zu beschleunigen und dadurch Latenz sowie Energiebedarf im Edge-Umfeld zu reduzieren. Als Referenzmodell wurde MobileNetV1 herangezogen, da dieses Netzwerk durch *depthwise separable convolutions* ein günstiges Verhältnis aus Genauigkeit, Rechenaufwand und Modellgröße bietet und daher häufig in ressourcenbeschränkten Edge-Systemen eingesetzt wird.

Für eine systemfähige Einbindung in ein SoC setzt der Beschleuniger auf eine standardisierte AXI-Anbindung. Die Laufzeitkonfiguration erfolgt über speichergemappede Konfigurationsregister (CSR), die via AXI4-Lite beschrieben werden und die wesentlichen Layerparameter in die Hardware übernehmen. Der Transfer großer Datenmengen wird über ein AXI4-Master-Interface in Kombination mit einem integrierten DMA-Controller realisiert, der Aktivierungen, Gewichte und Requantisierungsparameter blockweise und weitgehend autonom zwischen Host-Speicher und internen SRAM-Strukturen bewegt. Dadurch wird die Datenbewegung von der Rechenpipeline entkoppelt, der Host-Prozessor bei einer späteren SoC-Integration entlastet und die Ausführung über CSR-Programmierung deterministisch und reproduzierbar gesteuert.

Im zweiten Schritt wurden die charakteristischen Eigenschaften von CNN-basierten KI-Anwendungen analysiert. Dabei stehen insbesondere zwei Wiederverwendungsmechanismen im Vordergrund: *Input-Reuse*, da Eingabepixel aufgrund überlappender Fenster mehrfach in benachbarten Faltungspositionen genutzt werden, sowie *Weight-Reuse*, da ein Filter über viele Fensterpositionen hinweg wiederholt angewendet wird. Diese Reuse-Eigenschaften lassen sich in einem *systolischen Array* besonders effizient ausnutzen, da Operanden lokal zwischen benachbarten Processing Elements weitergereicht werden und Teilsummen im Array verbleiben.

Die resultierende Mikroarchitektur adressiert zentrale Herausforderungen der Datenbereitstellung. Aktivierungen werden zeilenweise verarbeitet, sodass gültige $K \times K$ -Fenster identifiziert und in einer nachfolgenden Streaming-Phase takt synchron an das systolische Array ausgegeben werden können. Für die Gewichte wurde ein speicherlayout gewählt, das die spaltenparallele Verarbeitung unterstützt und eine stalfreie Versorgung des Arrays mit Gewichtsvektoren ermöglicht.

Für die Verifikation der entwickelten Architektur wurde eine stufenweise Validierungsstrategie umgesetzt. Zunächst wurden die wesentlichen Funktionsblöcke des Datenpfads isoliert geprüft: das Arithmetikmodul, die Aktivierungsfunktion *ReLU* sowie das 2×2 -Pooling in beiden Varianten *Average-Pooling* und *Max-Pooling*. Darüber hinaus wurden unterschiedliche Konfigurationen von *Stride* und *Padding* getestet, um die Robustheit der Steuer- und Datenpfade gegenüber variierenden Layerparametern abzusichern. Ergänzend wurde durch *Integrationstests* nachgewiesen, dass die implementierten Module nicht nur isoliert korrekt arbeiten, sondern auch im Gesamtdatenpfad konsistent zusammenwirken: Insbesondere wurden die Handshake- und Synchronisationsmechanismen zwischen Fetching-, Arithmetik- und Writeback-/Pooling-Pfad verifiziert, sodass die Übergabe von Operanden sowie die Rückschreibung der Ergebnisse deterministisch und

ohne Datenverlust erfolgt. Abschließend wurde die Gesamtfunktionalität des Beschleunigers durch eine End-to-End-Ausführung des vollständigen MobileNetV1-Datenpfads validiert.

In der Evaluation wurden die zentralen Kenngrößen *Timing*, *Fläche* und *Performance* des KI-Beschleunigers analysiert. Die Syntheseergebnisse zeigen, dass das vorgegebene Timing-Constraint von $T_{\text{clk}} = 600$ ps in beiden Array-Konfigurationen eingehalten wird; damit ergeben sich maximale Taktfrequenzen von etwa 1.72 GHz für die 4×4 -Variante und etwa 1.71 GHz für die 8×8 -Variante. Die Fläche steigt erwartungsgemäß mit der Arraygröße an: Für 4×4 beträgt sie $19,725 \mu\text{m}^2$, während für 8×8 $49,579 \mu\text{m}^2$ erreicht werden, was einer Skalierung um den Faktor ≈ 2.51 entspricht. Die Performanceanalyse belegt einen deutlich höheren Durchsatz der größeren Konfiguration: Über alle MobileNetV1-Layer ergibt sich ein mittlerer MAC-Throughput von etwa 6.72 GMAC/s (4×4) bzw. 14.19 GMAC/s (8×8). Der daraus resultierende Laufzeitgewinn gegenüber Ibex ist signifikant, wird jedoch durch einen erhöhten Logikflächenbedarf erkauft.

Aus den Validierungs- und Evaluationsergebnissen liefert diese Arbeit einen konsistenten Nachweis, dass der entwickelte systolische Array-Beschleuniger als systemfähig ausgelegtes Hardwaremodul konzipiert ist. Die Architektur besitzt eine AXI-basierte Steuer- und Datenanbindung: Konfigurations- und Kontrollregister werden über ein memory-mapped Interface adressiert, während ein integrierter DMA-Controller die Übertragung von Aktivierungen, Gewichten sowie Requantisierungsparametern autonom aus dem Hauptspeicher in lokale On-Chip-SRAMs übernimmt. Dieses Kopplungskonzept entspricht dem typischen Integrationsmodell eines Coprozessors in einer RISC-V-basierten SoC-Architektur und entkoppelt die Datenbewegung von der Rechenpipeline. Darüber hinaus wurde die Synthese unter demselben Timing-Constraint von $T_{\text{clk}} = 600$ ps durchgeführt, das auch für den betrachteten RISC-V-Prozessor gilt, wodurch die prinzipielle Ko-Existenz beider Einheiten unter einem gemeinsamen Taktbudget abgesichert ist. Insgesamt zeigt die Arbeit damit, dass ein systolischer Array-Ansatz in Kombination mit einer tile- und blockbasierten Datenbereitstellung sowie standardisierten AXI/DMA-Schnittstellen einen praktikablen Weg darstellt, um MobileNet-ähnliche CNN-Workloads in ressourcenbeschränkten Edge-SoCs signifikant zu beschleunigen.

Als Ausblick ergeben sich mehrere Ansatzpunkte zur Erweiterung des Anwendungsbereichs. Dazu zählen insbesondere die Unterstützung weiterer Aktivierungsfunktionen, die Erweiterung auf größere Kernel sowie Mechanismen zur Blockverarbeitung und Akkumulation von Partialsummen für reguläre Convolutions mit $C_{\text{in}} > \text{LANES_MAX}$. Darüber hinaus bietet die beobachtete Auslastungscharakteristik in MobileNetV1 Hinweise auf Optimierungspotential in den Modi mit teilweiser Arrayaktivierung, etwa durch reduzierte Overheadanteile, alternative Tiling-Strategien oder eine weitergehende Parallelisierung in Pointwise-Schichten.

Literaturverzeichnis

- [1] A. Hassan, M. Ali, N. M. Durrani, and M. A. Tahir, "An empirical analysis of deep learning architectures for vehicle make and model recognition," 2021, abbildung: "The MobileNet architecture" (ResearchGate). [Online]. Available: https://www.researchgate.net/figure/The-MobileNet-architecture-15-MobileNet-is-built-on-depthwise-separable-convolutions_fig5_352621942
- [2] Z. Ehtesham. (2022, Oct.) Explained: Mobilenet v1. Medium, Zugriff am 22.12.2025. [Online]. Available: <https://medium.com/@zainehtesham/explained-mobilenet-v1-bb700abfe824>
- [3] L. A. Aranda, P. Reviriego, and J. A. Maestro, "Protecting image processing pipelines against configuration memory errors in sram-based fpgas," *Electronics*, vol. 7, no. 11, p. 322, 2018.
- [4] Arm Ltd., "Amba axi and ace protocol specification," Technical Specification, 2023, version 2.0. [Online]. Available: <https://developer.arm.com/documentation/ih0022/latest/>
- [5] blurredmachine. (n.d.) Vggnet-16 architecture: A complete guide. Kaggle Notebook. Accessed: 2026-01-25. [Online]. Available: <https://www.kaggle.com/code/blurredmachine/vggnet-16-architecture-a-complete-guide>
- [6] S. B. (2024) Lenet-5 architecture explained. Medium. Accessed: 2026-01-25. [Online]. Available: <https://medium.com/@siddheshb008/lenet-5-architecture-explained-3b559cb2d52b>
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [9] N. P. Jouppi, C. Young, N. Patil, and D. Patterson, "A domain-specific architecture for deep neural networks," *Communications of the ACM*, vol. 61, no. 9, pp. 50–59, 2018.
- [10] N. P. Jouppi, C. Young, N. Patil *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [11] Y. Wang, G.-Y. Wei, and D. Brooks, "Benchmarking tpu, gpu, and cpu platforms for deep learning," *arXiv preprint arXiv:1907.10701*, 2019.
- [12] R. Xu, S. Ma, Y. Guo, and D. Li, "A survey of design and optimization for systolic array-based dnn accelerators," *ACM Computing Surveys*, vol. 56, no. 1, 2023.
- [13] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982.
- [14] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>

- [15] Z. Yang, L. Wang, D. Ding, X. Zhang, Y. Deng, S. Li, and Q. Dou, "Systolic array based accelerator and algorithm mapping for deep learning algorithms," in *Network and Parallel Computing*, ser. Lecture Notes in Computer Science, F. Zhang, J. Zhai, M. Snir, H. Jin, H. Kasahara, and M. Valero, Eds., vol. 11276. Springer International Publishing, 2018, pp. 153–158.
- [16] R. Xu, S. Ma, Y. Wang, and Y. Guo, "Cmsa: Configurable multi-directional systolic array for convolutional neural networks," in *2020 IEEE 38th International Conference on Computer Design (ICCD)*, 2020.
- [17] H. T. Kung, B. McDanel, and S. Q. Zhang, "Packing sparse convolutional neural networks for efficient systolic array implementations: Column combining under joint optimization," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Association for Computing Machinery, 2019, pp. 821–834.
- [18] S. Selvam, V. Ganesan, and P. Kumar, "Fuseconv: Fully separable convolutions for fast inference on systolic arrays," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 651–656.
- [19] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [20] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of dnn accelerators using scale-sim," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020, pp. 58–68.
- [21] H. Genc, S. Kim, A. Amid *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 769–774.
- [22] Vishal, "Understanding fp32, fp16 and int8 precision in deep learning models — why int8 calibration is important," Medium, 2024, accessed: 2025-11-06. [Online]. Available: <https://medium.com/@vishalinddev/understanding-fp32-fp16-and-int8-precision-in-deep-learning-models-why-int8-calibration-is-5406b1c815a8>
- [23] AMD Xilinx, "Axi dma v7.1 logicore ip product guide (pg021)," Product Guide, 2022. [Online]. Available: https://docs.xilinx.com/r/en-US/pg021_axi_dma
- [24] T. Benz, M. Schaffner, and L. Benini, "A high-performance, energy-efficient modular dma engine architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [25] J. Fornt, P. Fontova-Musté, M. Caro, J. Abella, F. Moll, J. Altet, and C. Studer, "An energy-efficient gemm-based convolution accelerator with on-the-fly im2col," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 11, pp. 1874–1878, 2023.
- [26] TensorFlow Team, "Tensorflow lite: On-device machine learning," Documentation, n.d. [Online]. Available: <https://www.tensorflow.org/lite>
- [27] M. Mathew, K. Desappan, P. K. Swami, and S. Nagori, "Sparse, quantized, full frame cnn for low power embedded devices," in *2017 IEEE Conference on Computer Vision and Pattern*

- Recognition Workshops (CVPRW)*, 2017, pp. 328–336.
- [28] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, and H. Yang, “Angel-eye: A complete design flow for mapping cnn onto embedded fpga,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 35–47, 2018.
 - [29] E. Baek, D. Kwon, and J. Kim, “A multi-neural network acceleration architecture,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 940–953.
 - [30] A. Samajdar, E. Qin, M. Pellauer, and T. Krishna, “Self adaptive reconfigurable arrays (sara): Learning flexible gemm accelerator configuration and mapping-space using ml,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC ’22)*, 2022, pp. 583–588.
 - [31] M. Putic, A. Buyuktosunoglu, S. Venkataramani, P. Bose, S. Eldridge, and M. Stan, “Dyhard-dnn: Even more dnn acceleration with dynamic hardware reconfiguration,” in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, 2018, pp. 1–6.
 - [32] G. Shomron, T. Horowitz, and U. Weiser, “Smt-sa: Simultaneous multithreading in systolic arrays,” *IEEE Computer Architecture Letters*, vol. 18, no. 2, pp. 99–102, 2019.
 - [33] B. Wang, S. Ma, Z. Liu, L. Huang, Y. Yuan, and Y. Dai, “Sadd: A novel systolic array accelerator with dynamic dataflow for sparse gemm in deep learning,” in *Network and Parallel Computing (NPC 2022)*, 2022, pp. 42–53.
 - [34] Arm Ltd., “Amba axi4-lite protocol specification,” Technical Specification, 2023, part of AMBA AXI specification. [Online]. Available: <https://developer.arm.com/architectures/system-architectures/amba/amba-specifications>
 - [35] Arm Ltd., “Corelink dma-330: High-performance data transfer — features and benefits,” Product Page, n.d. [Online]. Available: <https://www.arm.com/products/silicon-ip-system/embedded-system-design/dma-330>
 - [36] Arm Ltd., “Corelink dma-350: Next generation direct memory access for endpoint ai,” Arm Community Blog, 2022. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/internet-of-things-blog/posts/arm-corelink-dma-350-next-generation-direct-memory-access-for-endpoint-ai>
 - [37] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>